GAMEPLAY FRAMEWORK • 源码解析 + CODE REVIEW + 面试追问

从 Unity Gameplay Framework 到游戏客户端面试

以 MyGamePlayArchitecture（Unity 2022.3）的真实源码为唯一依据：World / Actor / Pawn / Controller / GameMode / GameState 怎么组织，TimerSystem 的 Slot Map 为什么这样设计、哪里有漏洞，SoulDecay 夺舍 Demo 如何把这些模块串起来；再从每一处设计往下追到数据结构、GC、操作系统与引擎知识，练到能扛住 30～60 分钟的连续追问。

已写：第 0～13、11B、15 章（新增 11B 对象池落地设计）

2 个交互实验（Slot Map / 掉帧追赶）

C++ 移植实测 E1~E14 · F1~F5 · G1~G10 · H1~H5 + 基准 2 组

每章：四层答案 · 追问树 · 错误回答 · 发散题

CHAPTER 00

读前须知与面试地图

这本书怎么读、书里的话哪些是事实哪些是推测，以及——面试官会从这个项目的哪一行代码，一路追问到哪里。

0.1 事实等级：每一句话都要分清楚

标记	含义	例子
【源码事实】	当前 commit（427d7c1）真实的行为，逐行读过	generation 初始值是 0（TimerSystem.cs:197）
【设计意图】	从命名、注释、提交信息推测的目标，不等于已经实现	注释说 TimerData 是 “Blittable”
【问题】	缺陷、边界漏洞、注释与实现不一致	default(TimerHandle) 会命中第一个计时器
【改进方案】	建议的修改方向（本书只讲思路，不给修复代码）	generation 从 1 开始
【扩展知识】	项目没实现，但面试会从这里追下去的知识	UE 的 FTimerHandle 是 24 位下标 + 40 位序列号
【实测 Ex / Bx】	把 C# 逻辑逐行移植成 C++ 后跑出的结果	`demos/run_output.txt`、`demos/bench_output.txt`

关于“实测”的边界

本机没有 Unity，C# 编译器在当前环境被禁用，所以纯逻辑（句柄、队列、遍历、触发次数）用逐行移植的 C++ 验证；凡是依赖 C# 运行时的结论——闭包到底分配多少字节、`string.GetHashCode` 的具体值、Unity 生命周期的先后——都只标【源码事实】或【扩展知识】，**不冒充实测**。这些请在 Unity Profiler（GC Alloc 列）里自己补测，面试时说“我推断会分配，并用 Profiler 验证过”比报一个没测过的数字安全得多。

0.2 回答公式：面试官要的不是“是什么”

每道项目题都按这五步组织

① 结论　　一句话：它是什么、解决什么问题
② 机制　　数据结构 / 调用流程（能画出来）
③ 代价　　复杂度、内存、GC、顺序、线程
④ 项目事实　我的代码里具体怎么做的，哪里做对了，哪里有坑（主动说）
⑤ 改进　　如果规模扩大十倍 / 做成联网 / 做成多线程，我会怎么改

面试官攻击项目通常只有五种姿势，本书每章的“压力题”都按它们来出：

质疑

“你说 Zero-GC，证据是什么？”——考你有没有夸大。

反例

“default 句柄会不会被当成合法句柄？”——考你有没有想过边界。

边界

“回调里把自己删了呢？2001 个事件呢？”——考你的代码在极端输入下的行为。

性能

“一万个计时器呢？数组一定比链表快吗？”——考你能不能用数据说话。

替代设计

“为什么不用小顶堆 / 时间轮 / 协程？”——考你知不知道还有别的解法，以及为什么没选。

0.3 面试地图：每一章会被问什么、会被追到哪里

下表覆盖全书 27 章（已写的章节加粗）。第三列是“为什么面试官会从这个项目追到那个知识点”——**能说出这条因果链，发散题就不再是背题。**

章	面试官最可能问	会被追到哪里（发散）	为什么会追过去
Ch1 项目全景	介绍一下项目；为什么在 Unity 里仿 UE；解决了什么 Unity 原生解决不了的问题	游戏引擎架构分层、组合 vs 继承、ECS	你自称“Gamep构”，面试官会觉不懂“架构”这个的取舍
Ch2 生命周期	Awake/Start/BeginPlay 谁先；先 Possess 还是先 BeginPlay	Unity PlayerLoop、脚本执行顺序、UE 的 Spawn → PostInitializeComponents → BeginPlay	你重新包装了生命就必须说清楚两套如何对齐
Ch3 World Tick	统一 Tick 的好处；遍历中删除为什么安全；新 Actor 何时开始 Tick	TickGroup、Tick 依赖、fixed timestep、迭代器失效	你的倒序遍历只防况，面试官会找别（本书实测：会重Tick）
Ch4 Possess	为什么 Controller 和 Pawn 分开；双向引用怎么保证一致；Pawn 死了 Controller 怎么办	不变式、事务、所有权、UE 的 PlayerController/PlayerState 生命周期	夺舍 Demo 就是 Possess 的压力测问天然落在这里
Ch5 GameMode/State	为什么 UI 不读 GameMode；单机里的 Authority 是什么	Server/Client、网络复制、PlayerState、帧同步与状态同步	GameMode/Ga的分离本来就是为计的
Ch6 Singleton	子类 override Awake 为什么不调 base；Singleton 的缺点	初始化顺序、Service Locator、DI、UE Subsystem	三个核心单例都没base.Awake，这性 bug
Ch7 EventSystem	事件里发事件为什么不会栈溢出；队列满了怎么办；闭包分配了什么	Observer/PubSub、背压、消息队列、委托与多播实现	双缓冲是亮点，没照/异常是漏洞，会被问
Ch8 RingBuffer	head/tail/size 怎么维护；满和空怎么区分	取模 vs 位与、cache line、无锁 SPSC	基础数据结构题，热身
Ch9 Timer 结构	为什么不用 List；Swap-and-Pop 后句柄怎么	ABA、悬垂引用、ECS Entity、UE FTimerHandle、GPU 资源句柄、PID/fd 复用	全项目技术密度最问可以连续 8～1

	找；generation 解决什么；溢出怎么办		
Ch10 Timer 行为/GC	回调里删自己；掉帧触发几次；是不是 Zero-GC	Fix Your Timestep、spiral of death、Boehm GC、闭包、时间轮、帧同步里的定时器	“Zero-GC”是最质疑的说法
Ch11 ObjectPool	池真的用了吗；PoolObject 为什么没有 generation	内存碎片、GC 压力、UnityEngine.Pool、对象复位	池化会把 Timer ABA 问题重新带
Ch11B 对象池落地	如果要把对象池真正接进框架你会怎么设计；出池后怎么重新 BeginPlay；Tick 里回收会怎样；延迟回收被复用怎么办	UE SpawnActor/DestroyActor/EndPlay、FWeakObjectPtr、遍历中修改、帧末统一销毁	“写了没启用”的追问一定是“那你接”
Ch12 线程安全队列	lock 还是 lock-free；contention 是什么	mutex/自旋、内存序、false sharing、SPSC/MPSC	名字叫 ThreadSa试官会确认你懂什么程度”
Ch13 Log	[Conditional] 做了什么；落盘为什么要缓冲	编译期剥离、IO 线程、崩溃时丢日志	日志是线上排查的施
Ch14 GamePlayTest2	开局流程怎么串；UI 怎么知道倒计时	Push vs Pull、MVC/MVVM	这是 GameMod Timer → Event 最小闭环
Ch15 SoulDecay	介绍你的 Demo；夺舍时发生了什么；碰撞双方都触发怎么只结算一次；游戏结束后发生了什么	联网夺舍的权威校验、对象池接入、物理与 Transform 冲突、材质实例	Demo 是面试官“架是否真的好用”
Ch16 数据流	从输入到 UI 走了哪些模块	输入缓冲、命令模式、回放	考你能不能把模块条线

Ch17 ~ 21 Code Review	你现在会怎么改；性能瓶颈在哪；测试覆盖了什么	Profiler、Unity Test Framework、CI	“复盘能力”是核重的软素质之一
Ch22 ~ 27 面试	1/3 分钟介绍、压力面、UE 对比、C++/OS 延伸	UWorld/UGameInstance/Subsystem、虚函数、内存模型	收束全书

0.4 这个项目最该先背的 8 句话

1. 这是一个**学习型原型**：把 UE 的 World / Actor / Pawn / Controller / GameMode / GameState 分层思想落到 Unity 的 MonoBehaviour 上，不是 UE 的复刻，也不是生产框架。
2. `AActor` 本身就是一个 MonoBehaviour 组件；World 是**每个场景懒创建**的单例，统一驱动 Actor 的 BeginPlay 与 Tick。
3. TimerSystem 是 Slot Map：dense 数组连续遍历、稀疏映射做 O(1) 定位、swap-and-pop 做 O(1) 删除、generation 拦截旧句柄、FIFO free list 复用槽位。
4. 它的 Tick 稳态不分配，但**不是 Zero-GC**：TimerData 含两个委托引用，调用方的委托和闭包会分配。
5. generation 从 0 开始，所以 `default(TimerHandle)` = (0,0) 会被当成合法句柄——SoulDecay 里 AI 肉体的寿命句柄恰好是默认值。
6. 回调重入处理得对：先收集句柄、再统一执行、执行前二次校验，Demo 里“寿命到期 → Die → StopTimer(自己)”这条真实路径就是靠它保证安全。
7. EventSystem 用双缓冲做延迟派发，事件里发事件不会递归；代价是 0 ~ 1 帧延迟，以及溢出静默丢弃、异常中断派发这两个漏洞。
8. SoulDecay 用“夺舍”把 Possess / UnPossess 当作核心玩法反复触发，是对 Controller ⇄ Pawn 分离最直接的验证。

本章速记

- 先分清**源码事实 / 设计意图 / 问题 / 改进 / 扩展知识**，面试里说错等级比说错细节更致命。
- 回答五步：**结论 → 机制 → 代价 → 项目事实 → 改进。**
- 五种攻击：**质疑、反例、边界、性能、替代设计**；发散题要能讲出“为什么会追过去”。

PART ONE

第一部分　项目全景

先把“这个项目到底是什么”说准：做了什么、没做什么、哪些只是写了没用上，以及两个 Demo 分别在验证什么。

CHAPTER 01

项目目标、事实基线与真实架构

不照抄类继承图，而是从源码和场景文件里重建“谁持有谁、谁驱动谁、一帧里发生了什么”。

本章目标

- 能用 30 秒 / 1 分钟 / 3 分钟三个版本介绍项目，而且不夸大
- 不看代码画出真实架构：静态持有关系 + 一帧里的四个独立 Update
- 说清两个 Demo（GamePlayTest2、SoulDecay）各自在验证框架的哪一部分
- 知道哪些模块“已实现并被使用”、哪些“实现了但没启用”、哪些“没有充分验证”

1.1 这一章解决什么问题

面试的第一个问题几乎一定是“介绍一下你的项目”。这个问题的陷阱是：你说出的每一个名词（World、统一 Tick、Slot Map、Zero-GC、单元测试）都会变成下一个追问的入口。所以第一步不是“包装”，而是**把事实说准**——说准了，后面的追问都是你熟悉的地盘。

1.2 项目在做什么：一句话与三层

【设计意图】把 UE Gameplay Framework 的分层思想——*谁存在于世界（Actor/Pawn）*、*谁做决定（Controller）*、*谁定规则（GameMode）*、*谁对外公布状态（GameState）*——落到 Unity 的 MonoBehaviour 体系上，并配套计时器、事件、对象池、日志等基础设施，用两个可玩 Demo 验证。

Gameplay 层（GamePlayCore/）

World、AActor、APawn、AController、AUnityController、AGameModeBase/AGameMode、AGameStateBase/AGameState、GameInstance

基础设施层（Framework/ + DataStructure/）

TimerSystem、EventSystem（+ ThreadSafeEventSystem）、MonoSingleton/Singleton、ObjectPool、Log、RingBuffer、ThreadSafeRingBuffer

验证层（Demo + UnitTest/）

GamePlayTest2（开局流程）、SoulDecay（夺舍生存战）；6 个挂在 MonoBehaviour 上的运行时测试脚本

1.3 真实架构图

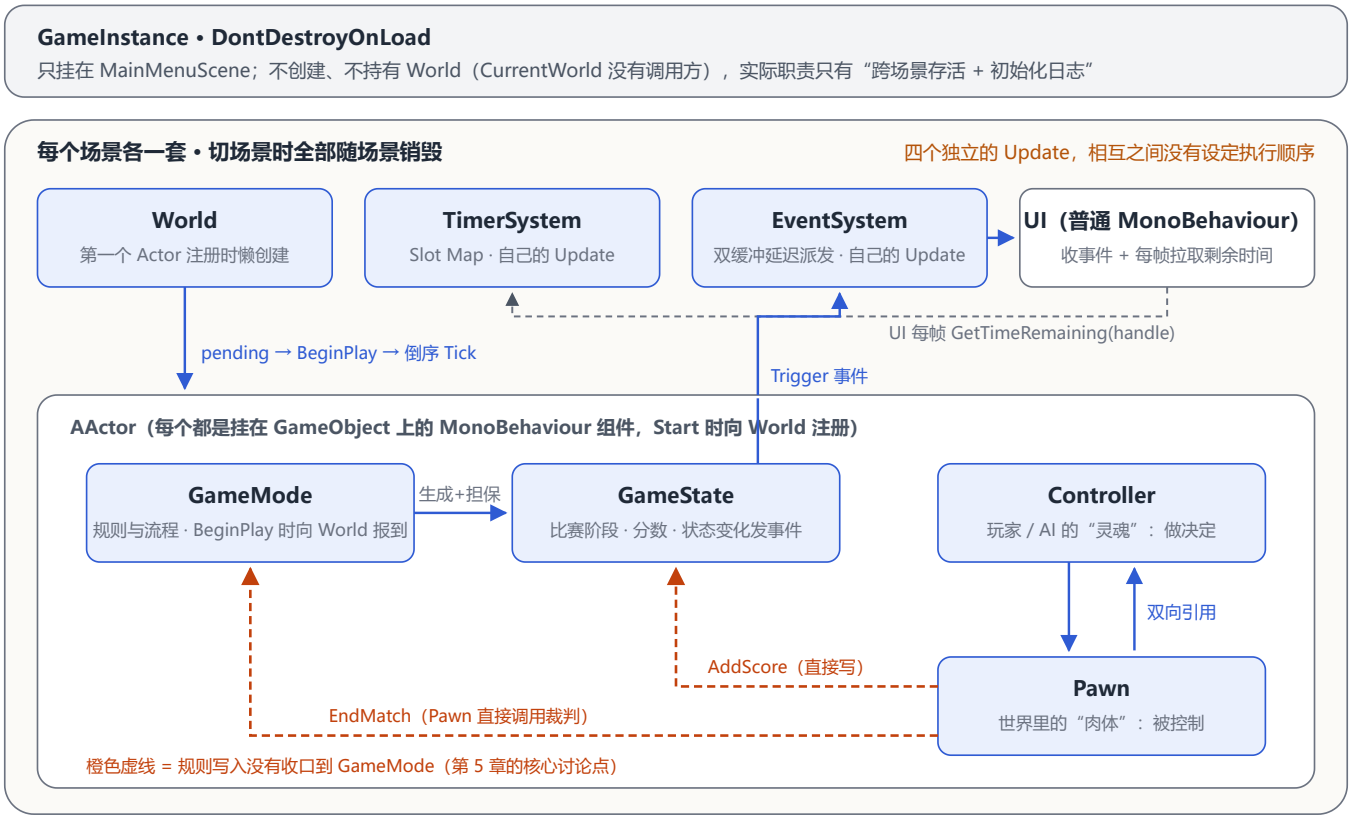


图 1-1 按源码重建的真实架构。与“GameInstance → World → GameMode/GameState”的理想图不同：GameInstance 不持有 World，World 是场景级懒单例，Timer / Event / UI 都不经过 World 驱动

继承树（源码事实）只是辅助理解：

继承关系（源码事实）

```
MonoBehaviour
 ├── MonoSingleton<T> ── World / TimerSystem / EventSystem / ThreadSafeEventSystem / GameInsta
 └── AActor
      ├── APawn ──────────── CombatPawn / DemoHeroPawn / MyHeroPawn
      ├── AController
      │    ├── AUnityController ── SoulPlayerController (+IFactionMember)
      │    └── SimpleAIController (+IFactionMember)
      ├── AGameModeBase ── AGameMode ── SurvivalGameMode / DemoGameMode
      └── AGameStateBase ── AGameState ── SurvivalGameState
```

面试注意：三个最容易说错的地方

- **“World 统一驱动一切”**——错。World 只驱动 Actor；TimerSystem、EventSystem、UI 各有自己的 `Update`，而且项目没有用 `DefaultExecutionOrder` 规定先后。
- **“GameInstance 管理 World”**——错。`CurrentWorld ⇒ World.Instance` 没有任何调用方；World 是第一个 Actor 在 `Start` 里注册时懒创建的。
- **“AActor 对应 UE 的 AActor”**——只对一半。UE 的 AActor 是组件的容器，这里的 AActor 本身就是组件；一个 GameObject 上挂两个 AActor 子类，会各自注册、各自 Tick。

1.4 一帧里发生了什么

Unity Update 阶段（四个脚本之间顺序未定义）

```
World.Update          ① _pendingAActors → _actors，逐个 BeginPlay
                      ② 倒序遍历 _actors：HasBegunPlay && IsTickEnabled → Tick(Time.deltaTime)
                         └─ Controller.Tick 读输入 → 驱动 Pawn 移动 / 发起夺舍
TimerSystem.Update    Tick(deltaTime, unscaledDeltaTime)：先执行所有 interval 回调，再执行所有完成回
EventSystem.Update    swap(front, back) → 按优先级 Logic → Appearance 清空 front
UI.Update             向 TimerSystem 拉取剩余时间，刷新文本
物理阶段              OnCollisionEnter（CombatPawn 的“大吃小”与加分）——不经过 World
```

这张时间线决定了很多“偶发 1 帧”的现象：同一帧 Trigger 的事件，如果 EventSystem 排在 World 后面，本帧就派发；排在前面，就要等下一帧。第 3、7 章会展开。

1.5 两个 Demo 分别在验证什么

框架写完之后，用两个场景（从 MainMenuScene 进入）验证。两者覆盖的模块几乎互补：

框架能力	GamePlayTest2（开局流程）	SoulDecay（夺舍生存战）
World 懒创建 + Actor 注册/BeginPlay/Tick	DemoGameMode 预置在场景中；Pawn/Controller 由 Prefab 实例化	GameMode 预置；所有肉体和灵魂都是运行时 `AddComponent` 动态生成
GameMode → GameState 生成与注册	AGameState 的比赛阶段：WaitingToStart → InProgress	子类 SurvivalGameState：三阵营计分
Possess / UnPossess	点按钮后一次性附身（DefaultPawnClass + DefaultControllerClass）	**核心玩法**：鼠标点敌人即夺舍，AI 灵魂被踢出后自毁，旧肉体枯萎
TimerSystem	命名计时器 `MatchWarmupTimer`，UI 按名字查剩余时间	每具玩家肉体一个寿命计时器（句柄经事件交给 UI）；循环刷怪计时器 `AISpawner`
EventSystem	双向：GameState → UI（阶段变化）；UI → GameMode（请求控制）	GameState → UI（分数，Appearance 优先级）；Pawn → UI（寿命句柄，Logic）
输入（Input System）	AUnityController 默认移动	SoulPlayerController 重写移动 + 鼠标射线
ObjectPool	—	写了接入代码，但**初始化被注释**，实际走的是 new + Destroy
暂停（timeScale = 0）	InGameMenuController	同左；寿命计时器与刷怪计时器都是 scaled，暂停时冻结

SoulDecay 是本书的主案例，第 15 章会完整拆开它：规则、开局时间线、夺舍调用链、碰撞结算、寿命计时器和 UI 的协作、游戏结束后发生了什么，以及它暴露出的框架问题。

1.6 事实基线：实现了 / 没启用 / 没验证

状态	模块	依据
已实现并被 Demo 使用	World、AActor、APawn、AController、AUnityController、AGameMode(Base)、AGameState(Base)、TimerSystem、EventSystem、RingBuffer、MonoSingleton、Log	两个场景的脚本挂载 + 调用链
实现了但没启用	ObjectPool（SoulDecay 里池子没初始化）、ThreadSafeEventSystem（零调用方）、`Singleton<T>`（零使用）、`GameInstance.CurrentWorld`	全仓库检索
只在测试脚本里用	ThreadSafeRingBuffer（RingBufferTests / StressTest）	同上
没有充分验证	Timer 的非追赶分支、EventSystem 溢出/异常、多线程 FIFO 顺序、Log 落盘、所有 Demo 级流程	测试用例名不副实或缺失；测试脚本没有挂在任何场景

1.7 借鉴了 UE 什么，省略了什么

UE 概念	本项目	语义差异
UWorld	World（MonoSingleton）	UE 的 World 拥有 Level、Actor 列表、TickTaskManager、TimerManager、物理场景；这里只是 Actor 名单 + 两个注册口
AActor / 组件	AActor : MonoBehaviour	这里没有组件容器、没有 TickGroup、没有 Tick 依赖、没有复制
APawn / AController	APawn / AController / AUnityController	核心的双向绑定与“抢占”语义一致；没有 PlayerState、没有 Possess 时的网络角色切换
AGameModeBase / AGameMode	同名	UE 的 GameMode 只存在于服务器；这里是单机，Authority 只表示“规则的唯一裁决者”（而且写入没有收口）
AGameStateBase / AGameState	同名，`ServerWorldTimeSeconds` 只是本地累加	UE 的 GameState 会复制到所有客户端；这里靠 EventSystem 通知 UI
UGameInstance	GameInstance	UE 的 GameInstance 持有 Subsystem、跨关卡数据、TimerManager 等；这里几乎为空
FTimerManager	TimerSystem	UE 用按到期时间排序的堆 + 64 位句柄；这里用线性扫描的 Slot Map（第 9、10 章）

1.8 项目介绍：30 秒 / 1 分钟 / 3 分钟

结构固定：**为什么做 → 解决什么问题 → 核心设计 → 最有技术含量的模块 → Demo 怎么验证 → 现在看来有什么不足。**不要罗列模块名。

30 秒版

为了真正理解 UE Gameplay Framework 的分层思想，我在 Unity 里实现了一个小框架：用 World 统一驱动 Actor 的生命周期，把“做决定的 Controller”和“存在于世界的 Pawn”分开，GameMode 管规则、GameState 对外公布状态。基础设施里最有含量的是一个 Slot Map 计时

器，用 generation 句柄防止旧句柄误操作。我用一个“夺舍”玩法的 Demo 去高频触发 Possess 来验证它。复盘时我也发现了几处边界问题，比如默认句柄会被当成合法句柄。

1 分钟版（在 30 秒版基础上补三点）

① **为什么包装生命周期**：Unity 每个 MonoBehaviour 自己 Update，顺序和开关都不好控制；我让 Actor 在 Start 时进入 World 的待加入队列，下一帧统一 BeginPlay、统一 Tick，并且可以单独关掉某个 Actor 的 Tick。

② **计时器怎么做**：数据放在连续数组里每帧线性遍历；句柄是“槽位下标 + 代数”，通过稀疏映射找到数组位置；删除用 swap-and-pop，O(1)；槽位回收时代数加一，旧句柄自然失效。回调采用“先收集、后执行、执行前再校验”，所以回调里删除自己或别人都安全。

③ **不足**：Tick 稳态不分配，但委托和闭包还会分配，不能叫 Zero-GC；统一 Tick 只覆盖了 Actor；事件队列满了会静默丢弃。

3 分钟版（按“问题—设计—验证—反思”展开）

背景（20 秒）：学 UE 时我发现最难的不是 API，而是“为什么要把 Pawn 和 Controller 拆开、GameMode 和 GameState 拆开”。所以我选择在 Unity 里从零搭一遍，逼自己把每个边界想清楚。

架构（40 秒）：三层。Gameplay 层有 World、Actor、Pawn、Controller、GameMode、GameState；基础设施层有计时器、事件系统、单例模板、对象池、日志；验证层是两个 Demo 和一组运行时测试脚本。World 是场景级的单例，Actor 注册后统一 BeginPlay/Tick；GameMode 在 BeginPlay 时向 World 报到，再按 `GameStateClass` 反射生成对应的 GameState 并替它向 World 担保注册；GameState 状态变化时通过事件系统通知 UI，UI 不直接读 GameMode。

重点模块（60 秒）：计时器。需求是大量计时器的增删要 O(1)、每帧遍历要快、外部持有的句柄要安全。我用了 Slot Map：dense 数组放数据，handle→dense 与 dense→handle 两个映射，删除时把末尾元素搬到洞里并修两条映射；句柄带 generation，槽位复用时加一，旧句柄比对失败就被拒绝。Tick 分两阶段，先把到期的句柄收集起来，再统一回调，每次回调前重新校验，解决回调重入。另外支持 interval 回调和掉帧追赶，追赶最多 5 次防止雪崩。事件系统用了按优先级的双缓冲环形队列，事件回调里再发事件只会进入下一帧，不会递归。

验证（30 秒）：SoulDecay 是一个夺舍生存战：玩家的灵魂每占据一具肉体就开始寿命倒计时，必须不停夺舍别人的身体活下去；体型大的撞碎体型小的得分。它高频触发 Possess/UnPossess、计时器的创建停止和回调里销毁自身，正好压测这几个模块。

反思（30 秒）：复盘时我发现 generation 从 0 开始，导致默认句柄等于第一个计时器的句柄，Demo 里 AI 肉体的空句柄恰好会命中它，只是当前参数下不会触发；三个单例子类没有调用基类的 Awake；事件队列溢出静默丢弃；对象池写了但没启用；测试是运行时脚本而不是 Unity Test Framework。如果重做，我会让 World 显式按顺序驱动计时器和事件，把句柄改成从 1 开始，并把纯逻辑拆成可单测的 C# 类。

1.9 面试追问树

追问树 · 项目全景

```
Q1 为什么要在 Unity 里仿 UE 的 Gameplay Framework?
 ├── Q1.1 Unity 原生没有这些概念吗？你解决了什么 Unity 解决不了的问题？
 │    └── Q1.1.1 那为什么不直接用 Unity 的 ECS / DOTS?
 ├── Q1.2 AActor 为什么继承 MonoBehaviour，而不是纯 C# 类？
 │    └── Q1.2.1 那 Tick 和 Unity 的 Update 是什么关系？性能更好吗？
 └── Q1.3 你说“统一 Tick”，TimerSystem 和 EventSystem 也被统一了吗？
      └── Q1.3.1 顺序不确定会造成什么现象？
Q2 这个框架最大的问题是什么？如果重做你会怎么改？
 └── Q2.1 为什么当时没发现？（测试为什么没覆盖）
```

Q1.1　Unity 原生没有这些概念吗？你的框架解决了什么?

L1 • 30 秒　Unity 给的是“GameObject + 组件 + 场景”这一层，没有规定玩法代码的分层。我补的是一套约定：谁控制谁（Controller/Pawn）、规则写在哪（GameMode）、状态从哪对外公布（GameState），以及统一的 Actor 生命周期。

L2 • 正常回答　具体有三个收益：一是 Actor 的 BeginPlay/Tick 由 World 驱动，可以统一开关、统一顺序（倒序遍历）、统一传 dt；二是 Controller 和 Pawn 解耦后，“换身体”只是改两条引用，玩家输入与 AI 逻辑可以替换同一具身体；三是 UI 只订阅 GameState 的事件，不依赖规则对象。

L3 • 深入　这些都不是“Unity 做不到”，而是“Unity 不替你约定”。项目的价值在于把约定固化成基类和注册流程，例如 GameMode 必须先 `RegisterGameMode` 再生成 GameState，World

只接受当前裁判担保的 GameState。

L4 • 面试官攻击　“那你这不就是换个名字的 MonoBehaviour 吗？” → **修正回答**：承认底层仍是 MonoBehaviour，区别在“控制权和数据流的约束”：Actor 的逻辑入口从 Update 换成了由 World 调用的 Tick，生命周期多了一个可控的 BeginPlay 时机；但我也必须承认这层约束目前只覆盖 Actor，计时器和事件还是各自 Update，这是我想改的第一件事。

Q1.1.1　为什么不直接用 Unity DOTS / ECS?

L2　目标不同：这个项目要学的是“面向对象的 Gameplay 分层”，也就是 UE 那一套；ECS 解决的是大规模同构实体的数据布局与并行。两者也不矛盾——我的 TimerSystem 本身就借用了 ECS 常见的稀疏集思想（dense 数组 + 稀疏映射 + 带版本的 ID）。

L4 攻击　“那你懂 ECS 的 Entity 为什么要带 Version 吗？” → 和我的 TimerHandle 一样：Entity 下标会被复用，Version 用来让旧 Entity 引用失效（第 9 章）。

Q1.2　AActor 为什么继承 MonoBehaviour?

L2　为了直接复用 Unity 的 Transform、物理回调（OnCollisionEnter）、Inspector 序列化和 Prefab；代价是生命周期仍然受 Unity 支配（Awake/Start/OnDestroy 的时机由引擎决定），而且 Unity 对象有“假 null”语义。

L3　纯 C# 的 Actor 更容易单元测试、也能完全掌控创建与销毁，但需要自己同步 Transform 和物理，工作量大很多。折中方案是“表现层 MonoBehaviour + 逻辑层纯 C#”，逻辑层由 World 持有并驱动。

Q1.2.1　Tick 比 Update 性能更好吗?

L2　理论上，Unity 每个 MonoBehaviour 的 Update 都是一次从引擎原生层到托管层的调用，Unity 官方博客也讨论过“大量 Update 不如一个管理器循环调用”的问题；World 的统一 Tick 把 N 次跨层调用变成 1 次 + N 次普通虚调用。

L4 攻击　“你测过吗？” → **修正回答**：没有在 Unity 里实测，所以我不会把它当成主要收益。我做统一 Tick 的主要目的是**控制时机**（BeginPlay 统一在下一帧、可按 Actor 关 Tick），性能是次

要收益，而且 World 里的 `List.Contains` 注册是 O(n)，规模大了反而会成为瓶颈。

Q1.3.1　执行顺序不确定会造成什么现象?

L2　最典型的是“偶发晚一帧”：Controller 在 World.Update 里触发事件，如果 EventSystem.Update 先于 World 执行，事件要到下一帧才派发；计时器在 World.Update 里创建，如果 TimerSystem 排在后面，它创建的当帧就会被加上一整帧的 dt。SoulDecay 里夺舍后 UI 可能闪 1 帧 “DIE”（第 15 章分析）。

L3　解决方法：让一个顶层 GameLoop（或 World）按固定顺序显式调用 `Timer.Tick → Actor.Tick → Event.Pump → UI`，基础设施不再自己 Update；或者至少用 `[DefaultExecutionOrder]` 固定顺序。

常见错误回答

错误：“我实现了一个类 UE 的完整 Gameplay 框架，支持 Zero-GC 计时器和完整的单元测试。”

问题在哪：三个词全部可以被一句追问击穿——“完整”（没有 PlayerState、复制、TickGroup）、“Zero-GC”（TimerData 含委托，调用方会分配）、“单元测试”（是挂在 MonoBehaviour 上的运行时脚本，没有挂进任何场景）。

更好的回答：“我实现了一个借鉴 UE 分层思想的学习型 Gameplay 框架，计时器的 Tick 稳态零分配，配了 22 个运行时测试用例；复盘后我知道它离 Unity Test Framework 的自动化测试还有差距。”

1.10 发散：面试官会从“架构”这个词追到哪里

组合 vs 继承

你的 Pawn/Controller/GameMode 都是继承体系；UE 也是深继承 + 组件。追问：阵营为什么用接口 `IFactionMember` 而不是基类字段？——这是 Demo 里组合思想的体现，可以主动讲。

引擎分层

平台层 → 核心（内存、容器、数学）→ 资源 → 渲染/物理/音频 → Gameplay 框架 → 玩法。你的项目位于“Gameplay 框架”这一层，下面依赖 Unity。

数据驱动

可测试性

SurvivalGameMode 里重/轻肉体的速度、寿命写死在代码里；追问“策划要改数值怎么办”→ ScriptableObject / 配置表，与 HOK 这类项目的配置表体系对应。

所有核心类都是 MonoBehaviour 单例，导致无法在 EditMode 下纯逻辑测试——这是“架构影响测试”的典型例子（第 20 章）。

本章速记

- 真实架构：**GameInstance 不持有 World**；World 是场景级懒单例，只驱动 Actor；Timer / Event / UI **各自 Update，顺序未定义。**
- AActor 本身是 MonoBehaviour **组件**，不是组件容器。
- GamePlayTest2 验证“GameMode → 计时器 → 事件 → UI → Possess”的开局闭环；SoulDecay 用夺舍**高频压测 Possess、计时器与回调重入。**
- ObjectPool、ThreadSafeEventSystem、`Singleton<T>` **写了没用**；测试脚本**没挂场景。**
- 介绍项目：为什么做 → 解决什么 → 核心设计 → 最难模块 → Demo 验证 → 不足。

▶ **自测 1：一帧里有几个独立的 Update 参与 Gameplay？各自做什么？**

▶ **自测 2：为什么说“ObjectPool 已经在 SoulDecay 中使用”是错的？**

CHAPTER 02

从 Unity 生命周期到 Gameplay 生命周期

项目在 MonoBehaviour 外面又包了一层 BeginPlay / Tick。包这一层是为了什么，两套时间线是怎么对齐的，没对齐的地方又会出什么事——比如“先 Possess、后 BeginPlay”。

本章目标

- 说清 Unity 的 Awake / OnEnable / Start / Update / OnDestroy 各自什么时候调用，以及运行时生成的对象和场景预置对象有什么不同
- 画出一个 AActor 从创建到 BeginPlay、Tick、注销的完整时间线，能指出“BeginPlay 至少晚一帧”的原因
- 解释三个 Demo 的 Possess 与 BeginPlay 谁先谁后，以及“先 Possess”带来的四个后果
- 知道 Destroy 是延迟的、没有 EndPlay 意味着什么，以及切场景时各单例的销毁顺序为什么不能依赖
- 能和 UE 的 Spawn → PostInitializeComponents → BeginPlay → EndPlay 逐项对照

2.1 这一章解决什么问题

面试官听到“我在 Unity 里仿照 UE 做了 BeginPlay 和 Tick”，第一反应几乎都是：**Unity 已经有 Start 和 Update 了，你为什么还要再包一层？包完之后，两套回调谁先谁后？**这两个问题如果答不清，后面讲 World、Possess、GameMode 都会显得“只是换了个名字”。

这一章先把 Unity 自己的规则讲准（哪些是官方保证的，哪些只是“通常如此”），再看项目怎么在上面搭出 Gameplay 生命周期，最后逐个检查两套时间线没有对齐的地方。

2.2 Unity 生命周期：只讲项目用到的部分

回调	什么时候调用	项目里谁用了
`Awake`	对象被加载（场景加载）或被 `AddComponent` / `Instantiate` 创建时，**在创建调用返回之前**立即执行；对象未激活则推迟到激活时	三个单例的 override（都没调 base，第 6 章）；CombatPawn / DemoHeroPawn 在这里取 Renderer
`OnEnable`	紧跟在同一个对象的 Awake 之后；之后每次重新启用都会调用	SurvivalUIController 在这里注册事件监听（第 6 章会讲它的隐患）
`Start`	该脚本第一次 Update 之前调用一次，**不会在创建调用内部执行**；脚本未启用时推迟到启用时	**AActor.Start → World.RegisterAActor**（整个 Gameplay 生命周期的入口）
`Update`	每帧一次；不同脚本之间的先后由 Script Execution Order 决定，没配置就**不保证**	World、TimerSystem、EventSystem、各 UI 各有一个
`OnCollisionEnter`	物理阶段（FixedUpdate 之后的物理模拟里）	CombatPawn 的“大吃小”——不经过 World
`OnDestroy`	对象真正被销毁时；`Destroy()` 是延迟的，要到本帧 Update 循环之后；只对曾经激活过的对象调用	AActor 在这里注销；GameMode 在这里停计时器、退订事件
`OnApplicationQuit`	应用退出前，对所有存活对象调用	MonoSingleton 设置退出标志（第 6 章）

本章和下一章的推演依赖下面四条规则。前三条来自 Unity 手册的“事件函数执行顺序”，第二条中“同一帧稍后执行”的细节来自社区实测（手册只保证“第一次 Update 之前”）。本机没有 Unity，书中凡是用到这四条的结论都标为【模型假设】，并在附录给出验证方法。

四条规则（demos/framework_verify.cpp 按它们建模）

```
R1  AddComponent / Instantiate：Awake → OnEnable 在调用返回之前执行完（对象激活时）
R2  Start 永远不在创建调用里执行。
      场景加载的对象：所有对象的 Awake/OnEnable 都执行完，才开始调用任何 Start
      Update 阶段里创建的对象：Start 在本帧 Update 阶段结束后执行，第一次 Update 在下一帧
R3  Destroy(obj) 只做标记，本帧 Update 循环结束后才真正销毁、调用 OnDestroy；
      DestroyImmediate(obj) 立即销毁
R4  被销毁的 UnityEngine.Object，用 == null 判断为 true（"fake null"），
      但 C# 对象本身还在：is / ?. / ?? / ReferenceEquals 都不经过 Unity 的 == 重载
```

面试注意：“Awake 一定在 Start 之前”只对同一个对象成立

官方保证的是：同一个脚本实例 Awake 先于 Start；场景加载时所有对象的 Awake/OnEnable 先于任何一个 Start。**不同对象之间的 Awake 顺序不保证**，运行时生成的对象和场景里已有对象之间的 Start 顺序也不保证。“A 的 Awake 里访问 B 的字段”能不能工作，取决于 B 是否已经 Awake——这是第 6 章 EventSystem 隐患的根源。

2.3 项目怎么在上面搭出 Gameplay 生命周期

AActor.cs:14-47（节选）

```
protected virtual void Start()
{
    World.RegisterAActor(this);          // 不是直接 BeginPlay，而是去 World 排队
}
protected virtual void OnDestroy()
{
    if (World.HasInstance) World.UnregisterAActor(this);
}
public virtual void BeginPlay() { HasBegunPlay = true; }   // 由 World 调用
public virtual void Tick(float deltaTime) { }              // 由 World 调用
```

Gameplay 概念	由谁触发	对应的 Unity 时机	代码位置
注册（进入 pending）	AActor 自己	Start	AActor.cs:17 → World.cs:92-102
BeginPlay	World	注册之后的**下一次 World.Update**，开头处理 pending 时	World.cs:54-68
Tick	World	每次 World.Update，倒序遍历，要求 HasBegunPlay 且 IsTickEnabled	World.cs:71-87
注销	AActor 自己	OnDestroy	AActor.cs:20-25 → World.cs:118-130
EndPlay	—	**不存在**，清理逻辑都写在 OnDestroy	—

包这一层得到了三样东西：

- **统一的开始时刻**：BeginPlay 只由 World 发出，所有 Actor 都“从 World 开始运转的那一刻”进入游戏，而不是各自在不确定的 Start 里开始。
- **统一的驱动入口**：只有 World 一个 Update，Actor 的 Tick 由它遍历调用，可以统一做暂停、开关 Tick、排序（第 3 章）。
- **UE 风格的概念**：BeginPlay / Tick / Possess / GameMode 这套词汇能和 UE 一一对照，便于团队交流和面试表达。

但它**没有替换掉** Unity 生命周期：Awake 仍然会先执行，物理回调仍然不经过 World，TimerSystem / EventSystem / UI 仍然有自己的 Update。所以项目里实际有两条时间线并行，下面逐个看它们对不齐的地方。

2.4 一个 Actor 的完整时间线

2.4.1 场景里预置的 Actor（例：SurvivalGameMode）

```
SoulDecayScene，帧 0（按 R1~R3 推演）

帧 0 · 场景加载     所有对象 Awake / OnEnable（彼此顺序不保证）
帧 0 · Start 阶段   SurvivalGameMode.Start → World.RegisterAActor
                      └ World.Instance 为空 → 懒创建 World（Awake 立即执行，Start 排到本帧稍后）
                      └ GameMode 进入 _pendingAActors
                    World.Start：遍历 _actors —— 此时 _actors 一定是空的（所有人都先进 pending）
帧 0 · Update 阶段 World.Update：pending → _actors → GameMode.BeginPlay → 本帧就 Tick
```

两个源码事实：

- **World.Start 是死代码。**它遍历 `_actors` 调 BeginPlay，但任何 Actor 都只能经由 `RegisterAActor` 进入 `_pendingAActors`，`_actors` 只在 Update 里被填充。World.Start 执行时列表一定为空。
- **RegisterAActor 的注释“只有 World 存在时才注册”不成立。**`if (Instance == null) return;` 里的 `Instance` 是懒创建的 getter，只要不是在退出过程中，它永远不会返回 null——这一句实际是在**创建** World。

2.4.2 运行时生成的 Actor（例：GameMode 在 BeginPlay 里生成的玩家肉体）

```
运行时生成：BeginPlay 至少晚一帧

帧 k · Update 阶段   某处 AddComponent<CombatPawn>()     → Awake 立即执行（R1）
                       ……调用方继续执行，可能立刻 Possess 它
帧 k · Update 之后   CombatPawn.Start → RegisterAActor → 进入 pending（R2）
帧 k+1 · Update      World.Update：pending → _actors → BeginPlay → 同一次 Update 里接着第一次 Tic
```

移植版实测（F2-C7，第 3 章详述）：第 2 帧的 Tick 里生成的 Actor N，**第 3 帧才 BeginPlay、第 3 帧才第一次 Tick。**只要创建发生在 World.Update 之内（GameMode 的 BeginPlay、计时器回调、事件回调都是），BeginPlay 至少晚一帧，这是结构决定的，与帧率无关。

为什么 BeginPlay 不能在 Start 里直接调用

如果 AActor.Start 直接调用 BeginPlay，那它就退化成了“换个名字的 Start”，开始时刻仍然由 Unity 决定。走 pending 的好处是：BeginPlay 一定发生在 World.Update 的开头、在本帧所有 Tick 之前，而且同一帧注册的 Actor 按注册顺序依次 BeginPlay。代价就是这一帧的延迟。UE 里关卡里的 Actor 也是“等 World 开始游戏时统一 BeginPlay”，运行时 SpawnActor 的 Actor 则在 Spawn 调用内部就 BeginPlay——这一点和本项目正好相反（2.7 节）。

2.5 先 Possess，后 BeginPlay

这是本项目生命周期最典型的“对不齐”。三个 Demo 的顺序各不相同：

Demo	Possess 发生在	Pawn / Controller 的 BeginPlay	结论
SoulDecay 开局（SurvivalGameMode.BeginPlay）	帧 0，AddComponent 之后立刻 Possess	帧 1	**先 Possess**
SoulDecay 刷 AI（AISpawner 回调）	计时器回调里，生成后立刻 Possess	下一帧	**先 Possess**
SoulDecay 夺舍（ExecuteSoulTransfer）	玩家点击时，目标早已在场	早已执行	**先 BeginPlay**
GamePlayTest2（DemoGameMode）	StartMatch 只生成，等 UI 按钮发事件后才 Possess	生成后的下一帧	**先 BeginPlay（靠玩家点按钮的时间差）**
GamePlayTest1（DemoStarter）	`Invoke(StartPossess, 0.1f)`	帧 0	**先 BeginPlay（靠 0.1 秒延时）**

“先 Possess”带来四个后果，前两个在项目里真实出现过：

① OnPossess 里拿不到 BeginPlay 初始化的东西

MyHeroPawn 在 BeginPlay 里取 Renderer，OnPossess 里改颜色。被提前 Possess 就是空引用。CombatPawn 和 DemoHeroPawn 都把 `GetComponent<Renderer>` 挪到了 Awake，DemoHeroPawn 的注释记录了这次修复；DemoStarter 则用 0.1 秒延时绕开。

② 半绑定：异常发生在 Possess 中间

移植版实测 F3-P3：Pawn.OnPossess 抛异常时，`Pawn.Controller` 已经赋值，`Controller.ControlledPawn` 还没赋值——Pawn 认为自己有主人，Controller 认为自己空手。下一帧重试 Possess 可以恢复（第 4 章）。

③ 输入在 BeginPlay 之前不可用

AUnityController 在 BeginPlay 里才 `new GameInput()`。TestPossess 在自己的 Update 里直接读 `_moveAction`，而它创建的 Controller 要下一帧才 BeginPlay——第一帧就是空引用（审计 P3-10）。

④ 寿命从 BeginPlay 之前开始计时

CombatPawn.OnPossess 里创建 5 秒寿命计时器，而 Pawn 要下一帧才开始 Tick。差一帧对玩法无影响，但说明“计时器的时间线”和“Actor 的时间线”不是同一条。

面试注意：把顺序说成“设计选择”，再说代价

好的说法是：“我的 World 用 pending 队列统一发 BeginPlay，所以运行时生成的 Actor 要下一帧才 BeginPlay；GameMode 生成玩家后立即 Possess，于是 Possess 早于 BeginPlay。我的应对是：Pawn 的自身引用在 Awake 里获取，OnPossess 只依赖 Awake 阶段的数据。更彻底的做法是让 Possess 也走队列，或者生成时同步 BeginPlay。”——先陈述事实，再说规则，再说取舍，比只说“UE 是先 BeginPlay”有说服力得多。

2.6 结束：Destroy、OnDestroy 与切场景

2.6.1 Destroy 是延迟的

R3 决定了：在 Tick 里 `Destroy(other)` 之后，other 本帧**仍然会被 Tick**（只要它排在后面）。移植版实测 F2-C5：A3 在 Tick 里 Destroy(A1)，A1 第 2 帧照样 Tick 了 1 次，第 3 帧才消失。放到游戏里就

是“被打死的怪本帧还能出手一次”。项目里 CombatPawn 用 `isDying` 标志挡住重复死亡，但它只在 OnUnPossess 里用于决定是否延迟销毁，并没有挡住其它逻辑。

2.6.2 没有 EndPlay

UE 把“离开游戏”（EndPlay，带原因：被销毁、切关卡、退出、PIE 结束……）和“对象内存回收”（BeginDestroy / FinishDestroy）分开。本项目只有 OnDestroy，所以：

- 无法区分“被打死”和“切场景”：DemoGameMode.OnDestroy 停计时器、退订事件，在两种情况下做的是同一件事；而真正需要区分的逻辑（例如“只有被打死才掉落、加分”）只能写到别处（CombatPawn.Die）。
- 对象池复用时没有“重新开始”的回调：池子只调 `SetActive`，Start 不会再执行，`HasBegunPlay` 一直为 true，BeginPlay 不会第二次调用（第 11 章）。

2.6.3 切场景时，谁先被销毁不确定

World、TimerSystem、EventSystem 都是场景级对象，切场景时和普通 Actor 一起销毁，**OnDestroy 的先后不保证**。项目在这里做得对：AActor.OnDestroy 用 `World.HasInstance` 而不是 `World.Instance`，GameMode、GameState、UI 也都先判断 `HasInstance`。如果写成 `Instance`，World 先被销毁时，后销毁的 Actor 会在 OnDestroy 里**重新创建一个 World**，它会被留到下一个场景（编辑器里会提示“Some objects were not cleaned up when closing the scene”）。

> **【问题】HasInstance 的两个盲区（第 6 章展开）**
> ① 三个单例没调 `base.Awake()`，场景预置的 TimerSystem / EventSystem 在第一次有人访问 `Instance` 之前，`HasInstance` 为 false；② 退出标志是每个泛型类型一份，只置不复位，关闭 Domain Reload 后第二次 Play 会出问题。

2.7 与 UE 的生命周期对照

阶段	UE（AActor）	本项目	差异
构造	构造函数 / CDO；SpawnActor → PostActorCreated → OnConstruction	AddComponent → Awake	Unity 没有 CDO，“默认值”来自 Inspector 序列化
组件初始化	PreInitializeComponents → 组件 InitializeComponent → PostInitializeComponents	无（AActor 本身就是组件）	本项目的 Actor 不是组件容器
开始游戏	关卡里的 Actor：World 开始游戏时统一 BeginPlay；运行时 Spawn：**在 SpawnActor 调用内部** BeginPlay	一律下一次 World.Update 统一 BeginPlay	UE 的 Spawn 返回时 Actor 已经 BeginPlay
附身	GameMode 的 RestartPlayer：先生成 Pawn（已 BeginPlay），再 Possess	先 Possess，下一帧 BeginPlay	正好相反（2.5 节）
每帧	TickFunction，按 TickGroup 和前置依赖排序；BeginPlay 时才注册 Tick	World 倒序遍历，HasBegunPlay 才 Tick	“BeginPlay 之后才 Tick”一致；本项目没有分组和依赖
离开游戏	EndPlay(EEndPlayReason)	无	清理逻辑和销毁混在一起
销毁	Destroy 标记 → GC 时 BeginDestroy / FinishDestroy	Destroy 延迟到帧末 → OnDestroy → 注销	都有“标记后才真正回收”的延迟

2.8 Code Review：生命周期相关问题

当前设计	问题	影响	改进方向（不写代码）
GameMode 生成 Actor 后立即 Possess	Possess 早于 BeginPlay	OnPossess 不能依赖 BeginPlay 的数据；可能半绑定	生成接口同步完成 BeginPlay；或 Possess 也进入队列，在双方 BeginPlay 之后执行
World.Start 遍历 _actors	列表永远为空，死代码	误导读者以为“场景 Actor 在 World.Start 里 BeginPlay”	删除，或改成处理 pending
RegisterAActor 里 `if (Instance == null)`	注释与行为不符，实际是懒创建	任何 Actor 的 Start 都可能创建 World	明确“World 由谁创建”：由 GameInstance / 场景引导对象显式创建
只有 OnDestroy	没有 EndPlay 及原因	切场景和被杀走同一段清理；对象池没有重新开始的回调	加 EndPlay(reason)，池化对象出池时重新进入 BeginPlay 流程
Destroy 延迟	被销毁的 Actor 本帧仍会 Tick	“死后还能出手一次”	销毁时立即 `SetActorTickEnabled(false)` 或标记 PendingKill 并在遍历时跳过

2.9 面试追问树

追问树 · 生命周期

```
Q2-1 Unity 已经有 Start/Update，你为什么还要包 BeginPlay/Tick?
 ├── Q2-2 BeginPlay 是在什么时候调用的？和 Start 谁先？
 │    └── Q2-3 运行时 Spawn 的 Actor，什么时候 BeginPlay?        ← 至少晚一帧
 ├── Q2-4 先 Possess 还是先 BeginPlay？有什么后果？              ← 项目真实踩过的坑
 │    └── Q2-5 UE 是什么顺序？为什么？
 └── Q2-6 统一驱动之后，Awake 里还能写什么？
Q2-7 在 Tick 里 Destroy 了另一个 Actor，它这一帧还会 Tick 吗？   ← R3
Q2-8 切场景时 World 和 Actor 谁先 OnDestroy？你怎么保证不出错？
 └── Q2-9 为什么用 HasInstance 而不是 Instance?
Q2-10 你的框架有 EndPlay 吗？没有会怎样？                       ← 对象池、切关
Q2-11 Unity 的 Script Execution Order / PlayerLoop 了解吗？     ← 发散
```

Q2-1 → Q2-3　为什么包一层？BeginPlay 什么时候调用?

L1 • 30 秒　为了让“开始游戏”的时刻由 World 统一决定，而不是由 Unity 在不确定的时机调用 Start。Actor 在 Start 里只向 World 登记，World 在下一次 Update 的开头统一 BeginPlay，然后统一 Tick。

L2 • 正常回答　Start 只做注册，进入 pending 队列；World.Update 先把 pending 里的 Actor 移到正式列表并逐个 BeginPlay，再倒序遍历做 Tick。所以场景里的 Actor 在第一帧就 BeginPlay；运行时生成的 Actor，它的 Start 不会在 AddComponent 里执行，在 Update 里生成的话要到本帧稍后才 Start、下一帧才 BeginPlay——我用移植版测过，第 2 帧生成的 Actor 第 3 帧才 BeginPlay 和第一次 Tick。

L3 • 深入　好处是三点：开始时刻统一、同帧注册的 Actor 按注册顺序 BeginPlay、Tick 可以统一暂停和开关。代价是一帧延迟和两条时间线并存：Awake 仍然最先执行，物理回调不经过 World，Timer/Event/UI 有自己的 Update。所以我的约定是：Awake 只取自身组件引用，不访问别的对象；跨对象的初始化放在 BeginPlay。

L4 • 面试官攻击　“那你不就是把 Start 改了个名字？” → 不只是改名。Start 由 Unity 调用，不同对象之间的顺序不保证，而且在 Update 中生成的对象 Start 时机还和生成位置有关；BeginPlay 一定发生在 World.Update 的开头、本帧所有 Tick 之前，顺序等于注册顺序。另外 Tick 由 World 统一遍历，我能在一处实现暂停、开关 Tick 和排序，这是 Update 做不到的。

Q2-4 → Q2-5　先 Possess 还是先 BeginPlay?

L1　SoulDecay 里是先 Possess：GameMode 在 BeginPlay 里生成肉体和灵魂后立刻 Possess，而它们要下一帧才 BeginPlay。

L2　后果是 OnPossess 里不能依赖 BeginPlay 初始化的字段。我最早的 MyHeroPawn 在 BeginPlay 里取 Renderer、OnPossess 里改颜色，被提前 Possess 就空引用；后来把取组件挪到 Awake。GamePlayTest1 当时用 Invoke 延时 0.1 秒绕过去，那是个时间上的补丁，不是结构上的保证。

L3　更隐蔽的是异常发生在 Possess 中间：Pawn 那一侧已经记下了 Controller，Controller 这一侧还没记下 Pawn，形成半绑定。UE 的 RestartPlayer 是先生成 Pawn（SpawnActor 返回时已经 BeginPlay）、再 Possess，所以 OnPossess 里可以放心用 BeginPlay 准备好的东西。

L4 攻击　“那你为什么不改成先 BeginPlay？” → 两种改法：一是生成接口同步完成注册和 BeginPlay，像 UE 的 SpawnActor 那样，代价是 BeginPlay 可能在别人的 Tick 中间发生，要重新考虑遍历安全；二是让 Possess 也进队列，等双方 BeginPlay 之后再执行，代价是多一帧延迟。我倾向第一种，因为“生成后立刻可用”对写玩法更直观。

Q2-7　Tick 里 Destroy 了另一个 Actor，它本帧还会 Tick 吗?

L2　会，只要它在遍历顺序里排在后面。Unity 的 Destroy 只做标记，要等本帧 Update 循环结束才真正销毁、调 OnDestroy，那时才从 World 注销；在此之前它不是 fake null，World 照常 Tick 它。移植版实测：A3 在 Tick 里 Destroy(A1)，A1 当帧还 Tick 了一次。要避免“死后出手”，销毁时应该同步关掉它的 Tick，或者在逻辑上标记死亡并在 Tick 开头检查。

Q2-8 → Q2-9　切场景时谁先 OnDestroy?

L2　不保证。所以所有 OnDestroy 里访问单例都用 HasInstance：如果 World 已经先销毁，HasInstance 为 false，直接跳过注销；如果写成 Instance，懒创建的 getter 会在场景卸载过程中重新造一个 World，成为泄漏到下一个场景的“幽灵单例”。

L3　HasInstance 也有盲区：子类没调 base.Awake，场景预置的单例在第一次被访问 Instance 之前 HasInstance 是 false，这时候的 OnDestroy 清理会被跳过。这是第 6 章要讲的确定性 bug。

Q2-10　没有 EndPlay 会怎样?

L2　清理逻辑只能写在 OnDestroy，切场景、被打死、退出游戏走的是同一段代码，无法根据原因做不同处理；对象池复用时只 SetActive，Start 不会再跑，HasBegunPlay 一直是 true，BeginPlay 不会第二次调用，所以池化对象没有“重新开始”的入口。UE 的 EndPlay 带 EEndPlayReason，和内存回收（BeginDestroy）是分开的，这是我想补上的。

常见错误回答 ①

错误：“BeginPlay 就是 Start，只是我们统一叫 BeginPlay。”

问题：项目里 Start 只负责注册，BeginPlay 由 World 在下一次 Update 时调用，时机不同；说成“改名”会被追问“那你包这一层有什么意义”。

更好的回答：“Start 里向 World 登记，World 在 Update 开头统一 BeginPlay，所以运行时生成的 Actor 会晚一帧开始，而且同帧注册的按注册顺序开始。”

常见错误回答 ②

错误：“Awake 一定先于 Start，所以我在 Awake 里初始化、在 Start 里访问别人就安全。”

问题：只对“场景加载时的对象”成立。运行时生成的对象、以及 Awake 里访问别的对象，都没有这种保证；本项目 UI 在 OnEnable 里访问 EventSystem 就依赖了不保证的顺序（第 6 章）。

更好的回答：“Awake 只初始化自己；跨对象的依赖放到 BeginPlay，由 World 统一保证时机。”

2.10 发散：面试官会从这里追到的知识

Unity PlayerLoop

一帧被拆成 Initialization → EarlyUpdate → FixedUpdate → PreUpdate → Update → PreLateUpdate → PostLateUpdate 等阶段，可以用 `PlayerLoop.SetPlayerLoop` 插入自定义阶段。把 World.Tick 注册成一个明确的阶段，就不再依赖“World 的 Update 和别人谁先”。为什么会追过去：你说“统一驱动”，面试官会问“统一到谁之前、谁之后”。

Script Execution Order

项目设置里或 `[DefaultExecutionOrder(n)]` 指定脚本的相对顺序，对 Awake / OnEnable / Update 都生效。本项目没有配置任何顺序，所以四个 Update 之间、Awake 之间的先后都不保证。

Enter Play Mode Options

关闭 Domain Reload 能让进入 Play 快很多，但静态字段不会重置——单例的 `_instance`、退出标志都会残留到下一次 Play，需要用 `[RuntimeInitializeOnLoadMethod]` 手动清理。

UE 的 EndPlay 与 GC

EndPlay 负责“离开游戏”的逻辑；对象真正释放要等 GC 把它标记为不可达后，经过 BeginDestroy / FinishDestroy。这也是为什么 UE 里对已

为什么会追过去：生命周期一问到“进入/退出”，就会问到静态状态。

Destroy 的 Actor 要用 IsValid 判断——和 Unity 的 fake null 是同一类问题。

C++ 的初始化顺序问题

不同编译单元的全局对象构造顺序不确定（static initialization order fiasco），常见解法是函数内静态变量（首次使用时构造）。这和“Awake 之间顺序不保证、所以用懒加载单例”是同一个问题的两个版本。

ECS 的系统顺序

ECS 框架通常显式声明系统顺序（UpdateBefore / UpdateAfter、阶段分组），而不是依赖对象各自的回调。把“谁先谁后”写成数据，是解决生命周期歧义的另一种思路。

本章速记

- Gameplay 生命周期：**Start → 进 pending → 下一次 World.Update 开头 BeginPlay → 同一次 Update 开始 Tick → OnDestroy 注销**；没有 EndPlay。
- 运行时生成的 Actor **至少晚一帧 BeginPlay**（F2-C7：第 2 帧生成，第 3 帧 BeginPlay 并第一次 Tick）；World.Start 是死代码。
- SoulDecay 开局和刷 AI 都是**先 Possess、后 BeginPlay**；后果：OnPossess 不能依赖 BeginPlay 数据、可能半绑定（F3-P3）、输入未就绪、计时器先跑。
- Destroy 延迟到帧末：**被 Destroy 的 Actor 本帧仍会 Tick**（F2-C5）。
- 切场景时 OnDestroy 顺序不保证，所以 OnDestroy 里一律用 `HasInstance`；UE 是先 Spawn（已 BeginPlay）再 Possess。

▶ **自测 1：TimerSystem 的回调里 AddComponent 了一个 AActor 子类，它的 Awake、Start、BeginPlay、第一次 Tick 分别在什么时候?**

▶ **自测 2：为什么 World.Start 里的循环永远不会执行?**

▶ **自测 3：把 AActor.OnDestroy 里的 `World.HasInstance` 改成 `World.Instance ≠ null`，切场景时可能出现什么?**

CHAPTER 03

World 与 Actor 统一 Tick

一个 List、一个 pending 队列、一次倒序遍历——这就是整个 World 的驱动核心。它解决了什么，倒序遍历到底防住了哪种修改、没防住哪几种，以及它离 UE 的 TickGroup 还差什么。

本章目标

- 能逐行讲清 World.Update 的两步：处理 pending、倒序 Tick
- 说出“统一 Tick”相对每个 MonoBehaviour 自己 Update 的好处与代价
- 用 10 种实测情况回答“遍历时删除为什么安全”，并指出倒序遍历的三个真实漏洞
- 说清 IsTickEnabled 与 Unity 的 enabled / activeSelf 为什么是脱钩的
- 知道注册与注销是 $O(N^2)$，以及什么规模下它会成为问题
- 能对照 UE 的 TickGroup、Tick 依赖、TickInterval 讲出改进方向

3.1 这一章解决什么问题

“统一 Tick”几乎是每个自研框架都会做的事，面试官对它的追问也非常固定：**好处是什么、新加入的什么时候开始 Tick、遍历中删除怎么办、顺序谁说了算。**项目代码里有一句注释“倒序遍历，安全删除，保证补位索引的元素是已经被遍历过的元素，不会漏读”——这句话只在一种情况下成立。本章用移植版跑了 10 种情况，把它的边界讲清楚。

3.2 World.Update 逐行看

World.cs:51-88

```
private void Update()
{
    // 1. 处理新增的 AActor
    if (_pendingAActors.Count > 0)
    {
        var newAActors = new List<AActor>(_pendingAActors);    // 拷贝：BeginPlay 里可能再注册新
        _pendingAActors.Clear();
        foreach (var newAActor in newAActors)
        {
            if (!_actors.Contains(newAActor))                  // 线性查找
            {
                _actors.Add(newAActor);
                newAActor.BeginPlay();
            }
        }
    }
    // 2. 驱动所有 AActor 的 Tick
    float dt = Time.deltaTime;                                 // scaled，且被 maximumDeltaTim
    for (int i = _actors.Count - 1; i ≥ 0; i--)                // 倒序
    {
        if (_actors[i] == null) { _actors.RemoveAt(i); continue; }    // Unity 的 == 能识别已销
        if (_actors[i].HasBegunPlay && _actors[i].IsTickEnabled)
            _actors[i].Tick(dt);
    }
}
```

细节	作用	注意
先拷贝 pending 再 Clear	BeginPlay 里新注册的 Actor 进入新的 pending，下一帧处理，不会边遍历边修改	每帧有新 Actor 时分配一个 List（GC）
`!_actors.Contains`	防止重复加入	O(N)，见 3.6 节
BeginPlay 之后同一次 Update 就 Tick	新 Actor 不用再等一帧	BeginPlay 与第一次 Tick 之间没有任何其它系统的 Update
`_actors[i] == null`	清理“已销毁但没注销”的条目（例如子类 OnDestroy 没调 base）	依赖 Unity 的 fake null；纯 C# 对象没有这个效果
`HasBegunPlay`	没 BeginPlay 的不 Tick	pending 里的 Actor 不在 _actors，这个条件实际总为 true
`Time.deltaTime`	所有 Actor 共用同一个 dt	暂停时 dt=0 但 Tick 照常调用；Controller 自己判断 `timeScale == 0`

3.3 统一 Tick：好处与代价

	每个 MonoBehaviour 自己 Update	World 统一 Tick（本项目）
调用开销	每个 Update 都是引擎原生层到托管层的一次消息调用	只有 World 一次原生调用，其余是托管层的虚函数调用
开始时刻	各自的 Start 之后	统一在 BeginPlay 之后
顺序	Script Execution Order，按**脚本类型**配置	按注册顺序倒序；可以改成按优先级排序（目前没有）
开关	`enabled` / `SetActive`	`IsTickEnabled`（与前两者脱钩，3.5 节）
统一暂停 / 慢动作 / 分帧	每个脚本自己处理	可以在 World 一处实现（目前只是传 dt）
覆盖范围	—	**只覆盖 AActor**：TimerSystem、EventSystem、UI、物理回调都不经过 World

【扩展知识】“10000 次 Update”

Unity 官方博客《10000 Update() calls》测过：大量 MonoBehaviour 各自 Update 时，开销主要在引擎原生层逐个调用托管方法的过程，而不是 Update 里的代码；改成一个管理器用托管循环调用所有对象，能明显降低这部分开销。本项目的 World 正是这种结构。面试时可以把它作为“统一 Tick 的性能理由”，但要说清：**本项目的 Actor 数量很少，这个收益没有被测量过**，统一 Tick 在这里的主要价值是控制时机和顺序。

3.4 遍历中修改列表：倒序遍历到底防住了什么

移植版 F2 在一个按 R1 ~ R3 建模的迷你帧循环里复刻 World.Update：场景里预置 A0 ~ A4（注册顺序即列表顺序，倒序遍历先 Tick A4），第 2 帧触发动作，共跑 3 帧。表中“Tick 次数”依次是第 1/2/3 帧（demo 从 1 计帧）。

#	第 2 帧在 Tick 里做了什么	实测 Tick 次数（第 1/2/3 帧）	结论
C1	A2 `Destroy(自己)`（Unity 默认，延迟）	A2 = 1/1/0，其余 1/1/1	**安全：帧末才注销，不在遍历中修改**
C2	A2 `DestroyImmediate(自己)`	A2 = 1/1/0，其余 1/1/1	**安全：删的是当前下标，后面的都已遍历**
C3	A2 `DestroyImmediate(A3)`（下标更高，已 Tick）	A3 = 1/1/0，其余 1/1/1	**安全**
C4	A3 `DestroyImmediate(A1)`（下标更低，未 Tick）	**A3 = 1/2/1**，A1 = 1/0/0	**当前 Actor 被 Tick 两次**
C5	A3 `Destroy(A1)`（延迟）	**A1 = 1/1/0**	**被“打死”的 A1 本帧仍 Tick**
C6	A4 一次 `DestroyImmediate` A0、A1、A2	A3 = 1/**0**/1；日志：“第 2 帧下标 3 越界（List 只剩 2 个）”	**C# 抛 ArgumentOutOfRangeException，本帧剩余 Actor 全部不 Tick**
C7	A2 生成新 Actor N	N：BeginPlay@3，Tick = 0/0/1	**安全：下一帧 BeginPlay 并 Tick**
C8	A4 把 A1 `SetActive(false)`、A2 `enabled=false`	A1、A2 = 1/1/1	**照样 Tick（3.5 节）**
C9	同一批 pending：P.BeginPlay 里 `DestroyImmediate(Q)`	Q 仍执行了 BeginPlay、被加入 _actors，第 3 帧被 null 检查清掉	**已销毁的对象被“复活”执行 BeginPlay**
C10	A2 的子类 OnDestroy 没调 base，然后 Destroy(自己)	A2 = 1/1/0；第 3 帧末列表里没有残留	**被 `== null` 检查兜住**

3.4.1 为什么只对“删自己 / 删已遍历的”安全

倒序遍历的下标推演（C2 与 C4）

```
C2：i=2 时删自己
  删前 [A0 A1 A2 A3 A4]   i=2
  删后 [A0 A1 A3 A4]
  下一步 i=1 → A1  ✓
  被移动的 A3、A4 都已经遍历过

C4：i=3 时删 A1（下标更低）
  删前 [A0 A1 A2 A3 A4]   i=3
  删后 [A0 A2 A3 A4]      ← A3 从下标 3 移到 2
  下一步 i=2 → A3 又被 Tick 一次 ✗
  A1 本帧没有被 Tick
```

倒序遍历的本质是：**删除某个下标只会移动它后面的元素，而倒序时“后面”都已经访问过。**所以删自己、删更高下标都安全；删更低下标会把当前元素（以及中间的元素）整体前移一格，下一次 `i--` 又落到当前元素上。一次删多个更低下标的元素时，列表长度可能小于 `i`，直接越界。

3.4.2 那项目里实际会不会发生

看真实的调用路径：项目里所有 Actor 都用 `Destroy`（延迟），注销发生在帧末的 OnDestroy，**不在遍历之内**；整个仓库唯一的 `DestroyImmediate` 在 SphereSpawner 的计时器回调里，那个对象也不是 AActor，SphereSpawner 也没有挂在任何场景。所以**当前代码里，遍历中真正发生的删除只有循环自己那行** `RemoveAt`，倒序遍历在“现在”是够用的。问题在于它给人“随便删都安全”的错觉：以后有人在 Tick 里 DestroyImmediate、或者直接调 UnregisterAActor，就会遇到 C4、C6、C9。

【问题】C9：BeginPlay 里销毁同批的另一个 Actor

处理 pending 时遍历的是拷贝出来的 `newAActors`。P 的 BeginPlay 里 DestroyImmediate(Q)：Q 的 OnDestroy 调 UnregisterAActor，但 Q 此时既不在 `_actors`（还没轮到它），也不在 `_pendingAActors`（已经 Clear），注销什么也没做；随后循环轮到 Q，`Contains` 为 false，于是 Q 被加入列表并执行 BeginPlay——在一个已经销毁的对象上。C# 对象还在，BeginPlay 能执行，但访问 `transform` 之类就会抛 MissingReferenceException。

面试注意：说清楚“现在安全”和“结构安全”

不要说“倒序遍历所以删除是安全的”。准确的说法是：“倒序遍历保证删除当前元素和已遍历元素是安全的；我们的 Actor 都用延迟销毁，注销发生在帧末，所以当前不会在遍历中删除别人。但如果有人在 Tick 里同步删除更低下标的 Actor，当前 Actor 会被 Tick 两次，批量删除还会越界——我用移植版测过。结构上更稳妥的做法是延迟删除：遍历中只做标记，遍历结束后统一清理。”

3.5 IsTickEnabled 与 enabled / activeSelf 脱钩

F2-C8：把 A1 的 GameObject 设为不激活、A2 的组件 `enabled = false`，World 照样 Tick 它们。原因很简单：World 只检查自己定义的 `IsTickEnabled`，从不读 Unity 的 `isActiveAndEnabled`。

- **好处**：Tick 开关和 Unity 的显示 / 组件开关可以独立控制，例如一个隐藏起来的管理者仍然需要 Tick。
- **代价**：习惯 Unity 的人会以为“禁用组件 = 不再更新”。对象池的包装类 `GameObjectWrapper` 就意识到了这一点：回收时先 `SetActorTickEnabled(false)` 再 `SetActive(false)`，出池时反过来——字段注释写着“兼容对象池”。但对象池目前没有启用（第 11 章），所以这段配合没有被真实运行过。

> **【问题】三种“停用”语义并存**
>
> SetActive(false)：Unity 停止该对象所有 Update 和渲染，但 World 照常 Tick；enabled=false：同上；SetActorTickEnabled(false)：World 停止 Tick，但 Unity 的其它回调（物理碰撞）照常。一个对象可以处在“看不见但在 Tick”或“在 Tick 但不碰撞”的组合状态，排查时很容易漏掉其中一个开关。

3.6 注册与注销：O(N²) 在什么规模下成为问题

`List.Contains` 和 `List.Remove` 都是线性查找。注册 N 个 Actor 时，每次都要扫描 `_actors` 和 `_pendingAActors`；首帧处理 pending 时每个都要再扫一遍 `_actors`；切场景时 N 个 OnDestroy 各做一次查找加搬移。三者都是 O(N²)。移植版 F1 的实测：

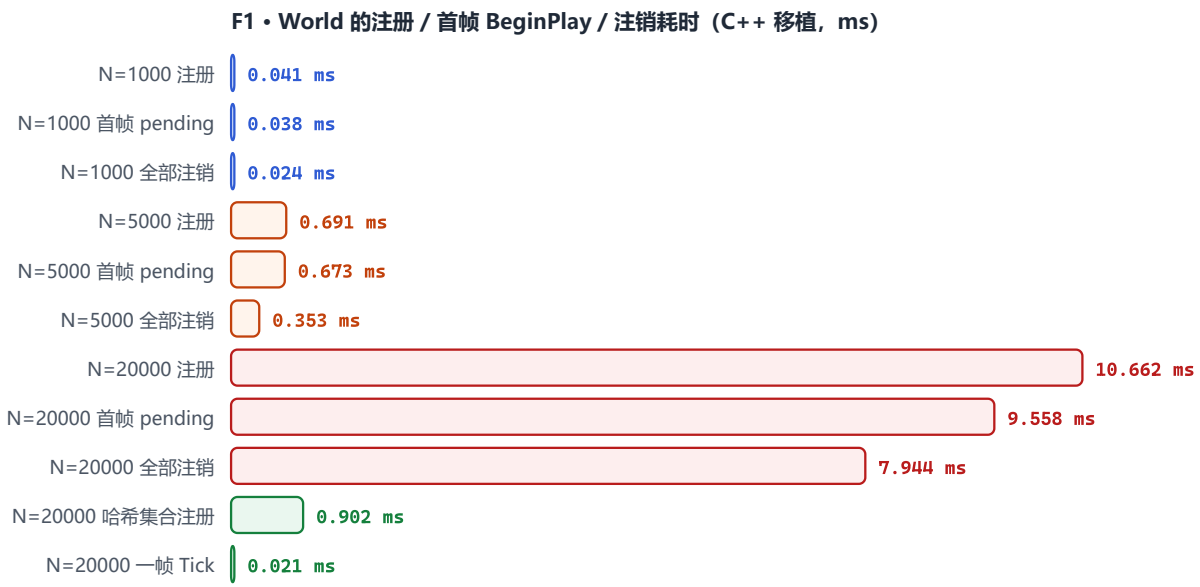


图 3-1 实测 F1（demos/framework_output.txt）。N 从 5000 增加到 20000（4 倍），三项分别增加 15、14、22 倍，与 $O(N^2)$ 的 16 倍同一量级。2 万个 Actor 时，开局注册和首帧 BeginPlay 各约 10 ms，切场景注销约 8 ms；换成哈希集合判重，注册降到 0.9 ms。相比之下一帧 Tick 2 万个只要 0.021 ms

结论要分规模说：SoulDecay 同时存在的 Actor 只有几十个，这里完全不是瓶颈；但如果拿这个 World 去做“开局生成上万个单位”的玩法（弹幕、RTS、大量子弹），开局那一帧会多出十几毫秒，切场景也会卡一下。C++ 移植的绝对数字比 C# 小，但增长趋势是一样的。

3.7 顺序：谁先 Tick

- **Actor 之间**：注册顺序的倒序。SoulDecay 每刷一个 AI 都是先生成肉体、再生成灵魂，所以列表里是 [..., AI 肉体, AI 灵魂]，倒序遍历时**灵魂（Controller）先于肉体（Pawn）Tick**。这恰好是“先读输入再移动”的顺序，但它是生成顺序的副作用，不是规则；同一帧内多个对象的 Start 顺序（从而注册顺序）Unity 也不保证。
- **World 与其它系统**：World、TimerSystem、EventSystem、UI 四个 Update 之间没有配置 Script Execution Order，**不保证先后**。同一帧里 Trigger 的事件是本帧派发还是下一帧派发，取决于 EventSystem 排在 World 前面还是后面（第 7 章）。
- **Tick 与物理**：CombatPawn 带 Rigidbody，却由 Controller 在 Tick（Update 阶段）里直接改 Transform；碰撞回调在物理阶段，不经过 World（第 15 章）。

3.8 离 UE 的 Tick 系统还差什么

UE 的机制	作用	本项目
TickGroup（PrePhysics / DuringPhysics / PostPhysics / PostUpdateWork …）	按与物理的先后分阶段	只有一个阶段（Update），物理在外面
Tick 前置依赖（AddTickPrerequisiteActor / Component）	显式声明“A 在 B 之后 Tick”	靠注册顺序的副作用
TickInterval	每 N 秒 Tick 一次，降低远处对象开销	无
BeginPlay 时注册 Tick、EndPlay 时注销	Tick 与生命周期绑定	HasBegunPlay 检查（一致）
组件各自的 TickFunction	Actor 和组件可以分开 Tick	AActor 本身就是组件，没有子组件 Tick
TickTaskManager 可并行执行部分 Tick	多核利用	单线程
Significance Manager / Tick LOD	按重要性降频	无

3.9 Code Review：World 遍历相关问题

当前设计	问题	影响	改进方向（不写代码）
倒序遍历 + 遍历中直接删除	只防删自己 / 已遍历元素	同步删更低下标：重复 Tick（C4）、越界中止整帧（C6）	延迟删除：遍历中只标记，遍历后统一清理；或 swap-and-pop + 墓碑
处理 pending 时遍历拷贝	不检查 Actor 是否已销毁	已销毁对象被 BeginPlay（C9）	BeginPlay 前判断 `== null`
List.Contains / Remove 判重	O(N²)	2 万个 Actor 时开局约 10 ms × 2、注销约 8 ms（F1）	Actor 上记录自己在列表中的下标或状态；或哈希集合判重
IsTickEnabled 独立于 enabled / activeSelf	三种停用语义并存	“禁用了还在 Tick”（C8）	明确文档约定，或在 World 里同时检查 `isActiveAndEnabled`
Destroy 延迟	被销毁的 Actor 本帧仍 Tick	“死后出手”（C5）	Destroy 时同步关闭 Tick / 标记 PendingKill
单一 Update 阶段、注册顺序决定先后	没有分组和依赖	Controller 先于 Pawn 只是巧合	引入 TickGroup / 优先级；World 自己用 PlayerLoop 注册到确定阶段
每帧有新 Actor 时 `new List`	分配	刷怪频繁时的 GC	复用一个缓存 List（TimerSystem 已经这样做）

3.10 面试追问树

追问树 · World Tick

```
Q3-1 为什么要统一 Tick？好处是什么？
  ├── Q3-2 统一之后，谁先 Tick？Controller 和 Pawn 谁先？        ← 注册顺序的副作用
  └── Q3-3 TimerSystem / UI 也是 World 驱动的吗？                ← 覆盖范围
Q3-4 新加入的 Actor 什么时候开始 Tick？
Q3-5 Tick 中删除 Actor 为什么不会出错？                          ← 本章核心
  ├── Q3-6 删一个下标更低的会怎样？删三个呢？                    ← C4 / C6
  ├── Q3-7 那项目里到底会不会遇到？                              ← 只有延迟销毁
  └── Q3-8 你会怎么改？标记删除和 swap-and-pop 各有什么代价？
Q3-9 SetActive(false) 的 Actor 还会 Tick 吗？                    ← 脱钩
Q3-10 有一万个 Actor 时，World 的哪一步最慢？                    ← O(N²) 注册
Q3-11 物理对象为什么不应该在 Tick 里改 Transform？               ← 发散到 FixedUpdate
```

Q3-1 → Q3-3　为什么统一 Tick?

L1 • 30 秒　让所有 Actor 在同一个入口、同一个开始时刻被驱动：World 先统一 BeginPlay 新 Actor，再遍历 Tick，暂停、开关 Tick、排序都可以集中处理。

L2 • 正常回答　每个 MonoBehaviour 自己 Update 时，顺序只能按脚本类型配置，开始时刻由各自的 Start 决定；统一之后，World 一个 Update 驱动全部 Actor，新 Actor 在下一帧开头统一 BeginPlay，Tick 只发给已 BeginPlay 且 IsTickEnabled 的。另外 Unity 官方测过，大量 Update 的开销主要在原生层到托管层的消息调用，一个管理器循环调用能省掉这部分——不过我的 Demo 规模太小，没有测这个收益。

L3 • 深入　要说清它覆盖的范围：只覆盖 AActor。TimerSystem、EventSystem、UI 各有自己的 Update，四者之间没有配置执行顺序；物理回调也不经过 World。所以“统一”目前只统一了 Actor 之间，没有统一系统之间——这是我想改的：用 PlayerLoop 把 World、Timer、Event 注册成固定顺序的阶段。

L4 • 面试官攻击　“你说可以统一暂停，那你实现了吗？” → 没有。现在暂停靠 timeScale=0 让 dt 变成 0，Tick 照常调用，Controller 里自己判断 timeScale 后 return。真正的统一暂停应该在 World 里按对象类别决定是否 Tick（比如 UI 动画继续、游戏逻辑停止），这一点我的框架还没做到。

Q3-5 → Q3-8　Tick 中删除为什么不出错?

L1 倒序遍历：删除一个元素只会移动它后面的元素，倒序时后面的都已经遍历过，所以删自己是安全的。

L2 但它只对删自己和删已遍历元素成立。我用移植版测了 10 种情况：同步删一个下标更低的，当前 Actor 会被 Tick 两次；一次删三个，下标越界，C# 抛异常，本帧剩下的 Actor 全部不 Tick；BeginPlay 里销毁同批 pending 的另一个 Actor，那个已经销毁的对象还会被 BeginPlay。

L3 为什么项目没出事：所有 Actor 都用 Destroy，Unity 把销毁推迟到本帧 Update 之后，注销发生在 OnDestroy，不在遍历里；整个仓库唯一的 DestroyImmediate 也不作用于 Actor。所以“现在安全”，但不是“结构安全”。

L4 攻击 “你会怎么改？” → 延迟删除：注销只标记，遍历结束后统一清理，代价是一个标记位和一次额外遍历。如果还想 O(1) 删除，可以用 swap-and-pop，但它会打乱 Tick 顺序——我的 TimerSystem 就是这样，同帧回调顺序不等于创建顺序；对于需要稳定顺序的 Actor Tick，标记删除更合适。

Q3-9 SetActive(false) 的 Actor 还会 Tick 吗?

L2 会。World 只看自己定义的 IsTickEnabled，不看 Unity 的 isActiveAndEnabled，实测把 GameObject 设成不激活、把组件 enabled 设成 false，都照样 Tick。这是有意脱钩的：对象池的包装类在回收时会先关 Tick 再 SetActive(false)。但这意味着项目里有三个独立的开关，排查“为什么它还在动”时要三个都看。

Q3-10 一万个 Actor 时哪一步最慢?

L2 不是 Tick，而是注册和注销。List.Contains 和 Remove 都是线性查找，注册 N 个、首帧 BeginPlay N 个、切场景注销 N 个都是 $O(N^2)$。移植版实测 2 万个时三项分别约 10.7、9.6、7.9 毫秒，而一帧 Tick 2 万个只要 0.02 毫秒。改进是让 Actor 记住自己在列表里的位置或状态，或者用哈希集合判重，同样规模注册降到 0.9 毫秒。

常见错误回答 ①

错误：“我用倒序遍历，所以 Tick 里删除任何 Actor 都是安全的。”

问题：删更低下标会重复 Tick，批量删除会越界（实测 C4、C6）。

更好的回答：“倒序遍历保证删除当前和已遍历的元素安全；项目里统一用延迟销毁，注销在帧末发生。如果要支持遍历中同步删除，应该改成标记删除。”

常见错误回答 ②

错误：“统一 Tick 之后，整个游戏的更新顺序就确定了。”

问题：只确定了 Actor 之间（而且是注册顺序的副作用）；World、Timer、Event、UI 四个 Update 之间仍然不确定，物理回调也在外面。

更好的回答：“Actor 之间按注册顺序倒序 Tick；系统之间没有配置执行顺序，这是我想用 PlayerLoop 显式固定的。”

3.11 发散：面试官会从这里追到的知识

C++ 迭代器失效

`vector::erase` 使被删位置及之后的迭代器失效，正确写法是 `it = v.erase(it)`，批量删除用 erase-remove 惯用法（C++20 有 `std::erase_if`）。为什么会追过去：遍历中删除是所有语言共同的问题，C++ 岗一定会问到这里。

C# 的 foreach 版本号

List 内部有一个版本号，每次增删都会 +1；foreach 的枚举器每次 MoveNext 都会检查，不一致就抛 InvalidOperationException。下标循环没有这个检查，所以是“静默出错”。

ECS 的结构性修改

遍历组件时不能增删实体或组件，通常记录到命令缓冲（Command Buffer），在系统之间统一回放。这和 World 的 pending 队列是同一种思路：把结构修改推迟到遍历之外。

固定步长

物理和需要确定性的逻辑用固定 dt（FixedUpdate / 累加器），渲染和表现用可变 dt。带 Rigidbody 的 Pawn 在 Update 里改 Transform，会和物理模拟互相覆盖（第 15 章）。

Tick 降频与分帧

数据导向的更新

远处的 AI 每 0.2 秒更新一次、一万个单位分 10 帧轮流更新——这是 TickInterval 和 Significance 的思想，也是“统一 Tick”之后才容易实现的优化。

同类对象的数据放进连续数组批量更新（第 9 章 B1 的 cache 实测），比遍历一个存放各种 Actor 的 List 并逐个虚函数调用更友好。统一 Tick 是向这个方向演进的第一步。

本章速记

- World.Update = **拷贝 pending → 判重加入 → BeginPlay；倒序遍历 → 清理 fake null → HasBegunPlay && IsTickEnabled 才 Tick。**
- 倒序遍历只防**删自己 / 删已遍历**；同步删更低下标重复 Tick（C4：A3 = 2 次），一次删三个越界中止整帧（C6），BeginPlay 中销毁同批对象会被“复活”（C9）。
- 项目里所有 Actor 用延迟 Destroy，注销在帧末，所以**现在安全、结构上不安全**；被 Destroy 的 Actor 本帧仍 Tick（C5）。
- IsTickEnabled 与 enabled / activeSelf **脱钩**（C8）。注册、首帧、注销都是 **O(N²)**：2 万个约 10.7 / 9.6 / 7.9 ms（F1）。
- 顺序：Actor 之间是注册顺序倒序（Controller 先于 Pawn 只是副作用），系统之间不确定；没有 TickGroup、依赖、TickInterval。

▶ **自测 1：列表是 [A0..A4]，A4 在 Tick 中 DestroyImmediate(A3)，本帧每个 Actor 被 Tick 几次?**

▶ **自测 2：为什么 `_actors[i] == null` 这一行对纯 C# 对象没用、对 MonoBehaviour 有用?**

▶ **自测 3：要让“同帧删除”在结构上安全、又保持 Tick 顺序稳定，用 swap-and-pop 还是标记删除？为什么?**

PART TWO

第二部分　Gameplay Framework

灵魂与肉体（Controller / Pawn）、裁判与计分板（GameMode / GameState），以及把它们串起来的单例。这是 Gameplay 岗位“框架理解”的必考区，也最容易被追问到联网。

CHAPTER 04

Pawn / Controller / Possess：灵魂与肉体

为什么要把“谁在操控”和“被操控的身体”拆成两个对象；Possess 的五个步骤怎么维护双向引用；以及在回调里再次 Possess、Pawn 被直接销毁时，这对引用会怎样悄悄失去一致。

本章目标

- 能用三个理由解释 Controller / Pawn 为什么分开，并用 SoulDecay 举例
- 背出 Possess 与 UnPossess 的步骤和回调顺序（实测 trace）
- 定义双向引用的不变式，指出项目里有哪些写入口能单边破坏它
- 说清 6 种实测情况：正常切换、抢夺、回调抛异常、回调中再 Possess（两种）、Pawn 被直接销毁
- 知道 Unity 的 fake null 与 C# 的 `is` 模式匹配在这里的差别
- 能和 UE 的 AController::OnPossess、APawn::PossessedBy、PlayerState 对照

4.1 这一章解决什么问题

Gameplay 岗位的面试几乎一定会问“Controller 和 Pawn 为什么分开”。背出“输入和身体解耦”不难，难的是接下来的追问：**两个对象互相引用，怎么保证它们一致？回调里改了怎么办？其中一个被销毁了怎么办？** SoulDecay 的夺舍玩法会反复触发 Possess，是这套机制最好的压力测试，本章用逐行移植的代码把每种情况都跑了一遍。

4.2 为什么要分开

① 生命周期不同

身体会死、会被替换，操控者要一直存在。SoulDecay 里玩家的 SoulPlayerController 从开局活到结束，中间换了很多具肉体；旧肉体变黑、3

② 同一具身体可以被不同的“大脑”控制

同一个 CombatPawn 可以被 SimpleAIController（柏林噪声瞎逛）或 SoulPlayerController（键盘

秒后销毁，灵魂不受影响。分数、阵营这类“属于玩家”的数据也就跟着灵魂走。

输入）控制，Pawn 的代码完全不用知道是谁在操控——夺舍就是在运行时换大脑。

③ 职责边界

Controller 负责“想做什么”（读输入、AI 决策），Pawn 负责“怎么做到”（移动速度、碰撞、寿命）。SoulPlayerController 重写 HandleMovement，只把方向交给 `CombatPawn.Move`，速度差异由 Pawn 自己的 MoveSpeed 决定。

项目里的一个好设计：**阵营挂在 Controller 上**（`IFactionMember` 由两个 Controller 实现），而不是挂在 Pawn 上。夺舍之后，肉体的颜色和敌我判断自动跟着新灵魂变，Pawn 不需要“改阵营”。UE 里队伍信息通常放在 PlayerState / Controller 上，是同样的思路。

【扩展知识】UE 还有第四个理由：网络

在 UE 里 PlayerController 代表一条客户端连接，只存在于服务器和拥有它的客户端；Pawn 会复制给所有客户端。把“谁拥有输入和连接”放在 Controller、把“大家都要看到的身体”放在 Pawn，复制规则才能清晰。本项目是单机，没有这一层（第 5 章）。

4.3 源码：Possess 与 UnPossess 的步骤

AController.cs:15-63（节选）

```
public void Possess(APawn pawnToPossess)
{
    if (pawnToPossess == null) return;
    if (ControlledPawn == pawnToPossess) return;
    if (ControlledPawn ≠ null) UnPossess();                                    // ① 先抛弃自己的
    if (pawnToPossess.Controller ≠ null) pawnToPossess.Controller.UnPossess(); // ② 踢掉目标的
    pawnToPossess.PossessedBy(this);                                           // ③ Pawn 侧赋值
    ControlledPawn = pawnToPossess;                                            // ④ Controller
    OnPossess(pawnToPossess);                                                  // ⑤ Controller.
}
public void UnPossess()
{
    if (ControlledPawn == null) { Log.D("不存在控制的对象"); return; }
    ControlledPawn.UnPossessed();          // ① Pawn.OnUnPossess（此时 Pawn.Controller 还在）→
    OnUnPossess(ControlledPawn);           // ② Controller.OnUnPossess
    ControlledPawn = null;                 // ③ 断开
}
```

移植版 F3-P1（C 已控制 P1，再 Possess(P2)）打印出的真实顺序：

实测 F3-P1 · 回调顺序

```
C.Possess(P2) 开始
C.UnPossess(P1) 开始
  P1.OnUnPossess()（此时 Controller 仍=C）      ← Pawn 先收到通知，还能读到是谁离开
P1.Controller=null
  C.OnUnPossess(P1)
C.ControlledPawn=null
P2.PossessedBy(C) → P2.Controller=C
  P2.OnPossess(C)                              ← 此时 C.ControlledPawn 还是 null
C.ControlledPawn=P2
  C.OnPossess(P2)
```

两个值得记住的设计细节：

- **UnPossessed 先回调、后清空**：APawn.UnPossessed 先调 OnUnPossess 再把 Controller 置空，注释写着“拿到当前控制器的引用做一些判断”。CombatPawn 正是依赖这一点：`var leavingController = this.Controller;` 判断离开的是不是玩家，玩家离开才让旧肉体变黑、3 秒后销毁。

- **第 ② 步“踢掉原主人”是框架自动做的**：SoulPlayerController 里那行 `targetPawn.Controller.UnPossess()` 被注释掉了，旁边写着“这一步 UE 框架下会默认执行”。和 UE 的 `AController::OnPossess` 一样：先放弃自己当前的 Pawn，再让目标 Pawn 的原 Controller UnPossess，然后 `PossessedBy`、`SetPawn`。

4.4 双向引用的不变式

不变式

```
对任意 Controller c、Pawn p：
    c.ControlledPawn == p   ⇔   p.Controller == c
推论：一个 Pawn 至多被一个 Controller 控制；一个 Controller 至多控制一个 Pawn
```

一份关系存了两份，就必须保证两份同时改、同时成功。项目里能写这两个字段的入口：

字段	setter	能改它的公开方法	问题
`AController.ControlledPawn`	private	`Possess` / `UnPossess`	—
`APawn.Controller`	private	`PossessedBy` / `UnPossessed`（**public**）	任何代码都能 `pawn.PossessedBy(x)`，只改 Pawn 一侧

所以不变式靠的是“大家只调 Controller.Possess”的约定，而不是代码结构。更关键的是，即使只调 Possess，**步骤 ③ 和 ④ 之间夹着用户回调**：Pawn 侧先赋值、再调 Pawn.OnPossess、然后 Controller 侧才赋值。回调里发生的任何事，都发生在“半个关系已经建立”的时刻。

4.5 实测：6 种情况

#	情况	结束时的状态（实测 F3）	不变式
P1	C 控制 P1，再 Possess(P2)	C→P2，P1←null，P2←C	✓
P2	C1 控制 P，C2.Possess(P)（夺舍）	C1→null，C2→P，P←C2	✓
P3	Pawn.OnPossess 抛异常（Possess 早于 BeginPlay，字段未初始化）	**C→null，P←C**	✗ **半绑定**
P3'	下一帧重试 C.Possess(P)	C→P，P←C（中途打印“不存在控制的对象”）	✓ **可恢复**
P4	【构造反例】Seat.OnPossess 里 C.Possess(Car)	**C→Seat，Seat←C，Car←C**	✗ **两个 Pawn 都认 C 为主人**
P5	【构造反例】Body.OnUnPossess 里 C.Possess(Home)	**C→null，Home←C**；Controller.OnUnPossess 被调用 2 次	✗
P6	Old 被直接 Destroy（没先 UnPossess），之后 C.Possess(New)	C→New；Old 的 OnUnPossess 执行 0 次	**引用对、清理丢了**

4.5.1 P3：半绑定是怎么恢复的

异常从 Pawn.OnPossess 冒出来，步骤 ④ 没执行。此时 Controller 认为自己空手（`AUnityController.Tick` 里 `ControlledPawn == null` 直接 return，玩家动不了），Pawn 认为自己有主人（SoulPlayerController 的夺舍判断 `targetPawn.Controller ≠ null` 为真）。下一帧重试时：步骤 ① 跳过；步骤 ② 发现 Pawn 的主人就是自己，调用自己的 UnPossess，因为 ControlledPawn 为空直接 return；然后 ③④⑤ 正常完成。**恢复是“碰巧”的**——如果 Pawn 的 OnUnPossess 有副作用（比如 CombatPawn 会停计时器、改颜色），而它在这条路径上没有被调用，状态就会更乱。

4.5.2 P4 / P5：回调里再 Possess，字段被“重新读取”

实测 F3-P4 · OnPossess 里再 Possess

```
C.Possess(Seat) 开始
Seat.PossessedBy(C) → Seat.Controller=C
  Seat.OnPossess(C)                    ← 回调里：C.Possess(Car)
  C.Possess(Car) 开始                  ← 此时 C.ControlledPawn 还是 null，步骤①跳过，Seat 没有被
  Car.PossessedBy(C) → Car.Controller=C
  C.ControlledPawn=Car
C.ControlledPawn=Seat                  ← 外层继续执行步骤④，把 Car 覆盖掉
结果：C→Seat，Seat←C，Car←C
```

实测 F3-P5 · OnUnPossess 里再 Possess

```
C.UnPossess(Body) 开始
  Body.OnUnPossess()                   ← 回调里：C.Possess(Home)
  C.Possess(Home)：C.ControlledPawn 仍是 Body → 嵌套 UnPossess(Body)
      Body.OnUnPossess()（第二次，回调里的一次性标志挡住了递归）
      Body.Controller=null；C.OnUnPossess(Body)；C.ControlledPawn=null
  Home.PossessedBy(C)；C.ControlledPawn=Home；C.OnPossess(Home)
Body.Controller=null
  C.OnUnPossess(Home)                  ← 外层重新读取 ControlledPawn，拿到的是 Home
C.ControlledPawn=null                  ← 外层步骤③把刚建立的关系断开
结果：C→null，Home←C；Controller.OnUnPossess 调用 2 次，第二次参数是错的
```

两种情况的共同原因：**Possess / UnPossess 在用户回调之后，还会继续读写同一个字段。**回调里的嵌套调用已经把字段改了，外层不知道，按自己的进度继续执行，于是覆盖或断开了嵌套调用的结果。P5 如果没有那个一次性标志，就是无限递归、栈溢出。

面试注意：这是构造出来的反例，要说清楚

项目里没有任何回调会再次 Possess，P4、P5 不会在当前 Demo 里发生。但“上车自动转控载具”“死亡后灵魂回到大本营”都是很常见的玩法需求，第一个写这种需求的人就会踩到。面试时说“我分析过回调重入，当前代码不支持在 Possess 回调里再次 Possess，需要延迟到下一帧或者改成先完成状态修改再统一回调”，比说“我的 Possess 是安全的”好得多。

4.6 生命周期边界：一方被销毁

4.6.1 Pawn 被直接销毁（P6）

如果有人直接 `Destroy(pawn.gameObject)` 而不是调用 `CombatPawn.Die()`，Controller 不会收到任何通知。Unity 的 fake null 让 `ControlledPawn == null` 在帧末之后变成 true，所以 Controller 下一次 Possess 时会跳过“放弃旧 Pawn”，引用关系本身能自动恢复；但**旧 Pawn 的 OnUnPossess 永远不会执行**——对 CombatPawn 来说，就是寿命计时器没有停。按源码推演，后果比“少清理一次”更严重：计时器到期后对这个已销毁的 Pawn 调用 OnBodyExploded → `Die(true)`，而它的 `Controller` 字段从没被清空、仍指向玩家灵魂，于是 ① 判定“玩家死亡”并 EndMatch；② 执行 `Controller.UnPossess()`——把玩家**当前**的新肉体踢掉。项目靠 `Die()` 里先 UnPossess 再 Destroy 的约定避免了这个问题，框架本身没有在 Pawn 的 OnDestroy 里自动解绑。

4.6.2 Controller 被销毁：fake null 与 is 模式匹配

AController 的 OnDestroy 也不会自动 UnPossess（AUnityController 只是关掉输入）。假设某个 Controller 被直接销毁，Pawn 的 `Controller` 字段还引用着它，这时同一个对象会得到两种判断：

写法（都出现在项目里）	走不走 Unity 的 == 重载	对已销毁 Controller 的结果
`targetPawn.Controller ≠ null`（SoulPlayerController:44）	走	false：认为“没有灵魂”
`Controller is IFactionMember mySoul`（CombatPawn:113）	**不走**：`is` 只看 C# 引用和类型	true：认为“有灵魂，还能碾压别人”

同一个 Pawn 在夺舍判断里是“无主”，在碰撞判断里是“有主”。当前 Demo 没有触发它，因为 SimpleAIController 只在 OnUnPossess 之后才销毁自己、玩家 Controller 从不销毁；但这是 Unity 面试里非常经典的追问：`?.`、`??`、`is`、`ReferenceEquals` **都绕过 fake null**。

4.7 SoulDecay：夺舍就是 Possess 的压力测试

一次夺舍的完整调用链在第 15 章 15.5 节，这里只看它覆盖了 Possess 的哪些分支：

Possess 分支	夺舍时发生了什么
① 放弃自己的 Pawn	玩家旧肉体 OnUnPossess：离开的是玩家 → 变黑、改名、关碰撞、停寿命计时器、3 秒后 Destroy
② 踢掉目标的原主人	AI 灵魂 UnPossess → AI 肉体 OnUnPossess（离开的是 AI → 保持“新鲜”）→ SimpleAIController.OnUnPossess 里 `Destroy(gameObject)`
③ PossessedBy	新肉体 OnPossess：玩家阵营 → 变青色、新建寿命计时器、发事件把句柄给 UI
④⑤	Controller 侧赋值；SoulPlayerController 没有重写 OnPossess
Possess 之后	Controller 直接调 `gameState.AddScore`——规则写在了 Controller 里（第 5 章）

值得在面试里讲的两点：

- **AI 灵魂“失去肉体就自毁”**：SimpleAIController 在 OnUnPossess 里 Destroy 自己，所以不会留下空转的 AI Controller。它用的是延迟 Destroy，回调返回后 Possess 剩下的步骤照常执行，没有问题。
- **Possess 早于 BeginPlay**：开局和刷 AI 时，Possess 发生在双方 BeginPlay 之前（第 2 章），CombatPawn 因此把取 Renderer 放到了 Awake。

4.8 PlayerController 与 AIController

	本项目	UE
玩家控制器	AUnityController：BeginPlay 里 `new GameInput()` 并启用；Tick 里读 Move，`HandleMovement` 可重写；SoulPlayerController 加鼠标射线夺舍	APlayerController：输入组件栈（Enhanced Input 映射上下文）、PlayerCameraManager、HUD、拥有网络连接、持有 PlayerState
AI 控制器	SimpleAIController：柏林噪声随机移动；失去肉体自毁	AAIController：BrainComponent（行为树 / StateTree）、感知、寻路跟随
输入资源	每个实例 new 一个 GameInput，OnDestroy 只 Disable 不 Dispose（审计 P3-11）	LocalPlayer 级别的输入子系统
移动方式	直接改 Transform（AUnityController 注释：“以后应该调用 AddMovementInput”）	Pawn 的 AddMovementInput → MovementComponent 消费

4.9 与 UE 对照

环节	UE	本项目
入口	`AController::Possess`（非虚，只在服务器调用）→ 虚函数 `OnPossess`	`Possess` 非虚，回调 `OnPossess` 为 protected virtual
步骤	放弃当前 Pawn → 目标原 Controller UnPossess → `Pawn->PossessedBy(this)` → `SetPawn`	完全相同的顺序
Pawn 侧	`PossessedBy` 设置 Controller、同步 PlayerState、触发蓝图事件	只设置 Controller、回调 OnPossess
Pawn 销毁	Pawn 销毁流程会通知 Controller 解除控制（DetachFromControllerPendingDestroy）	不通知（P6）
PlayerState	每个玩家一份，跨 Pawn 保存名字、分数、队伍，复制给所有客户端	没有；分数按阵营存在 GameState，阵营存在 Controller
朝向	ControlRotation 存在 Controller，Pawn 可以选择是否跟随	没有

4.10 Code Review：Possess 相关问题

当前设计	问题	影响	改进方向（不写代码）
`PossessedBy` / `UnPossessed` 是 public	可以只改 Pawn 一侧	不变式靠约定维持	改成 internal / 只允许 AController 调用
赋值与回调交错	回调发生在半个关系已建立的时刻；异常留下半绑定（P3）	状态不一致，恢复靠碰巧	先完成两侧字段修改，再统一触发回调；或回调异常时回滚
回调之后继续读写字段	回调中嵌套 Possess 被覆盖（P4、P5）	两个 Pawn 认同一个主人；OnUnPossess 调两次	检测重入（正在 Possess 时拒绝或排队到下一帧）；回调前把要用的值存进局部变量
OnDestroy 不自动解绑	任意一方被直接销毁，另一方不知道	清理逻辑丢失（P6）；fake null 与 is 判断不一致	Pawn / Controller 的 OnDestroy 里自动 UnPossess
Controller 销毁后，Pawn 侧用 `is` 判断有无灵魂	绕过 fake null	“死灵魂”仍能碾压加分	判断前先 `Controller ≠ null`，或提供 `IsControlled` 属性统一判断
没有 PlayerState	“属于玩家的数据”散在 Controller 和 GameState	扩展到多人时无处存放个人分数	引入 PlayerState，由 GameMode 在玩家加入时创建

4.11 面试追问树

追问树 · Possess

```
Q4-1 为什么 Controller 和 Pawn 要分开？
 ├── Q4-2 阵营 / 分数应该放在哪一边？为什么？                ← 灵魂 vs 肉体
 └── Q4-3 UE 为什么还要 PlayerState？                        ← 发散到网络
Q4-4 讲一下 Possess 的步骤。Pawn 已经有主人了怎么办？
 ├── Q4-5 两边互相引用，怎么保证一致？                       ← 不变式
 │    ├── Q4-6 OnPossess 抛异常呢？                          ← P3 半绑定
 │    └── Q4-7 OnPossess 里再 Possess 另一个 Pawn 呢？       ← P4
 └── Q4-8 UnPossess 时 Pawn 的回调还能拿到 Controller 吗？   ← 先回调后清空
Q4-9 Pawn 死了 Controller 怎么办？Controller 被销毁了 Pawn 怎么办？  ← P6 / fake null
 └── Q4-10 Unity 里 is 和 == null 判断已销毁对象有什么区别？
Q4-11 如果做成联网，Possess 应该在哪一端执行？              ← 发散
```

Q4-1 → Q4-3　为什么分开？阵营放哪？

L1 • 30 秒　Controller 是灵魂，负责决策；Pawn 是肉体，负责表现和物理。分开之后身体可以死、可以换，灵魂和属于玩家的数据一直存在；同一具身体也可以被玩家或 AI 控制。

L2 • 正常回答　我的 SoulDecay 就是这个思想的玩法化：玩家的 SoulPlayerController 从头到尾只有一个，不断夺舍 AI 的肉体；旧肉体销毁不影响灵魂。阵营我放在 Controller 上，用 IFactionMember 接口，夺舍后肉体的颜色和敌我判断自动跟着新灵魂走，Pawn 不需要改阵营。

L3 • 深入　UE 在此基础上还有 PlayerState：每个玩家一份、跨 Pawn 保存分数和队伍，并且复制给所有客户端，因为 PlayerController 只在服务器和拥有者客户端上存在。我的项目是单机，分数按阵营存在 GameState 里，扩展到多人时需要补一个 PlayerState。

L4 • 面试官攻击　“那 Pawn 自己的血量、寿命放哪？” → 放在 Pawn。区分标准是“换身体时这个数据要不要跟走”：寿命是这具肉体的属性，换身体就重新计时，所以 CombatPawn 自己持有寿命计时器；分数和阵营属于玩家，跟着灵魂。

Q4-4 → Q4-8　Possess 的步骤与一致性

L1　五步：先放弃自己当前的 Pawn；如果目标 Pawn 已经有主人，让那个主人 UnPossess；然后 Pawn 侧记录 Controller 并回调；Controller 侧记录 Pawn 并回调。和 UE 的 OnPossess 顺序一样。

L2 一致性指“Controller 指向 Pawn 当且仅当 Pawn 指向 Controller”。两个字段的 setter 都是私有的，但 Pawn 的 PossessedBy 是 public，所以靠约定只走 Controller.Possess。UnPossess 时 Pawn 先回调再清空，这样 Pawn 的回调里能知道是谁离开——CombatPawn 用它判断“离开的是玩家才让肉体枯萎”。

L3 问题在于赋值和回调是交错的。我用移植版测过：Pawn 的 OnPossess 抛异常，Pawn 已经记下主人、Controller 还没记下，形成半绑定，下一帧重试能恢复，但那是碰巧；如果在 OnPossess 回调里再 Possess 另一个 Pawn，外层会在回调返回后把字段覆盖，最后两个 Pawn 都认这个 Controller 为主人。

L4 攻击 “那你怎么改？” → 两步提交：先把两侧所有字段改好、确认不变式成立，再依次触发回调；回调里如果又发起 Possess，检测到正在 Possess 就排队到当前操作结束之后执行。这和 TimerSystem 的“先收集、再回调、回调前二次校验”是同一个思想。

Q4-9 → Q4-10 一方被销毁怎么办?

L2 项目里靠约定：CombatPawn.Die 先 UnPossess 再 Destroy；AI Controller 在 OnUnPossess 里才销毁自己。框架层面两边的 OnDestroy 都不会自动解绑。如果 Pawn 被直接 Destroy，Controller 那边靠 Unity 的 fake null 让 `ControlledPawn == null` 变成 true，下次 Possess 能自动恢复，但 Pawn 的 OnUnPossess 不会执行，寿命计时器之类的清理就丢了。

L3 反过来 Controller 被直接销毁更隐蔽：Pawn 的 Controller 字段用 `≠ null` 判断是“没有灵魂”，用 `is IFactionMember` 判断却是“有灵魂”，因为 `is`、`?.`、`??` 都不走 UnityEngine.Object 重载的 ==。我的 CombatPawn 碰撞判断用的就是 is，SoulPlayerController 的夺舍判断用的是 !=。当前不会触发，但修复方向很明确：在两边的 OnDestroy 里自动 UnPossess，判断“有没有灵魂”统一走一个属性。

Q4-11 做成联网，Possess 在哪一端执行? （发散）

L2 在服务器执行。Possess 改变的是“谁有权操控这具身体”，属于权威状态：客户端点击目标后发一个请求（RPC），服务器校验距离、阵营、目标是否存活，通过后执行 Possess，再把 Pawn 的所有者和 Controller 的引用复制下去。UE 的 Possess 就只允许在有权威的一端调用。

L3 表现可以先行：客户端可以先播放夺舍特效、预测性地切换镜头，服务器拒绝时再回滚。这又会引出输入延迟、客户端预测和服务器校验的话题，和第 5 章 GameMode 的“权威”是同一条

线。

常见错误回答 ①

错误：“Controller 和 Pawn 分开就是为了解耦输入。”

问题：只说了一个理由，而且没说“解耦之后得到了什么”。

更好的回答：“三个理由：生命周期不同（身体会死、灵魂常驻）、同一身体可以换大脑（玩家 / AI / 夺舍）、职责边界（决策 vs 执行）；UE 里还关系到网络复制。”

常见错误回答 ②

错误：“我的 Possess 会先 UnPossess 旧的，所以双向引用一定是一致的。”

问题：回调异常会留下半绑定，回调中再 Possess 会被覆盖，一方被直接销毁时另一方不知道（实测 P3、P4、P6）。

更好的回答：“正常路径下一致；但赋值和回调交错，回调里的异常或重入会破坏一致性，我测过这几种情况，改进方向是先改状态再统一回调，并在 OnDestroy 里自动解绑。”

4.12 发散：面试官会从这里追到的知识

不变式与两阶段提交

修改一组相关状态时，先计算并校验，再一次性提交，最后通知观察者。数据库事务、UI 的批量更新、ECS 的命令缓冲都是这个模式。为什么会追过去：双向引用一致性本质上就是一个微型事务。

C++ 里的双向引用

两个对象互相持有指针时，谁负责释放？常见做法是一方拥有（`unique_ptr`）、另一方观察（裸指针或 `weak_ptr`），析构时由拥有者通知对方置空，否则就是悬垂指针——Unity 的 fake null 帮项目挡住了这一类崩溃。

命令模式与输入

Controller 把输入翻译成“移动、夺舍”这样的命令再交给 Pawn，命令可以被记录（回放）、被发送（联网）、被 AI 生成。为什么会追过去：面试官想知道你的 Controller 能不能扩展到回放和联网。

Unity 的 fake null

UnityEngine.Object 的 C# 包装对象和原生对象分开管理，原生对象销毁后，包装对象仍然存在，== 被重载为“原生对象是否还在”。Unity 官方博客专

门讨论过这个设计的取舍，也解释了为什么 `?.` 和 `??` 不应该用在 UnityEngine.Object 上。

网络里的所有权

谁可以发送某个 Pawn 的输入、谁的预测会被服务器纠正，都由“这个 Pawn 归哪个连接所有”决定。UE 的 Owner / Autonomous Proxy / Simulated Proxy 就是在 Possess 之后才确定的。

ECS 里的“附身”

ECS 没有对象引用，通常用一个关系组件表示“这个实体被谁控制”，换控制者就是改一个组件值，一致性问题自然消失——代价是查询“我控制着谁”需要反向索引。

本章速记

- 分开的理由：**生命周期不同、同一身体可换大脑、决策与执行分离**（UE 还有网络）；阵营挂在 Controller 上，夺舍后自动跟着灵魂走。
- Possess 五步：**放弃自己的 → 踢掉目标原主人 → Pawn.PossessedBy（赋值+回调）→ Controller 赋值 → Controller 回调**，与 UE 的 OnPossess 顺序一致；UnPossess 时 Pawn 先回调后清空。
- 不变式 `c.ControlledPawn == p ⇔ p.Controller == c`；PossessedBy 是 public，靠约定维持。
- 实测（F3）：OnPossess 抛异常 → **半绑定**（可重试恢复）；回调中再 Possess → **覆盖 / 断开**，OnUnPossess 调 2 次；Pawn 被直接 Destroy → **清理丢失**。
- `≠ null` 走 fake null，`is / ?. / ??` 不走：同一个已销毁的 Controller，夺舍判断认为“无主”，碰撞判断认为“有主”。

▶ **自测 1：CombatPawn.OnUnPossess 里为什么要在 `base.OnUnPossess()` 之前读 `this.Controller`？如果框架改成“先清空再回调”，它会出什么问题？**

▶ **自测 2：P3 半绑定之后，玩家点击这具肉体想夺舍它，会发生什么？**

▶ **自测 3：为什么说 P4 和 TimerSystem 的“回调里删自己”是同一类问题？两者的解法有什么共同点？**

CHAPTER 05

GameMode / GameState：规则、权威与可观察状态

裁判和计分板为什么要分成两个对象；“Authority”在一个单机项目里到底意味着什么；为什么场景里放两个 GameMode 并不会“忽略后注册的裁判”；以及名叫 ServerWorldTimeSeconds 的字段，离“服务器世界时间”有多远。

本章目标

- 说清 GameModeBase / GameMode / GameStateBase / GameState 四个类各自负责什么，以及与 UE 的对应关系
- 逐步讲出开局流程：注册裁判 → 反射生成计分板 → 担保注册 → 比赛状态机
- 区分“注册时的权威校验”和“写入路径的权威收口”，并列出项目里绕过 GameMode 的写入口
- 用实测说明两个 GameMode 并存时会发生什么、ServerWorldTimeSeconds 会偏多少
- 回答“为什么 UI 读 GameState 不读 GameMode”，并延伸到状态同步、帧同步与 PlayerState

5.1 这一章解决什么问题

“GameMode 和 GameState 的区别”是 UE 面试的高频题，标准答案是“GameMode 只在服务器、管规则；GameState 复制给所有人、存状态”。但这个项目是单机，面试官一定会追问：**单机里为什么还要分？你说 GameMode 是“权威”，权威体现在哪里？**本章用源码回答：分开的价值是真实的，但当前实现里“权威”只体现在注册环节，写入路径并没有收口。

5.2 四个类的分工

类	职责（源码事实）	UE 对应
`AGameModeBase`	BeginPlay 时向 World 注册为裁判 → InitGameState → StartPlay；持有 DefaultPawnClass / DefaultControllerClass；空的 EndMatch	AGameModeBase：只在服务器，负责生成玩家、选择 Pawn 类、登录流程
`AGameMode`	用 `GameStateClass`（Type）反射生成 GameState 并担保注册；StartPlay → WaitingToStart；StartMatch → InProgress；EndMatch → WaitingPostMatch	AGameMode：在 GameModeBase 上增加比赛状态机
`AGameStateBase`	每帧累加 `ServerWorldTimeSeconds`	AGameStateBase：复制给所有客户端；提供与服务器同步的世界时间
`AGameState`	`EMatchState` 四个状态；`SetMatchState` 改状态并通过 EventSystem 广播；OnDestroy 时向 World 注销	AGameState：复制 MatchState，客户端据此切换 UI

UE 的 MatchState 有六个：EnteringMap、WaitingToStart、InProgress、WaitingPostMatch、LeavingMap、Aborted。项目实现了前四个，没有“离开地图”和“异常中止”。两个 Demo 的用法：

Demo	GameMode 子类做了什么	GameState 子类
GamePlayTest2	StartPlay 里建 3 秒热身计时器 → StartMatch 生成灰色方块 → 收到 UI 请求事件后 Possess	直接用 AGameState；DemoMatchUI 监听 MatchStateChanged 显示阶段
SoulDecay	BeginPlay 里直接 StartMatch、生成玩家并 Possess、建循环刷怪计时器；EndMatch 停刷怪	SurvivalGameState：三个阵营的分数 + AddScore + 开局广播 0 分；**没有人监听 MatchStateChanged**

5.3 开局流程逐步看

AGameModeBase.cs:18-26 + AGameMode.cs:16-35（节选）

```
// AGameModeBase
public override void BeginPlay()
{
    base.BeginPlay();
    World.RegisterGameMode(this);    // “一定要先注册自身，再去注册 GameState”
    InitGameState();
    StartPlay();
}
// AGameMode
override protected void InitGameState()
{
    Type stateType = GameStateClass;                                    // 子类 override：typeo
    GameObject gsObj = new GameObject($"GameState_{GetType().Name}");
    base.GameState = gsObj.AddComponent(stateType) as AGameState;       // 反射版 AddComponent
    if (World.HasInstance) World.Instance.RegisterGameState(this, GameState);    // 裁判亲自担保
}
```

SoulDecay 开局（帧 0，World.Update 里 SurvivalGameMode.BeginPlay）

```
SurvivalGameMode.BeginPlay
 └ AGameModeBase.BeginPlay
    ├ World.RegisterGameMode(this)          → AuthorityGameMode = this
    ├ AGameMode.InitGameState               → new GameObject + AddComponent(SurvivalGameState)
    │   └ World.RegisterGameState(this, gs) → authority == AuthorityGameMode ✓ → World.GameSta
    └ AGameMode.StartPlay                   → gs.SetMatchState(WaitingToStart) → Trigger(Match
 ├ StartMatch()                             → gs.SetMatchState(InProgress)     → Trigger(Match
 ├ 生成玩家肉体 + 灵魂，Possess
 └ CreateTimer("AISpawner", 3 s, loop)
帧 1：SurvivalGameState 才 BeginPlay（广播三个阵营 0 分），才开始累加 ServerWorldTimeSeconds
```

三个容易被问到的细节：

- **为什么“先注册自己”**：RegisterGameState 会检查 `authority == AuthorityGameMode`，裁判没登记，计分板就会被当成越权注册而销毁。顺序依赖写在注释里，没有写在代码结构里。
- **GameState 在 BeginPlay 之前就被改了状态**：EnteringMap → WaitingToStart → InProgress 都发生在帧 0，而 GameState 自己的 BeginPlay 在帧 1。和第 2 章的“先 Possess 后 BeginPlay”是同一个现象。

- **同一帧连发两次状态事件**：如果有 UI 监听，它会在派发时依次收到 WaitingToStart 和 InProgress，中间状态只停留了“零帧”。SoulDecay 没有监听者，EventSystem 的 Trigger 在找不到监听者时直接 return，这两个事件根本没进队列。

5.4 GameStateClass：Type + 反射，以及三个同名属性

5.4.1 用 Type 指定类，没有编译期约束

`public virtual Type GameStateClass => typeof(AGameState);` 返回的是任意 `System.Type`。如果子类写成了一个不继承 AGameState 的类型，`AddComponent(stateType) as AGameState` 会得到 null（不是 MonoBehaviour 的类型则直接抛异常），随后 RegisterGameState 在打印日志时访问 `newGameState.GetType()` 抛空引用。**错误要到运行时才暴露。**

写法	约束发生在	例子
`Type` + 反射（本项目）	运行时	`override Type GameStateClass => typeof(SurvivalGameState)`
泛型参数 + where 约束	编译期	GameMode 带一个“TState : AGameState”的类型参数
UE 的 `TSubclassOf<AGameStateBase>`	编辑器选择时 + 编译期	GameMode 的 GameStateClass 属性，只能选派生类

5.4.2 三个同名属性：`new` 隐藏而不是 override

三层 GameState 属性

```
AGameModeBase      public AGameStateBase      GameState { get; protected set; }
AGameMode          public new AGameState      GameState => base.GameState as AGameState;
SurvivalGameMode   public new SurvivalGameState GameState => base.GameState as SurvivalGameSta
```

`new` 是“隐藏”：调用哪个属性取决于引用的**静态类型**，不是对象的实际类型。`World.Instance.AuthorityGameMode.GameState` 的静态类型是 AGameModeBase，拿到的永远是

AGameStateBase。项目里大家都改用 `World.Instance.GameState is SurvivalGameState gs` 来取，这又引出了第二个问题：World 和 GameMode 各存了一份 GameState 引用（5.5 节）。UE 用模板函数 `GetGameState<T>()` 解决“按需向下转型”，只有一份存储。

5.5 “Authority”：校验了注册，没有收口写入

World 的字段名是 `AuthorityGameMode`，RegisterGameState 里有“越权警告”。但看一下 SoulDecay 里所有改变比赛结果的地方：

改了什么	谁改的	代码位置	经过 GameMode 吗
玩家夺舍得分	SoulPlayerController	`gameState.AddScore(EFaction.Player, 1)`（SoulPlayerController.cs:59-63）	**否**
碾压得分	CombatPawn	`gameState.AddScore(mySoul.FactionId, 1)`（CombatPawn.cs:145-148）	**否**
结束比赛	CombatPawn	`World.Instance.AuthorityGameMode.EndMatch()`（CombatPawn.cs:164）	**调用了，但“该不该结束”是 Pawn 决定的**
比赛阶段	任何人	`AGameState.SetMatchState` 是 public（注释：“供裁判调用的唯一修改接口”）	**没有限制**
World 的计分板引用	任何人	`World.GameState { get; set; }`（World.cs:21）	**setter 是 public，可绕过担保注册**

结果就是第 15 章 15.8 节记录的现象：玩家死亡、比赛进入 WaitingPostMatch 之后，AI 之间仍然在碾压、仍然在加分——因为“比赛结束后不再计分”这条规则**没有地方可以写**：AddScore 不校验阶段，加分的决定也不是 GameMode 做的。

面试注意：单机里的 Authority 该怎么说

不要说成“网络权威”。准确的说法：“在单机里，GameMode 的 Authority 指它是**规则的唯一裁决者**：其它对象只上报事实（我撞到了谁、我的寿命到了），由 GameMode 判断要不要加分、要不要结束比赛，再写入 GameState。我的实现里只做到了注册环节的校验，写入路径还散在 Pawn 和 Controller 里，这是我复盘时最想改的设计问题。”——同时承认现状、给出原则，面试官通常会接着问“那联网之后呢”，正好进入 5.9 节。

5.6 实测：场景里放了两个 GameMode

RegisterGameMode 里写着“场景中存在多个裁判！将忽略后注册的裁判！”。但它只是 `return` 了自己，**调用它的 BeginPlay 并没有停下来。**移植版 F4 复刻了 AGameModeBase / AGameMode / World 的注册逻辑和 SurvivalGameMode.BeginPlay 的后续步骤：

实测 F4 · 两个 SurvivalGameMode

```
GM_A.BeginPlay
  World.RegisterGameMode：成为 AuthorityGameMode
  World.RegisterGameState：注册 GameState_GM_A
  GameState_GM_A：EnteringMap → WaitingToStart → EventSystem.Trigger
  GameState_GM_A：WaitingToStart → InProgress → EventSystem.Trigger
  生成玩家肉体 + SoulPlayerController 并 Possess
  CreateTimer("AISpawner")：新建
GM_B.BeginPlay
  World.RegisterGameMode：已有裁判 → Log.E 并 return（但 BeginPlay 继续执行）
  World.RegisterGameState：越权 → Destroy(GameState_GM_B)（延迟），return
  GameState_GM_B：EnteringMap → WaitingToStart（已被拒绝，帧末才消失）→ EventSystem.Trigger
  GameState_GM_B：WaitingToStart → InProgress（已被拒绝，帧末才消失）→ EventSystem.Trigger
  生成玩家肉体 + SoulPlayerController 并 Possess
  CreateTimer("AISpawner")：同名计时器仍有效 → 先 Stop 旧的（属于 GM_A）再新建
```

“被忽略”的 GM_B 实际造成了	原因
MatchStateChanged 多 Trigger 2 次（共 4 次）	被拒绝的 GameState 因为 Destroy 延迟，本帧仍能 SetMatchState 并发事件
多出一具玩家肉体和一个 SoulPlayerController	BeginPlay 照常执行；两个 Controller 读同一份输入，两具肉体同步移动
AISpawner 的回调换成了 GM_B 的	同名 CreateTimer 会停掉旧的再新建（第 10 章 10.2 #7）
GM_B 的“比赛结束就不刷怪”检查永远不成立	它的 GameState 帧末已销毁，`base.GameState ≠ null` 为 false，跳过了阶段判断（好在 EndMatch 按名字停掉了 AISpawner）

这说明“拒绝注册”和“让这个对象停止工作”是两回事。拒绝之后至少要让 BeginPlay 停止执行（返回一个是否成功的结果），或者直接禁用 / 销毁多余的 GameMode。

5.7 为什么 UI 读 GameState，而不读 GameMode

	GameMode（规则）	GameState（可观察状态）
回答的问题	“该怎么办”：何时开始、谁得分、何时结束、生成什么	“现在怎样”：第几阶段、几分、过了多久
谁需要它	只有做决策的一方	所有要显示或响应状态的一方（UI、音效、AI）
联网时	只在服务器存在，客户端拿不到	复制给所有客户端
本项目里 UI 怎么拿	不读	SurvivalUIController 监听分数事件（push）；寿命则拿到句柄后每帧向 TimerSystem 拉取（pull）

单机里分开的价值是**依赖方向**：UI 只依赖“状态”，规则怎么改都不影响 UI；以后改成联网，UI 代码不用动。项目的 SoulDecay UI 确实只依赖 GameState 发出的事件和 TimerSystem，这一点是做对了的。

【问题】计分板的“状态”其实不完整

寿命剩余时间不在 GameState 里，UI 要先从事件里拿到计时器句柄，再每帧问 TimerSystem；比赛结束（WaitingPostMatch）SoulDecay 的 UI 也没有监听，界面上只能靠寿命文字变成“DIE”间接看出来。“所有 UI 需要的状态都在 GameState 上”这个原则只做到了一半。

5.8 实测：ServerWorldTimeSeconds 离“服务器时间”有多远

`AGameStateBase.Tick` 里只有一行：`ServerWorldTimeSeconds += deltaTime;`。名字借自 UE（那里是服务器通过复制同步给客户端的世界时间），这里却是：本地的、scaled 的、被 maximumDeltaTime 截断的、从 GameState BeginPlay 才开始的 float 累加。移植版 F5：

实测 F5

```
真实经过 120.017 s → ServerWorldTimeSeconds = 109.476 s
    （差 10.541 s：暂停 10 s + 两次 0.6 s 卡顿各被截断成 0.333 s + 少算第 1 帧）
连续    1 小时 60fps：float 累加 = 3597.21 s，double = 3600.00 s，误差 2.79 s（0.078%）
连续   10 小时 60fps：float 累加 = 36544.82 s，double = 36000.00 s，误差 -544.82 s（-1.513%）
连续  100 小时 60fps：float 累加 = 418445.66 s，double = 360000.00 s，误差 -58445.66 s（-16.235%
```

这里有两类偏差：**语义偏差**（暂停、截断、晚一帧开始），这是选择了 scaled 时间的结果，作为“游戏内时间”可能正是想要的，但名字会误导；**数值偏差**，float 有效位只有 24 位，数值越大，每次加 1/60 被舍入得越厉害，而且误差方向随数值所在区间变化（10 小时时反而多算了 545 秒）。第 10 章 E14 的计时器在约 6 天处“停止前进”，是同一个原因。

【扩展知识】UE 的做法

UE 的 GameStateBase 在服务器上取世界时间，周期性地复制给客户端，客户端记录本地与服务器的时间差，`GetServerWorldTimeSeconds()` 返回“本地时间 + 差值”。时间本身由引擎用 double 维护（UE5 的世界时间是 double），不是每帧 float 累加。

5.9 延伸：做成联网之后，GameMode / GameState 怎么变

	状态同步（服务器权威，UE 默认）	帧同步（lockstep，许多 MOBA 采用）
规则在哪里执行	只在服务器的 GameMode	每个客户端都执行同一份确定性逻辑，服务器只转发输入
GameState 怎么到达客户端	属性复制（变化时同步）	不同步状态，每端自己算出来，靠校验和检查是否一致
Authority 的含义	服务器	“确定性模拟”本身；作弊靠校验和与服务器复算发现
对本项目的要求	规则写入收口到 GameMode（5.5 节），Pawn 只发请求	逻辑用整数帧和定点数；不能依赖 Unity 物理回调和 float deltaTime；同帧回调顺序必须确定
需要补的	PlayerState、RPC、复制属性	输入帧缓冲、确定性的计时器和随机数（第 10 章 Q10-14）

两种方案都要求同一件事：**规则只能有一个执行入口。**所以 5.5 节的“写入路径没有收口”，在单机里是代码组织问题，到了联网就是正确性问题。

5.10 Code Review：GameMode / GameState 相关问题

当前设计	问题	影响	改进方向（不写代码）
AddScore / SetMatchState 是 public，Pawn 和 Controller 直接调用	规则散落，权威只体现在注册时	比赛结束后仍计分；联网时无法收口	Pawn / Controller 只上报事实（事件或调用 GameMode 的接口），GameMode 裁决后写入；GameState 的写方法只对 GameMode 开放
`World.GameState` 的 setter 是 public	可以绕过 RegisterGameState	“担保注册”形同虚设	setter 私有，只能经注册接口修改
RegisterGameMode 失败只 return	调用方不知道失败	多余的 GameMode 照常生成玩家、发事件、接管计时器（F4）	返回结果，失败时中止 BeginPlay 并禁用自己
GameStateClass 用 Type	无编译期约束	类型写错到运行时才报空引用	泛型约束，或至少在注册前校验类型
三层 `new GameState`	结果取决于静态类型	同一个对象在不同引用下拿到不同类型的属性	只保留一份存储 + 泛型取值方法
World 和 GameMode 各存一份 GameState	两个真相来源	可能指向不同对象	只由一方持有，另一方转发
ServerWorldTimeSeconds 是 scaled float 累加	名不副实，长时间运行精度下降	暂停 10 秒就差 10 秒；10 小时误差 545 秒（F5）	改名为游戏时间，或改用 double / 引擎提供的时间源
没有 LeavingMap / Aborted、SoulDecay 无人监听比赛阶段	结束流程不完整	结算界面只能间接判断	补全状态，UI 监听 MatchState

5.11 面试追问树

追问树 · GameMode / GameState

```
Q5-1 GameMode 和 GameState 有什么区别？单机里为什么还要分？
 ├── Q5-2 UI 为什么不直接读 GameMode？                          ← 依赖方向
 └── Q5-3 PlayerState 是干什么的？你为什么没有？
Q5-4 你的 World 里叫 AuthorityGameMode，权威体现在哪？          ← 本章核心
 ├── Q5-5 那 Pawn 直接 AddScore 算不算越权？                    ← 规则散落
 └── Q5-6 比赛结束后还会加分吗？为什么？                        ← 15.8
Q5-7 场景里放了两个 GameMode 会怎样？                           ← F4
Q5-8 GameStateClass 为什么用 Type？有没有更好的写法？           ← 反射 vs 泛型 vs TSubclassOf
Q5-9 ServerWorldTimeSeconds 准吗？                              ← F5
Q5-10 做成联网之后，这套结构哪里要改？状态同步和帧同步有什么不同？  ← 发散
```

Q5-1 → Q5-3　区别？单机为什么还要分？

L1 • 30 秒　GameMode 是裁判，决定规则：何时开始、谁得分、何时结束；GameState 是计分板，存所有人都需要看到的状态：阶段、分数、时间。

L2 • 正常回答　我的实现里 GameMode 在 BeginPlay 时先向 World 注册为裁判，再用 GameStateClass 反射生成 GameState 并亲自去 World 担保注册，然后驱动比赛状态机；GameState 的状态变化通过 EventSystem 广播给 UI。单机里分开的价值是依赖方向：UI 只依赖状态，不依赖规则，规则改了 UI 不用动。

L3 • 深入　在 UE 里这还关系到网络：GameMode 只在服务器存在，客户端根本没有，所以 UI 只能读复制下来的 GameState；每个玩家的个人数据放在 PlayerState，也复制给所有人。我的项目是单机，分数按阵营存在 GameState，阵营存在 Controller，所以没有 PlayerState；扩展到多人对战时需要补上。

L4 • 面试官攻击　“单机里分开就是过度设计吧？” → 如果只看 SoulDecay 的规模，确实可以合在一起。但我分开的目的是验证这套结构能不能承载“规则可替换”：GamePlayTest2 和 SoulDecay 用的是同一个 AGameMode 基类，只换了子类和 GameStateClass，UI 都不依赖 GameMode。代价是多了注册流程和两份引用，这部分我承认还不够干净。

Q5-4 → Q5-6　权威体现在哪？

L1　目前只体现在注册：World 只接受当前裁判担保的 GameState。

L2 写入路径没有收口：玩家夺舍得分是 Controller 直接调 AddScore，碾压得分是 Pawn 直接调，比赛结束是 Pawn 判断“死的是玩家”后调 EndMatch；SetMatchState 和 World.GameState 的 setter 也都是 public。所以比赛结束后 AI 之间还在加分——“结束后不计分”这条规则没有地方写。

L3 改法是让 Pawn 和 Controller 只上报事实：“我撞碎了谁”“我的寿命到了”“我夺舍了谁”，GameMode 根据当前阶段和规则决定是否加分、是否结束，再写 GameState；GameState 的写接口只对 GameMode 开放。这样联网时只要把“上报”换成发给服务器的请求，结构不用变。

Q5-7 场景里放了两个 GameMode?

L2 注释说会忽略后注册的，但实测不是：RegisterGameMode 只是 return，BeginPlay 继续执行。第二个 GameMode 生成的 GameState 会被 World 拒绝并 Destroy，但 Destroy 是延迟的，它本帧照样切了两次状态、发了两次事件；然后它还生成了第二个玩家、第二个 PlayerController，两个 Controller 读同一份输入，两具肉体一起动；它还用同名 CreateTimer 把刷怪计时器的回调换成了自己的。拒绝注册不等于让它停止工作，至少要让注册返回失败、BeginPlay 停止并禁用自己。

Q5-8 GameStateClass 为什么用 Type?

L2 为了像 UE 的 GameStateClass 那样“子类只声明类型，父类负责生成”。代价是没有编译期约束：写一个不继承 AGameState 的类型，要到运行时 `as` 出 null、注册时空引用才发现。更好的写法是泛型基类加 where 约束，编译期就能检查；UE 用 TSubclassOf，在编辑器里只能选派生类。另外我有三层用 new 隐藏的 GameState 属性，拿到哪个取决于引用的静态类型，这也是想合并成一份存储加泛型取值方法的原因。

Q5-9 → Q5-10 世界时间准吗？联网怎么改?

L2 不准，而且名字有误导性：它是本地 scaled deltaTime 的 float 累加，暂停时不走、卡顿被截断、从 GameState BeginPlay 才开始；我用移植版测过，真实 120 秒里有 10 秒暂停和两次卡

顿，它只走了 109.5 秒；连续跑 10 小时 float 累加会差 545 秒。UE 的服务器世界时间是服务器用 double 维护、复制给客户端再补偿时间差。

L3 联网方面：状态同步下，规则只在服务器 GameMode 执行，GameState 和 PlayerState 复制给客户端，客户端只发请求；帧同步下，每个客户端都执行同一份确定性逻辑，GameState 不同步而是各自算出来，所以逻辑必须用整数帧和定点数，不能依赖 float deltaTime 和物理回调。两种都要求规则只有一个执行入口，这正是我现在缺的那一步。

常见错误回答 ①

错误：“GameMode 是权威，所以所有规则都由它执行。”

问题：与源码不符——加分和结束比赛的决定都在 Pawn / Controller 里，被追问一行代码就会露馅。

更好的回答：“设计上 GameMode 是规则的唯一裁决者；当前实现只在注册环节做了校验，写入路径还散在 Pawn 和 Controller，我知道怎么收口。”

常见错误回答 ②

错误：“GameState 存的是服务器时间，所以可以用来做倒计时同步。”

问题：本项目的 ServerWorldTimeSeconds 只是本地 scaled 时间的 float 累加，没有服务器，也不同步（实测 F5）。

更好的回答：“它目前是本地游戏时间，名字借自 UE；真要做同步倒计时，应该由服务器下发开始时刻，客户端用对时后的时间计算剩余量。”

5.12 发散：面试官会从这里追到的知识

命令-查询分离

“改状态的方法”和“读状态的方法”分开，并且只给特定的一方改的权限。GameState 只读暴露给所有人、写接口只给 GameMode，就是这个原则。为什么会追过去：规则散落本质上是“谁都能写”。

MVC / MVVM

GameState 像 Model，UI 像 View，事件是变更通知。UI 只观察 Model，不调用 Controller 的逻辑——和“UI 不读 GameMode”是同一个依赖方向。

属性复制与 RepNotify

UE 里 GameState 的属性标记为复制，变化时客户端收到并触发 OnRep 回调更新 UI。它和本项目“SetMatchState 后 Trigger 事件”是同一个模式，只是事件来源从本地变成了网络。

对时

客户端发出请求时记下本地时间 t0，服务器回复它的时间 ts，客户端收到时是 t1，估算偏移 ts + (t1 − t0)/2 − t1；多次采样取延迟最小的一次。NTP 用的是同一思路。

帧同步的确定性

浮点运算在不同 CPU / 编译选项下可能不同，所以逻辑层常用定点数；随机数用同一个种子的伪随机；遍历容器的顺序也必须确定（哈希表遍历顺序就不能依赖）。

反射与类型系统

C# 的 `AddComponent(Type)` 依赖运行时反射；C++ 没有内建反射，UE 靠 UHT 生成的 UClass 信息实现 `TSubclassOf` 和按类生成对象。为什么会追过去：GameStateClass 一问就会问到“C++ 里怎么按类型名创建对象”。

本章速记

- GameMode = 规则（何时开始、谁得分、何时结束），GameState = 可观察状态（阶段、分数、时间）；单机里分开的价值是**依赖方向**，联网里还关系到“GameMode 只在服务器”。
- 开局：**RegisterGameMode → 反射生成 GameState → 担保注册 → WaitingToStart → InProgress**，都在 GameState 的 BeginPlay 之前。
- Authority **只校验了注册，没有收口写入**：AddScore / SetMatchState / World.GameState 都是 public，加分和结束比赛由 Pawn / Controller 决定 → 比赛结束后仍计分。
- 两个 GameMode（F4）：后注册的**没有被忽略**——多发 2 次事件、多一个玩家、接管刷怪计时器。
- ServerWorldTimeSeconds（F5）：120 s 里只走 **109.5 s**；10 小时 float 累加误差 **545 s**。

▶ **自测 1：为什么 AGameModeBase.BeginPlay 里必须先 RegisterGameMode 再 InitGameState？调换顺序会怎样？**

▶ **自测 2：`World.Instance.AuthorityGameMode.GameState` 在 SoulDecay 里拿到的是什么类型？为什么？**

▶ **自测 3：把“比赛结束后不再计分”这条规则加进当前代码，最小的改动放在哪里？为什么这仍然不是好的设计？**

CHAPTER 06

Singleton / GameInstance / 初始化顺序

五个基础设施都继承同一个 MonoSingleton。它的懒查找、防重和退出标志各自解决了什么；为什么五个子类里只有一个调了 base.Awake；以及一个比它更隐蔽的问题——UI 在 OnEnable 里注册事件时，EventSystem 可能还没 Awake。

本章目标

- 讲清 MonoSingleton 的四个部分：懒查找 / 懒创建、Awake 防重、OnDestroy 清理、退出标志
- 说出“为什么每个子类各有一份单例”（C# 封闭泛型类型各自的静态字段）
- 准确描述漏调 base.Awake 的后果，不夸大也不缩小
- 解释 Awake / OnEnable 跨对象顺序不保证带来的初始化隐患，以及为什么 TimerSystem 没事而 EventSystem 有风险
- 说清单例的代价，并对照 Service Locator、依赖注入、UE Subsystem 给出替代方案

6.1 这一章解决什么问题

单例是最常被追问“缺点是什么”的设计模式。这个项目让它更具体：**有一个确定性的 bug（base.Awake 没调）、一个依赖未定义顺序的隐患（Awake 与 OnEnable 之间）、一个只在特定编辑器设置下出现的问题（关闭 Domain Reload）**。能把这三件事讲清楚，“单例的缺点”就不再是背书。

6.2 两个模板

Singleton.cs:34-94（节选）

```
public class MonoSingleton<T> : MonoBehaviour where T : MonoBehaviour
{
    private static T _instance;
    private static bool _isApplicationQuitting = false;
    public static bool HasInstance => (_instance != null && !_isApplicationQuitting);

    protected virtual void Awake()                          // ② 防重：场景里预置了多个时，多余的自杀
    {
        if (_instance == null) _instance = this as T;
        else if (_instance != this) Destroy(gameObject);
    }
    public static T Instance
    {
        get
        {
            if (_isApplicationQuitting) return null;     // ④ 退出时不再创建
            if (_instance == null)
            {
                _instance = FindObjectOfType<T>();       // ① 懒查找
                if (_instance == null)
                    _instance = new GameObject(typeof(T).Name).AddComponent<T>();   // 懒创建
            }
            return _instance;
        }
    }
    protected virtual void OnDestroy() { if (_instance == this) _instance = null; }   // ③
    protected virtual void OnApplicationQuit() { _isApplicationQuitting = true; }     // ④
}
```

部分	解决什么	注意
① 懒查找 / 懒创建	没人放也能用；放了就用场景里那个	任何代码第一次访问 Instance 都可能**创建**对象，包括 OnDestroy 里
② Awake 防重	场景里预置了多个时只留一个	需要子类调 base.Awake 才生效（6.4 节）；Destroy 的是整个 GameObject
③ OnDestroy 清理	切场景后不留下指向已销毁对象的静态引用	基类这一步做对了，World 的 OnDestroy 也调了 base
④ 退出标志	退出过程中不再懒创建，避免“幽灵单例”	每个 T 一份、只置不复位（6.6 节）
`HasInstance`	只查询、不创建，给 OnDestroy 用	`_instance ≠ null` 走的是 Unity 重载的 ==，能识别已销毁

为什么“每个子类各有一份单例”

`World : MonoSingleton<World>` 这种写法常被叫作 CRTP（源码注释也这么写）。在 C# 里真正起作用的是：**每个封闭泛型类型（MonoSingleton<World>、MonoSingleton<TimerSystem>……）都是独立的类型，各有一份静态字段。**所以 `_instance` 和 `_isApplicationQuitting` 都是按 T 分开的。C++ 的 CRTP 也有同样效果（每个模板实例各有静态成员），但 C++ 的 CRTP 更常用于静态多态（基类通过 `static_cast<T*>(this)` 调用派生类）。

另一个模板 `Singleton<T> where T : new()` 是纯 C# 版，**项目里没有任何类使用它。**它有两个可以被追问的问题：`new()` 约束要求 T 有公开的无参构造函数，所以任何人都能 `new T()` 出第二个实例；`_instance ??= new T()` 不是线程安全的，两个线程同时首次访问可能各建一个（C# 的 `Lazy<T>` 或静态构造函数能解决）。

6.3 谁在用、怎么用

单例	override Awake 调了 base 吗	在哪些场景预置	DontDestroyOnLoad
World	**否（Awake 是空的，只有注释）**	无，永远懒创建	否（场景级）
TimerSystem	**否（只调 AllocateCapacity）**	SoulDecayScene	否
EventSystem	**否（在 Awake 里创建缓冲区和监听字典）**	SoulDecayScene（Managers 下的子物体）	否
ThreadSafeEventSystem	**否**	无；没有任何代码访问它	否
GameInstance	**是，而且检查了自己是不是多余的**	MainMenuScene	是

两个小事实：SoulDecayScene 里有**两个都叫 “EventSystem” 的 GameObject**——一个是 Unity UI 的事件系统，一个是项目的 EventSystem（挂在 Managers 下），层级面板里只能靠位置区分；GamePlayTest2Scene 没有预置任何单例，三个系统都在第一次被访问时懒创建。

6.4 确定性 bug：漏调 base.Awake

四个子类 override 了 Awake 却没调 `base.Awake()`，基类的防重逻辑对它们**完全不执行**。准确的后果是三条：

后果	具体表现	当前是否发生
预置实例不会在 Awake 登记	在第一次有人访问 `Instance`（触发 FindObjectOfType）之前，`_instance` 为 null、`HasInstance` 为 false	发生，但影响很小：SoulDecay 的 UI 在 OnEnable 里就访问了 EventSystem.Instance，GameMode 开局就访问了 TimerSystem.Instance
多余实例不会自杀	场景里有两个 TimerSystem（复制了预制体、叠加加载两个场景）时，两个都会 Update；`Instance` 指向 FindObjectOfType 找到的那一个	不发生：目前每个场景至多一个
先懒创建、后加载预置对象时出现双份	如果某段代码在预置对象加载前访问了 Instance（例如上一个场景的对象在卸载时），会懒创建一个；之后加载的预置对象也不会自杀	不发生：OnDestroy 里一律用 HasInstance

对比唯一做对的 GameInstance：

GameInstance.cs:6-14

```
protected override void Awake()
{
    base.Awake();                                   // 先调基类，如果我是多余的，基类会把我杀掉
    if (this == null || Instance ≠ this) return;    // 多余的就不要继续执行
    DontDestroyOnLoad(this.gameObject);
}
```

这里有一个可以加分的细节：`this == null` **这个判断不会生效。**基类调的是 `Destroy(gameObject)`，按规则 R3，销毁要等到本帧 Update 循环之后，Awake 返回时对象还没销毁，`this == null` 为 false。真正起作用的是第二个条件 `Instance ≠ this`——它让多余的实例跳过 DontDestroyOnLoad。代码是对的，但注释里的理由不准确。

面试注意：说“确定性 bug”，但别夸大成“单例全坏了”

准确的说法：“World、TimerSystem、EventSystem 都 override 了 Awake 却没调 base，所以基类的防重和提前登记对它们不生效；现在每个场景至多一个实例，而且懒查找会兜底找到预置对象，所以没有表现出来。如果叠加加载两个带 Managers 的场景，就会有两个 TimerSystem 同时 Update。修复是子类第一行调 base 并在多余时 return，或者基类把 Awake 改成非虚、另给子类一个初始化钩子。”

6.5 更隐蔽的隐患：Awake 与 OnEnable 的跨对象顺序

SurvivalUIController.cs:19-26 与 EventSystem.cs:37-47

```
// SurvivalUIController（普通 MonoBehaviour，场景预置）
private void OnEnable()
{
    if (EventSystem.Instance)                                    // 懒查找：找到场景里预置的
        EventSystem.Instance.Register<ScoreChangedEventArgs>(OnScoreChanged);
}
// EventSystem
override protected void Awake()
{
    _frontBuffers = ...; _backBuffers = ...;
    _listenersDic = new Dictionary<Type, Delegate>();            // 监听字典在 Awake 里才创建
}
public void Register<T>(Action<T> listener) where T : IEventArgs
{
    if (_listenersDic.ContainsKey(type)) ...                     // Awake 之前调用 → 空引用
}
```

场景加载时，Unity 对每个对象依次调用 Awake 和 OnEnable，**不同对象之间的先后没有保证**（没有配置 Script Execution Order 时）。如果 SurvivalUIController 排在 EventSystem 前面：

按源码推演（【推断】，验证方法见下）

```
SurvivalUIController.Awake → OnEnable
  └ EventSystem.Instance：_instance 为空 → FindObjectOfType 找到场景里的 EventSystem（它还没 Awa
  └ Register：_listenersDic == null → NullReferenceException
     → 两个监听都没注册成功：分数文字永远不更新，寿命文字因为拿不到句柄一直显示 “DIE”
EventSystem.Awake（稍后）→ 创建字典，一切正常——但 UI 已经错过了注册
```

当前 Demo 能正常运行，说明在这个场景里 EventSystem 恰好先 Awake；这是加载顺序的偶然结果，不是代码保证的。**验证方法**：在 Project Settings → Script Execution Order 里把 SurvivalUIController 设为 −100（早于默认），进入 Play，应当在 Console 看到 Register 处的空引用异常。

> **为什么 TimerSystem 没有同样的风险**
>
> TimerSystem 的核心数组是**字段初始化器**创建的：`private TimerData[] _timers = new TimerData[128];`、`_handleToIndex = new List<int>(128)`……C# 的字段初始化器在构造函数里执行，Unity 反序列化创建对象时就已经完成，早于任何 Awake。Awake 里的 AllocateCapacity 只是再确认一次容量。所以即使在 TimerSystem.Awake 之前调用 CreateTimer，数据结构也是就绪的。
>
> EventSystem 把初始化全部放在 Awake 里，才有了这个窗口。

消除这类问题的方向	原理	代价
能用字段初始化器的就用	构造时完成，早于所有 Awake	不能访问其它对象和序列化字段
在 Instance getter 或方法内部做一次性初始化	谁先访问谁触发，与 Awake 顺序无关	每次访问多一次判断
注册放到 Start	场景加载时所有 Awake 都先于任何 Start	Start 之前发出的事件收不到（SoulDecay 的开局事件在 World.Update 里发，晚于 Start，没问题）
指定执行顺序	`[DefaultExecutionOrder(-100)]` 让基础设施先初始化	顺序成了隐式的全局配置
显式引导	一个引导对象按顺序创建并初始化所有系统	需要改造成“不依赖场景预置”

6.6 退出标志与 Domain Reload

- **只置不复位**：`_isApplicationQuitting` 在 OnApplicationQuit 里设为 true，之后没有任何地方设回 false。正常构建里无所谓（进程就要结束了）。

- **每个 T 一份，只有存活实例才会置位**：退出时没有存活实例的单例类型，标志不会被设置；如果退出过程中有代码访问它的 Instance，仍然会懒创建一个对象。项目里 OnDestroy 统一用 HasInstance，避开了这个问题。
- **关闭 Domain Reload（Enter Play Mode Options）之后**：静态字段在两次 Play 之间不会重置。第一次 Play 退出时被置位的类型，第二次 Play 时 `Instance` 直接返回 null——`World.RegisterAActor` 里 `if (Instance == null) return;` 会让所有 Actor 都不注册，**整个游戏没有任何 BeginPlay 和 Tick**（按源码推演）。修复方向是用 `[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.SubsystemRegistration)]` 在每次进入 Play 时重置静态状态。

【扩展知识】两个与版本相关的细节

① `FindObjectOfType` 在 Unity 2023.1 起被标记为过时，替代为 `FindFirstObjectByType` / `FindAnyObjectByType`；项目用的是 2022.3，没有警告。② `_instance == null` 走的是 UnityEngine.Object 重载的 ==，要检查原生对象是否存活，比纯引用比较慢一些；UI 的 Update 里每帧多次调用 `TimerSystem.Instance`，规模小时可以忽略。

6.7 GameInstance：名字有了，职责还没有

GameInstance 只放在 MainMenuScene，Awake 里 DontDestroyOnLoad 并打一行日志，唯一的成员是 `CurrentWorld ⇒ World.Instance`。它不持有 World、不管理任何系统；而且 `CurrentWorld` 如果在没有 Actor 的场景（比如主菜单）里被访问，会**懒创建一个 World**。从 SoulDecay 直接开始 Play（不经过主菜单）时，GameInstance 根本不存在。

	UE 的 UGameInstance	本项目的 GameInstance
生命周期	引擎创建，跨关卡存在，整个进程一份	只在从主菜单进入时存在
与 World 的关系	World 由引擎创建和切换，GameInstance 可以访问当前 World	World 各场景懒创建，GameInstance 只是转发 World.Instance
典型职责	跨关卡数据、在线会话、持有 GameInstanceSubsystem	无（日志落盘初始化被宏关闭）

6.8 单例的代价与替代方案

隐藏依赖

CombatPawn 依赖 TimerSystem、EventSystem、World 三个单例，这在它的字段和构造参数里看不出来，只能读代码发现。

初始化顺序

谁先访问谁创建，顺序随调用路径变化；6.5 节的隐患就是顺序问题的一个实例。

生命周期含糊

名字叫“单例”，实际是“每个场景一个”。TimerSystem 随场景销毁，计时器也一起消失——这是想要的行为，但不是从类型上看得出来的。

难以测试

MonoSingleton 必须挂在 GameObject 上，EditMode 单元测试里不能直接 new，也很难替换成假的实现（第 20 章）。

替代方案	做法	适合
Service Locator	一个注册表按接口存取服务，启动时注册具体实现	改造成本低，测试时能注册假实现；依赖仍然是隐式的
依赖注入（如 VContainer、Zenject）	容器按作用域（项目 / 场景）创建服务，通过构造函数或字段注入	依赖显式、作用域清晰；引入框架和学习成本
显式引导（Composition Root）	一个引导对象按固定顺序创建、初始化、Tick 所有系统	顺序完全可控，最适合解决 6.5 节的问题
UE Subsystem	按宿主划分生命周期：Engine / GameInstance / World / LocalPlayer Subsystem，由引擎自动创建和销毁	World 级的计时器和事件系统对应 WorldSubsystem；UE 的 FTimerManager 本来就挂在 World 上

6.9 Code Review：单例相关问题

当前设计	问题	影响	改进方向（不写代码）
子类 override Awake 不调 base	防重与提前登记失效	多实例并存；HasInstance 在首次访问前为 false	子类首行调 base 并检查；或基类 Awake 非虚 + 子类初始化钩子
EventSystem 在 Awake 里创建字典，UI 在 OnEnable 里注册	依赖不保证的跨对象 Awake 顺序	顺序反过来时监听注册失败，UI 不更新	字段初始化器 / 方法内惰性初始化 / 注册移到 Start / 指定执行顺序
Instance getter 懒创建	任何访问都可能创建对象	OnDestroy、主菜单里意外创建 World	区分“获取”和“创建”，创建只在引导阶段进行
退出标志只置不复位	关闭 Domain Reload 后第二次 Play 失效	所有 Actor 不注册	进入 Play 时重置静态状态
多余实例 Destroy(gameObject)	销毁整个物体	如果和其它组件放在同一个物体上会被连带销毁	只销毁组件，或约定单例独占一个物体
GameInstance 空壳	名不副实	面试时容易被追问“它管理什么”	让它持有跨场景服务，或删掉
Singleton<T> 未使用	公开构造、非线程安全	死代码	删除，或用 Lazy<T>

6.10 面试追问树

追问树 · Singleton

```
Q6-1 讲一下你的 MonoSingleton。为什么每个子类各有一个实例？
 ├── Q6-2 场景里放了两个 TimerSystem 会怎样？                    ← base.Awake
 │    └── Q6-3 GameInstance 里的 this == null 判断有用吗？        ← Destroy 延迟
 └── Q6-4 HasInstance 和 Instance 有什么区别？什么时候用哪个？
Q6-5 Awake 里能访问别的单例吗？OnEnable 呢？                     ← 6.5 隐患
 └── Q6-6 为什么 TimerSystem 没这个问题？                        ← 字段初始化器
Q6-7 关闭 Domain Reload 之后你的单例会怎样？
Q6-8 单例的缺点是什么？你会用什么替代？UE 是怎么做的？          ← Subsystem
Q6-9 C++ 里怎么写线程安全的单例？                                ← 发散
```

Q6-1 → Q6-4　MonoSingleton 与 base.Awake

L1 • 30 秒　泛型基类，四部分：懒查找场景里的实例、找不到就创建；Awake 里登记自己、多余的自杀；OnDestroy 清空静态引用；退出时设置标志不再创建。每个子类各有一份，因为 C# 每个封闭泛型类型有独立的静态字段。

L2 • 正常回答　有一个我复盘时发现的确定性 bug：World、TimerSystem、EventSystem 都 override 了 Awake 但没调 base，防重和提前登记对它们不生效。现在每个场景至多一个实例，懒查找又会兜底找到预置对象，所以没表现出来；但叠加加载两个带 Managers 的场景，就会有两个 TimerSystem 同时 Update。唯一做对的是 GameInstance。

L3 • 深入　GameInstance 里还有个细节：它在 base.Awake 之后判断 `this == null`，但基类调的是 Destroy，销毁是延迟到帧末的，这时 this 还不是 null，真正起作用的是后面的 `Instance ≠ this`。HasInstance 只查询不创建，所以所有 OnDestroy 里都用它——切场景时如果单例先被销毁，用 Instance 就会重新创建一个泄漏到下一个场景的对象。

L4 • 面试官攻击　“既然没表现出来，为什么叫 bug？” → 因为它让基类承诺的行为失效了，而失效的条件（多场景叠加、预制体复制）是正常的工程操作，不需要任何错误代码就能触发；而且 HasInstance 在首次访问前返回 false，会让依赖它的清理逻辑静默跳过。修复成本只有一行，没有理由留着。

Q6-5 → Q6-6　Awake / OnEnable 里能访问别的单例吗？

L2　不安全。Unity 只保证同一个对象 Awake 在 OnEnable 和 Start 之前，场景加载时所有 Awake 在所有 Start 之前；不同对象之间的 Awake 顺序不保证。我的 SurvivalUIController 在

OnEnable 里调 EventSystem.Instance.Register，而 EventSystem 的监听字典在它自己的 Awake 里才创建——如果 UI 先 Awake，懒查找能找到 EventSystem 对象，但字典还是 null，注册直接空引用，UI 永远收不到分数和寿命事件。现在能跑，是因为这个场景里 EventSystem 恰好先初始化。

L3 TimerSystem 没这个问题，因为它的数组是字段初始化器创建的，构造时就完成了，早于任何 Awake。所以修复可以是：EventSystem 也用字段初始化器；或者在 Register 内部做一次性初始化；或者 UI 改在 Start 里注册；或者给基础设施加 DefaultExecutionOrder。我更倾向于显式引导：一个启动对象按顺序初始化所有系统，不依赖场景里谁先被加载。

Q6-7 关闭 Domain Reload 之后?

L2 静态字段在两次 Play 之间不重置。退出标志在第一次退出时被置为 true、之后从不复位，第二次 Play 时 Instance 直接返回 null；World.RegisterAActor 遇到 null 就 return，所有 Actor 都不会注册，整个游戏没有 BeginPlay 和 Tick。解决办法是用 RuntimeInitializeOnLoadMethod 在每次进入 Play 时重置所有单例的静态状态。

Q6-8 单例的缺点？怎么替代?

L2 隐藏依赖（CombatPawn 用了三个单例，从接口上看不出来）、初始化顺序不确定、生命周期含糊（叫单例，实际每个场景一个）、难以测试（MonoBehaviour 不能在 EditMode 里直接 new，也不好换成假实现）。

L3 替代方案按改造成本排：Service Locator（按接口注册，测试时能替换）、显式引导（一个启动对象按顺序创建和驱动所有系统）、依赖注入框架（按项目 / 场景作用域管理）。UE 的思路是按宿主划分生命周期的 Subsystem：Engine、GameInstance、World、LocalPlayer 各有一类，由引擎自动创建销毁。我的 TimerSystem 和 EventSystem 本质上是“每个 World 一份”，对应 UE 的 WorldSubsystem——UE 的 FTimerManager 本来就挂在 World 上。

常见错误回答 ①

错误：“我的单例在 Awake 里会把多余的实例销毁，所以不会有多个实例。”

问题：四个子类都没调 base.Awake，这段逻辑对它们不执行。

更好的回答：“基类有防重逻辑，但 World / TimerSystem / EventSystem 漏调了 base.Awake，目前靠每个场景只放一个来保证唯一。”

常见错误回答 ②

错误：“Awake 一定比 OnEnable 和 Start 早，所以在 OnEnable 里访问别的系统是安全的。”

问题：顺序保证只针对同一个对象；跨对象时，别人的 Awake 可能还没执行。

更好的回答：“跨对象的依赖放在 Start 或 BeginPlay，或者让被依赖的系统在被访问时惰性初始化。”

6.11 发散：面试官会从这里追到的知识

C++ 的 Meyers 单例

函数内静态局部变量：`static T& Get() { static T inst; return inst; }`。C++11 起标准保证局部静态变量的初始化是线程安全的（编译器生成带锁或原子标志的“magic statics”）。为什么会追过去：游戏客户端岗大多是 C++，单例题几乎一定会转到 C++ 实现。

双重检查锁

“先判空、加锁、再判空、再创建”在没有内存屏障时可能让别的线程看到未构造完的对象；C++ 要用 `std::atomic` 配合 acquire / release，或者直接用 `std::call_once`。第 13 章的 Log 会遇到同样的问题。

静态初始化顺序

不同编译单元的全局对象构造顺序不确定，析构顺序与构造相反；一个全局对象的析构函数里访问另一个已析构的单例就是未定义行为——和“切场景时单例谁先 OnDestroy”是同一个问题。

C# 的 Lazy<T> 与静态构造函数

静态构造函数由运行时保证只执行一次且线程安全；`Lazy<T>` 提供可选的线程安全模式。它们解决了 `Singleton<T>` 的并发问题，但解决不了“公开构造函数”。

UE 的 Subsystem

依赖注入的作用域

继承 UWorldSubsystem 等基类即可，引擎在宿主创建时自动实例化、销毁时自动清理，通过 `GetWorld()->GetSubsystem<T>()` 访问。它把“全局唯一”换成了“宿主内唯一”，生命周期由类型直接表达。

Singleton（整个容器一份）、Scoped（每个作用域一份，例如每个场景）、Transient（每次注入新建）。项目里 TimerSystem 实际需要的是“场景作用域”，这正是 DI 框架能直接表达、而 MonoSingleton 只能靠约定实现的东西。

本章速记

- MonoSingleton = **懒查找/懒创建 + Awake 防重 + OnDestroy 清空 + 退出标志**；每个封闭泛型类型一份静态字段。
- **五个子类只有 GameInstance 调了 base.Awake**：防重和提前登记对 World / TimerSystem / EventSystem 不生效；现在靠“每场景至多一个 + 懒查找兜底”没有表现出来。
- GameInstance 的 `this == null` 不生效（Destroy 延迟），起作用的是 `Instance ≠ this`。
- **EventSystem 在 Awake 里建字典、UI 在 OnEnable 里注册**：依赖不保证的跨对象 Awake 顺序；TimerSystem 用字段初始化器所以没这个问题。
- 退出标志只置不复位：关闭 Domain Reload 后第二次 Play，`Instance` 返回 null，所有 Actor 都不注册。替代方案：Service Locator、显式引导、DI、UE Subsystem。

▶ **自测 1：在 GamePlayTest2 场景（没有预置任何单例）里，TimerSystem 第一次被创建是在什么时候？它的 Awake 漏调 base 会有影响吗？**

▶ **自测 2：把 SurvivalUIController 的注册从 OnEnable 移到 Start，为什么能消除 6.5 节的隐患？会不会漏掉开局事件？**

▶ **自测 3：为什么说 TimerSystem 对应 UE 的 WorldSubsystem，而不是 GameInstanceSubsystem？**

PART THREE

第三部分　基础设施

事件、环形队列、计时器、对象池、线程安全队列、日志。TimerSystem 是全书重点，分上下两章（第 9、10 章）；本部分其余各章都围绕“它解决了什么、在哪种情况下会失效”展开。

CHAPTER 07

EventSystem：从 Observer 到双缓冲延迟派发

项目里所有“状态变了，UI 去更新”的路径都经过它。双缓冲让事件里发事件不会递归，这是亮点；溢出静默丢弃、异常中断派发、注销后仍被调用、闭包分配，是面试官最容易打的四个点。

本章目标

- 能画出 Trigger → back → swap → front → Dispatch 的流程，并解释“A 的回调里发 B 为什么不会递归”
- 对比立即派发与延迟派发的取舍：递归深度、状态一致性、延迟、调试栈
- 说清 Priority 到底保证了什么、会不会饿死
- 说清队列满了当前怎么处理，以及不同事件该用哪种溢出策略
- 逐项说出一次 Trigger 有哪些堆分配，并给出低 GC 事件队列的设计空间
- 知道 4 个隐藏行为：按运行时类型分发、快照语义、多播异常中断、事件对象被延迟读取

7.1 这一章解决什么问题

没有事件系统时，GameState 加了分要通知 UI，就得持有 UI 的引用；UI 换了、多了一个成就系统，GameState 也得改。事件系统把“谁关心”从发送方挪走：发送方只说“分数变了”，订阅方自己登记。这就是 **Observer / Pub-Sub**。它带来的第二个问题是**派发时机**：通知是当场执行（立即派发），还是先排队、稍后统一执行（延迟派发）？项目选了后者，并用双缓冲实现。

7.2 当前项目怎么做：三个结构

EventSystem.cs:23-114（节选）

```
public class EventSystem : MonoSingleton<EventSystem> {
    private readonly int PRIORITY_LEVEL_NUM = (int)Priority.Max + 1;      // = 3（Logic / Appe
    private readonly int MAX_EVENTQUEUE_LENGTH = 2000;
    private RingBuffer<Action>[] _frontBuffers;                           // 本帧要执行的
    private RingBuffer<Action>[] _backBuffers;                            // 新触发的先放这里
    private Dictionary<Type, Delegate> _listenersDic;                     // 事件类型 → 多播委

    public void Register<T>(Action<T> listener) where T : IEventArgs {
        Type type = typeof(T);                                            // ① 注册用“静态类型
        _listenersDic[type] = _listenersDic.ContainsKey(type)
            ? Delegate.Combine(_listenersDic[type], listener) : listener;
    }
    public void Trigger<T>(T e) where T : IEventArgs {
        if (!_listenersDic.TryGetValue(e.GetType(), out var rawDelegate)) return;   // ② 查找用
        if (rawDelegate is Action<T> callback) {
            Action task = () => callback(e);                              // ③ 闭包：此刻抓住委
            _backBuffers[(int)e.Priority].TryEnqueue(task);               // ④ 返回值被忽略
        }
    }
    private void Update() {
        var temp = _frontBuffers; _frontBuffers = _backBuffers; _backBuffers = temp;    // ⑤ 交
        for (int i = 0; i < PRIORITY_LEVEL_NUM; ++i)
            while (_frontBuffers[i].TryDequeue(out Action task)) task.Invoke();         // ⑥ 没有
    }
}
```

结构	内容	容量 / 内存
`_listenersDic`	事件类型 → 一个多播委托（内部是调用列表）	按事件类型数增长；每次 Register / UnRegister 都生成新的委托对象
`_backBuffers[3]`	本帧新触发的任务，按优先级分三个环形队列	每级 2000 个槽
`_frontBuffers[3]`	本帧正在派发的任务	同上；两组共 6 × 2000 = 12000 个引用槽（64 位下约 94 KB），其中 Max 级的 4000 个永远为空（实测 G3）

7.3 双缓冲：A 的回调里发 B，为什么不会递归

Frame N / N+1 的状态变化

```
Frame N（World.Update 里）    GameState.SetMatchState → Trigger(A)
                              front = []      back = [A]

Frame N · EventSystem.Update  swap            front = [A]     back = []
                              执行 A 的回调 → 回调里 Trigger(B) → B 进入 back
                              front = []      back = [B]       ← B 没有在本次 while 循环里被执行

Frame N+1 · EventSystem.Update swap           front = [B]     back = []
                              执行 B 的回调 → 回调里 Trigger(A) → A 进入 back …
```

关键在于：**Dispatch 只消费 front，而 Trigger 只写 back。**派发过程中新产生的事件进不了正在被消费的队列，于是“回调里再发事件”被拆成了“下一帧再处理”。【实测 E7-a】A、B 两个事件互相触发，5 帧里每帧恰好派发一个（A B A B A），最大调用深度始终是 1。如果是立即派发，这会是无限递归直到栈溢出。

和 World 的 pending 列表是同一个思想

World 把“Tick 中新注册的 Actor”放进 pending，下一帧才加入；EventSystem 把“派发中新触发的事件”放进 back，下一帧才派发。两者解决的都是**遍历中修改被遍历的集合**，代价都是“新东西晚一帧生效”。第 3 章的结论在这里同样成立。

7.4 立即派发 vs 延迟派发

维度	立即派发（C# event / UnityEvent 直接 Invoke）	延迟派发（本项目）
回调里发事件	嵌套调用，栈越来越深；互相触发就是无限递归 → StackOverflow	进入下一帧，栈深度恒为 1（E7-a）
状态重入	发送方还在修改自己的状态时，监听者已经回调进来读写它（例如分数加到一半就被 UI 读）	回调执行时发送方的那一步已经结束
顺序	深度优先，嵌套越深越难推理	广度优先，按帧分层：第 k 级连锁事件在第 k 帧处理（G2）
延迟	0	0～1 帧起步，每多一级连锁多 1 帧（G2）
读到的状态	事件发生那一刻的状态	派发时的状态——可能已经又变了（例如 Pawn 已经被 Destroy）
调试	调用栈能看到是谁发的	栈断开：断点在回调里只能看到 EventSystem.Update
适合	需要同步结果的（“能不能放技能”的校验、伤害修正链）	通知类（UI 刷新、音效、成就统计）

【实测 G2】一条 A→B→C 的三级事件链，Trigger 发生在第 0 帧：如果 World.Update 先于 EventSystem.Update 执行，A、B、C 分别在第 0、1、2 帧处理；顺序反过来，就是第 1、2、3 帧。而项目没有固定这两个脚本的执行顺序（第 1 章 1.4），所以起点延迟是 0 还是 1 帧**由 Unity 决定。**

> **面试注意：不要说“延迟派发更好”**
> 它是用“晚一帧 + 栈断开 + 读到的不是当时的状态”换“不递归 + 不重入”。正确的说法是：通知类事件适合延迟派发；需要立刻得到结果的请求（“我能不能夺舍这个目标”）应该是直接的函数调用，不该走事件。

7.5 四个隐藏行为

7.5.1 注册用 typeof(T)，查找用 e.GetType()

【源码事实】Register 用编译期类型做键，Trigger 用对象的运行时类型查找，然后再用 `is Action<T>` 检查。两者不一致时会**静默丢弃**，没有任何日志：

写法	查到的委托	is Action<T>	结果
`Register<ScoreChangedEventArgs>` + `Trigger(new ScoreChangedEventArgs(...))`	Action<ScoreChanged>	true	**正常**
同上注册，但 `AbstractEventArgs e = new ScoreChangedEventArgs(); Trigger(e)`	Action<ScoreChanged>（按运行时类型找到了）	false：T 是基类，Action<派生> 不能当 Action<基类> 用	**丢弃**
`Register<AbstractEventArgs>`（想监听所有事件）+ 发派生事件	按派生类型查找，找不到	—	**丢弃**

Demo 里所有调用都用具体类型，所以没触发；但“写一个通用的事件转发 / 录制工具，用基类变量 Trigger”时一定会踩到。要么统一用 `typeof(T)`，要么明确规定“只按具体类型分发、不支持继承”。

7.5.2 快照语义：注销之后还会被调用一次

闭包在 Trigger 那一刻抓住了当时的多播委托。C# 的委托是不可变的，之后 UnRegister 生成的是新委托，不会影响已经入队的旧快照。【实测 E7-c】Trigger 之后、派发之前注销的监听者，下一帧仍被调用了 1 次。

什么时候会出事：SurvivalUIController 在 `OnDisable` 里注销。如果同一帧先有人 Trigger 了分数事件、然后 UI 被关掉或场景开始卸载，下一帧派发时会调用到一个已经被 Destroy 的 MonoBehaviour——访问 `PlayerScoreText.text` 就是 MissingReferenceException，而 EventSystem 没有 try/catch（7.5.3）。

7.5.3 异常：一个监听者抛异常，影响三层

【实测 G1】同一事件注册了“UI 分数 → 成就系统 → 音效”三个监听者，成就系统抛异常：

1. **同一事件**：多播委托按注册顺序调用，遇到异常立即中断——**音效永远收不到这次事件。**
2. **同一帧的其他事件**：异常直接逃出 `Update`，while 循环中断，front 里剩下的事件都没派发。

3. **之后的帧**：下一帧 swap 时，剩下的事件所在的旧 front 变成了 back，这一帧没有任何派发；再下一帧才轮到它们——**晚了 2 帧，而且排在第 2 帧新触发的事件后面**（E7-d：E2、E3 排到了 E4 之后）。

对比：TimerSystem 的每个回调都包了 try/catch（第 10 章）。同一个作者在两个基础设施里做了不同的选择，面试时值得主动指出并说明应该统一。

7.5.4 事件对象被延迟读取

闭包捕获的是事件对象的**引用**，派发时才读字段。现在每次都 `new` 一个事件对象，没有问题；但如果为了省 GC 把事件对象做成复用的（比如一个静态的 `ScoreChangedEventArgs` 改字段后反复 Trigger），同一帧的三次 Trigger 到下一帧派发时会读到**同一个最后的值**。“对象池化 + 延迟派发”是一对天然冲突，第 11 章还会遇到。

7.6 Priority：它到底保证了什么

优先级定义与 Demo 用法

```
enum Priority { Logic = 0, Appearance = 1, Max = 2 }
MatchStateChangedEventArgs     Logic        比赛阶段切换
PlayerRequestControlEventArgs  Logic        UI 请求控制权
PlayerLifespanTimerEventArgs   Logic        寿命计时器句柄
ScoreChangedEventArgs          Appearance   分数刷新（表现层）
```

问题	答案（源码事实 / 实测 G3）
保证了什么	**同一批次**（同一次 swap 得到的 front）里，所有 Logic 事件先于所有 Appearance 事件执行
不保证什么	跨帧顺序：Logic 回调里新触发的 Appearance 事件进入下一帧（E7-e）；同级内按 Trigger 顺序
会饿死吗	不会。每帧清空 front 的全部三级；单帧 Trigger 2500 个 Logic + 10 个 Appearance，Appearance 10 个全部派发
容量是否独立	是。上例只丢了 Logic 级溢出的 500 个，Appearance 不受影响
Max 是什么	本意是“级数哨兵”，却被算进了级数（`Max + 1 = 3`），多分配了一级永远为空的队列
能限流吗	不能。每帧必须清空，没有“每帧最多处理 N 个、剩下的留到下一帧”的预算机制——事件洪水会直接变成一帧的卡顿

面试注意：优先级会不会饿死，取决于“每帧清空”还是“每帧有预算”

当前实现每帧清空所以不饿死，但也意味着没有削峰能力。如果加上“每帧最多派发 N 个”，低优先级就可能被持续的高优先级洪水饿死，这时要配老化（等待越久优先级越高）或者给每级单独预算。这就是操作系统调度里“优先级反转 / 饥饿 / aging”在游戏里的样子。

7.7 队列满了怎么办：溢出策略

【源码事实】`TryEnqueue` 满了返回 false，Trigger 忽略返回值。【实测 E7-b / G3】单帧单级第 2001 个事件开始被静默丢弃，无日志、无计数、调用方不知道。2000 个对 Demo 绰绰有余，但这类 bug 出现时最难查：现象是“偶尔 UI 没刷新”。

策略	行为	适合的事件
DropNewest（当前，但是静默的）	丢掉新来的	可合并的状态通知：下一次分数事件会带来最新值
DropOldest	挤掉最老的	只关心最新的：鼠标位置、调试信息
合并（Coalesce）	同类型同 key 的事件只保留最后一个	分数、血量这类“状态型”事件——比扩容更根本
Resize	扩容	不能丢的：比赛阶段切换、玩家死亡。代价是偶发分配
Fallback 队列	溢出部分进一个可增长的 List	同上，正常路径保持零扩容
Assert / 报错	开发期直接中断	“理论上不可能满”的队列，用来尽早发现逻辑错误（例如事件互相触发的环）
Telemetry	记录每帧峰值、丢弃数	任何策略都应该配上；容量 2000 是拍脑袋还是有数据，靠它回答

重点在于**按事件语义选策略**：“分数变了”丢一个无所谓，“玩家死了”丢了游戏就卡死在进行中。一个队列用一种策略处理所有事件，是当前设计的根本问题。

7.8 GC：一次 Trigger 分配了什么

分配	在哪	什么时候
事件对象	调用方 `new ScoreChangedEventArgs( ... )`（class）	每次
闭包对象（编译器生成的 display class，字段 = callback、e）	Trigger 内部	**每次，即使没有监听者**——见下方说明
Action 委托	`() ⇒ callback(e)`	找到监听者时
多播委托 + 调用列表数组	Register / UnRegister 的 Delegate.Combine / Remove	每次注册注销（不在每帧路径上）
Dictionary 扩容、RingBuffer	Awake	一次（RingBuffer 容量固定，永不扩容）

为什么“没有监听者也会分配闭包”

【扩展知识，需用 IL 或 Profiler 确认】C# 编译器把被 lambda 捕获的局部变量和参数“提升”到一个编译器生成的类里，这个类的实例在**被捕获变量所在作用域的入口**创建。`e` 是参数，作用域是整个方法，所以闭包对象在 Trigger 第一行之前就 new 出来了，哪怕后面 `TryGetValue` 失败直接 return。SoulDecay 的 MatchStateChanged 没人监听，每次切阶段照样分配。可以用 sharplab.io 看编译结果，或在 Profiler 的 GC Alloc 列确认。

SurvivalGameState 的 BeginPlay 一次发 3 个分数事件，每次加分发 1 个；按上表每个事件约 3 次托管分配。频率很低，不是性能问题——但它说明了“框架宣称低 GC”和“框架的每次使用都在分配”之间的距离。

真正低 GC 的事件队列，设计空间有哪些

按类型的泛型队列

每种事件一个 `EventQueue<T>`（静态泛型类），队列里直接存 T 而不是闭包；派发时取出 T 调 `Action<T>`。没有闭包、没有类型查找，代价是派发顺序只在同类型内有保证。

struct 事件 + 预分配

事件定义为 struct，存进预分配的数组；结合上一条可以做到稳态零分配。注意 struct 里如果有引用，出队时要清槽（第 8 章 G5）。

EventEnvelope

统一的信封 `{ int typeId; int payloadHandle; }`，payload 存在按类型分池的数组里，信封里只放句柄——这又回到了第 9 章的 Slot Map + generation。

合并状态事件

分数、血量这类事件根本不需要排队：标脏，帧末统一推最新值（或者 UI 直接绑定可观察属性）。最省的分配是不发事件。

7.9 ThreadSafeEventSystem：实现了，没人用

【源码事实】同一文件里还有一个加锁版本：Register / UnRegister / Trigger 查字典时加 `_listenerDicLock`，入队和交换时加 `_swapLock`，底层用 ThreadSafeRingBuffer（每个操作自己又加一次锁）；派发仍在主线程的 Update 里。全仓库没有调用方。设计意图是“工作线程可以发事件，回调在主线程执行”，这个方向是对的（Unity API 只能在主线程调用），但有三个问题：

- **拿不到实例**：工作线程第一次访问 `ThreadSafeEventSystem.Instance` 时，如果实例还没缓存，会走 `FindObjectOfType` / `new GameObject`，这些 Unity API 只能在主线程调用；正确用法是在主线程先取到引用再交给工作线程。
- **双重加锁**：外层 `_swapLock` 已经保证了入队和交换的互斥，内层 ThreadSafeRingBuffer 的锁是多余的开销（第 12 章 G7：同样的 lock 环形队列在 4 生产者时吞吐降到 1 生产者的约 1/4）。
- **继承了单线程版的所有问题**：溢出静默丢弃、无 try/catch、闭包分配（而且是在工作线程分配）。

7.10 与 Unity / UE 的原生机制对比

机制	派发	与本项目的区别
C# `event Action<T>`	立即，同步	零基础设施，强类型；发送方必须暴露 event，订阅方要能拿到发送方
UnityEvent	立即	可在 Inspector 里配置，基于反射与序列化，比 C# event 慢
SendMessage / BroadcastMessage	立即，按方法名字符串	反射查找、拼错名字运行时才知道；只在 GameObject 层级内
UE 动态多播委托（`DECLARE_DYNAMIC_MULTICAST_DELEGATE`）	立即 Broadcast	可被蓝图绑定，带 UObject 弱引用，绑定对象被 GC 后自动失效——正好解决本项目 7.5.2 的“调用已销毁对象”
UE Gameplay Message Subsystem（Lyra 示例项目中的插件）	立即，按 GameplayTag 频道	解耦方式与本项目相同（按“频道”而非按发送者订阅），但用 Tag 层级代替 C# 类型
UE GAS 的 GameplayEvent	立即，发给 AbilitySystemComponent	用于触发技能，带 payload 结构体

值得注意：主流引擎的事件 / 委托默认都是**立即派发**，延迟派发通常由业务自己按需加一层队列。项目把所有事件都做成延迟派发，是一个可以讨论的整体选择。

7.11 Code Review：EventSystem 问题表

级别	问题	为什么 / 触发场景	最小修复（思路）	成熟设计
P0	派发无异常隔离	一个监听者抛异常，同事件后续监听者丢失、整帧中断、剩余事件晚 2 帧且乱序（G1、E7-d）	每个 task 外包 try/catch	逐个监听者调用（遍历调用列表），单个失败不影响其他
P0	溢出静默丢弃	第 2001 个起消失（E7-b、G3）	失败时打错误日志 + 计数	按事件语义选策略（7.7）+ 峰值统计
P1	快照语义	注销后仍被调用一次（E7-c），叠加 P0 会抛异常	闭包只捕获事件，派发时再查字典	订阅返回 token，注销即作废；或绑定 owner 自动失效
P1	注册 / 查找类型不一致	基类变量 Trigger 静默丢弃（7.5.1）	统一用 typeof(T)	明确“不支持继承分发”或按类型层级派发
P1	字典在 Awake 创建	OnEnable 里注册可能早于它（第 6 章 6.5）	改为字段初始化器	显式引导顺序
P1	派发时机依赖未定义的脚本顺序	事件延迟 0 或 1 帧（G2）	`DefaultExecutionOrder`	由 GameLoop 在固定时机 Pump
P2	每次 Trigger 约 3 次分配	无监听者也分配闭包（7.8）	先查字典、再进入构造闭包的局部函数	按类型的泛型队列（7.8）
P2	无削峰能力	事件洪水变成单帧卡顿	—	每帧预算 + 老化
P3	Max 级多分配一组空队列；类名与 `UnityEngine.EventSystems.EventSystem` 相同	浪费 4000 槽；场景里两个同名物体（第 6 章）	级数用 `(int)Max`；改名	—

7.12 测试现状

`EventSystemTest.cs` 注册 EventA / EventB 两个互相触发的回调，每帧打一行日志，**没有任何断言**，也没有挂在场景里——它演示的正好是 E7-a 的“不递归”，但只能靠人眼看日志。溢出、异常、注销时序、类型分发、优先级顺序全部没有测试。

7.13 面试追问树

追问树 · EventSystem

```
Q7-1 讲一下你的事件系统
├── Q7-2 回调里再 Trigger 一个事件会怎样？为什么不会栈溢出？
│   ├── Q7-3 那延迟派发有什么代价？                          ← 延迟、栈断开、读到的状态
│   └── Q7-4 和直接用 C# event 比，你为什么要做延迟？
├── Q7-5 Priority 保证了什么？会饿死吗？
├── Q7-6 RingBuffer 满了怎么办？                             ← 压力题：静默丢弃
│   └── Q7-7 那你觉得哪种策略对？
├── Q7-8 () ⇒ callback(e) 分配了什么？没有监听者也分配吗？    ← GC
│   └── Q7-9 怎么做一个真正低 GC 的事件队列？
├── Q7-10 一个监听者抛异常会怎样？                           ← 反例
├── Q7-11 UI 被销毁了还会收到事件吗？                        ← 快照语义
└── Q7-12 能用基类监听所有派生事件吗？                       ← 类型分发
Q7-13 做成多线程呢？工作线程能直接 Trigger 吗？              ← 发散到第 12 章
Q7-14 联网游戏里事件系统和 RPC / 复制是什么关系？            ← 发散
```

Q7-1 → Q7-4　讲一下事件系统；回调里发事件为什么不会栈溢出？

L1 • 30 秒　按事件类型订阅的发布订阅系统，派发是延迟的：Trigger 只把任务放进 back 缓冲，EventSystem 每帧交换 front/back 再清空 front。回调里再发的事件只能进 back，下一帧才执行，所以不会递归。

L2 • 正常回答　字典存“事件类型 → 多播委托”，缓冲是按优先级分级的环形队列，每级 2000 个。我测过 A、B 两个事件互相触发，每帧恰好派发一个，调用深度始终是 1；立即派发的话就是无限递归。延迟派发的另一个好处是监听者执行时发送方那一步已经做完，不会读到改了一半的状态。

L3 • 深入　代价是：起步延迟 0 到 1 帧，而且取决于 EventSystem 和 World 谁先 Update，我没有固定这个顺序；每多一级连锁就多一帧；回调里读到的是派发时的状态而不是事件发生时的；调试栈断开。所以我会把它定位成“通知”通道，需要立即结果的请求还是直接函数调用。

L4 • 面试官攻击　“UE 的委托都是立即 Broadcast 的，你这是不是过度设计？” → 部分同意。全部事件都延迟是我当时的统一选择，好处是一致、不会递归；但像“玩家请求控制权”这种请求式事件，立即调用更合适。更好的分层是：默认立即派发，业务按需把某些事件放进延迟队列，并且给延迟队列配异常隔离和溢出策略。

Q7-6 / Q7-7　RingBuffer 满了怎么办？（压力题）

L1　现在是静默丢弃：TryEnqueue 返回 false，Trigger 没有处理。这是一个问题。

L2　单帧单级超过 2000 个才会触发，Demo 远到不了；但一旦出现就是“偶尔 UI 不刷新”这种很难查的 bug，至少要打错误日志和计数。

L3　策略要按事件语义分：状态型事件（分数、血量）应该合并，只保留最新值，比扩容更根本；关键流程事件（死亡、阶段切换）不能丢，要么扩容、要么溢出到备用队列；只关心最新的可以挤掉最老的；“理论上不可能满”的队列开发期直接断言。再配上峰值统计，容量就有了依据。

L4 攻击　“那你为什么不直接用 Queue<T>，让它自动扩容？” → 可以，自动扩容只在峰值时分配一次，之后容量保持。固定容量的价值是把“异常的事件量”暴露出来——但前提是溢出时要报出来，静默丢弃把这个价值也丢掉了。

Q7-8 / Q7-9　闭包分配了什么?

L2　编译器为被捕获的 callback 和 e 生成一个类，每次 Trigger 实例化一次；lambda 本身再生成一个 Action 委托。加上调用方 new 的事件对象，每次 Trigger 约三次分配。而且因为 e 是参数，闭包对象是在方法入口创建的，没有监听者的事件也会分配——比如 SoulDecay 里比赛阶段事件没人听，照样分配。

L3　低 GC 的思路：按事件类型的泛型静态队列，队列里直接存事件本身而不是闭包；事件定义成 struct 存进预分配数组；状态型事件干脆不排队，标脏后帧末推送最新值。各有代价：泛型队列会丢失跨类型的全局顺序，struct 事件要注意出队清理里面的引用。

Q7-10 / Q7-11　监听者抛异常？UI 销毁后还会收到吗?

L2　抛异常影响三层：同一事件排在后面的监听者收不到，因为多播委托遇到异常就中断；这一帧剩下的事件都没派发，因为异常逃出了 Update；剩下的事件要晚两帧，还排在新事件后面——我把逻辑移植出来测过。TimerSystem 每个回调都有 try/catch，EventSystem 没有，这是我要统一的。

L3　UI 销毁后还可能收到一次：闭包在 Trigger 时抓住了当时的委托，之后 OnDisable 里注销不影响已经入队的任务。同一帧“先发事件、后关 UI”就会调用到已销毁的组件，访问 TMP 文本抛 MissingReferenceException，再叠加上面的异常问题。修法是派发时再查字典，或者订阅绑定 owner、owner 销毁自动失效——UE 的动态委托对 UObject 就是这么做的。

Q7-14　联网游戏里事件系统和 RPC / 属性复制是什么关系？（发散）

L2　本地事件系统只在一个进程里传递。联网后，“分数变了”这类状态应该走属性复制：服务器改 GameState / PlayerState 的属性，客户端收到后在 OnRep 回调里发一个本地事件给 UI——事件系统变成了“复制属性 → 本地表现”的最后一环。“播放一个特效”这类一次性的表现走不可靠多播 RPC。不要把本地事件直接透传成网络消息：网络消息需要权威校验、序列化、带宽控制，本地事件系统不负责这些。

常见错误回答 ①

错误：“双缓冲保证了事件的执行顺序和触发顺序完全一致。”

问题：只在同一批次、同一优先级内成立；跨优先级 Logic 先于 Appearance；回调里触发的进入下一帧；异常后剩余事件会被排到新事件之后（E7-d）。

更好的回答：“同一批次内先按优先级、再按触发顺序；连锁事件逐帧展开；异常会打乱顺序，这是当前实现缺 try/catch 导致的。”

常见错误回答 ②

错误：“用了 RingBuffer，所以事件系统是零 GC 的。”

问题：RingBuffer 只是不扩容；队列里存的是闭包委托，每次 Trigger 都要新建闭包和委托，事件对象本身也是 class。

7.14 发散：面试官会从这里追到的知识

消息队列与背压

生产速度大于消费速度时，要么阻塞生产者（有界阻塞队列）、要么丢、要么扩容。Kafka、TCP 滑动窗口、音频缓冲区都是同一个问题的不同答案。游戏主线程不能阻塞，所以只剩“丢”和“扩容”。

C# 委托的实现

委托是对象：目标 + 方法指针；多播委托内部持有调用列表数组，Combine/Remove 返回新对象（不可变），所以在多线程下读取是安全的。`event` 关键字只是限制外部只能 += / -=。

C++ 里的对应物

`std::function` 有小对象优化，捕获少时不分配；信号槽（Qt、boost.signals2）解决的也是“槽被销毁后不要再调用”——用连接对象 / 弱引用追踪生命周期。

命令模式与回放

如果把输入也做成事件并记录帧号，延迟派发的事件流就是一份可回放的录像；帧同步游戏的“输入即事件”就是这个思路。前提是派发顺序完全确定。

UI 架构

事件推送是 MVC/MVP 里 Model → View 的通知；MVVM 用可观察属性和数据绑定代替手写事件。分数这类状态，绑定比事件更不容易出错（不会丢、不会重复）。

操作系统的消息循环

Win32 的 GetMessage/DispatchMessage、PostMessage（异步入队）vs SendMessage（同步调用），就是延迟派发 vs 立即派发；Unity 自己的 PlayerLoop 也是一个大的消息循环。

本章速记

- 结构：**Dictionary<Type, Delegate>** + 按优先级的 **front/back 两组 RingBuffer<Action>（每级 2000）**；Trigger 写 back，Update 交换后清空 front。
- 双缓冲让回调里发的事件进入下一帧：**不递归、不重入**（E7-a 深度恒为 1）；代价是**延迟 0 ~ 1 帧起步、每级连锁 +1 帧**（G2）、栈断开、读到派发时的状态。
- Priority：同批次内 Logic 先于 Appearance；**每帧清空所以不饿死，但也无法限流**；Max 级多分配一组空队列（G3）。
- 四个隐藏行为：**按运行时类型查找导致基类 Trigger 丢失**、**快照语义**（E7-c）、**多播异常中断 + 整帧中断 + 晚 2 帧乱序**（G1/E7-d）、**事件对象被延迟读取**。
- 溢出：**静默丢弃**（E7-b）；按语义选 合并 / 扩容 / 丢最老 / 断言，并加统计。

- GC：事件对象 + 闭包（**无监听者也分配**）+ 委托；低 GC 方向是按类型的泛型队列 / struct 事件 / 状态合并。

▶ **自测 1：第 N 帧依次 Trigger 了 Appearance 事件 X、Logic 事件 Y，Y 的回调里 Trigger 了 Logic 事件 Z。按派发顺序写出 X、Y、Z 在哪一帧、谁先谁后（假设 EventSystem 在 World 之后 Update）。**

▶ **自测 2：为什么 UnRegister 之后，监听者在下一帧仍可能被调用一次？怎么改？**

▶ **自测 3：想把 ScoreChangedEventArgs 做成复用的单个对象来省 GC，会出什么问题？**

CHAPTER 08

RingBuffer：环形队列、满与空、清槽与回绕

70 行代码，面试里常作为热身题出现。真正被追问的不是怎么写，而是：满和空怎么区分、为什么出队要清槽、取模和位与差多少、为什么它是无锁队列的起点。

本章目标

- 手推 head / tail / size 在入队、出队、回绕时的变化，说清满与空的三种判别方式
- 说清“出队清槽”的目的，以及 `typeof(T).IsValueType` 这个判断漏掉了什么
- 用实测回答“取模和位与差多少”“为什么容量常取 2 的幂”
- 对比 RingBuffer 与 `Queue<T>`、链表队列的取舍
- 知道测试为什么没覆盖到它，以及要补哪些用例

8.1 这一章解决什么问题

EventSystem 需要一个 FIFO 队列，要求：入队出队 O(1)、不在每帧路径上分配、容量可控。链表每个节点一次分配；`List<T>` 从头部删除是 O(n)；`Queue<T>` 本身就是环形数组，只是会自动扩容。项目选择了自己写一个**固定容量**的环形数组。

8.2 当前实现

RingBuffer.cs（全文节选）

```
public class RingBuffer<T> {                          // 注意：不在 GamePlayArchitecture 命名空间里
    private readonly T[] _buffer; private readonly int _capacity;
    private int _head, _tail, _size;                  // head：下一个出队位置；tail：下一个入队位置
    public bool TryEnqueue(T e) {
        if (_size ≥ _capacity) return false;          // 满：拒绝（不覆盖、不扩容）
        _buffer[_tail++] = e;
        _tail %= _capacity;                           // 回绕
        _size++; return true;
    }
    public bool TryDequeue(out T e) {
        if (_size == 0) { e = default; return false; }
        e = _buffer[_head];
        if (!typeof(T).IsValueType) _buffer[_head] = default;    // 引用类型：清槽，让 GC 能回收
        _head = (_head + 1) % _capacity;
        --_size; return true;
    }
    public bool TryPeek(out T e) { ... }              // 只读队头
    public int Count ⇒ _size;
}
```

8.3 手推一遍：回绕与满 / 空

capacity = 3（实测 G4，与源码逐行一致）

操作	buffer	head	tail	size	
初始	[_ _ _]	0	0	0	← head == tail，空
入队 1,2,3	[1 2 3]	0	0	3	← head == tail，满
再入队 4	→ false（size ≥ capacity）				
出队 → 1	[_ 2 3]	1	0	2	（引用类型时 1 号槽被清成 default）
入队 4	[4 2 3]	1	1	3	← tail 回绕到 0 后写入，再 +1
全部出队 → 2 3 4	[_ _ _]	1	1	0	← head == tail，又是空

“满”和“空”时都有 `head == tail`，光看两个指针分不出来。三种常见解法：

方案	做法	代价	典型使用
额外计数 size（本项目）	size==0 空，size==capacity 满	多一个字段；入队出队都要改 size——多线程下 size 成了生产者和消费者**共同写**的变量	单线程队列
浪费一个槽	(tail+1)%cap == head 视为满	实际只能存 cap−1 个	教科书写法
单调递增计数	head、tail 只增不回绕，下标用 `tail & (cap-1)`；`tail-head` 就是元素数	容量必须是 2 的幂；计数溢出要靠无符号回绕保证正确	Linux kfifo、无锁 SPSC（第 12 章 G7）

面试注意：为什么无锁队列不用 size

size 被生产者 ++、被消费者 −−，两个线程写同一个变量，必须用原子操作或锁。单调递增计数让 head 只被消费者写、tail 只被生产者写，各写各的，这正是 SPSC 无锁的前提。所以“满空用 size 判断”这个设计在单线程下最简单，到了多线程就是瓶颈——ThreadSafeRingBuffer 沿用了它，只能整体加锁。

8.4 出队清槽：为了 GC，但判断条件不完整

如果出队不清槽，被取走的元素仍然被数组引用着，GC 认为它还活着。对 EventSystem 来说，队列里存的是闭包委托，闭包又引用着事件对象和多播委托（进而引用所有监听者）——不清槽的话，这些对象要等到这一格被下一轮覆盖才释放。作者在这里是清楚这个问题的（对比第 10 章 E13：TimerSystem 停止时没有清尾部槽位）。

但判断条件 `!typeof(T).IsValueType` 漏了一种情况：**T 是含引用字段的结构体**。比如第 7 章提到的低 GC 方案 `struct EventEnvelope { int typeId; object payload; }`，IsValueType 为 true，不清槽，payload 被钉住。【实测 G5】按源码逻辑，出队并丢弃后 payload 仍然存活；改成“含引用就清槽”后被释放。

- 【改进方案】判断改为“是否是引用或含引用”：.NET 提供 `RuntimeHelpers.IsReferenceOrContainsReferences<T>()`（Unity 2021 起的 .NET Standard 2.1 配置可用，使用前请在目标版本确认）；或者干脆无条件清槽——对纯值类型写一次 default 的成本可以忽略。
- 【扩展知识】.NET Core 的 `Queue<T>` 源码就是用这个判断决定出队是否清槽的；Mono 沿用的 .NET Framework 参考源码则是无条件清槽。

C++ 视角：同一个问题叫“析构时机”

C++ 的环形队列如果存的是 `std::shared_ptr` 或带资源的对象，出队时不 reset / 不调用析构，资源同样不会释放。`std::deque` 的 pop_front 会析构元素；自己用裸数组 + placement new 实现时，要在出队时手动调用析构函数。“容器负责元素的生命周期”在两种语言里是同一件事。

8.5 取模 vs 位与：为什么容量常取 2 的幂

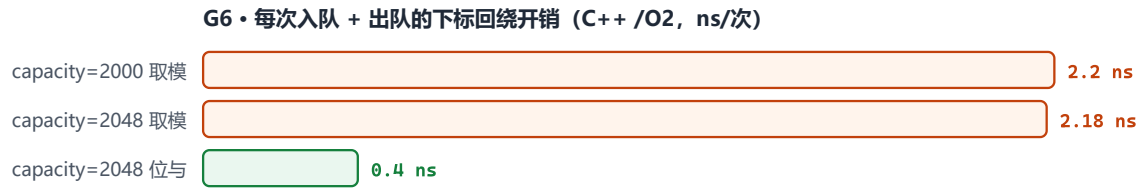


图 8-1 实测 G6（demos/infra_output.txt）。容量是运行期变量时，取模要用整数除法指令；容量是 2 的幂时可以用 `& (cap-1)` 代替，约快 5 倍。但这是每次 2 ns 和 0.4 ns 的差别——EventSystem 每帧几十个事件，完全无关紧要

- 整数除法在 x86 上要几十个周期，位与只要 1 个周期。编译器只有在除数是编译期常量时才能自动把 % 换成乘法 / 位运算。
- 源码的 2000 不是 2 的幂；改成 2048 能用位与，但前提是用位与重写下标计算，单改数字没用（实测：2048 取模仍是 2.18 ns）。
- 2 的幂的另一个好处是配合“单调递增计数”（8.3），计数器溢出回绕时 `& mask` 仍然正确。

面试注意：说“我把取模换成位与做了优化”之前

先说清楚规模：这个队列的瓶颈从来不是下标计算，而是每个元素的闭包分配和回调执行。面试官更想听到“我知道这个技巧，也知道它在这里不值得”。

8.6 和其他队列比

	RingBuffer（本项目）	`Queue<T>`	链表队列 / `LinkedList<T>`
底层	固定数组	可扩容的环形数组	节点 + 指针
入队 / 出队	O(1)	O(1) 均摊（扩容时 O(n)）	O(1)
分配	只在构造时	扩容时	每个节点一次
满了	拒绝（返回 false）	扩容	无上限
cache	连续	连续	节点散落，pointer chasing
清槽	只对引用类型（有漏洞）	引用或含引用（.NET Core）/ 无条件（.NET Framework、Mono）	节点整体被回收
迭代 / 调试	无迭代器	有	有

坦白说，`Queue<T>(2000)` 在不超过容量时和这个 RingBuffer 行为几乎一样，而且清槽更正确。自己写的价值在于“容量固定、满了显式拒绝”这一语义，以及作为 ThreadSafeRingBuffer 和无锁版本的起点——前提是满了要报出来（第 7 章 7.7）。

8.7 Code Review 与测试

级别	问题	说明
P2	含引用的结构体出队不清槽	G5；改用“是否含引用”的判断或无条件清槽
P2	每次出队都执行 `typeof(T).IsValueType`	泛型按值类型特化时 JIT 通常能常量折叠；引用类型共享代码（Mono / IL2CPP 的泛型共享）时可能是运行期调用。可缓存为静态只读字段
P3	不在命名空间	全局命名空间里的 `RingBuffer<T>` 容易和第三方库冲突
P3	没有 Clear / 迭代 / 容量查询	切场景时无法统一清空并释放引用
测试	**没有直接测试**	`RingBufferTests.cs` 的类名叫 RingBufferTest，测的却是 ThreadSafeRingBuffer，而且 T 都是 int——清槽逻辑根本不会执行；EventSystem 真正用的这个类零覆盖

应补的用例：引用类型出队后被 GC 回收（用 WeakReference 验证）；含引用结构体同上；容量 1 的边界；连续回绕多圈后顺序正确；满时 TryEnqueue 返回 false 且不改变状态；构造容量 ≤ 0 抛异常。

8.8 面试追问树

追问树 · RingBuffer

```
Q8-1 手写一个环形队列
 ├── Q8-2 满和空怎么区分？
 │    └── Q8-3 为什么无锁队列不用 size？                ← 引到第 12 章
 ├── Q8-4 为什么出队要把槽位清成 default？              ← GC
 │    └── Q8-5 值类型就不用清了吗？                     ← G5 反例
 ├── Q8-6 % 和 & 差多少？为什么容量常取 2 的幂？
 └── Q8-7 满了为什么不覆盖最老的？为什么不扩容？
Q8-8 为什么不直接用 Queue<T>？
Q8-9 环形缓冲区在游戏里还用在哪？                      ← 发散
```

Q8-1 → Q8-3　手写环形队列；满和空怎么区分？

L1 • 30 秒　固定数组加 head、tail 两个下标，入队写 tail 再前进，出队读 head 再前进，到末尾取模回绕。满和空时 head 都等于 tail，我用一个 size 计数区分。

L2 • 正常回答　另外两种做法是浪费一个槽（tail 的下一个是 head 就算满），或者 head、tail 只增不回绕、用位与取下标、用差值算元素个数，后者要求容量是 2 的幂。

L3 • 深入　单线程下 size 最简单；多线程下 size 被生产者和消费者同时修改，必须加锁或原子操作。单调递增计数让 head 只归消费者写、tail 只归生产者写，单生产者单消费者时可以不加锁，只用 acquire/release 内存序保证可见性——我实测过，同样传 400 万个数，加锁版约 168 ms，无锁 SPSC 约 18 ms。

L4 • 面试官攻击　“单调递增的计数器溢出了怎么办？” → 用无符号整数，溢出按 2^{64}（或 2^{32}）取模回绕；只要容量是 2 的幂、且元素数不超过容量，`tail - head` 的无符号差和 `& mask` 取下标在回绕后都仍然正确。

Q8-4 / Q8-5　为什么要清槽？值类型就不用了吗？

L2　出队后数组还引用着这个元素，GC 认为它活着，要等这一格被覆盖才释放。EventSystem 队列里放的是闭包，闭包引用事件对象和监听者，不清槽就会延长它们的寿命。

L3　源码只对引用类型清槽，这有漏洞：结构体里有引用字段时 IsValueType 是 true，但引用照样被钉住，我用移植版验证过。应该判断“是否是引用或含引用”，.NET Core 的 Queue 就是这么做的；或者像 .NET Framework 的 Queue 那样无条件清。

Q8-7　满了为什么不覆盖最老的？为什么不扩容？

L2　覆盖最老的适合日志、录像、网络抖动缓冲这类“只要最近 N 个”的场景；扩容适合不能丢的。事件队列里两种事件都有，所以单一策略都不对。我选择“满了拒绝”，本意是让异常的事件量暴露出来，但 Trigger 没处理返回值，暴露这一步没做——这是第 7 章的 P0。

Q8-9　环形缓冲区在游戏里还用在哪？（发散）

L2　很多地方：网络的收发缓冲和 jitter buffer；音频混音缓冲区（生产者是解码线程、消费者是声卡回调，典型 SPSC）；输入历史（格斗游戏的指令缓冲、帧同步回滚时保留最近 N 帧状态）；

帧时间统计（最近 120 帧的平均 FPS）；GPU 每帧的动态常量缓冲（按帧轮转，避免 CPU 改 GPU 正在读的那块）；日志的最近 N 条，崩溃时一起上报。共同点是“只关心最近一段”或“一个写一个读”。

常见错误回答

错误：“环形队列 head==tail 就是空。”

问题：满的时候 head 也等于 tail（G4）。必须说出用什么额外信息区分。

8.9 发散：面试官会从这里追到的知识

Linux kfifo

容量 2 的幂、in/out 两个无符号计数只增不减、单生产者单消费者无锁，配合内存屏障——是 8.3 第三种方案的工业实现。

LMAX Disruptor

预分配的环形数组 + 序列号 + 缓存行填充，避免 false sharing 和分配；多消费者按依赖关系排队。第 12 章 G8 的 false sharing 就是它要解决的问题之一。

std::deque 与 std::queue

deque 是分段数组（块 + 中控数组），不是环形数组；std::queue 是适配器，默认底层 deque。追问“deque 头部插入为什么是 O(1)”时会用到。

GPU 的 ring buffer

命令缓冲、上传堆按帧轮转；用 fence 判断 GPU 是否已经消费完某一段，没消费完 CPU 就等——本质是“满了怎么办”的另一种答案：阻塞生产者。

本章速记

- head 出、tail 入、取模回绕；**满和空都是 head==tail**，项目用 size 区分（G4）；另两法：浪费一槽、单调递增计数 + 2 的幂。
- size 是两端共写的变量 → 多线程只能加锁；单调计数让两端各写各的 → SPSC 可无锁。
- 出队清槽是为了 GC；**只判 IsValueType 会漏掉“含引用的结构体”**（G5）。
- 取模 2.2 ns vs 位与 0.4 ns（G6），**只是数量级上快，在这里无意义。**
- RingBufferTests 测的是 ThreadSafe 版、T=int，**EventSystem 真正用的这个类零覆盖。**

▶ **自测 1：capacity=4，依次入队 a b c，出队 2 次，再入队 d e f，写出 head、tail、size 和数组内容。**

▶ **自测 2：把容量从 2000 改成 2048，下标计算就自动变快了吗?**

CHAPTER 09

TimerSystem（上）：Slot Map 与 generation 句柄

从“为什么 List<Timer> 不够”开始，一步步推出 dense 数组、稀疏映射、swap-and-pop、free list 和 generation；再用实测找出这套句柄的四个漏洞。

本章目标

- 不看代码画出 5 个数组的关系，并手推“删除中间元素”后每个数组的变化
- 用具体例子讲清 stale handle、ABA 式槽位复用，以及为什么只有 index 不够
- 知道当前实现的四个句柄漏洞：默认句柄、残留映射/伪造句柄、跨实例、代数溢出
- 能用实测数据回答“数组一定比链表快吗”“swap-and-pop 比 RemoveAt 快多少”
- 能对比 UE 的 FTimerManager / FTimerHandle，并说出 production 版本的设计

9.1 这一章解决什么问题

Gameplay 里到处是“过一会儿再做”：技能冷却、Buff 持续时间、刷怪间隔、UI 倒计时、延迟销毁。一个计时器系统至少要满足五个要求：

#	要求	为什么
R1	创建、删除都是 O(1)	技能和 Buff 会在一帧里大量增删
R2	每帧遍历要快	所有计时器每帧都要累加时间
R3	外部持有一个“票据”，可以随时暂停、查询、停止	UI 要查剩余时间，Pawn 死亡要取消自己的计时器
R4	票据过期后不能误操作别人的计时器	计时器结束后，持有方往往还留着那张票据
R5	回调里可以安全地增删计时器	回调里停止自己、开启下一个计时器非常常见（第 10 章）

9.2 从 List<Timer> 出发：朴素方案为什么不够

方案	删除	遍历	外部票据	问题
`List<TimerData>` + 下标当票据	RemoveAt O(n)	连续，快	下标	删一个元素，后面所有元素的下标都变了，票据全部失效
`List<Timer>`（class）+ 对象引用当票据	Remove O(n)	指针数组，元素散在堆上	引用	每个计时器一次堆分配；删除后持有方的引用依然“能用”，只能靠标志位判断；一旦为了省 GC 把 Timer 对象池化，旧引用就会指向别人的计时器（ABA）
`Dictionary<int, TimerData>` + 自增 id	O(1) 均摊	遍历哈希桶，且 struct 值不能原地修改	id	每帧修改 elapsed 都要“读出—改—写回”；遍历中不能修改字典
Slot Map（本项目）	O(1)	dense 连续数组 + `ref` 原地修改	(index, generation)	需要维护三份映射；删除打乱顺序

Slot Map 的核心思路是**一次间接寻址**：外部拿到的不是数据的位置，而是一个“槽位号”；槽位号通过一张表找到数据当前在哪。数据可以随便搬家（为了保持紧凑），只要更新这张表即可。

Handle → 稀疏映射 → 紧凑数组

```
TimerHandle(Index=1, Generation=0)          ← 外部持有，永远不变
      │  _generations[1] == 0 ?              ← 版本校验：不相等就是过期票据
      ▼
_handleToIndex[1] = 3                       ← 稀疏映射：槽位 1 的数据现在在 dense[3]
      │
      ▼
_timers[3] = { duration, timeElapsed, ... }  ← 紧凑数组：每帧从 0 线性遍历到 _timerCount-1
      ▲
_indexToHandle[3] = 1                       ← 反向映射：搬家时用来找到“它属于哪个槽位”
```

9.3 当前项目怎么做：5 个数组 + 3 个辅助结构

TimerSystem.cs:14-35 / 44-63 / 80-105（节选）

```
public readonly struct TimerHandle : IEquatable<TimerHandle> {
    public readonly int Index;       // 槽位号（不是 dense 下标！）
    public readonly int Generation;  // 槽位的代数
    public static readonly TimerHandle Invalid = new TimerHandle(-1, -1);
}
public struct TimerData {
    public float duration, timeElapsed, interval, intervalElapsed;
    public Action onTrigger, onComplete;              // ← 两个引用类型字段
    public bool isLoop, isPaused, isUnScaled, isChase;
    public int timerId;                               // 名字的 hash，0 = 匿名
}
private TimerData[] _timers = new TimerData[128];     // ① dense：热数据
private int _timerCount = 0;
private readonly List<int> _handleToIndex;            // ② 槽位 → dense 下标
private readonly List<int> _indexToHandle;            // ③ dense 下标 → 槽位
private readonly List<int> _generations;              // ④ 槽位 → 当前代数
private readonly Queue<int> _freeIndices;             // ⑤ 空闲槽位（FIFO）
private readonly Dictionary<int, TimerHandle> _idToHandle;   // 名字 hash → 句柄
private readonly List<TimerHandle> _expiredTimersCache;      // 本帧到期
private readonly List<TimerHandle> _intervalTimersCache;     // 本帧 interval 触发
```



图 9-1　Slot Map 的结构：外部句柄 → 比对代数 → 稀疏映射拿到 dense 下标 → 访问紧凑数组。橙色是访问 Handle(1,0) 时经过的格子

为什么 dense 用数组 TimerData[] 而不是 List<TimerData>

【源码事实】Tick 里写的是 `ref TimerData timer = ref _timers[i];`，直接改数组里的那一份。如果换成 `List<TimerData>`，索引器返回的是**副本**，`list[i].timeElapsed += dt` 连编译都过不了（修改的是临时值）；只能“读出、改、写回”，多两次 40 字节的拷贝。Unity 2022 的 .NET Standard 2.1 里也没有 `CollectionsMarshal.AsSpan` 可以拿到 List 内部数组。这是“用数组”最实在的理由，比“数组更快”更站得住。

【问题】同一个文件里 `SetPaused` 却用了“读出—改—写回”，风格不统一（功能没错）。

9.4 CreateTimer：逐行看分配一个计时器

TimerSystem.cs:156-223（节选）

```
if (!string.IsNullOrEmpty(timerName)) {
    timerId = timerName.GetHashCode();                          // ① 名字 → int hash
    if (_idToHandle.TryGetValue(timerId, out var oldHandle)) {
        if (IsHandleValid(oldHandle)) StopTimer(oldHandle);  // ② 同名覆盖：停掉旧的
        _idToHandle.Remove(timerId);
    }
}
int handleIndex;
if (_freeIndices.Count > 0) handleIndex = _freeIndices.Dequeue();    // ③ 优先复用空闲槽
else {
    handleIndex = _handleToIndex.Count;                         //     否则新开一个槽
    _handleToIndex.Add(-1); _indexToHandle.Add(-1); _generations.Add(0);  // ← 代数从 0 开始
}
TimerHandle handle = new TimerHandle(handleIndex, _generations[handleIndex]); // ④ 代数在创建时
EnsureCapacity(_timerCount + 1);                                // ⑤ 不够就翻倍 Array.Resize
_timers[_timerCount] = data; int realIndex = _timerCount; _timerCount++;     // ⑥ 追加到 dense
_handleToIndex[handleIndex] = realIndex;                        // ⑦ 建立双向映射
if (realIndex ≥ _indexToHandle.Count) _indexToHandle.Add(handleIndex);
else _indexToHandle[realIndex] = handleIndex;
if (timerId ≠ 0) _idToHandle[timerId] = handle;                 // ⑧ 登记名字
```

- ③④：代数在**停止时**加一（StopTimer 里），创建时只读取。所以一个从没被回收过的槽位，代数永远是初始值 0——这是 9.8 节默认句柄漏洞的根源。
- ⑦【源码事实】`_indexToHandle` 在新开槽位时就已经 `Add(-1)`，它的长度永远等于槽位数，而 `realIndex < _timerCount ≤ 槽位数`，所以 `Add` 分支实际上**不可达**。这是一个隐含不变式，写成断言比写成分支更清楚。
- ②【问题】同名覆盖是**静默**的：调用方不知道旧计时器被停了，也拿不到任何回调；加上 hash 碰撞（9.8.5），两个不同名字也可能互相覆盖。

9.5 StopTimer：Swap-and-Pop 的完整推导

TimerSystem.cs:243-283（节选）

```
if (!IsHandleValid(handle)) return;
int handleIndex = handle.Index;
int realIndex = _handleToIndex[handleIndex];              // 要删的数据在哪
int lastRealIndex = _timerCount - 1;
if (realIndex ≠ lastRealIndex) {
    _timers[realIndex] = _timers[lastRealIndex];          // ① 末尾数据填洞（40 字节结构体拷贝）
    int lastHandleIndex = _indexToHandle[lastRealIndex];  // ② 末尾数据属于哪个槽位
    _indexToHandle[realIndex] = lastHandleIndex;          // ③ 修反向映射
    _handleToIndex[lastHandleIndex] = realIndex;          // ④ 修正向映射
}
_timerCount--;                                            // ⑤ 逻辑删除末尾（数组不清空）
_generations[handleIndex]++;                              // ⑥ 旧句柄失效
_freeIndices.Enqueue(handleIndex);                        // ⑦ 槽位进入 free list
```

下面用构建脚本**按源码逐行复刻** CreateTimer/StopTimer，画出 [A][B][C][D] 删除 B、再新建 E、最后用 B 的旧句柄访问的真实状态：

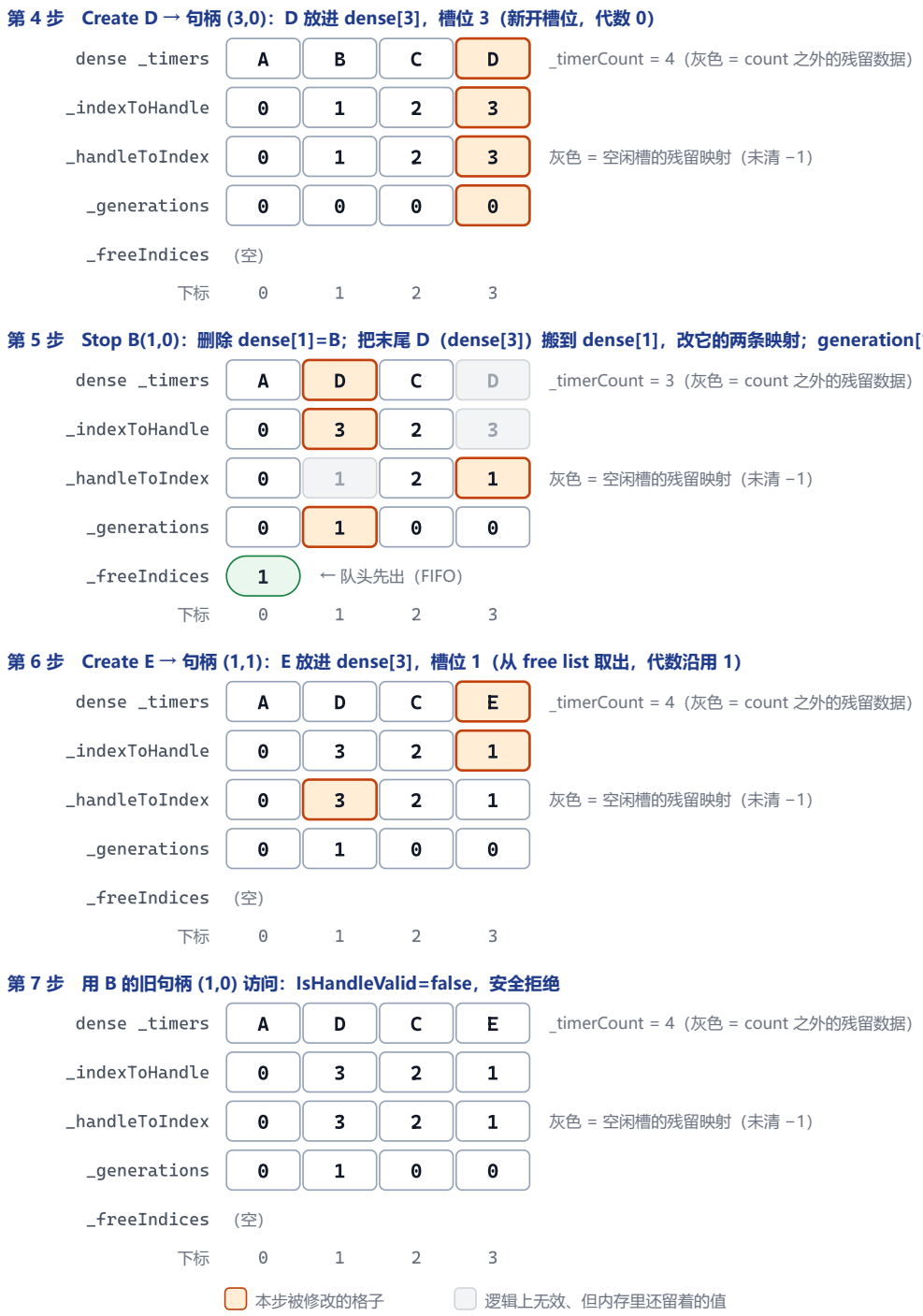


图 9-2　Swap-and-Pop + free list + generation 的完整过程（构建时由 Python 逐行复刻 TimerSystem.cs 计算）。注意第 5 步：E 复用了 B 的槽位 1，但代数已经是 1，B 的旧句柄 (1,0) 被拒绝

逐步解释第 5 步“Stop B(1,0)”：

1. `realIndex = _handleToIndex[1] = 1`，`last = 3`，不是末尾，需要搬家。
2. `_timers[1] = _timers[3]`：D 的数据被拷到 1 号位置。

3. `_indexToHandle[3] = 3`：D 属于槽位 3；于是 `_indexToHandle[1] = 3`、`_handleToIndex[3] = 1`。**D 的句柄 (3,0) 没有变，但它现在能找到 dense[1]。**
4. `_timerCount = 3`；`_generations[1] = 1`；槽位 1 入队。
5. 灰色格子：`_timers[3]` 还留着 D 的副本、`_handleToIndex[1]` 还留着 1——它们**逻辑上无效，但内存里还在**，9.8.2 和第 10 章的 GC 审计都会用到这两个“残留”。

对比	List.RemoveAt（保序）	Swap-and-Pop
时间复杂度	O(n − i)：后面的元素整体前移	O(1)：一次拷贝 + 两次映射修改
顺序	保持插入顺序	**打乱**：末尾元素跳到中间
映射维护	后面所有元素的 dense 下标都变了，要改 n − i 条映射	只改被搬动的那一个元素的两条映射
内存行为	memmove 一大段，大 n 时会扫过很多 cache line	只碰两个位置
什么时候选它	需要稳定的执行顺序（例如按创建顺序回调、帧同步要求确定顺序且直观）	顺序无所谓、删除频繁

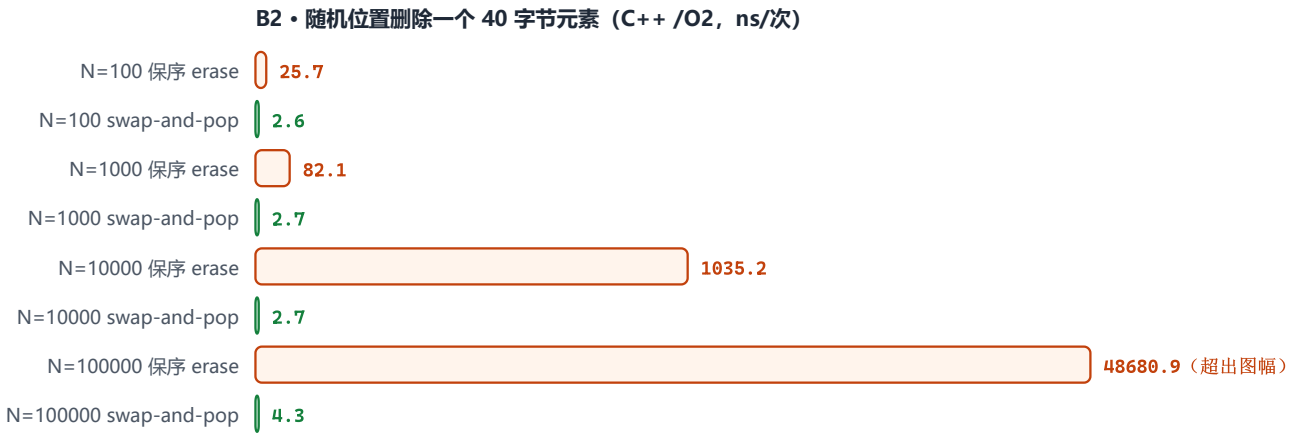


图 9-3 实测 B2（demos/bench_output.txt）。横轴截断在 2000 ns，N=100000 的保序删除远超图幅。客户端常见的几十到几百个计时器下，二者差距只有几十纳秒——**swap-and-pop 的价值在规模上去之后才明显**

面试注意：swap-and-pop 打乱顺序的后果

【实测 E4】按 1、2、3、0（重建）的顺序创建、同一帧到期的四个计时器，回调顺序是 `0 1 2 3`——和创建顺序不同，因为删除/重建把元素搬到了别的位置。如果业务依赖“先创建先回调”（例如两个 Buff 同帧到期，先移除哪个决定了最终属性），就必须显式排序（例如按 `(到期时间, 创建序号)`），不能依赖 dense 顺序。

9.6 Free List：为什么需要它，为什么是队列

没有 free list，每次 CreateTimer 都要新开槽位，`_generations` 等三张表只增不减，一个跑了一小时的战斗会积累成千上万个空槽。free list 让槽位被复用，三张稀疏表的长度等于“历史上同时存在的计时器数量峰值”。

它和 generation 是一对：**free list 负责复用，generation 负责让复用安全**。复用意味着同一个槽位号先后属于不同的计时器，必须有东西区分“这是第几任主人”。

空闲结构	复用顺序	对句柄安全的影响	本项目
Queue（FIFO）	最早释放的槽最先复用	同一槽位两次复用间隔更长；代数自增被分摊到更多槽位，单个槽位回绕更慢	✓
Stack（LIFO）	刚释放的槽立刻复用	热点槽位代数涨得快；但稀疏表访问更集中（cache 友好）	ObjectPool 用的是它
侵入式链表（槽位里存下一个空闲槽号）	任意	不需要额外容器，零分配	常见于 C++ 的 slot map

【实测 E5】FIFO 让测试用例变得“依赖执行顺序”

仓库里的 `Test_GenerationIncrement` 和 `Test_IndexReuse` 断言“停止后再创建，会拿到同一个槽位、代数 +1”。这只在 free list 里**恰好没有其他空闲槽**时成立：预留 0 或 1 个空闲槽时两个用例 PASS，预留 2 个时**都 FAIL**（新计时器拿到的是更早释放的槽）。它们在当前测试顺序下能过，是因为前面的用例恰好没有留下多余的空闲槽——这是“测试依赖全局状态”的典型反面教材（第 20 章）。

9.7 Index + Generation：它到底解决了什么

9.7.1 用 (5,3) → (5,4) 讲清 stale handle

一个槽位的三任主人

```
时刻 t0  Timer A 创建      → 拿到槽位 5，_generations[5] = 3 → 句柄 Handle(5, 3)
         Pawn 把 Handle(5,3) 存进字段 _decayTimer
时刻 t1  A 到期 / 被停止    → _generations[5]++ → 4，槽位 5 进 free list
         Pawn 不知道，字段里还是 Handle(5,3)          ← 这就是 stale handle（过期句柄）
时刻 t2  Timer B 创建      → 从 free list 取到槽位 5 → 句柄 Handle(5, 4)
时刻 t3  Pawn 死亡，调用 StopTimer(Handle(5,3))
         IsHandleValid: _generations[5] = 4 ≠ 3 → false → 直接 return，B 安然无恙
如果没有 generation（句柄只有 Index=5）：
         StopTimer(5) → 找到槽位 5 → 删掉的是 B ！ —— A 的主人误杀了 B 的计时器
```

这类问题叫做**ABA 式槽位复用**：持有方看到的“槽位 5”从 A 变成了 B（中间经历了“空”），但只看槽位号无法察觉变化。名字借自无锁编程里的 ABA 问题：CAS 只比较值，值从 A 变 B 再变回 A，CAS 以为什么都没发生。两者的共同解法都是**给值附加一个只增不减的版本号。**

只用这个当票据	为什么不够
dense 下标	swap-and-pop 一次，别人的 dense 下标就变了
槽位号 Index	槽位会被复用，旧票据会指向新主人（上面的例子）
对象引用（C#）	不会悬垂（GC 保证对象还活着），但会“逻辑过期”：对象已被移出系统，持有方仍可调用它；一旦为了省 GC 池化对象，就退化成槽位复用问题
原生指针（C++）	对象释放后成为悬垂指针（use-after-free）；内存被重新分配给同类对象时还是 ABA
Index + Generation	Index 定位，Generation 确认“还是不是同一任主人”；代价是每次访问多一次比较

9.7.2 IsHandleValid 只做了两件事

TimerSystem.cs:349-355

```
public bool IsHandleValid(TimerHandle handle) {
    if (handle.Index < 0 || handle.Index ≥ _generations.Count) return false;  // 越界
    return _generations[handle.Index] == handle.Generation;                   // 代数相等
}
```

注意它**没有**检查“这个槽位现在是不是正在被使用”。正常情况下这两件事等价：槽位被释放时代数已经 +1，旧句柄的代数必然对不上。但只要能构造出“空闲槽位的当前代数”，校验就会通过——下一节的漏洞 2 就是这个。

9.8 这套句柄的四个漏洞（实测）

9.8.1 漏洞 1：default(TimerHandle) = (0,0) 是合法句柄

C# 的结构体字段默认值是全零。`CombatPawn` 里写的是 `private TimerHandle _decayTimer;`，没有初始化为 `Invalid`。只有被**玩家**附身时才会创建寿命计时器，所以 **AI 肉体的 _decayTimer 一直是 (0,0)**。而代数从 0 开始，第一个被创建的计时器恰好也是 (0,0)。



图 9-4　SoulDecay 开局顺序：先为玩家肉体创建寿命计时器 P=(0,0)，再创建刷怪器 S=(1,0)。任意一只 AI 肉体死亡时调用 StopTimer(default)，删掉的是玩家的寿命计时器——UI 显示 DIE，而玩家永远不会因寿命耗尽而死（实测 E1-A）

【实测 E1】三种情况：

- **A（当前 Demo 顺序）**：玩家寿命计时器 (0,0) 被 AI 的死亡停掉 → UI 显示 DIE，玩家永不爆炸。
- **B（如果先创建刷怪器）**：刷怪器变成 (0,0)，任意 AI 死亡后刷怪停止。
- **C（槽位 0 被复用过一次之后）**：代数已经变成 1，(0,0) 永久失效——**这就是当前 Demo 能侥幸跑通的原因**：玩家第一具肉体寿命 5 秒，第一只 AI 3 秒出现、第二只 6 秒出现，5 秒内不可能发生 AI 之间的碰撞；而玩家一旦夺舍或爆炸，槽位 0 就被回收过了。



图 9-5　最小修复：让代数从 1 开始（构建脚本把初始代数改为 1 重新计算）。此时第一个计时器是 (0,1)，default (0,0) 永远不合法——这也是 UE 的做法：FTimerHandle 的 Handle 为 0 表示“从未被初始化”

【改进方案】两层防线

① 系统侧：代数从 1 开始，让“全零”永远无效（也可以把 Index 存成 index+1）。② 使用侧：句柄字段一律显式初始化为 `TimerHandle.Invalid`，停止后立刻重置（CombatPawn.OnUnPossess 里已经这样做了，但声明处漏了）。只改使用侧不够——任何一个新写的类忘了初始化，问题就会回来。

9.8.2 漏洞 2：残留映射 + 不检查槽位在用 → 伪造句柄能删掉别人

StopTimer 之后，`_handleToIndex[槽位]` 没有被重置为 −1（图 9-2 的灰色格子）。如果有人构造出 (空闲槽位, 当前代数) 的句柄，IsHandleValid 会通过，StopTimer 会顺着残留映射去删一个**不属于它**的 dense 元素：



图 9-6　实测 E2 的逐步状态：第 5 步伪造句柄 (1,1) 通过校验，按残留映射删掉了 dense[1]（C 的数据），C 的句柄 (2,0) 仍“有效”却指向 count 之外，成了僵尸；同时槽位 1 被第二次压入 free list。第 6 步新建 D 落在 dense[1]，此后用 C 的句柄查询剩余时间读到的是 D 的数据；free list 里还剩一个重复的 1，下一次 Create 会再次拿到槽位 1——两个活着的计时器共用一个槽位

正常使用拿不到这样的句柄，所以这是一个**“防御深度”问题**而不是日常 bug。但它会在两种情况下自然出现：代数回绕（9.8.4）之后，以及跨实例使用句柄（9.8.3）时。面试里说清“IsHandleValid 只比代数，不检查槽位是否在用，停止后也不清映射”，再给出修复——停止时把映射置 −1，校验时额外检查 `h2i[index] ≥ 0 && h2i[index] < count`——就足够了。

9.8.3 漏洞 3：句柄不知道自己属于哪个 TimerSystem

TimerSystem 是场景级的懒单例，切场景时会被销毁、在新场景里重新创建，新实例的槽位和代数又从 0 开始。【实测 E10】旧场景里保存下来的句柄 (0,0)，在新场景的 TimerSystem 上 `IsHandleValid =`

`true`，读到的是新场景第一个计时器的剩余时间；用它 StopTimer 会把新计时器停掉。谁会跨场景持有句柄？挂了 DontDestroyOnLoad 的对象——比如以后真正有职责的 GameInstance。

【改进方案】句柄里带一个“系统实例 id”（或者每个实例的代数起点随机化 / 全局递增），校验时一起比较。UE 的做法是序列号由 TimerManager 全局递增分配（`LastAssignedSerialNumber`），不同槽位之间也不会重号。

9.8.4 漏洞 4：generation 溢出

C# 的 `int++` 默认是 unchecked，到 `int.MaxValue` 之后回绕成 `int.MinValue`【实测 E6：-2147483648】。同一个槽位要复用 2^{32} 次，代数才会回到原值：

同一槽位每秒复用	回到同一代数所需时间（实测 E6）	判断
1 次	约 4.97 万天（136 年）	可忽略
60 次（每帧一次）	约 829 天	客户端可忽略；7×24 服务器需要考虑
1000 次	约 49.7 天	需要处理

而且回绕后要真出事，还需要“某个持有方恰好在 2^{32} 次复用之前拿了句柄、一直没用、刚好在回绕那一刻使用”。所以正确的面试回答是：**先估算，再说明风险窗口，最后给方案**——64 位代数；或者像 UE 那样 64 位句柄拆成 24 位下标 + 40 位序列号（2^{40} 次分配才回绕）；或者代数到达上限的槽位直接“退役”不再复用。

9.8.5 附：名字 → string.GetHashCode() 的问题

- **碰撞**：32 位 hash 必然有碰撞。两个不同名字 hash 相同时，第二次 CreateTimer 会把第一个“同名覆盖”掉，`StopTimer("X")` 可能停掉 Y。
- **不稳定**：.NET 文档明确说字符串 hash 不保证跨进程、跨运行时版本一致（.NET Core 默认按进程随机化）；Mono / IL2CPP 下碰巧稳定，也不能拿去持久化或网络同步。
- **0 的歧义**：`timerId == 0` 表示匿名，若某个名字的 hash 恰好是 0，它就无法按名查找。
- **注释不符**：注释说“使用 Unity 内置的高效 Hash”，代码用的是 `string.GetHashCode()`。

【改进方案】`Dictionary<string, TimerHandle>` 直接以字符串为键（字典内部会处理碰撞）；或者把名字收敛成编译期常量 / 枚举 / 预先注册的 id 表。

9.9 Dense Array 与 cache：什么时候“数组更快”才成立

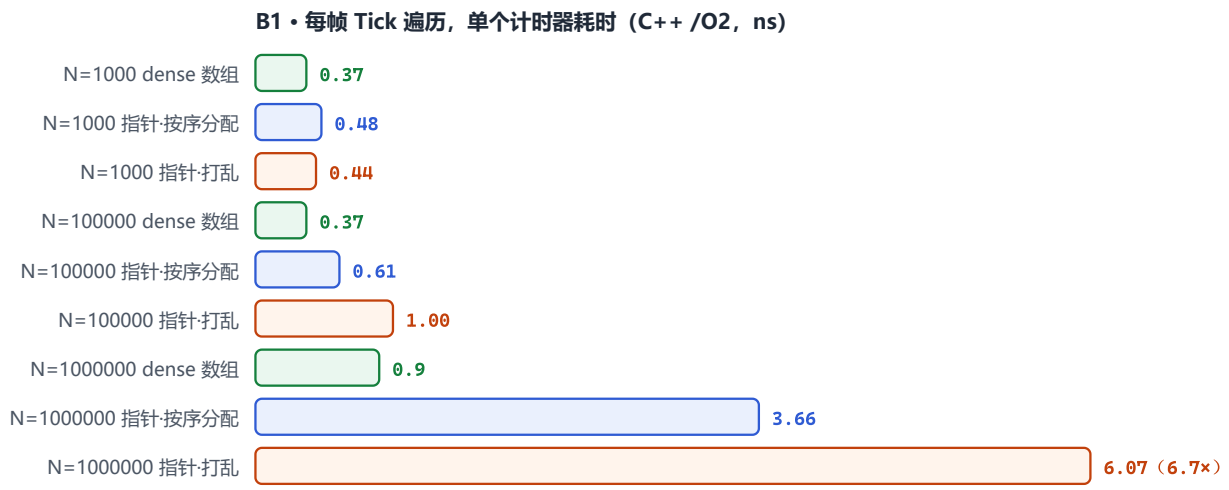


图 9-7 实测 B1（demos/bench_output.txt）：结构体 40 字节，与 C# TimerData 字段一致。1000 个计时器（39 KB，装得进 L2）时三种布局几乎没差别；10 万个（3.9 MB）开始拉开；100 万个（39 MB，超出缓存）时打乱的指针数组慢 6.7 倍

所以“数组一定比链表快”是错的说法。正确的说法有三个前提：

1. **访问模式是顺序遍历**：Tick 从 0 扫到 count−1，硬件预取器能提前把下一批 cache line 拉进来。随机按句柄访问单个计时器时，dense 数组和指针数组都是一次（或两次）随机访问，差别不大。
2. **数据量超过缓存**：数据全部在 L1/L2 时，指针追逐（pointer chasing）的代价被缓存掩盖了。几百个计时器的客户端场景，布局带来的差异是纳秒级的。
3. **对象确实是散的**：C# 的 class 对象分配在托管堆上，刚分配时往往是连续的（“按序”那一列）；长时间运行、分配释放交错后才会散开。Unity 用的 Boehm GC **不压缩（non-moving）**，散开之后不会被整理回来——这是“托管语言里也要关心数据布局”的理由。

更进一步：热冷分离

每帧 Tick 真正要读写的只有 `timeElapsed / duration / isPaused / interval ...`，两个委托引用只在触发时才用到，却占了 40 字节里的 16 字节。把回调挪到另一个按 dense 下标对齐的数组（Structure of Arrays 的思路），Tick 每个 cache line 能装下更多计时器；顺带让热数据数组不含引用，GC 标记时可以跳过（第 10 章）。

9.10 交互实验：自己操作一次 Slot Map

建议操作：新建 A、B、C、D → 选中 B 停止（看 swap-and-pop）→ 再新建 E（看槽位复用、代数 +1）→ 用 B 的旧句柄再停一次（被拒绝）→ 点“StopTimer(default)”（删掉了 A！）→ 勾选“generation 从 1 开始”后重复一遍。

交互实验 • Slot Map：新建 / 停止 / 旧句柄 / 默认句柄

下标	0	1	2	3
dense _timers	A	B	C	D
_indexToHandle	0	1	2	3
_handleToIndex	0	1	2	3
_generations	0	0	0	0
_freeIndices	（空）			

已发出的句柄：`A(0,0) → dense[0]=A` `B(1,0) → dense[1]=B` `C(2,0) → dense[2]=C` `D(3,0) → dense[3]=D`

已预先创建 A、B、C、D。default(0,0) 当前有效 ⚠（会命中 A）。试试：选中 B 停止 → 新建 → 再用 B 的旧句柄停止 → StopTimer(default)

9.11 替代方案：为什么不是堆、时间轮或协程

方案	插入	删除	每帧 Tick	适合
Slot Map + 线性扫描（本项目）	O(1)	O(1)	O(N)：所有计时器都累加一次	计时器数量中等（几百～几千）、需要逐帧累加 elapsed、需要暂停/scaled 混用
小顶堆（按到期时间）	O(log N)	O(log N)（需要句柄→堆位置的映射）	O(K log N)：只处理到期的 K 个	大量长计时器，大多数帧没有到期；UE FTimerManager 就是这种
（分层）时间轮	O(1)	O(1)	O(当前格子里的计时器)	海量、精度固定（按 tick 粒度）的计时器：Linux 内核定时器、Netty HashedWheelTimer、服务器
协程 `WaitForSeconds`	分配 IEnumerator 对象	StopCoroutine	由 Unity 调度	写起来简单；但绑定 MonoBehaviour 生命周期（物体失活即停止），没有统一暂停/查询剩余时间
`Invoke` / `InvokeRepeating`	按方法名字符串	CancelInvoke	由 Unity 调度	原型；没有句柄、不能传参、拼写错误到运行时才暴露

线性扫描的另一个好处常被忽略：**scaled / unscaled、暂停这些“时间流速不同”的计时器可以放在一起。**堆按绝对到期时间排序，一旦某个计时器暂停或时间缩放改变，它的到期时间就要重算并调整堆。UE 的做法是把暂停的计时器移到单独的 `PausedTimerSet`。

9.12 与 Unreal 的 FTimerManager / FTimerHandle 对比

	本项目 TimerSystem	UE FTimerManager【扩展知识】
数据存储	dense 数组 + 三张稀疏表	`TSparseArray<FTimerData> Timers`（带空洞的数组 + 空闲链表）
调度	每帧线性扫描所有计时器	`ActiveTimerHeap`：按 ExpireTime 排序的堆，只弹出到期的；另有 `PendingTimerSet`（本帧新加的）、`PausedTimerSet`
句柄	(int Index, int Generation)，代数从 0 开始	`uint64 Handle` = 24 位下标 + 40 位序列号；序列号由管理器全局递增；Handle == 0 表示无效
时间	每个计时器累加 float elapsed	管理器维护 double `InternalTime`，计时器存绝对到期时间，不累加误差
循环计时器掉帧	每帧最多补 1 次（第 10 章）	默认会在一帧里连续补调多次；UE 5.4 起可设 `bMaxOncePerFrame`
回调对象销毁	不感知，回调照样调用	委托绑定 UObject 时，对象失效后计时器不会再执行；还有 `ClearAllTimersForObject`

对比的落点：**思想一致（间接句柄 + 版本号），工程取舍不同。**UE 需要支撑成千上万的计时器和长时间运行的服务器，所以用堆 + 64 位句柄 + 绝对时间；本项目面向单机 Demo，线性扫描简单直观。

9.13 Production 版本会怎么设计（只讲思路）

1. **句柄**：64 位，包含“系统实例 id + 槽位 + 代数”，代数从 1 开始；停止时清空映射；校验时同时检查“槽位在用”。
2. **名字**：用 `Dictionary<string, Handle>` 或预注册 id，绝不用 hash 当唯一键；同名覆盖要么报错、要么返回旧句柄让调用方决定。
3. **存储**：热冷分离；停止时清掉回调引用（第 10 章 E13）。
4. **调度**：少量计时器线性扫描；数量大时按“到期时间”建堆或用时间轮；暂停的计时器移出活动集合。
5. **所有者**：创建时登记 owner，owner 销毁时批量取消（对应 UE 的 ClearAllTimersForObject），避免回调到已销毁的 MonoBehaviour。

6. **时机**：由 World / GameLoop 显式调用 Tick，固定在 Actor Tick 之前或之后，而不是自己 Update。

9.14 面试追问树

追问树 · TimerSystem 数据结构（模拟真实面试的连续追问）

```
Q9-1 计时器为什么不用 List<Timer>?
 └── Q9-2 删除复杂度是多少?
      └── Q9-3 你说 O(1)，但 TimerHandle 怎么找到实际元素?
           └── Q9-4 Swap-and-Pop 以后，被搬走那个元素的 Handle 不就失效了吗?
                └── Q9-5 那你怎么维护映射?
                     └── Q9-6 slot 被新 Timer 复用怎么办?
                          ├── Q9-7 为什么 generation 能解决？这是不是 ABA?
                          │    └── Q9-8 generation 溢出怎么办?
                          └── Q9-9 default(TimerHandle) 是合法句柄吗?          ← 压力题：项目里的真
                               └── Q9-10 还有别的方式能骗过 IsHandleValid 吗?
Q9-11 数组一定比链表快吗?                                                      ← 性能题
 └── Q9-12 那你为什么不用 Dictionary<int, Timer>?                              ← 替代设计
Q9-13 为什么不用小顶堆？UE 用的是什么?                                         ← 替代设计 / UE 对比
Q9-14 string.GetHashCode 能当唯一 ID 吗?                                       ← 质疑
```

Q9-1 → Q9-5　为什么不用 List<Timer>？句柄怎么找到元素？搬家后怎么维护？

L1 • 30 秒　List 删除中间元素是 O(n)，而且下标会变，没法当票据。我用 Slot Map：外部拿“槽位号 + 代数”，槽位号经稀疏映射找到 dense 数组位置；删除时把末尾元素搬进空洞，只改被搬那个元素的两条映射，整体 O(1)。

L2 • 正常回答　有三张表：`_handleToIndex`（槽位 → dense 下标）、`_indexToHandle`（dense 下标 → 槽位）、`_generations`（槽位 → 代数）。删除 B 时：用 B 的槽位查到它在 dense[1]；把 dense[3] 的 D 拷过来；用反向表查到 D 属于槽位 3，于是把 `_handleToIndex[3]` 改成 1、`_indexToHandle[1]` 改成 3；count 减一。D 的句柄 (3,0) 没变，但现在指向 dense[1]——外部完全无感。

L3 • 深入　反向表是 swap-and-pop 必需的：没有它，搬动末尾元素时不知道该改哪个槽位的映射，只能 O(n) 搜索。维护的不变式是 `h2i[i2h[k]] == k`（k 在 count 内）。代价是删除会打乱顺序，同帧回调顺序不再等于创建顺序（实测 E4）。

L4 • 面试官攻击　“你这三张表都是 List<int>，每次访问都有边界检查和间接寻址，真的比直接 List.RemoveAt 快吗？”　→ **修正回答**：几十个计时器时确实差不多（实测 N=100 时两者分别约 26 ns 和 3 ns，都可以忽略）；swap-and-pop 的价值在规模上去之后——N=1 万时保序删除约 1 µs、N=10 万时约 49 µs，而 swap-and-pop 始终约 3～4 ns。更重要的是语义：RemoveAt 会让所有后续元素的位置改变，句柄体系要么跟着改 n 条映射，要么根本做不了。

Q9-6 / Q9-7　槽位被复用怎么办？generation 为什么能解决？是不是 ABA？

L1　槽位回收时代数加一；句柄里存着创建时的代数，校验时不相等就拒绝。比如 A 的句柄是 (5,3)，A 停止后槽位 5 的代数变成 4，B 复用槽位 5 拿到 (5,4)；A 的主人再用 (5,3) 访问会被拒绝，不会误伤 B。

L2　这和无锁编程里的 ABA 是同一类问题：只比较“位置/值”无法察觉中间发生过的变化，所以给它附加一个单调递增的版本号。区别在于：无锁 ABA 是并发下的竞态，这里是单线程里“持有方的信息过期”。

L3　严格说，generation 防住的是“旧句柄访问新主人”；它不能防住“伪造句柄”，因为 IsHandleValid 只比代数、不检查槽位是否在用，停止后映射也没清（实测 E2）。

L4 攻击　“那为什么不直接用自增的全局 id 当句柄，再用 Dictionary 找？”　→ 也可以，这就是“id + 哈希表”方案：id 全局唯一天然没有 ABA，代价是每次访问一次哈希查找、遍历时要走哈希桶。Slot Map 用数组下标代替哈希，用代数代替“全局唯一”，是用一点安全余量换常数级性能。UE 的 FTimerHandle 其实是两者结合：下标 + 全局递增的序列号。

Q9-8　generation 溢出怎么办？

L2　先估算：32 位代数要同一个槽位复用 2^{32} 次才回绕，每帧复用一次也要约 829 天；然后指出风险窗口——还需要有人恰好持有一个 2^{32} 次之前的句柄。所以客户端可以接受。

L3　要彻底解决有三种做法：64 位代数；UE 式 24 位下标 + 40 位序列号；代数到上限的槽位退役不再进入 free list（代价是槽位数缓慢增长）。另外要注意 C# 默认 unchecked，`int.MaxValue + 1` 会静默变成负数，不会抛异常。

Q9-9　default(TimerHandle) 是合法句柄吗？（压力题）

L1 在我的实现里是，这是我复盘时发现的 bug。代数从 0 开始，第一个计时器的句柄就是 (0,0)，而结构体字段的默认值也是 (0,0)。

L2 Demo 里正好踩到：CombatPawn 的 `_decayTimer` 没初始化成 Invalid，AI 肉体从不创建计时器，死亡时 `StopTimer(_decayTimer)` 实际是 StopTimer(0,0)，会停掉槽位 0 上的计时器。我把逻辑移植出来测过：按 Demo 的创建顺序，被停掉的是玩家的寿命计时器。当前参数下不会触发，是因为槽位 0 在第一次碰撞前一定已经被回收过一次。

L3 修复分两层：系统侧代数从 1 开始，让全零永远无效（UE 的 FTimerHandle 也是 0 表示无效）；使用侧句柄字段显式初始化为 Invalid。

L4 攻击 “那你的测试为什么没发现？” → 测试里的句柄都来自 CreateTimer 的返回值，没有“从未赋值的句柄”这种用例；而且测试是运行时脚本，没有覆盖 Demo 的组合流程。这类 bug 需要“默认值 / 未初始化”的边界用例，我会补成 EditMode 单元测试。

Q9-11 数组一定比链表快吗?

L1 不一定。顺序遍历、数据量超过缓存、对象在堆上分散，这三个条件同时成立时，连续数组才明显更快。

L2 我用 C++ 按 TimerData 的布局测过：1000 个计时器时，dense 数组、按序分配的指针数组、打乱的指针数组每个元素都是 0.4 ns 左右，没有区别；100 万个时分别是 0.9、3.7、6.1 ns。客户端里同时存在的计时器一般是几百个，布局不是瓶颈，选数组更多是为了 `ref` 原地修改、零额外分配和 swap-and-pop。

L3 C# 里还要加一层：class 对象之间是引用，Unity 的 Boehm GC 不压缩堆，长时间运行后对象会越来越散；而 struct 数组天然连续，并且只占一次分配。

Q9-13 为什么不用小顶堆?

L2 堆的优势是 Tick 只处理到期的计时器，适合“很多长计时器、大多数帧没有到期”的场景，UE 就是按到期时间建堆。我的场景计时器少、每帧都要累加 elapsed（因为支持暂停、scaled/unscaled 混用、interval），线性扫描更简单。

L4 攻击 “那如果计时器多到 10 万个呢？” → 那我会把“绝对到期时间”作为存储方式（不再逐个累加），按时间流速分组（scaled 一组、unscaled 一组，暂停的移出），每组一个堆或时间

轮；句柄体系不用变，堆里存句柄即可。

常见错误回答 ①

错误：“数组肯定比 List 快，因为数组是连续内存。”

问题：C# 的 `List<T>` 内部就是数组，也是连续的；真正的差别在于 List 的索引器返回副本，不能 `ref` 原地修改 struct。至于“连续内存更快”，要看访问模式和数据规模（实测 B1）。

更好的回答：“在每帧线性遍历、需要原地修改 struct 的场景下，裸数组可以用 ref 直接改元素，避免 List 索引器的副本拷贝；连续布局在数据量超过缓存时能减少 pointer chasing，但几百个计时器的规模下这个收益很小。”

常见错误回答 ②

错误：“generation 解决了所有句柄安全问题。”

问题：它只防“旧句柄访问新主人”。默认句柄、伪造句柄、跨实例句柄、代数回绕都不在它的防御范围内（9.8 节四个漏洞）。

更好的回答：“generation 解决的是槽位复用带来的 stale handle；另外还需要让全零句柄无效、停止时清映射并检查槽位在用、句柄带系统实例 id，才能覆盖其他边界。”

常见错误回答 ③

错误：“C# 有 GC，不会有悬垂引用，所以不需要 generation。”

问题：GC 只保证内存不被提前释放，不保证“逻辑上它还是你的那个计时器”。本项目的句柄是值（槽位号），不是引用，复用后照样指向新主人；就算用 class 引用，一旦池化也是同样的问题。

9.15 发散：同一个思想在别处的样子

ECS 的 Entity

Unity Entities 的 Entity 是 (Index, Version)；实体销毁后 Version+1，旧 Entity 查询组件会失败。

UE 的弱指针

`FWeakObjectPtr` 存的是 ObjectIndex + ObjectSerialNumber：对象数组的槽位被复用后序列号不同，弱指针就失效。面试问

EnTT 把版本号编码在 32 位 id 的高位。和 TimerHandle 一模一样。

“TWeakObjectPtr 怎么知道对象没了”，答案就是 generation。

渲染资源句柄

纹理、缓冲区、渲染目标常放在资源池里，上层只拿“池下标 + 代数”的句柄；资源被异步释放、槽位复用后，旧句柄不会误用新资源。

操作系统：fd 与 PID 复用

close 一个文件描述符后，下一次 open 会拿到最小的空闲号——持有旧 fd 的线程会写错文件。PID 回绕复用导致“kill 错进程”，Linux 后来引入 pidfd 解决。都是“只有 index 没有 generation”的事故。

无锁编程的 ABA

无锁栈 pop 时 CAS 比较栈顶指针，若栈顶节点被弹出又被重新压入，CAS 误以为没变。解法：指针旁边带一个计数（tagged pointer / 双字 CAS），本质就是 generation。

数据库乐观锁

行上带 version 字段，更新时 `WHERE version = 旧值`，被别人改过就失败重试。也是“用版本号检测中间发生过的变化”。

本章速记

- 五个数组：**dense _timers**、**_handleToIndex**、**_indexToHandle**、**_generations**、**_freeIndices（FIFO）**；两套下标：槽位号对外不变，dense 下标对内会变。
- 删除 = 末尾填洞 + 修被搬元素的两条映射 + count−1 + 代数+1 + 槽位入队；O(1)，**不保序**（实测 E4）。
- generation 防的是**旧句柄访问新主人**（ABA 式复用）；防不住：**默认句柄 (0,0)**（E1）、**残留映射 + 伪造句柄**（E2）、**跨实例**（E10）、**回绕**（E6，829 天/每帧复用）。
- “数组更快”有前提：顺序遍历 + 超出缓存 + 对象分散；1000 个时无差别，100 万个时打乱指针慢 6.7×（B1）。用数组的实在理由是 **ref 原地改 struct**。
- UE：TSparseArray + 按到期时间的堆 + 64 位句柄（24 位下标 + 40 位序列号，0 = 无效）+ double 绝对时间。

▶ **自测 1：dense = [A,B,C,D]（槽位 0..3，代数全 0），依次 Stop(C)、Stop(A)、Create(E)，写出最后的 dense、_handleToIndex、_generations、_freeIndices 和 E 的句柄。**

▶ **自测 2：为什么 swap-and-pop 必须有反向映射 _indexToHandle?**

▶ **自测 3：只把 CombatPawn 的 _decayTimer 初始化为 Invalid，算不算修好了默认句柄问题?**

CHAPTER 10

TimerSystem（下）：Tick、重入、掉帧语义与 GC 审计

数据结构对了，行为也要对：回调里删自己、删别人、建新的会怎样；卡了一帧该补几次；scaled 与 unscaled 怎么选；以及最常被质疑的那句——“它真的是 Zero-GC 吗？”

本章目标

- 讲清 Tick 为什么分“收集”和“执行”两个阶段，以及二次校验防住了什么
- 能逐一回答回调重入的 7 种情况，并举出 SoulDecay 里真实存在的重入路径
- 说清 interval（追赶 / 不追赶）与 loop 三种掉帧语义的差异，并结合 Unity 的 maximumDeltaTime
- 按“初始化 / 扩容 / 创建 / 稳态 Tick / 业务回调”五类做一次严格的 GC 审计
- 给出完整的 Code Review 表：当前设计 → 问题 → 为什么 → 最小修复 → production 方案

10.1 Tick 的两阶段结构

TimerSystem.cs:374-505（节选）

```
_expiredTimersCache.Clear(); _intervalTimersCache.Clear();
for (int i = 0; i < _timerCount; i++) {                    // ── 阶段一：只改数据、只收集句柄
    ref TimerData timer = ref _timers[i];
    if (timer.isPaused) continue;
    float dt = timer.isUnScaled ? unscaledDeltaTime : scaledDeltaTime;
    timer.timeElapsed += dt;
    if (timer.interval > 0f) { /* 累加 intervalElapsed，按追赶规则 Add 若干次句柄（10.3 节） */ }
    if (timer.timeElapsed ≥ timer.duration) {
        int handleIndex = _indexToHandle[i];
        _expiredTimersCache.Add(new TimerHandle(handleIndex, _generations[handleIndex]));
        if (timer.isLoop) timer.timeElapsed -= timer.duration;     // 循环：只减一个周期
    }
}
for (...) {                                                // ── 阶段二 A：所有 interval 回调
    if (!IsHandleValid(handle)) continue;                  //    二次校验
    try { data.onTrigger?.Invoke(); } catch (Exception e) { Debug.LogException(e); }
}
for (...) {                                                // ── 阶段二 B：所有完成回调
    if (!IsHandleValid(handle)) continue;
    var data = _timers[realIndex];                         //    拷贝一份（回调可能改数组）
    try { data.onComplete?.Invoke(); } catch (Exception e) { Debug.LogException(e); }
    if (!data.isLoop && IsHandleValid(handle)) StopTimer(handle);  // 回调里可能已经停过了
}
```

如果在第一个循环里直接回调，会出什么事

假设 dense = [A, B, C]，遍历到 i=0 时调用 A 的回调，回调里 `StopTimer(A)`：swap-and-pop 把 C 搬到 dense[0]，count 变成 2。循环继续 i=1，处理 B——**C 这一帧被漏掉了**。反过来，回调里 CreateTimer 会把新元素追加到末尾、count 变大，新计时器在创建它的同一帧就被累加了时间。这就是遍历中的**结构性修改（structural modification）**：C# 的 `foreach` 遇到它会直接抛 InvalidOperationException，裸数组 + 下标循环则会“静默地错”。

两阶段结构把“改结构”推迟到遍历结束之后：阶段一只读写数据、只记录句柄（值类型，不受搬家影响）；阶段二再逐个执行，执行前用 `IsHandleValid` 确认句柄仍然有效。

同一帧内的时间线

```
阶段一   i=0..count-1    累加时间 → 记录 interval 句柄 ×k、到期句柄
                          （此时没有任何用户代码运行，数组结构不变）
阶段二A interval 回调   for each 句柄：IsHandleValid? → onTrigger()
                          ↑ 回调里可能 Stop / Create / 抛异常
阶段二B 完成回调        for each 句柄：IsHandleValid? → onComplete() → 非循环且仍有效 → StopTimer
注意：所有计时器的 interval 回调都先于任何一个完成回调
```

10.2 回调重入：7 种情况逐一检查

#	回调里做了什么	当前行为【源码事实 / 实测】	结论
1	停止自己	代数 +1；回调返回后 `IsHandleValid` 为 false，不会重复 Stop；同一帧剩余的追赶次数也被跳过	**安全**
2	停止同帧已进入缓存的另一个计时器 B	B 的句柄在缓存里，但执行到它时校验失败被跳过（实测 E4：B 的完成回调没有出现）	**安全**
3	停止后立刻新建，复用了刚释放的槽位	新计时器拿到同槽位、代数 +1；缓存里的旧句柄代数对不上，不会误触发新计时器（实测 E4：C=(0,1)）	**安全（这正是 generation 在重入场景的价值）**
4	新建计时器	追加到 dense 末尾；不在本帧缓存里，下一帧开始计时	**安全**
5	新建导致 `Array.Resize`	阶段二A 持有 `ref TimerData data`，扩容后它指向**旧数组**；回调返回后不再使用它，所以没出错。阶段二B 用的是拷贝，不受影响	**安全但脆弱：以后如果在 Invoke 之后再写 data，就会写进旧数组**
6	抛异常	try/catch 吞掉并 `Debug.LogException`；非循环计时器之后照样被 Stop，不会每帧重复抛	**安全（比 EventSystem 好：后者没有 try/catch）**
7	用同名 CreateTimer “重启自己”	CreateTimer 发现同名旧句柄仍有效 → 先 Stop 旧的 → 建新的；回调返回后旧句柄已失效，不会把新计时器误删	**安全**

Demo 里真实存在的重入路径（SoulDecay，第 15 章）

玩家肉体寿命到期 → TimerSystem 阶段二B 调用 `OnBodyExploded` → `Die(true)` → ① `AuthorityGameMode.EndMatch()` → `StopTimer("AISpawner")`（停掉另一个计时器）→ ② `Controller.UnPossess()` → `CombatPawn.OnUnPossess` → `StopTimer(_decayTimer)`（**停掉正在执行的自己**）→ ③ `Destroy(gameObject)`。回调返回后 TimerSystem 再校验，句柄已失效，跳过 StopTimer。三种重入在一个调用栈里同时出现，而系统行为正确——这是面试里最好的“我的设计经受住了验证”的例子。

【问题】“到期了却不回调”的语义

因为所有 interval 回调先于所有完成回调，如果 Y 的 interval 回调停掉了同帧到期的 X，X 的 onComplete 就永远不会执行。对“停止 = 取消”的语义这是合理的，但需要在 API 文档里写明：**StopTimer 不会触发 onComplete**，到期与取消是两种结局。

10.3 interval 与掉帧：一帧卡了 0.6 秒，该触发几次

TimerSystem.cs:394-437（节选）

```
if (timer.interval > 0f) {
    timer.intervalElapsed += dt;
    if (timer.isChase) {                                    // 追赶模式（默认 true）
        int chaseCount = 0;
        while (timer.intervalElapsed ≥ timer.interval && chaseCount < MAX_CHASE_COUNT) {  //
            _intervalTimersCache.Add(handle);  timer.intervalElapsed -= timer.interval;  chase
        }
    } else if (timer.intervalElapsed ≥ timer.interval) {  // 普通模式：最多 1 次
        _intervalTimersCache.Add(handle);  timer.intervalElapsed -= timer.interval;
    }
    if (timer.intervalElapsed ≥ timer.interval) timer.intervalElapsed = 0f;    // “防爆”：多余
}
```

“interval = 0.1 s，一帧 dt = 0.6 s”理论上欠了 6 次。三种可能的设计语义：

语义	触发次数	适合的业务	风险
只触发 1 次（丢弃欠账）	1	UI 刷新、心跳、AI 重新决策——只关心“最新状态”	依赖次数的逻辑会少算（DOT 少跳伤害）
全部补上（精确）	6	持续伤害、资源产出、回放/帧同步——次数本身有意义	卡得越久补得越多，补的过程又让帧更卡 → **死亡螺旋（spiral of death）**
最多补 N 次（封顶）	min(6, N)	折中	封顶后怎么处理余量：丢弃（丢相位）还是保留（下帧接着补）

本项目的实际行为（【实测 E3 / E12】，C++ 逐行移植）：

场景	本帧触发	余量 intervalElapsed	解读
isChase=true，dt=0.6	5	0	封顶 5 次；剩下 0.1 仍 ≥ interval → 被“防爆”清零，**丢 1 次**
isChase=false，dt=0.6	1	0	只触发 1 次，剩余 0.5 被清零，**丢 5 次**
isChase=true，dt=0.35	3	0.0500000045	没触顶，余量保留，节拍相位正确
isChase=true，dt=0.5	5	1.49e-08	理想余量 0，float 运算残留一个极小正数
Loop(duration=0.1)，dt=0.35 后接 16 ms 帧	每帧 1 1 1 0 1 0	—	循环计时器每帧**至多 1 次但不丢账**，欠账分摊到后续帧

【问题】同一个系统里有三种掉帧语义

interval 追赶：封顶 5 次 + 清零（丢相位）；interval 不追赶：1 次 + 清零（丢次数）；loop：1 次 + 不清零（拖延）。调用方很难预期。更合理的设计是把它做成显式参数，例如“掉帧策略 = 只触发一次 / 补齐（封顶 N，余量保留）/ 补齐不封顶”，loop 与 interval 共用同一套策略。

10.3.1 Unity 里根本不会出现 dt = 0.6 —— 对 scaled 计时器而言

【扩展知识】Unity 的 `Time.deltaTime` 会被 `Time.maximumDeltaTime`（Project Settings → Time → Maximum Allowed Timestep，默认 1/3 秒）截断；`Time.unscaledDeltaTime` 不截断。所以真实卡顿 0.6 秒时，scaled 计时器看到的 dt 只有 0.333。【实测 E12】：

真实卡顿	scaled 计时器（dt 被截断为 0.3333333）	unscaled 计时器（dt 不截断）
0.6 s	触发 3 次，余量 0.0333	触发 5 次（封顶），余量清零
2.0 s	触发 3 次，余量 0.0333	触发 5 次（封顶），余量清零，丢 15 次

结论：对 scaled 计时器，`MAX_CHASE_COUNT = 5` 只有在 interval < 0.067 s 时才可能起作用；真正需要“防爆”的是 unscaled 计时器。【实测 E12】unscaled 的 Loop(0.1) 在 2 秒卡顿后，连续 22 帧每帧补 1 次，然后才恢复正常节拍——总次数是对的（2.48 s 内理想 24 次、实际 24 次），但“补”的那 22 帧里回调频率是设计值的 6 倍。

发散：手机切后台再回来

如果切后台或长时间断点调试之后 `unscaledDeltaTime` 一帧变得很大（具体数值取决于平台和 Unity 版本，需要实测），unscaled 的 loop 计时器会在之后的每一帧都补一次，持续很久：一个 0.1 s 周期的计时器欠 60 秒，就会在接下来 600 帧里每帧触发——这就是为什么联网游戏的心跳、超时检测通常直接读真实时钟（`Time.realtimeSinceStartupAsDouble` / Stopwatch）并按“当前时间 − 上次时间”计算，而不是累加帧间隔。

交互实验 • 卡顿一帧之后，interval 与 loop 计时器各触发几次

帧	dt(s)	interval 回调	loop 回调	理想(累计)
0	0.3333	3	1	3
1	0.0167	0	1	3
2	0.0167	0	1	3
3	0.0167	0	0	3
4	0.0167	0	0	3
5	0.0167	1	1	4
6	0.0167	0	0	4
7	0.0167	0	0	4
8	0.0167	0	0	4
9	0.0167	0	0	4

```
10 帧累计：interval 4 次，loop 4 次，理想 4 次（总时长 0.4833 s）
isChase=true：一帧最多补 5 次（MAX_CHASE_COUNT），补完仍超出则把余量清零 → 丢相位
loop：一帧最多 1 次，但不清零 → 欠账分摊到后面的帧（语义与 interval 不一致）
```

10.4 scaled / unscaled：每个计时器选哪一种

场景	选择	原因
技能冷却、Buff 持续、刷怪、肉体寿命	scaled	暂停、子弹时间时应该一起变慢 / 停止（SoulDecay 的寿命计时器和 AISpawner 都是 scaled，按 ESC 暂停时冻结）
暂停菜单里的动画、Toast 自动消失、按钮冷却	unscaled	`timeScale = 0` 时 UI 仍要动
网络心跳、请求超时、断线重连	真实时钟	和游戏内时间无关；切后台 / 卡顿时要按真实流逝判断
防沉迷、活动倒计时	服务器时间	本地时间可被修改，且可能跨越数天（见 10.6 的 float 精度问题）
帧同步（lockstep）战斗逻辑	逻辑帧计数	必须确定性：所有客户端在第 N 帧触发，不能依赖 float dt

【实测 E11】timeScale = 0 时的边界

暂停 10 帧（scaled dt = 0）：普通 5 秒计时器剩余仍是 5.00 s，interval 计时器触发 0 次——符合预期；但 **duration = 0 的循环计时器触发了 10 次**，因为判断条件是 `timeElapsed ≥ duration`，0 ≥ 0 恒成立。CreateTimer 没有任何参数校验，duration ≤ 0 的循环计时器会变成“每帧回调”，暂停也挡不住（E3 也测到了 5 帧触发 5 次）。

10.5 float 累加：为什么 UE 存“到期时刻”而不是“已走时间”

本项目每个计时器每帧做 `timeElapsed += dt`（float）。float 只有 24 位有效数字，数值越大，能表示的最小增量越粗。【实测 E14】从某个已走时间开始再走 600 帧（每帧 0.016 s，理想 +9.6 s）：

已走时间	实际增加	说明
3600 s（1 小时）	9.668 s	误差 0.7%
86400 s（1 天）	9.375 s	每帧的 0.016 被舍入成 0.015625
262144 s（约 3 天）	18.75 s	每帧被舍入成 0.03125，**时间走快一倍**
524288 s（约 6 天）	0 s	0.016 小于半个最小增量，加了等于没加，**计时器永远不会到期**

普通 Gameplay 计时器只有几秒到几分钟，循环计时器每周期还会减回去，所以这不是 Demo 的问题；但如果有人用它做“7 天后活动结束”的倒计时就会出事。UE 的 FTimerManager 维护一个 double 的 `InternalTime`，每个计时器只存绝对到期时间 `ExpireTime`，比较时不累加误差。

10.6 GC 审计：它到底是不是 Zero-GC

先给结论：**不是。准确的说法是：在调用方缓存委托、计时器数量不超过预分配容量的前提下，Tick 的稳态是零分配的；CreateTimer 本身不分配，但几乎所有真实调用点都会产生委托或闭包分配。**

类别	来源（源码位置）	什么时候	是否在稳态每帧发生
① 初始化	`new TimerData[128]`、三个 `List<int>(128)`、`Queue<int>(128)`、`Dictionary(64)`、两个缓存 `List(64)`（字段初始化器，81-105 行）	TimerSystem 组件创建时一次	否
	`AllocateCapacity(128)`（Awake）：容量已是 128，不再分配	—	否
② 扩容	`Array.Resize(ref _timers, 2×)`（113 行）	同时存在的计时器第一次超过 128、256…	否（但会产生一个旧数组的垃圾，并拷贝全部数据）
	三个 List 的 `Add`（195-197 行）	槽位数第一次超过 128…	否
	`Queue.Enqueue`、`Dictionary` 插入	空闲槽 / 命名计时器数超过容量	否
	两个缓存 List 的 `Add`	单帧 interval 回调或到期数第一次超过 64（追赶模式一帧可加 5 次）	否（Clear 不释放容量）
③ 创建计时器（调用方）	无捕获的 lambda `() ⇒ {}`	编译器缓存为静态委托，首次之后不分配	—
	捕获变量的 lambda（SphereSpawner、TimerSystemTest 里大量使用）	每次：一个闭包对象 + 一个委托	每次创建都分配
	实例方法组 `CreateTimer(MaxLifespan, OnBodyExploded)`（CombatPawn.cs:44）	每次 new 一个委托（C# 9 不缓存实例方法组）	每次夺舍分配一次
	缓存在字段里的委托（DemoGameMode 的 `_startMatchAction`、SurvivalGameMode 的 `_spawnAIAction`）	只在缓存时分配一次	否 ✓（正确做法）

④ 稳态 Tick	`ref` 遍历、值类型句柄、`List.Clear`、`IsHandleValid`；`HashCode.Combine` 不装箱	—	**不分配**（除非抛异常时 `Debug.LogException`）
⑤ 业务回调	回调里做什么就分配什么：`Log.N($"...")` 的字符串插值、`new XxxEventArgs`、事件闭包（第 7 章）、Destroy	回调执行时	取决于业务——不应算在计时器头上，但也不能说“整个计时链路零 GC”

10.6.1 “Blittable、避免 GC 扫描”这句注释为什么不成立

- **Blittable** 指托管与非托管内存里表示完全相同、可以直接 memcpy 给原生代码的类型。含引用字段的结构体**不是** blittable，也不满足 C# 的 `unmanaged` 约束——所以 TimerData 不能放进 `NativeArray`，也不能进 Burst Job。
- TimerData 有**两个**委托字段（注释写“唯一的引用类型字段”也错了），整个 `_timers` 数组都需要被 GC 标记阶段扫描。
- “存 int hash 而不是 string”确实少了一个引用，但这个收益远小于两个委托的存在。

10.6.2 【实测 E13】停止后的“托管层泄漏”

StopTimer 只做 `_timerCount--`，不清空被移出范围的那一格。【实测 E13】回调捕获了一个对象，计时器停止后该对象**仍然存活**（引用残留在 `_timers[1]`），直到这一格被新计时器覆盖才释放。对比：RingBuffer 在出队时会 `_buffer[_head] = default`，作者在那里是知道这个问题的。

对 Unity 的影响：回调通常捕获 `this`（某个 MonoBehaviour）。GameObject 被 Destroy 后，原生对象没了，但 C# 包装对象被残留引用钉住，GC 回收不了；如果它又引用了贴图、Mesh 之类的资源，`Resources.UnloadUnusedAssets` 也会认为它们仍在使用。修复只要一行思路：swap-and-pop 之后把 `_timers[last]` 置为 default。

10.6.3 怎么证明“稳态零分配”

1. **Profiler**：CPU Usage 模块的 Hierarchy 视图看 `TimerSystem.Update` 一行的 GC Alloc 列，连续多帧为 0 B；打开 Deep Profile 或加 `ProfilerMarker` 精确到 Tick。
2. **ProfilerRecorder**：在测试里记录 “GC Allocated In Frame” 计数器。

3. **Unity Test Framework**：`Assert.That(() ⇒ ts.Tick(dt, dt), Is.Not.AllocatingGCMemory())`（需要先把 Tick 从 MonoBehaviour 中拆出来，第 20 章）。

> **面试注意：说“低分配”时附上边界条件**
>
> “Tick 稳态零分配，前提是计时器数量不超过预分配容量、单帧回调数不超过 64；创建计时器时调用方如果传闭包或实例方法组，每次会分配一个委托；这些我用 Profiler 看过 / 计划用 Profiler 验证。”——比“Zero-GC”多三句话，但不会被任何追问击穿。

10.6.4 如果真的要做到更低 GC，设计空间有哪些

- **热冷分离**：Tick 用的数值字段放一个纯值数组（可以变成 unmanaged，甚至进 Job）；回调放另一个数组，只在触发时访问。
- **状态参数代替闭包**：提供 `CreateTimer<TState>(…, Action<TState> cb, TState state)`，让调用方传静态方法 + 状态对象，避免每次生成闭包（状态若是值类型，存储时要避免装箱，需要泛型化存储或按类型分池）。
- **回调 id 表**：回调预先注册成整数 id，计时器只存 id。代价是失去 lambda 的便利。
- **接口回调**：`ITimerListener.OnTimer(handle)`，由长期存在的对象实现，计时器只存接口引用（仍是引用，但不会每次新分配）。

10.7 Code Review：完整问题表

主题	当前设计	问题 / 为什么	最小修复	Production 方案
默认句柄	代数从 0 开始	(0,0) 与 default 相同 → 命中首个计时器（E1，P0）	代数从 1 开始；字段初始化为 Invalid	0 永远代表无效（UE 同款）
槽位在用检查	只比代数	伪造/残留句柄可删他人、产生僵尸与重复空闲槽（E2）	Stop 时 h2i 置 −1；校验加 0 ≤ h2i < count	槽位带 active 位，并断言不变式
跨实例	句柄无系统标识	切场景后旧句柄命中新系统（E10）	每个实例代数起点不同（如全局计数）	句柄内含系统实例 id
代数溢出	int++ unchecked	2^{32} 次复用回绕（E6：每帧复用约 829 天）	说明风险窗口即可	64 位句柄 / 槽位退役
名字	`string.GetHashCode()` 当键	碰撞互相覆盖；不保证跨进程稳定；hash 为 0 时无法按名查找	`Dictionary<string, Handle>`	预注册 id / 常量
同名覆盖	静默停止旧计时器	调用方无感知	文档写明 / 返回旧句柄	覆盖策略作为参数（替换 / 忽略 / 报错）
duration ≤ 0	无校验	循环计时器每帧触发，暂停也挡不住（E3、E11）	loop 时要求 duration > 0	参数校验 + 断言
interval ≤ 0	静默当作“无 interval”	传 0 的意图可能是“每帧”	文档写明	每帧回调用单独 API
掉帧语义	三套规则	丢相位 / 丢次数 / 拖延（E3、E12）	统一并写入文档	掉帧策略枚举，loop 与 interval 共用

回调异常	try/catch + LogException	✓ 正确	—	附带计时器名/创建栈便于定位
回调重入	两阶段 + 二次校验	✓ 正确；ref 跨扩容脆弱	Invoke 前先取出委托到局部变量	Tick 期间的增删进入命令缓冲
回调顺序	dense 顺序	swap-and-pop 打乱，≠ 创建顺序（E4）	文档写明	按 (到期时间, 序号) 排序回调
尾部残留	Stop 不清空	捕获对象无法回收（E13）	Stop 后 `_timers[last] = default`	热冷分离，回调数组单独清理
owner 生命周期	不感知	MonoBehaviour 已 Destroy，回调仍会执行 → MissingReferenceException	调用方在 OnDestroy 里 Stop	创建时登记 owner，销毁时批量取消（UE ClearAllTimersForObject）
时间精度	float 累加	数天后精度崩溃（E14）	长计时器用别的方案	double 全局时间 + 绝对到期时刻
调度复杂度	每帧扫描全部（含暂停的）	O(N)，暂停的也遍历	暂停的移出 dense	按流速分组 + 堆 / 时间轮
执行时机	自己 Update	与 World / Event 顺序不定	`[DefaultExecutionOrder]`	由 GameLoop 显式调用
线程	无同步	只能主线程调用（Unity API 本来也要求）	文档写明 / 断言主线程	跨线程请求走队列投递回主线程

10.8 测试：22 个用例到底验证了什么

行为	是否覆盖	说明
句柄有效性、停止后失效、Invalid 永远无效	✓	Test_HandleValidity
Swap-and-Pop	**部分**	只检查三个句柄的有效性，没检查被搬动的计时器数据是否正确（剩余时间、回调是否还对得上）
代数自增 / 槽位复用	**依赖顺序**	FIFO 下预留 2 个空闲槽两例都失败（E5）
暂停 / 循环 / 命名覆盖 / 回调中停止	✓	协程按真实时间等待，断言区间宽松
追赶 vs 非追赶	✗	两个用例都用默认 isChase=true（注释反而说默认是 false）；稳定帧率下根本触发不了追赶
scaled vs unscaled	✗	没有修改 timeScale，只验证了“都会触发”
默认句柄、伪造句柄、跨实例、溢出、hash 碰撞、duration≤0、扩容 >128、回调异常、回调里新建	✗	批量用例只有 30 个，碰不到 128 的扩容

形式上的问题：这是挂在 MonoBehaviour 上的协程脚本，**没有挂进任何场景**；`WaitUntil` 没有超时，失败表现为卡住而不是报错；`Fail()` 只打日志，`RunTestCoroutine` 结束后照样打印“通过”。它是一个运行时测试 harness，不是 NUnit / Unity Test Framework 意义上的单元测试（第 20 章给出迁移方案）。

10.9 与 UE 的行为对比

行为	本项目	UE FTimerManager【扩展知识】
循环计时器一帧欠多次	每帧最多 1 次，欠账拖到后续帧	默认在同一帧里连续补调多次；UE 5.4 起可用 `bMaxOncePerFrame` 限制为一次
回调里清除自己	两阶段 + 二次校验	记录 `CurrentlyExecutingTimer`，执行中被清除会被识别，不再重新入堆
回调目标对象已销毁	照样调用	绑定 UObject 的委托失效后不再执行，计时器被移除
Tick 中新加的计时器	追加到 dense，下一帧开始累加	先放进 `PendingTimerSet`，本次 Tick 结束后再入堆
时间基准	每个计时器 float 累加	管理器 double 时钟 + 绝对到期时刻

10.10 面试追问树

追问树 · TimerSystem 行为与 GC

```
Q10-1 回调里 StopTimer(自己) 会怎样？
 ├── Q10-2 回调 A 里 StopTimer(B) 呢？B 也在本帧到期
 │    └── Q10-3 那你为什么不在遍历时直接回调？          ← 考结构性修改
 └── Q10-4 回调抛异常呢？会不会每帧都抛？
Q10-5 interval=0.1，一帧卡了 0.6 秒，触发几次？          ← 边界
 ├── Q10-6 为什么封顶 5 次？封顶之后多余的时间去哪了？
 ├── Q10-7 Unity 里真的会出现 dt=0.6 吗？               ← 考 maximumDeltaTime
 └── Q10-8 循环计时器呢？和 interval 一致吗？
Q10-9 你说 TimerSystem 是 Zero-GC，证据是什么？          ← 质疑
 ├── Q10-10 TimerData 里有 Action，它还能叫 blittable 吗？
 ├── Q10-11 CreateTimer(MaxLifespan, OnBodyExploded) 分配了什么？
 └── Q10-12 停止的计时器会不会导致内存泄漏？             ← E13
Q10-13 计时器的主人被 Destroy 了，回调还会执行吗？
Q10-14 如果做成帧同步（lockstep），这个计时器能直接用吗？  ← 发散
```

Q10-1 → Q10-3　回调里删自己 / 删别人，为什么安全？为什么不边遍历边回调？

L1 • 30 秒　Tick 分两阶段：先遍历累加时间、只收集到期句柄；遍历结束后再逐个回调，每次回调前用句柄重新校验。回调里删了自己或者删了别人，代数已经变了，后面轮到它时校验失败直接

跳过。

L2 • 正常回答　如果边遍历边回调，回调里的 StopTimer 会做 swap-and-pop，把末尾元素搬到当前位置，循环继续往后走就漏掉了被搬过来的那个；回调里 Create 会改变循环上界。所以要把“可能修改结构的用户代码”和“遍历”分开。缓存里存的是句柄而不是下标，搬家不影响它；二次校验处理“已经被删”的情况；generation 处理“被删后槽位又被新计时器复用”的情况。

L3 • 深入　还有两个细节：完成回调前先把 TimerData 拷贝出来，因为回调可能触发扩容或 swap-and-pop；非循环计时器回调后要再校验一次才 Stop，因为回调里可能已经 Stop 过。SoulDecay 里“寿命到期 → Die → StopTimer(自己) + StopTimer(AISpawner) + Destroy”就是三种重入同时发生，行为正确。

L4 • 面试官攻击　“二次校验只能说明句柄失效，如果 B 被删了又在同一帧被同名重建，缓存里 B 的旧句柄会不会触发新的 B？”　→ 不会。重建拿到的槽位要么不同，要么相同但代数 +1，旧句柄的代数对不上。这正是 generation 在重入场景下的价值，我用移植版测过“回调里停 A、停 B、再建 C 复用 A 的槽位”，B 的回调没有执行，C 拿到 (0,1)。

Q10-5 → Q10-8　卡了 0.6 秒触发几次?

L1　默认追赶模式触发 5 次——最多补 5 次，补完剩下的时间直接清零；非追赶模式触发 1 次并清零。

L2　这是在“补齐次数”和“防止死亡螺旋”之间的折中：卡顿后一帧里补太多回调，会让这一帧更慢，下一帧 dt 更大，越补越卡。但清零会丢相位：理论上应该在 0.6、0.7…触发的节拍，变成从清零那一刻重新计。

L3　而且在 Unity 里，scaled 计时器根本看不到 0.6：deltaTime 被 maximumDeltaTime 截断到 1/3 秒，所以只触发 3 次、余量保留；只有 unscaled 计时器才会真的封顶。循环计时器又是另一套：每帧最多 1 次但不清零，欠账拖到后面每帧补一次。三种语义并存是我想改的地方：应该做成一个显式的掉帧策略参数。

L4 攻击　“如果是 DOT 伤害呢？丢 1 跳可以接受吗？”　→ 不可以。伤害、资源产出这类次数有意义的逻辑，不应该依赖“回调次数”，而应该按时间结算：一次回调里根据流逝时间算出应跳几次（或者直接算总伤害），这样卡顿只影响表现的平滑度，不影响数值。

Q10-9 → Q10-11　Zero-GC 的证据？TimerData 还能叫 blittable 吗？

L1　严格说不是 Zero-GC。Tick 的稳态不分配，但创建计时器时调用方传的委托和闭包会分配，扩容也会分配。

L2　按五类拆：初始化分配一次；超过 128 个计时器或单帧回调超过 64 次会扩容；创建时，捕获变量的 lambda 每次一个闭包对象加一个委托，实例方法组在 C# 9 下每次 new 一个委托——比如 CombatPawn 每次被玩家附身都会分配一次；稳态 Tick 只有 ref 遍历和值类型操作，不分配；回调里业务做什么就分配什么。TimerData 含两个委托引用，不是 blittable，整个数组还要被 GC 扫描，那句注释是错的。

L3　证据要靠 Profiler 的 GC Alloc 列，或者用 Unity Test Framework 的 `Is.Not.AllocatingGCMemory()` 约束，前提是把 Tick 拆成可直接调用的纯 C# 方法。

L4 攻击　“那你项目里有没有地方做对了？” → 有：DemoGameMode 和 SurvivalGameMode 都把委托缓存成字段（`_startMatchAction = StartMatch`），之后每次传给 CreateTimer 都不再分配；CombatPawn 没有这样做，是一个可以直接改的点。

Q10-12　停止的计时器会不会导致内存泄漏？

L2　会有一种“托管层的滞留”：StopTimer 只把 count 减一，被移出范围的那一格还保存着委托引用，委托又引用着它捕获的对象。我用移植版验证过，停止后被捕获对象仍然存活，直到这一格被新计时器覆盖。在 Unity 里，被 Destroy 的 MonoBehaviour 的 C# 包装对象就会被它钉住。修复是 swap-and-pop 后把末尾那一格清成 default——RingBuffer 出队时就是这么做的。

Q10-13　主人被 Destroy 了，回调还会执行吗？

L2　会。TimerSystem 只存委托，不知道它属于谁；Destroy 之后委托照样被调用，回调里访问 `transform` 就会抛 MissingReferenceException（被 try/catch 接住并打日志）。项目里靠调用方自觉：CombatPawn.Die 和 OnUnPossess 里会 StopTimer，SphereSpawner 的回调里先判断 `if (sphere)`。

L3 系统性的做法是创建时登记 owner，owner 销毁时批量取消；或者执行前检查 owner 是否存活（Unity 对象可以用重载的 `==` 判断假 null）。UE 的委托绑定 UObject 时会自动失效，还提供 `ClearAllTimersForObject`。

Q10-14 做成帧同步（lockstep），这个计时器能直接用吗？（发散）

L2 不能直接用。帧同步要求所有客户端在同一逻辑帧得到完全相同的结果：① 时间不能用 float 的 deltaTime 累加，要用整数逻辑帧（或定点数）；② 不能有 scaled/unscaled 这种依赖本地渲染帧率的分支，逻辑计时器只跟逻辑帧走；③ 同帧多个计时器的回调顺序必须确定且所有端一致——swap-and-pop 的顺序虽然在相同操作序列下是确定的，但不直观，通常显式按 (到期帧, 创建序号) 排序；④ 追赶策略也必须确定，掉帧时逻辑帧照样一帧一帧推进，渲染帧可以跳。

L3 表现层的计时器（UI 动画、特效消失）仍然可以用现在这套，因为它不参与逻辑结果。所以帧同步项目里通常有两套计时：逻辑帧计时器和表现层计时器。

常见错误回答 ①

错误：“我的 TimerSystem 是 Zero-GC 的，因为 TimerData 是 struct。”

问题：struct 只说明 TimerData 本身不单独分配；它里面的两个 Action 是引用，调用方创建它们时会分配；数组也会扩容。

更好的回答：“TimerData 用 struct 存在连续数组里，Tick 稳态不分配；分配集中在创建时的委托和闭包，可以通过缓存委托、状态参数或回调 id 表进一步降低。”

常见错误回答 ②

错误：“掉帧时我会补齐所有次数，保证逻辑正确。”

问题：当前代码封顶 5 次并清零，并没有补齐；而且无限补齐会导致死亡螺旋。

更好的回答：“interval 追赶最多 5 次，超出部分丢弃，这是为了防止卡顿后回调爆发；对次数敏感的逻辑应该在回调里按流逝时间结算，而不是依赖回调次数。”

10.11 发散：面试官会从这里追到的知识

Fix Your Timestep

固定步长 + 累加器：`acc += frameDt; while (acc ≥ step) { Simulate(step); acc -= step; }`，渲染用 `acc/step` 插值；为防死亡螺旋，限制每帧最多模拟 N 步。Unity 的 FixedUpdate 就是这套，maximumDeltaTime 就是那个 N。

协程 vs 计时器

`yield return new WaitForSeconds(1)` 每次分配一个对象（可缓存复用）；协程随 GameObject 失活而停止、无法统一暂停或查询剩余时间；UniTask 等库基于 PlayerLoop 实现零分配的 await 延迟。

时间轮

环形数组，每格一个链表；指针每 tick 前进一格，处理该格的计时器；超过一圈的放到上层轮（分层时间轮）。插入 O(1)，适合服务器上数十万连接的超时管理。

float 精度与大世界

和 E14 同理：大世界坐标离原点越远精度越差，于是有 Floating Origin、UE5 的 Large World Coordinates（double）。“数值越大，float 越粗”是同一个知识点。

GC 机制

Unity（Mono / IL2CPP）使用 Boehm GC：非分代、不压缩，可开启增量模式把标记分摊到多帧。所以“减少分配次数”和“减少存活的引用”同样重要——前者减少 GC 触发，后者减少每次标记的工作量。

操作系统的定时器

Linux 内核用分层时间轮管理 timer，epoll 的超时、timerfd、高精度定时器 hrtimer（红黑树）；游戏服务器的“定时器线程 + 回调投递到逻辑线程”结构，正好对应第 12 章的线程安全队列。

本章速记

- Tick = **阶段一累加+收集句柄 → 阶段二A 全部 interval 回调 → 阶段二B 全部完成回调**；回调前二次校验；完成回调前先拷贝数据。
- 重入 7 种情况都安全，Array.Resize 下的 `ref` 属于“安全但脆弱”；SoulDecay 寿命到期路径同时包含停自己、停别人、Destroy。
- 掉帧（E3/E12）：追赶 **封顶 5 次 + 清零**；不追赶 **1 次 + 清零**；loop **每帧 1 次不丢账。** Unity 的 scaled dt 被截断到 **1/3 s**，unscaled 不截断。
- timeScale=0 时 duration=0 的循环计时器**每帧触发**（E11）；float 累加在约 6 天处**停止前进**（E14）。

- GC：**Tick 稳态零分配 ≠ Zero-GC**；分配来自初始化、扩容、调用方委托/闭包、业务回调；TimerData 不是 blittable；Stop 后尾部残留引用（E13）。

▶ **自测 1：dt=0.35、interval=0.1、isChase=true，本帧触发几次？intervalElapsed 剩多少？如果 isChase=false 呢？**

▶ **自测 2：为什么阶段二B 用 `var data = _timers[realIndex]` 拷贝，而不是 ref？**

▶ **自测 3：列出 CombatPawn 一次被玩家夺舍时，与计时器相关的所有托管分配。**

CHAPTER 11

ObjectPool：写好了，为什么没启用

对象池的数据结构很简单，难的是“借出去的东西怎么确认还是你的”和“还回来的东西怎么复位”。这个项目两件事都没做完——前者让第 9 章的 ABA 问题重新出现，后者让池子接不进 Actor 的生命周期。

本章目标

- 准确描述 ObjectPool 的使用状态：实现了、接入代码写了、但没有启用
- 讲清数组 + 可用下标栈的结构、LIFO 复用、翻倍扩容
- 说清 PoolObject 只有 index 带来的三类问题：陈旧拷贝、错配回收、默认值
- 能把第 9 章的 Index + Generation 迁移到对象池上，并说清还缺什么
- 回答“池化到底省了什么”：分配、GC、碎片、初始化成本，以及池化的代价
- 对比 Unity 内置的 `UnityEngine.Pool.ObjectPool<T>`

11.1 先把事实说准：它到底用了没有

位置	源码事实
Framework/ObjectPool.cs	泛型池 `ObjectPool<T> where T : class, IPoolable, new()`，105 行，无测试
SoulDecayDemo/GameObjectWrapper.cs	IPoolable 实现：出池 SetActive(true) + 恢复 Tick；入池关 Tick + SetActive(false) + 清刚体速度
SurvivalGameMode.cs	声明了 3 个池子字段；**初始化语句被注释**；`GetOrCreatePawn / GetOrCreateAIController / RecycleObject` 已写好，**无调用方**；SpawnRandomAI 里调用池子的两行也被注释
实际运行路径	`CreatePawn`（CreatePrimitive + AddComponent）与 `new GameObject().AddComponent<SimpleAIController>()`；死亡和被抛弃时 `Destroy`

面试注意

“我用对象池优化了 Demo”是错误说法。准确说法：“我实现了一个对象池并写了 Demo 的接入代码，但在接入时发现它和 Actor 生命周期冲突，最终没有启用。”——然后讲清冲突在哪（11.6），这比“用了”更有说服力。

11.2 结构：数组 + 可用下标栈

ObjectPool.cs（节选）

```
public struct PoolObject<T> { public T elem; public int index; }      // 借条：对象 + 槽位下标（
public class ObjectPool<T> where T : class, IPoolable, new() {
    struct Elem { public T elem; public bool IsAvailable; }
    private int _capacity; private Elem[] _pool; private Stack<int> _availableIndexStack;
    public ObjectPool(int n) { ... for (i < n) { _pool[i].elem = new T(); _pool[i].IsAvailable
    public bool Acquire(out PoolObject<T> obj) {
        if (stack.TryPop(out int index)) { obj = new(_pool[index].elem, index); _pool[index].I
        if (ReAllocate(_capacity * 2)) { ... 同上 ... }                    // 空了就翻倍
        obj = default; return false;
    }
    public bool Recycle(ref PoolObject<T> obj) {
        if (obj.index < 0 || obj.index ≥ _capacity) return false;
        if (_pool[obj.index].IsAvailable) return false;                    // 只检查"槽位是否已经
        obj.elem.OnRecycle(); obj.elem = default;                          // 只清空调用方手里这
        _pool[obj.index].IsAvailable = true; stack.Push(obj.index); return true;
    }
}
```

- **预先 new 满**：构造时和每次扩容时，把所有空位都 `new T()` 填满（【实测 G9】容量 2 连取 5 个，一共 new 了 8 个对象）。`new T()` 在泛型约束下编译为 `Activator.CreateInstance<T>()`，比直接 new 慢，但只在预热时发生。
- **LIFO 复用**：可用下标是 Stack，刚还回来的槽位下一次立刻被借出（G9：归还 a 后再借，拿到的还是 a 的槽位）。好处是“热”的对象更可能还在缓存里；坏处是旧借条立刻就能撞上新主人（11.3）。
- **翻倍扩容**：新建更大的数组、拷贝旧元素、为新增部分 new 对象并压栈。注释说“深拷贝”，实际是结构体拷贝 = 引用的浅拷贝（这正是想要的，但注释不对）。
- **容量 0**：【实测 G9】0×2 = 0，ReAllocate 返回 true 却什么也没分配，**永远借不到**，也不报错。
- **注释的说法**：“只有值类型可以申请连续内存，所以用 struct 包起来而不使用 Tuple”——引用数组本身也是连续的（存的是连续的引用），不连续的是引用指向的对象；用 struct 的真正收益是避免 `System.Tuple`（class）的额外分配，`ValueTuple` 同样可以。

11.3 PoolObject 只有 index：第 9 章的问题回来了

借条里只有“对象引用 + 槽位下标”。归还时，池子只能检查“这个槽位现在是不是已经空闲”，无法确认“这张借条是不是当前这一任的”。

情况	当前行为（实测 E8 / G9）	对应第 9 章的
陈旧拷贝：借条是 struct，存进 Dictionary 或别的字段时被拷贝；用一份归还后，另一份还在	归还 a 后槽位立刻被 b 借走（LIFO）；再用 a 的旧拷贝归还 → **成功**，b 正在用的对象被 OnRecycle（E8）	stale handle / ABA 式复用
错配回收：借条里的 elem 和 index 来自不同的租借	Recycle 用 index 判断槽位、却对 elem 调 OnRecycle：a 的对象被回收，b 的槽位被标记空闲而 b 还在用；再借一次，同一个对象被两人持有（G9）	伪造句柄
默认值：`default(PoolObject)` 的 index = 0、elem = null	若 0 号槽正在使用，`Recycle(default)` 通过两项检查后对 null 调 OnRecycle → NullReferenceException（E8）	default(TimerHandle) = (0,0)
重复归还同一份	第一次 Recycle 后调用方那份的 elem 被清空、槽位已空闲，第二次被 IsAvailable 拦截 ✓	—

把 Index + Generation 迁移过来

迁移思路（只讲设计，不给实现）

```
槽位：  { T obj; int generation; bool inUse; }
借出：  inUse = true；返回借条 Handle(index, generation)
归还：  校验 index 合法 && inUse && slot.generation == handle.generation
        → OnRecycle → inUse = false → generation++ → 入栈
访问：  pool.TryGet(handle, out obj)：校验同上，失败返回 false
generation 从 1 开始，让 default 借条永远无效
```

还缺什么：generation 只能保证“借条 → 池子”这条路径的安全。**业务代码手里如果直接存了对象引用（而不是借条）**，池子管不到——比如 SoulDecay 的 UI 持有某具 CombatPawn 的引用，Pawn 被回收再借出成另一只 AI，UI 拿着引用照样能读写它。所以对池化对象，业务层也应该只持有句柄，或者在对象内部也存一份 generation，访问时比对。这正是 UE 的 `FWeakObjectPtr`（ObjectIndex + SerialNumber）要解决的问题。

11.4 池化到底省了什么

收益	说明	在本 Demo 里的分量
托管分配与 GC	不再每次 new；GC 触发更少。Unity 的 Boehm GC 不分代，一次回收要扫描整个托管堆	小：每 3 秒一次刷怪
原生对象创建 / 销毁	GameObject、组件、碰撞体、材质实例的创建和 Destroy 都在原生层，比托管分配贵得多	**这才是主要收益**：CreatePrimitive + AddComponent×2 + Destroy
初始化成本	Awake、GetComponent、加载资源只做一次	中
内存碎片	Boehm GC 不压缩，频繁分配释放不同大小的对象会让堆碎片化；池化让对象长期驻留、地址稳定	小

代价	说明
常驻内存	池子只增不减（项目没有收缩）；峰值过后内存不还
复位负担	对象的每一个状态字段、每一个副作用都要在出池时撤销——漏一个就是“上一任的残留”（11.6）
生命周期变复杂	“销毁”变成“入池”，引擎提供的 Start / OnDestroy 语义不再适用
悬垂引用变成 ABA	不池化时，持有已 Destroy 对象的引用会得到“假 null”（Unity）或报错，问题显性；池化后对象复活成别人，问题静默（11.3）
存活对象变多	GC 每次要标记的对象更多——减少的是“分配次数”，增加的是“存活集合”

面试注意：先问“值不值得池化”

判断标准是创建频率 × 单次成本，以及是否在帧敏感的时刻创建。子弹、伤害数字、特效、音效源这类“每秒几十个、很快销毁”的对象值得；每 3 秒一个、活几十秒的 AI 收益很小，复位却很复杂。能说出“我当时判断这个 Demo 不值得”，也是一个合格的答案。

11.5 和 Unity 内置池对比

	本项目 ObjectPool	`UnityEngine.Pool.ObjectPool<T>`（Unity 2021+）【扩展知识】
创建方式	`new T()`，构造时预先 new 满	`createFunc` 委托，按需创建（可配 defaultCapacity）
回调	IPoolable.OnSpawn / OnRecycle（侵入式，T 必须实现接口）	actionOnGet / actionOnRelease / actionOnDestroy（非侵入，可池化任何类型，包括 GameObject）
上限	无，只增不减	`maxSize`：超出的归还对象直接销毁
重复归还	IsAvailable 拦截同一份借条的重复归还	`collectionCheck`：重复 Release 抛异常（文档说明只在编辑器中检查）
借条	PoolObject（index + elem），无 generation	直接返回对象本身，没有借条——同样没有 ABA 防护
其他	—	ListPool / DictionaryPool / HashSetPool / GenericPool，以及 `using (ListPool<T>.Get(out var list))` 的作用域借用

结论：数据结构上两者是一个量级的东西；本项目缺的是 maxSize、非侵入回调、按需创建。Unity 内置池同样不防 ABA，这说明业界的普遍做法是“靠约定：归还后不再使用”，需要更强保证时才上句柄。

11.6 接入 SoulDecay 时会遇到的生命周期冲突

第 15 章 15.9 列过清单，这里按“根因”重新归类，这才是面试时要讲的部分：

根因	具体表现	需要的框架能力
Actor 的注册挂在 Unity 的 Start 上，而 Start 一生只调用一次	第二次出池不会 BeginPlay；HasBegunPlay 永远为 true	Actor 增加“出池 / 入池”生命周期钩子（类似 UE 的 Actor 池化需要业务自己实现的 Reset 逻辑）
注销挂在 OnDestroy 上，而 SetActive(false) 不会触发 OnDestroy	入池对象仍在 World 名单里，只靠 IsTickEnabled 跳过	World 提供显式的 Unregister / Suspend
状态没有集中复位点	isDying、名字、材质颜色、碰撞体 enabled、寿命句柄都要逐个撤销；GetOrCreatePawn 里“碰撞体为空才 AddComponent”的判断永远不成立（碰撞体只是被 disable）	每个 Actor 一个 ResetForReuse，由池子统一调用
销毁路径写死了 Destroy	Die、OnUnPossess、SimpleAIController.OnUnPossess 都直接 Destroy	“结束生命”抽象成一个可替换的策略：销毁或归还
外部持有直接引用	Controller ⇄ Pawn 双向引用、UI 持有句柄、GameMode 的字典	入池前强制解绑（UnPossess）；对外只发句柄

11.7 Code Review：ObjectPool 问题表

级别	问题	说明
P1	借条无 generation	陈旧拷贝能回收别人的对象（E8）
P1	Recycle 不校验 elem 与槽位是否匹配	错配回收导致同一对象被两人持有（G9）
P2	default 借条 index = 0	回收时对 null 调 OnRecycle（E8）
P2	容量 0 永远借不到且不报错	G9
P2	只增不减、无 maxSize、扩容时预先 new 满	峰值后内存不还；扩容那一帧集中分配
P3	侵入式接口；注释“深拷贝” “只有值类型才连续”不准确；未启用的接入代码与实际参数已分叉（4.5 s / 8 s vs 5 s / 30 s）	—
测试	无	应覆盖：LIFO 顺序、扩容、重复归还、陈旧借条、默认借条、容量 0

11.8 面试追问树

追问树 · ObjectPool

```
Q11-1 你的项目用了对象池吗？                                  ← 先说准事实
 └── Q11-2 为什么没启用？                                     ← 生命周期冲突
      └── Q11-3 Start 只调用一次，那出池怎么重新初始化？
Q11-4 讲一下池子的结构。为什么用栈不用队列？
Q11-5 PoolObject 为什么没有 generation？会出什么问题？        ← 压力题
 └── Q11-6 加了 generation 就安全了吗？                       ← 业务层的直接引用
Q11-7 对象池到底省了什么？什么对象不值得池化？
Q11-8 Unity 自带的 ObjectPool 和你的有什么区别？
Q11-9 C++ 里对象池怎么做？和 malloc 有什么关系？              ← 发散
```

Q11-1 → Q11-3　用了对象池吗？为什么没启用？

L1 • 30 秒　实现了一个泛型对象池，也给 SoulDecay 的肉体和 AI 灵魂写了接入代码，但没有启用，现在走的是创建加 Destroy。原因是接入时发现它和我的 Actor 生命周期冲突。

L2 • 正常回答　我的 Actor 在 Unity 的 Start 里向 World 注册、在 OnDestroy 里注销。池化之后对象不会被销毁，只是 SetActive(false)，所以不会注销；再出池时 Start 不会再调用，也就不会重新 BeginPlay。我当时用 IsTickEnabled 临时挡住了入池对象的 Tick，但状态复位没做完：死亡标志、名字、颜色、碰撞体、寿命句柄，每一个都要撤销。

L3 • 深入　根本上需要框架层的支持：Actor 增加出池和入池两个钩子，World 提供显式的挂起和恢复，所有“结束生命”的地方走同一个可替换的出口（销毁或归还），入池前强制解除 Possess。没有这些，池化只是把 bug 从“对象没了”变成“对象带着上一任的状态复活”。

Q11-5 / Q11-6　PoolObject 为什么没有 generation?　（压力题）

L1　当时没想到，这是一个问题。借条只有对象和下标，归还时只能检查槽位是否已空闲，判断不了借条是不是当前这一任的。

L2　借条是 struct，存进字典会被拷贝；用一份归还后，另一份还能用。池子是栈，刚还的槽位下一次立刻借给别人，这时用旧拷贝归还会成功，把别人正在用的对象回收掉——我用移植版测过。另外 Recycle 不检查借条里的对象是否就是那个槽位的对象，错配时会出现同一个对象被两个人持有。修法和计时器一样：槽位带代数，借条带代数，归还时比对，代数从 1 开始。

L3 / L4 攻击　“加了就安全了吗？” → 只对“通过借条访问”的路径安全。业务代码如果直接存了对象引用，池子管不到，对象被回收再借出后，旧引用就指向了新主人。所以池化对象对外最好只发句柄；或者对象内部也带代数，访问前比对。UE 的弱对象指针就是用“对象数组下标 + 序列号”判断对象是否还是原来那个。

Q11-7　对象池省了什么？什么不值得池化?

L2　在 Unity 里主要省的是原生对象的创建销毁——GameObject、组件、碰撞体、材质的创建和 Destroy 比托管 new 贵得多；其次是 GC 次数和初始化成本。代价是常驻内存、复位负担、生命周期变复杂、悬垂引用变成静默的 ABA。值得池化的是创建频繁、寿命短、在帧敏感时刻创建的：子弹、伤害数字、特效、音效。SoulDecay 每 3 秒刷一只、活几十秒的 AI，收益小、复位复杂，不启用是合理的。

Q11-9　C++ 里对象池怎么做？和 malloc 有什么关系？　（发散）

L2　C++ 常见的是固定大小块的内存池（free list allocator）：预先申请一大块内存切成等大的块，空闲块用侵入式链表串起来，分配和释放都是 O(1)，用 placement new 构造、手动调用析构。和 malloc 比，省掉了通用分配器的元数据、锁和碎片；tcmalloc、jemalloc 内部也是按大小分级的线程本地缓存，本质上是同一个思想。游戏引擎里还常见帧分配器（每帧开头整体重置的线性分配器）和对象级的池。

L3　C++ 池的风险更大：归还后继续使用就是 use-after-free，而且因为内存被复用，往往不会崩溃而是静默读到新对象——所以引擎会配合句柄 + 代数，或者在调试版本里给释放的块填充特殊值（如 0xDD）来尽早暴露问题。

常见错误回答

错误：“对象池能完全消除 GC。”

问题：它减少的是分配次数；池里常驻的对象仍然每次被 GC 标记。扩容时会集中分配；借出时的 OnSpawn 里如果有字符串拼接、闭包，照样分配。

11.9 发散：面试官会从这里追到的知识

内存碎片

外部碎片：总空闲够但没有连续的大块；内部碎片：分配的块比需要的大。不压缩的 GC（Boehm）和 C/C++ 的 malloc 都会有外部碎片，等大块的池天然没有外部碎片。

分代与压缩 GC

.NET CLR 的 GC 分代 + 压缩，短命对象很便宜；Unity 的 Boehm 不分代、不移动，短命对象并不便宜——这是 Unity 项目比一般 .NET 项目更重视池化的原因。

UE 的对象复用

UE 没有通用的 Actor 池（Spawn/Destroy 很重），项目通常自己写池；Niagara 特效、音频组件有引擎内置的池；UObject 的 GC 是标记—清除，靠 UPROPERTY 引用图追踪。

ECS 的“池”

ECS 里实体本来就是数组下标 + 版本号，组件存在按类型的连续数组里，增删就是 swap-and-pop——第 9 章的 Slot Map 已经是“池 + 句柄”的完整形态。

本章速记

- **实现了、接入代码写了、没启用**；实际走 Create + Destroy。
- 结构：Elem[] + **可用下标栈（LIFO）** + 翻倍扩容（预先 new 满）；容量 0 永远借不到（G9）。
- 借条只有 index：**陈旧拷贝回收别人（E8）、错配回收导致一物两主（G9）、default 借条 NRE**；修法是 Index + Generation，但业务层的直接引用仍不受保护。
- 池化在 Unity 里主要省**原生对象创建销毁**，代价是常驻内存、复位负担、静默 ABA。
- 没启用的根因：**注册挂在 Start、注销挂在 OnDestroy**，池化绕开了这两个时机。

▶ **自测 1：容量 4 的池，借 a、b，归还 a，借 c，然后用 a 的旧拷贝再归还一次，会发生什么?**

▶ **自测 2：为什么说“池化让悬垂引用从显性变成静默”？**

CHAPTER 11B

对象池落地设计：把半成品接进框架和 SoulDecay

第 11 章说清了“为什么没启用”。这一章回答“具体怎么做”：池放在框架的哪一层、Actor 的生命周期要补哪几个钩子、World 怎样成为唯一的出生和死亡入口、延迟回收怎样避免 ABA，以及 SoulDecay 每个文件改哪几行。所有关键行为都用逐行移植的 C++ 跑过（H1 ~ H5）。

本章目标

- 指出原有半成品（ObjectPool / GameObjectWrapper / SurvivalGameMode 接入代码）每一处具体缺什么
- 给出一个分四层的设计：L0 带 generation 的 `ObjectPool<T>` → L1 `ActorPool` → L2 `World.SpawnActor / DestroyActor` → L3 Demo
- 说清 Actor 池化后的完整状态机，以及 `EndPlay`、`OnReturnToPool`、`OnTakeFromPool` 各自负责什么
- 理解为什么回收必须延迟到帧末，为什么延迟回收的计时器必须捕获借条而不是对象引用
- 能逐文件说出 SoulDecay 的改动，并用实测数据说明收益
- 知道怎么分步迁移、怎么测试，以及面试官会怎么追问这套设计

关于本章的代码

前面各章按你的要求没有给修复代码；这一章的问题本身就是“具体怎么实现”，所以给出**关键代码骨架**。本机无法编译 C#，这些代码**没有在 Unity 中编译运行过**；其中的逻辑（借条校验、帧末回收、延迟回收、状态复位、整局创建次数）已在 `demos/pool_design_verify.cpp` 中逐行移植验证，输出见 `demos/pool_design_output.txt`。依赖 Unity 引擎规则的部分标为【Unity 规则】，请以官方文档和实际运行为准。

11B.1 先看半成品：原代码的四个缺口

仓库里已经有三块对象池相关代码，思路是对的——“出池打开 Tick、入池关掉 Tick、重置刚体速度”说明作者意识到了生命周期问题——但每一块都停在半路：

原代码 · SurvivalGameMode.cs（节选，均未启用）

```
private ObjectPool<GameObjectWrapper> _heavyPawnPool, _lightPawnPool, _aiControllerPool;
private Dictionary<GameObject, PoolObject<GameObjectWrapper>> _activeWrappers = new  … ;   //

private CombatPawn GetOrCreatePawn(Vector3 position, bool isHeavy) {
    var pool = isHeavy ? _heavyPawnPool : _lightPawnPool;
    if (pool.Acquire(out var wrapper)) {
        if (wrapper.elem.Instance == null) {                // 池里存的是“空壳 wrapper”，第一次用
            GameObject bodyObj =  …  CreatePrimitive  … ;  …  AddComponent<CombatPawn>();
            p.MaxLifespan = isHeavy ? 8f : 4.5f;            // ← 与实际在用的 CreatePawn（30 s /
            wrapper.elem.Init(bodyObj);
        }
        GameObject go = wrapper.elem.Instance;
        go.name = isHeavy ? "HeavyPawn" : "LightPawn";      // 只恢复了名字
        if (go.GetComponent<Collider>() == null)  …         // 碰撞体只是被 disable，这个判断永远
        _activeWrappers[go] = wrapper;                      // 借条被拷贝进字典
        return go.GetComponent<CombatPawn>();
    }
    return null;
}
public void RecycleObject(GameObject go, bool isHeavyPawn) {  …  pool.Recycle(ref wrapper); _a
```

原代码 · GameObjectWrapper.cs（节选）

```
public void OnSpawn()   { Instance.SetActive(true);  actor.SetActorTickEnabled(true); }
public void OnRecycle() { actor.SetActorTickEnabled(false); Instance.SetActive(false); rb.velo
```

#	缺口	原代码的表现	后果（实测）
①	**借条不安全**	PoolObject 只有 index，没有 generation；借条是 struct，会被拷贝进字典	H1：旧拷贝能回收别人正在用的对象；default 借条对 null 调 OnRecycle；在 OnRecycle 里再回收自己会让同一槽位入栈两次，之后两次借出拿到**同一个对象**
②	**生命周期没接上**	Actor 只在 `Start` 注册、只在 `OnDestroy` 注销；池化用 SetActive，两者都不会再触发	H4：复用的肉体**不会再 BeginPlay**；闲置时**仍留在 World 名单**里，只靠 IsTickEnabled 跳过
③	**状态没有复位点**	wrapper 只管 SetActive 和速度；CombatPawn 的 isDying、碰撞体、名字、颜色、寿命句柄没人管	H4：第二轮起出池时 `isDying=true`、碰撞体是关的，被玩家抛弃后**不会再安排回收**（尸体永久残留）
④	**死亡出口写死了 Destroy**	CombatPawn.Die、OnUnPossess 和 SimpleAIController.OnUnPossess 直接 `Destroy`；要回收就得找到 GameMode 调 RecycleObject	池子与销毁路径完全脱节，所以 RecycleObject 一直没有调用方

①是数据结构问题，②③④是**框架问题。**所以只修 ObjectPool 是不够的：要让池化“接进框架”，必须改 Actor 的生命周期和 World 的职责。

11B.2 设计目标

#	目标	对应缺口
G1	借条安全：旧借条、默认借条、重入归还都被拒绝	①
G2	池化 Actor 每次出池都完整走一遍 BeginPlay → Tick → EndPlay，闲置时不在 World 名单里	②
G3	状态复位有固定的、由框架调用的位置	③
G4	业务代码只表达“这个 Actor 该死了”，不需要知道它是不是来自对象池	④
G5	在 Tick、物理回调、计时器回调里调用都安全（重入、遍历中修改）	第 3、10 章的教训
G6	改动尽量小；底层数据结构可以脱离 Unity 做单元测试	第 20 章

11B.3 池放在哪一层：三种方案

方案	谁拥有池	优点	问题
A · GameMode 自己持有（原代码的思路）	SurvivalGameMode	直观	每个 GameMode 重写一遍；Pawn / Controller 死亡时得反过来找 GameMode（RecycleObject + _activeWrappers 字典）；规则对象兼做工厂，职责混乱
B · 全局 PoolManager 单例	DontDestroyOnLoad 的单例	任何地方都能用	闲置对象是场景物体，切场景要么被销毁、要么全部跨场景常驻；World 是场景级的，池里的 Actor 会引用已销毁的 World；单例的所有问题（第 6 章）
C • World 拥有机制，GameMode 配置策略（推荐）	World（场景级）	生命周期和场景一致；World 本来就管 Actor 的注册注销，变成“唯一的出生和死亡入口”顺理成章；对应 UE 的 `UWorld::SpawnActor / DestroyActor`	World 职责变多；需要改 AActor

World.cs 里作者自己写过一句注释：“GameMode 注册，以后可以改成 World 直接 Spawn 一个对应的 GameMode 实例”——方案 C 就是把这个想法推广到所有 Actor：**World 提供机制（怎么生成、怎么回收），GameMode 决定策略（哪些类型池化、预热多少、闲置上限多少）。**

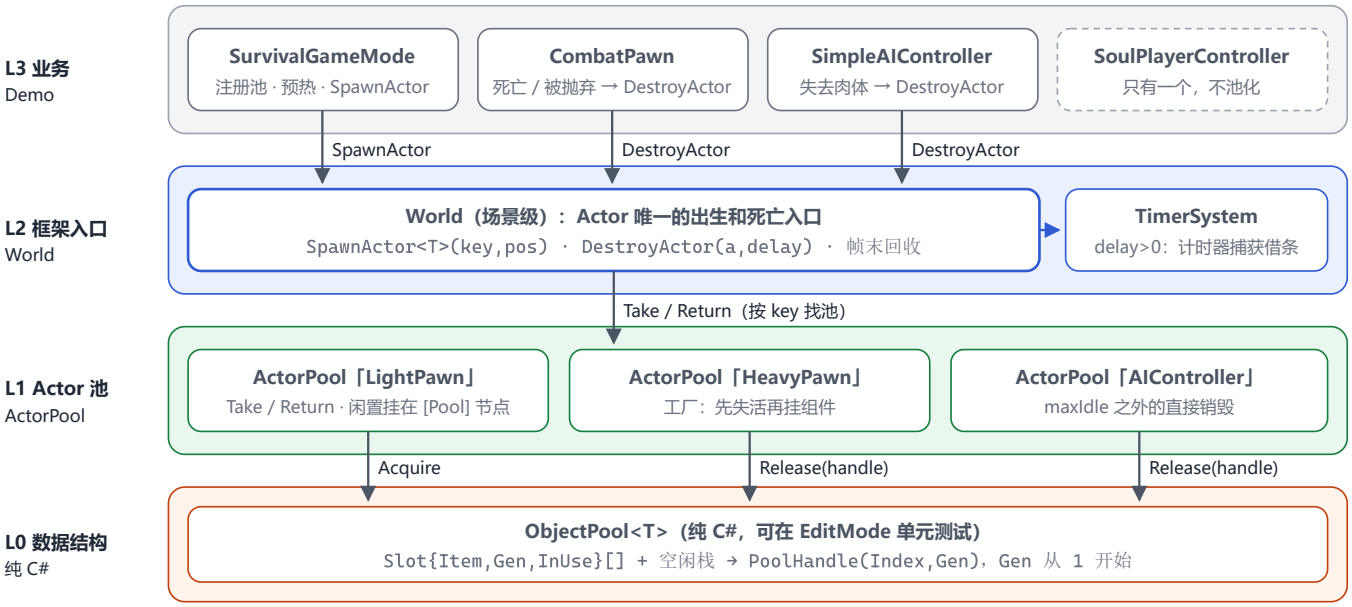


图 11-1　四层结构。业务代码只认识 L2 的两个入口；L1 负责“GameObject 怎么出池入池”；L0 只负责“槽位和借条”，不依赖 Unity，可以单独测试。原代码把 L0～L3 混在 SurvivalGameMode 和 GameObjectWrapper 里，这是它接不进去的根本原因

11B.4 L0：带 generation 的 ObjectPool<T>

在原 ObjectPool 的基础上改四件事：借条带代数；工厂由调用方提供（不再要求 `new()` 和 IPoolable，从而可以池化 MonoBehaviour）；归还时先失效再回调；闲置数有上限。思路和第 9 章 TimerSystem 的 Slot Map 一致，只是不需要 dense 数组（池子不需要每帧遍历）。

新 · Framework/ObjectPool.cs（骨架，未在 Unity 中编译）

```
public readonly struct PoolHandle : IEquatable<PoolHandle> {
    public readonly int Index, Generation;                  // Generation 从 1 开始：default(0,
    public PoolHandle(int index, int generation) { Index = index; Generation = generation; }
    public bool Equals(PoolHandle o) => Index == o.Index && Generation == o.Generation;
    public override int GetHashCode() => HashCode.Combine(Index, Generation);
}

public sealed class ObjectPool<T> where T : class {
    private struct Slot { public T Item; public int Generation; public bool InUse; }
    private Slot[] _slots = new Slot[8];
    private int _slotCount;
    private readonly Stack<int> _free = new Stack<int>();    // 仍是 LIFO：刚还的最先借出（缓存友
    private int _idleItems;
    private readonly Func<T> _create; private readonly Action<T> _onRelease, _onDestroy; priva

    public ObjectPool(Func<T> create, Action<T> onRelease = null, Action<T> onDestroy = null,
        _create = create ?? throw new ArgumentNullException(nameof(create));
        _onRelease = onRelease; _onDestroy = onDestroy; _maxIdle = maxIdle;
        for (int k = 0; k < prewarm; k++) { int i = NewSlot(); T item = _create(); _slots[i].I
    }
    private int NewSlot() {
        if (_slotCount == _slots.Length) Array.Resize(ref _slots, _slots.Length * 2);
        _slots[_slotCount].Generation = 1;
        return _slotCount++;
    }
    public PoolHandle Acquire(out T item) {
        int i = _free.Count > 0 ? _free.Pop() : NewSlot();
        if (_slots[i].Item ≠ null) _idleItems--;
        else { T created = _create(); _slots[i].Item = created; }   // 先求值再写回：见下方说明
        _slots[i].InUse = true;
        item = _slots[i].Item;
        return new PoolHandle(i, _slots[i].Generation);
    }
    public bool IsValid(PoolHandle h) =>
        (uint)h.Index < (uint)_slotCount && _slots[h.Index].InUse && _slots[h.Index].Generatic
    public bool TryGet(PoolHandle h, out T item) { bool ok = IsValid(h); item = ok ? _slots[h.
    public bool Release(PoolHandle h) {
        if (!IsValid(h)) return false;                          // 旧借条 / 默认借条 / 重复归还 → 拒
        T item = _slots[h.Index].Item;
        _slots[h.Index].InUse = false;
        if (++_slots[h.Index].Generation == 0) _slots[h.Index].Generation = 1;   // 回绕时跳过
        _onRelease?.Invoke(item);                               // 回调在失效之后：回调里重入 Releas
        if (_idleItems ≥ _maxIdle) { _slots[h.Index].Item = null; _onDestroy?.Invoke(item); }
```

```
            else _idleItems++;
            _free.Push(h.Index);
            return true;
        }
    }
```

原 ObjectPool.cs	新设计	为什么
`where T : class, IPoolable, new()`	`Func<T> create` + 回调	MonoBehaviour 不能 `new`；回调非侵入，T 不必实现接口（与 UnityEngine.Pool 的设计一致）
`PoolObject { elem, index }`	`PoolHandle { Index, Generation }`，对象由池按句柄取出	借条里不放对象引用，也就不会出现“elem 和 index 来自不同租借”的错配（第 11 章 G9）
只检查 `IsAvailable`	检查 InUse + Generation	拦截旧拷贝和默认借条（H1-a、H1-b）
先 OnRecycle，再标记可用	先失效（代数 +1），再回调	回调里重入归还不会让槽位入栈两次（H1-c）
容量翻倍时预先 new 满	按需创建 + 可选预热	扩容那一帧不再集中分配；预热数量由业务决定
只增不减	`maxIdle`：超出的直接销毁	峰值过后内存能还回去

两个容易写错的细节（都和第 10 章的“ref 跨扩容”是同一类问题）

① `_slots[i].Item = _create();` **不安全。**C# 对数组元素赋值时，会**先**读取 `_slots` 字段和下标 i，**再**执行右边的 `_create()`，最后写回。如果工厂里间接触发了同一个池的扩容（`Array.Resize` 替换了 `_slots`），结果会写进**旧数组**，新数组里这一格仍然是 null。所以先用局部变量接住，再写回。

② **不要在回调前后持有 `ref Slot`。**Release 里调用 `_onRelease` 之后，如果用之前取到的 `ref` 去写 `Item = null`，同样可能写进旧数组。上面的骨架每次都用 `_slots[h.Index]` 重新索引。

【实测 H1】同一组操作，原池与新池的对比：

操作	原 ObjectPool	新 ObjectPool
a) 借 a → 拷贝借条 → 归还 a → 借 b（LIFO 拿到同一槽）→ 用旧拷贝归还	**成功：b 正在用的对象被回收**	**拒绝：a=(0,1)、b=(0,2)，代数对不上**
b) 槽 0 在用时归还 default 借条	**通过检查，对 null 调 OnRecycle（C# 中 NullReferenceException）**	**拒绝：代数从 1 开始**
c) OnRecycle 回调里再归还同一个对象	**槽位入栈两次，接下来两次借出拿到同一对象**	**内层归还被拒绝**
d) 同一借条归还两次	**拒绝（IsAvailable）**	**拒绝**

11B.5 Actor 的池化生命周期

11B.5.1 先确认三条 Unity 规则

【Unity 规则】	对设计的影响
挂在**未激活** GameObject 上的脚本，Awake 要等到 GameObject 第一次被激活时才调用；OnEnable / OnDisable 随 SetActive 触发	工厂必须**先 SetActive(false) 再 AddComponent**，否则预热出来的对象会立刻 Awake、Start，并在 Start 里被 World 当成活 Actor 注册（原 AActor.Start 无条件注册）
Start 每个组件一生只调用一次	不能靠 Start 做“每次出池”的初始化，也不能靠它注册——池化 Actor 的注册要由 SpawnActor 显式完成
SetActive(false) 不会触发 OnDestroy	不能靠 OnDestroy 注销——回收时 World 要显式注销

11B.5.2 AActor 新增的成员

新 · GamePlayCore/AActor.cs（在原文件上增加的部分）

```
public enum EEndPlayReason { Destroyed, ReturnedToPool, LevelUnload }

public class AActor : MonoBehaviour {
    public bool HasBegunPlay { get; private set; }
    public bool IsTickEnabled { get; set; } = true;

    // ———————— 新增：销毁与池化 ————————
    public bool IsPendingKill { get; private set; }          // 已被 DestroyActor，等待帧末处理
    public PoolHandle PoolHandle { get; internal set; }      // default = 不来自对象池（或已归还
    internal ActorPool OwnerPool { get; set; }                // null = 普通 Actor，帧末走 Dest

    protected virtual void Start() {
        if (OwnerPool == null) World.RegisterAActor(this);    // ★ 池化 Actor 只由 SpawnActor
    }
    protected virtual void OnDestroy() { if (World.HasInstance) World.UnregisterAActor(this);

    public virtual void BeginPlay() { HasBegunPlay = true; }
    public virtual void EndPlay(EEndPlayReason reason) { HasBegunPlay = false; }   // ★ 与 Be
    protected internal virtual void OnTakeFromPool() { }      // 出池：SetActive(true) 之后、Be
    protected internal virtual void OnReturnToPool() { }      // 入池：EndPlay 之后、SetActive(

    internal void MarkPendingKill()  => IsPendingKill = true;
    internal void ClearPendingKill() => IsPendingKill = false;
}

// APawn：离开游戏前先解除附身（Pawn 死了，灵魂必须知道）
public override void EndPlay(EEndPlayReason r) { if (Controller != null) Controller.UnPossess(
// AController：灵魂离开游戏，肉体变回无主
public override void EndPlay(EEndPlayReason r) { if (ControlledPawn != null) UnPossess(); base
```

三个钩子分工明确，面试时最好能一句话说清：

钩子	何时调用	负责什么	SoulDecay 里放什么
`EndPlay(reason)`	销毁和入池**都会**调用	“离开游戏”：解除 Possess、停止计时器、注销事件——和是否池化无关的清理	CombatPawn 停寿命计时器；APawn / AController 解除附身
`OnReturnToPool()`	只在入池时	“清理残留”：复位字段、去掉对外引用，让闲置对象不钉住别的东西	isDying、碰撞体、名字、颜色、阵营
`OnTakeFromPool()`	只在出池时	“准备上场”：与位置相关、必须在激活后做的设置	清刚体速度
`BeginPlay()`	每次出池后的下一次 World.Update	游戏逻辑的初始化（和非池化 Actor 完全一样）	SimpleAIController 重新随机 Perlin 偏移

复位放在入池时还是出池时

两种都有人用。这里的建议是：**清引用放在入池时**（否则闲置对象会钉住它引用的东西，第 10 章 E13、第 8 章 G5 是同类问题），**与新位置、新主人相关的设置放在出池时。**只在一处复位也可以，但要保证覆盖所有字段——漏掉一个，就是“上一任的残留”。

11B.5.3 状态机

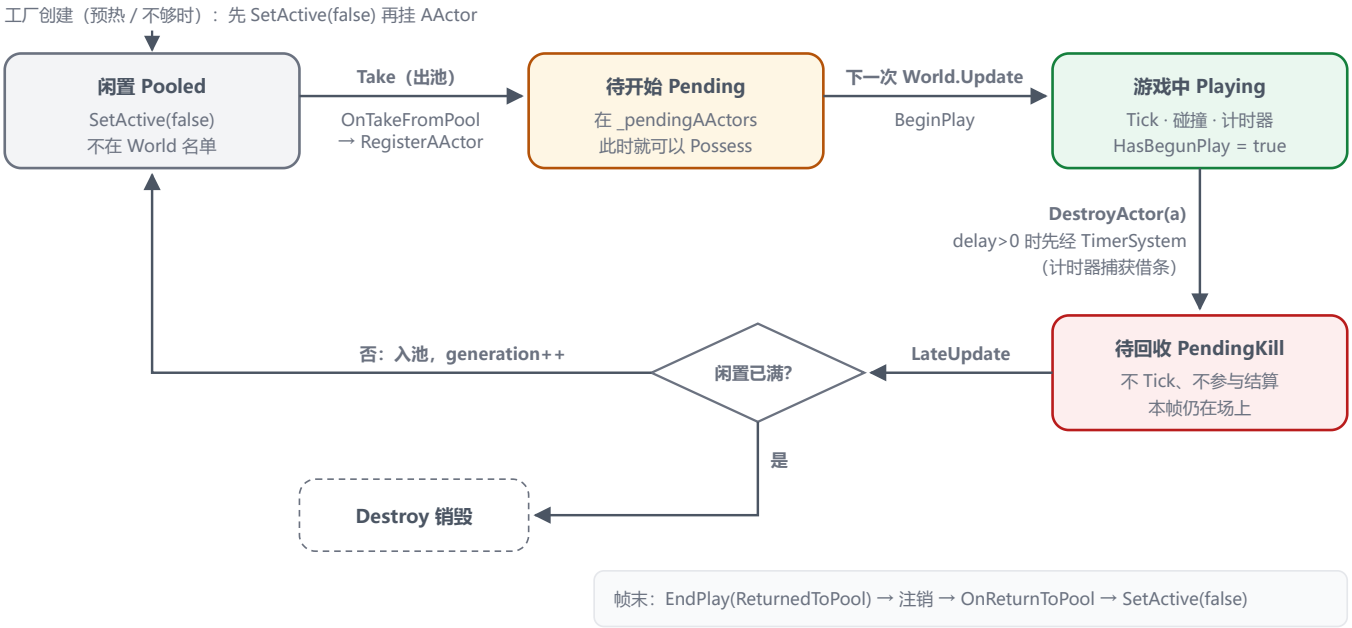


图 11-2 池化 Actor 的状态机。与原方案（SetActive + 开关 Tick）相比，多出了“待开始”和“待回收”两个状态：前者保证每次出池都重新 BeginPlay，后者保证“死亡”在帧内只是一个标记、真正的回收统一在帧末执行

【实测 H4】同一具肉体被复用 3 次：

轮次	原方案：出池时 isDying / 碰撞体	原方案：累计 BeginPlay / 闲置时在 World 名单	新设计：出池
1	false / 开	1 / 是	false / 开
2	**true / 关**	**1 / 是**	**false / 开**
3	**true / 关**	**1 / 是**	**false / 开**

原方案第二轮开始，出池的肉体已经“死了”：`isDying` 为 true，所以被玩家抛弃时 `if (!isDying) Destroy(gameObject, 3f)` 不执行，尸体永远留在场上；碰撞体是关的，不能碾压也不能被碾压，而且射线点不中它，无法被夺舍。

11B.6 World：唯一的出生和死亡入口

新 · GamePlayCore/World.cs（增加的部分）

```
private readonly Dictionary<string, ActorPool> _pools = new Dictionary<string, ActorPool>();
private readonly List<AActor> _pendingKill = new List<AActor>(32);

public static void RegisterPool(string key, Func<AActor> factory, int prewarm = 0, int maxIdle
    => Instance._pools[key] = new ActorPool(key, factory, prewarm, maxIdle);

public static T SpawnActor<T>(string poolKey, Vector3 pos, Quaternion rot) where T : AActor {
    var actor = (T)Instance._pools[poolKey].Take(pos, rot);    // 出池：设位置 → SetActive(true)
    RegisterAActor(actor);                                     // 进入 pending：下一次 World.Upd
    return actor;
}

public static void DestroyActor(AActor actor, float delay = 0f) {
    if (actor == null || actor.IsPendingKill) return;          // ★ 幂等：重复调用无害
    if (!HasInstance) { Destroy(actor.gameObject); return; }   // 场景卸载 / 退出中
    if (delay > 0f) { Instance.ScheduleDestroy(actor, delay); return; }
    actor.MarkPendingKill();                                   // 立刻"逻辑死亡"
    Instance._pendingKill.Add(actor);                          // 真正的回收推迟到帧末
}

private void ScheduleDestroy(AActor actor, float delay) {
    PoolHandle h = actor.PoolHandle; ActorPool pool = actor.OwnerPool;    // ★ 捕获"此刻的借条"
    TimerSystem.Instance.CreateTimer(delay, () => {
        bool stillSame = pool != null ? pool.IsAlive(h) : actor != null; // 池化对象比借条，不比
        if (stillSame) DestroyActor(actor);
    });
}

private void LateUpdate() {                                    // 所有 Update（含计时器、事件回调
    for (int i = 0; i < _pendingKill.Count; i++) {             // 用下标：EndPlay 里新标记的也在
        AActor a = _pendingKill[i];
        if (a == null) continue;                               // 已被 Unity 销毁（假 null）
        a.EndPlay(a.OwnerPool != null ? EEndPlayReason.ReturnedToPool : EEndPlayReason.Destroy
        UnregisterAActor(a);
        if (a.OwnerPool != null) a.OwnerPool.Return(a);        // 入池（或超过 maxIdle 时销毁）
        else Destroy(a.gameObject);
    }
    _pendingKill.Clear();
}
// World.Update 里两处小改：处理 pending 时跳过 IsPendingKill 的；Tick 条件加上 && !IsPendingKill
```

11B.6.1 为什么回收要延迟到帧末

原代码之所以“在 Tick 里 Destroy 没出事”，是因为 Unity 的 Destroy 本身就是延迟到帧末的——OnDestroy（进而 List.Remove）不会在 World 的遍历中途发生。池化之后如果立即 SetActive(false) + 注销，就等于在倒序遍历中途修改 `_actors`，第 3 章的问题全部回来。【实测 H2】：

场景（Tick 中回收）	立即回收	标记 PendingKill，帧末回收
A3 回收 A1（玩家夺舍 → 被踢出的 AI 灵魂回收）	**A3 被 Tick 2 次**	**每个 Actor 至多 1 次；A1 已“逻辑死亡”，本帧不再 Tick**
A4 一次回收 A0、A1、A2	**越界，本帧 World.Update 中止，A3 没有被 Tick**	**正常结束，A3、A4 各 1 次**
A2 回收自己	**正常**	**正常**

同样的道理也适用于另外两个调用时机：物理回调（OnCollisionEnter 里碾碎对方）和计时器回调（寿命到期 → Die）。在帧末统一处理，这三处都不用关心“我现在是不是在别人的遍历里”。这和 Unity 的 Destroy、UE 的“标记待销毁、之后再真正清理”是同一个思路。

> **IsPendingKill 顺带修好了第 15 章的“重复计分”**
> 第 15 章指出：同一物理步里两具大肉体撞同一具小肉体，Die 会被调用两次、加两次分。现在 Die 和 OnCollisionEnter 都先检查 `IsPendingKill`，第二次结算直接返回；DestroyActor 本身也是幂等的。

11B.7 延迟回收与 ABA：尸体为什么要用借条

原代码里玩家抛弃的旧肉体用 `Destroy(gameObject, 3.0f)` 延迟销毁。Unity 的延迟销毁没有 ABA 问题，因为被销毁的对象永远不会回来。**池化之后，“3 秒后回收这个对象”就变成了“3 秒后回收这个槽位上的东西”**——如果这 3 秒内它被提前回收并借给了别人，到期时回收的就是别人。



图 11-3 实测 H3 的时间线。原 Demo 里还没有“提前回收尸体”的路径，所以今天不会触发；但只要以后加一个“清场”或“尸体可以被吃掉”，而池又是 LIFO（刚还回来的对象最先被借出），这个 bug 几乎必然出现。借条让这类“延迟操作”天然安全——这正是第 9 章 generation 的价值，换了一个地方再次出现

同样需要用借条（或 generation）的地方，在这个项目里还有：UI 持有的寿命 `TimerHandle`（已经有 generation，所以安全）；以后如果 GameMode 用列表记录“场上所有 AI”，列表里也应该存句柄而不是引用。

11B.8 ActorPool：GameObject 怎么出池、入池

新 · Framework/ActorPool.cs（骨架）

```
public sealed class ActorPool {
    private readonly ObjectPool<AActor> _pool;
    private readonly Transform _root;                          // 闲置对象挂在 “[Pool] key” 下，
    public string Key { get; }

    public ActorPool(string key, Func<AActor> factory, int prewarm, int maxIdle) {
        Key = key;
        _root = new GameObject($"[Pool] {key}").transform;     // 场景物体：随场景卸载一起销毁
        _pool = new ObjectPool<AActor>(
            create:    () => { AActor a = factory(); a.transform.SetParent(_root, false); retu
            onRelease: a  => { a.OnReturnToPool(); a.gameObject.SetActive(false); a.transform.
            onDestroy: a  => { if (a) Object.Destroy(a.gameObject); },
            prewarm: prewarm, maxIdle: maxIdle);
    }

    internal AActor Take(Vector3 pos, Quaternion rot) {
        PoolHandle h = _pool.Acquire(out AActor a);
        a.PoolHandle = h; a.OwnerPool = this; a.ClearPendingKill();
        a.transform.SetParent(null, false);
        a.transform.SetPositionAndRotation(pos, rot);          // 先摆好位置再激活，避免物理在旧
        a.gameObject.SetActive(true);                          // 第一次：Awake → OnEnable（St
        a.OnTakeFromPool();
        return a;
    }
    internal void Return(AActor a) { PoolHandle h = a.PoolHandle; a.PoolHandle = default; _poo
    internal bool IsAlive(PoolHandle h) => _pool.IsValid(h);
}
```

ActorPool 只做三件和 Unity 有关的事：父节点、位置、激活状态。复位逻辑通过 `OnReturnToPool` 交给 Actor 自己——ActorPool 不知道 CombatPawn 有哪些字段。

11B.9 在 SoulDecay 中落地：逐文件改动

11B.9.1 SurvivalGameMode：配置池，生成走 SpawnActor

新 · SurvivalGameMode.cs（节选）

```
const string LightKey = "LightPawn", HeavyKey = "HeavyPawn", AIKey = "AIController";
[System.Serializable] public struct PawnConfig { public float Scale, MoveSpeed, Lifespan; }
public PawnConfig LightConfig = new PawnConfig { Scale = 0.8f, MoveSpeed = 8f, Lifespan = 5f }
public PawnConfig HeavyConfig = new PawnConfig { Scale = 1.5f, MoveSpeed = 3f, Lifespan = 30f

public override void BeginPlay() {
    base.BeginPlay();
    World.RegisterPool(LightKey, () ⇒ BuildPawn(false), prewarm: 8, maxIdle: 16);   // 预热数
    World.RegisterPool(HeavyKey, () ⇒ BuildPawn(true),  prewarm: 6, maxIdle: 16);
    World.RegisterPool(AIKey,    BuildAIController,     prewarm: 8, maxIdle: 16);
    _spawnAIAction = SpawnRandomAI;
    StartMatch();
    var playerBody = World.SpawnActor<CombatPawn>(LightKey, Vector3.zero, Quaternion.identity)
    var playerSoul = new GameObject("Player_Controller").AddComponent<SoulPlayerController>();
    playerSoul.FactionId = EFaction.Player;
    playerSoul.Possess(playerBody);
    TimerSystem.Instance.CreateTimer(3.0f, _spawnAIAction, timerName: "AISpawner", isLoop: tru
}

private void SpawnRandomAI() {
    if (GameState.MatchState == AGameState.EMatchState.WaitingPostMatch) return;
    Vector3 pos = new Vector3(Random.Range(-10f, 10f), 0, Random.Range(-10f, 10f));
    bool isHeavy = Random.value > 0.5f;
    var aiBody = World.SpawnActor<CombatPawn>(isHeavy ? HeavyKey : LightKey, pos, Quaternion.i
    var aiSoul = World.SpawnActor<SimpleAIController>(AIKey, Vector3.zero, Quaternion.identity
    aiSoul.FactionId = Random.value > 0.5f ? EFaction.RedAI : EFaction.BlueAI;
    aiSoul.Possess(aiBody);
}

private AActor BuildPawn(bool isHeavy) {                        // 工厂：只在预热或池空时调用
    GameObject go = GameObject.CreatePrimitive(isHeavy ? PrimitiveType.Cube : PrimitiveType.Sp
    go.SetActive(false);                                        // ★ 先失活：Awake / Start 推迟
    go.name = isHeavy ? HeavyKey : LightKey;
    var rb = go.AddComponent<Rigidbody>();
    rb.constraints = RigidbodyConstraints.FreezeRotation | RigidbodyConstraints.FreezePositior
    var p = go.AddComponent<CombatPawn>();
    PawnConfig c = isHeavy ? HeavyConfig : LightConfig;         // 这些值整个生命期都不变，创建时
    go.transform.localScale = Vector3.one * c.Scale; p.MoveSpeed = c.MoveSpeed; p.MaxLifespan
    return p;
}
private AActor BuildAIController() {
    var go = new GameObject(AIKey); go.SetActive(false);
    return go.AddComponent<SimpleAIController>();
```

```
}
// 删除：三个 ObjectPool 字段、_activeWrappers、GetOrCreatePawn、GetOrCreateAIController、Recycl
```

11B.9.2 CombatPawn：死亡出口换成 DestroyActor，补复位

位置	原代码	改为	为什么
字段	`private TimerHandle _decayTimer;`	`= TimerHandle.Invalid;`	顺手修掉第 9 章的 P0（默认句柄会命中首个计时器）
Awake	只取 Renderer	再缓存 Collider、Rigidbody 和 `_onExploded = OnBodyExploded`	对象常驻之后，缓存委托就是一次性的成本（第 10 章：方法组每次转换都会分配）
OnPossess	`CreateTimer(MaxLifespan, OnBodyExploded)`	`CreateTimer(MaxLifespan, _onExploded)`	同上
OnUnPossess	`if (!isDying) Destroy(gameObject, 3.0f);`	`if (!isDying) World.DestroyActor(this, 3.0f);`	延迟回收改为捕获借条的计时器（11B.7）
OnCollisionEnter	只判断 Controller 和体型	开头 `if (IsPendingKill || isDying) return;`，对方同样判断	防止同帧重复结算
Die	`isDying = true; … Destroy(gameObject);`	`if (isDying) return; isDying = true; … World.DestroyActor(this);`	幂等；出口统一
新增 EndPlay	—	停止寿命计时器，然后 `base.EndPlay`（APawn 负责解除附身）	销毁和入池共用的清理
新增 OnReturnToPool	—	`isDying = false;` 碰撞体 enabled；名字恢复；颜色恢复为灰色；`_decayTimer = Invalid`	H4 里原方案缺的全部状态
新增 OnTakeFromPool	—	刚体 `velocity / angularVelocity` 清零	原来在 GameObjectWrapper 里（Unity 6 中属性名改为

linearVelocity)

注意 EndPlay 与 OnUnPossess 的交互：入池时 APawn.EndPlay 会调用 `Controller.UnPossess()`，进而触发 CombatPawn.OnUnPossess——如果离开的是玩家，它会再调用 `World.DestroyActor(this, 3f)`。此时 `IsPendingKill` 已经是 true，这次调用直接返回。幂等让这条回调链不需要任何特判。

> **顺手可以改的：颜色用 MaterialPropertyBlock**
>
> `_renderer.material.color` 第一次访问时会为这个 Renderer 复制一份材质实例。池化后对象常驻，复制只发生一次，反而比原来“每具新肉体复制一份”省；用 `MaterialPropertyBlock` 改颜色可以完全不产生材质实例（项目用的是内置渲染管线；若换成 URP / HDRP，要注意 MaterialPropertyBlock 会让该 Renderer 退出 SRP Batcher）。

11B.9.3 SimpleAIController / GameObjectWrapper

文件	改动
SimpleAIController.cs	OnUnPossess 里 `Destroy(gameObject)` → `World.DestroyActor(this)`；新增 OnReturnToPool：`FactionId = EFaction.Neutral`。BeginPlay 不用改——每次出池都会重新执行，Perlin 偏移自动重新随机
GameObjectWrapper.cs、IPoolable	**删除。**它们的职责分别被 ActorPool（SetActive）、World（注册注销，替代 IsTickEnabled 开关）和 Actor 的复位钩子接管
SoulPlayerController / UI	不改。UI 只持有 TimerHandle，肉体回收时寿命计时器已停止，句柄自动失效显示 DIE

11B.9.4 EndPlay 清理清单

池化最常见的 bug 是“回收时漏清了某个对外关系”。按资源列一张表，每加一种资源就补一行：

对外关系	谁清理	漏清的后果
Possess 双向引用	APawn / AController.EndPlay	闲置的肉体仍被某个灵魂控制，灵魂每帧 Tick 去移动一个看不见的对象
寿命计时器	CombatPawn.EndPlay	闲置对象的计时器到期，回调 Die → 回收一个已经被别人借走的对象
延迟回收计时器	借条自动失效（11B.7）	H3：回收别人
事件订阅	订阅方在 EndPlay 中 UnRegister（Demo 中的 Actor 都没有订阅事件）	闲置对象继续收到事件；叠加第 7 章的快照语义
业务集合里的引用	存句柄，或在 EndPlay 时从集合移除	集合里出现“已经变成别人”的对象
物理与表现	OnTakeFromPool / OnReturnToPool	出池带着上一任的速度、颜色、名字

11B.10 两条调用链：夺舍与死亡

玩家夺舍红队 AI 的肉体 Target（第 N 帧）

```
World.Update（倒序 Tick）
  SoulPlayerController.Tick → ExecuteSoulTransfer → Possess(Target)
    ① UnPossess(Old)  → Old.OnUnPossess：变黑、关碰撞 → World.DestroyActor(Old, 3s)
                        → 建计时器，捕获 Old 当前借条 (i,g)
    ② AI.UnPossess()  → SimpleAIController.OnUnPossess → World.DestroyActor(AI)
                        → AI.IsPendingKill = true，加入 _pendingKill（不碰 _actors 列表）
    ③ Target.PossessedBy(玩家) → 新寿命计时器 + 事件
  ……倒序遍历继续：轮到 AI 时 IsPendingKill → 跳过 Tick
TimerSystem.Update / EventSystem.Update（照常）
World.LateUpdate
  AI.EndPlay(ReturnedToPool)：ControlledPawn 已为空 → 无事
  UnregisterAActor(AI) → ActorPool「AIController」.Return → generation++ → OnReturnToPool（阵营
第 N 帧 + 3 秒：计时器到期 → IsAlive((i,g))? 是 → DestroyActor(Old) → 当帧 LateUpdate 入池
```

寿命耗尽（TimerSystem 完成回调里）

```
TimerSystem 阶段二B → OnBodyExploded → Die(isExplosion: true)
  isDying = true → EndMatch()（停 AISpawner）→ Controller.UnPossess() → StopTimer(自己)
  → World.DestroyActor(this)：标记 PendingKill
回调返回 → TimerSystem 二次校验：句柄已失效，跳过（第 10 章）
World.LateUpdate → EndPlay → 注销 → Return → OnReturnToPool（isDying=false、碰撞体打开、恢复名字
```

11B.11 收益：一局 10 分钟要创建多少次

【实测 H5，C++ 事件模拟】规则按源码：每 3 秒刷一只 AI（一半重型）；假设每具 AI 肉体存活 8～60 秒（均匀分布）后被碾碎；玩家每 4 秒夺舍一次（被踢出的 AI 灵魂立即回收，旧肉体 3 秒后回收）。存活时间和夺舍频率是假设，结论看量级：

种类	借出次数	归还次数	峰值同时存在	有池时的创建次数
轻型肉体	117	115	8	8
重型肉体	84	78	6	6
AI 灵魂	200	194	7	7

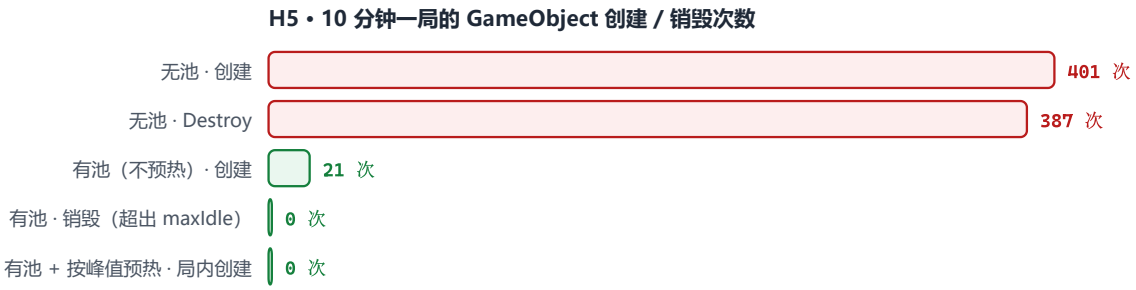


图 11-4 实测 H5（demos/pool_design_output.txt）。无池时每次刷怪都是 CreatePrimitive + AddComponent×2（肉体）和 new GameObject + AddComponent（灵魂），每次死亡都是 Destroy；有池时创建次数等于各池峰值之和，按峰值预热后局内为 0。这只是次数——每次创建在 Unity 里的真实耗时请用 Profiler 测

预热和上限怎么定：先不预热跑几局，记录每个池的峰值（在 ActorPool 里加一个统计字段即可）；预热取峰值，maxIdle 取峰值的 1.5～2 倍。**预热放在加载阶段**（GameMode.BeginPlay 发生在场景加载后的第一帧），如果预热数量大，可以分摊到加载界面的多帧里。

面试注意：收益要说得克制

SoulDecay 每 3 秒才刷一只，不池化也不会卡；池化的价值在“规则和框架已经支持了”，以后换成每秒几十发的子弹、伤害数字时可以直接复用。如果面试官问“这个 Demo 需要对象池吗”，诚实的回答是“不需要，我是借它验证框架对池化的支持”。

11B.12 还有哪些替代做法

做法	说明	取舍
OnEnable / OnDisable 自动注册注销	AActor 在 OnEnable 里注册、OnDisable 里 EndPlay + 注销，SetActive 自动驱动生命周期	改动更少；但会改变**所有** Actor 的注册时机（OnEnable 在创建调用内部就执行，比 Start 早），影响第 2 章“先 Possess 后 BeginPlay”的时序；场景卸载时 OnDisable 也会触发 EndPlay，需要区分原因
用 UnityEngine.Pool.ObjectPool 作为 L0	createFunc / actionOnRelease / maxSize 都现成	少写代码；但它直接返回对象、没有句柄。可以在 AActor 上加一个每次出池自增的 `SpawnSerial`，对外用 (对象, 序号) 作为句柄——这正是 UE 弱对象指针的思路
预制体为键	RegisterPool(prefab) 代替字符串 key	真实项目里 Actor 来自 Prefab，用 Prefab 引用做键更安全；Demo 是代码生成的原始体，用字符串更直接

11B.13 分四步迁移（每一步都能运行）

1. **替换 L0**：新 ObjectPool + PoolHandle，补 EditMode 单元测试（清单见 11B.14）。此时不接入任何东西。
2. **统一死亡出口**：AActor / APawn / AController 加 EndPlay 和 IsPendingKill，World 加 DestroyActor + LateUpdate；把项目里所有对 Actor 的 `Destroy` 换成 `World.DestroyActor`。这一步还没有池化，但已经修好了 Die 不幂等、Tick 中销毁的问题。

3. **接入池**：World 加 RegisterPool / SpawnActor 和 ActorPool；SurvivalGameMode 改为 SpawnActor（先不预热）；CombatPawn / SimpleAIController 实现 OnReturnToPool / OnTakeFromPool。
4. **调参与清理**：记录峰值后设定预热和 maxIdle；用 Profiler 确认 GC Alloc 和创建次数；删除 GameObjectWrapper 和 SurvivalGameMode 里的死代码。

11B.14 测试

层	测试方式	用例
L0 ObjectPool	EditMode（NUnit，纯 C#）	归还后句柄失效；旧拷贝归还被拒绝；default 句柄无效；LIFO 复用时代数 +1；onRelease 里重入归还被拒绝；超过 maxIdle 调 onDestroy；预热数量正确；工厂里触发扩容时结果仍正确
L2 World	PlayMode	SpawnActor 后下一帧 BeginPlay 恰好一次；Tick 中 DestroyActor 自己 / 更低下标 / 多个，每个 Actor 每帧至多 Tick 一次；DestroyActor 两次只回收一次；延迟回收期间对象被提前回收并复用，到期不误伤
L3 Demo	PlayMode 冒烟	固定随机种子跑 60 秒：无异常日志；每个池的创建次数 ≤ 预热数；所有闲置对象不在 World 名单中；复用的肉体出池时 isDying=false、碰撞体打开

11B.15 与 UE 对照

本设计	UE【扩展知识】
`World.SpawnActor / DestroyActor`	`UWorld::SpawnActor` / `AActor::Destroy`（内部走 `UWorld::DestroyActor`）
`EndPlay(EEndPlayReason)`	`AActor::EndPlay(EEndPlayReason)`：Destroyed、LevelTransition、EndPlayInEditor、RemovedFromWorld、Quit——本设计照搬了“销毁和其他离开方式共用一个出口、用原因区分”的思路
IsPendingKill + 帧末处理	Actor 被 Destroy 后标记为待销毁，对象真正释放由之后的 GC 完成；访问时用 `IsValid()` 判断
Actor 池	UE 没有通用的 Actor 池（Spawn 和 Destroy 都很重），项目通常自己实现；引擎内置的池化见于 Niagara 特效组件（`ENCPoolMethod`）等
PoolHandle / SpawnSerial	`FWeakObjectPtr`：对象数组下标 + 序列号

11B.16 面试追问树

追问树 · 对象池落地

```
Q11B-1 如果要把你的对象池真正接进框架，你会怎么设计？
  ├── Q11B-2 池放在哪？为什么不是全局单例？为什么不是 GameMode？
  ├── Q11B-3 Start 只调用一次，出池后怎么重新 BeginPlay？
  │     └── Q11B-4 为什么不用 OnEnable / OnDisable 自动注册？
  ├── Q11B-5 在 Tick 里回收一个 Actor 会怎样？                ← 遍历中修改
  │     └── Q11B-6 那 Unity 的 Destroy 为什么没这个问题？
  ├── Q11B-7 玩家的旧肉体 3 秒后回收，这 3 秒里它被复用了怎么办？   ← ABA
  ├── Q11B-8 复位放在出池还是入池？漏了一个字段怎么发现？
  └── Q11B-9 业务代码要知道对象来自池吗？
Q11B-10 这个 Demo 真的需要对象池吗？收益怎么证明？
Q11B-11 多线程能从池里借对象吗？                              ← 发散
```

Q11B-1 / Q11B-2　怎么设计？池放在哪？

L1 • 30 秒　分四层：底层是带代数的通用池，借条是槽位加代数；上面是按类型的 Actor 池，负责激活和父节点；再上面是 World 的 SpawnActor 和 DestroyActor，作为 Actor 唯一的出生和死亡入口；业务层只调用这两个函数，不知道对象是不是来自池。

L2 • 正常回答　池归 World 所有、由 GameMode 在 BeginPlay 里配置。归 World 是因为生命周期一致——都是场景级的，闲置对象是场景物体，切场景一起销毁；而且 World 本来就管 Actor 的注册注销。不放全局单例，是因为闲置对象跨场景会引用已经销毁的 World；不放 GameMode，是因为 Pawn 和 Controller 死亡时还得反过来找 GameMode，原来的接入代码就卡在这里，RecycleObject 一直没有调用方。

L3 • 深入　关键改动在 Actor 生命周期：加 EndPlay，与 BeginPlay 成对，销毁和入池都调用，负责解除附身、停计时器；加入池和出池两个钩子负责复位；加 IsPendingKill，DestroyActor 只做标记，真正的回收在 World 的 LateUpdate 统一执行。

L4 • 面试官攻击　“World 越来越胖了。” → 同意，它现在同时是注册表、调度器和工厂。我会把工厂和池拆成 World 持有的一个子系统（类似 UE 的 WorldSubsystem），World 只暴露 SpawnActor 和 DestroyActor 两个入口。接口不变，内部可以拆。

Q11B-3 / Q11B-4　出池后怎么重新 BeginPlay？为什么不用 OnEnable?

L2　EndPlay 把 HasBegunPlay 清回 false，入池时 World 把它从名单里注销；出池时 SpawnActor 重新注册到 pending，下一次 World.Update 就会再次 BeginPlay，和新生成的 Actor 走同一条路径。池化 Actor 的注册只走 SpawnActor，AActor.Start 里遇到池化对象就不注册，避免它被重复注册或者在闲置时被注册。

L3　OnEnable / OnDisable 自动注册改动最少，但会改变所有 Actor 的注册时机：OnEnable 在 AddComponent 调用内部就执行，比 Start 早，会影响现在“先 Possess 后 BeginPlay”的时序；而且场景卸载时 OnDisable 也会触发，要另外区分。显式的入口更容易推理。

Q11B-5 / Q11B-6　在 Tick 里回收会怎样?

L2　如果立即 SetActive(false) 并从名单里移除，就是在倒序遍历中途修改列表：回收一个下标更小的 Actor，当前 Actor 会被 Tick 两次；一次回收多个，会越界、整帧中止。我用移植版测过这两种情况。所以 DestroyActor 只做标记：被标记的 Actor 本帧不再 Tick、不参与碰撞结算，在 LateUpdate 统一回收。

L3 原来用 Destroy 没出事，是因为 Unity 的 Destroy 本身就推迟到帧末，OnDestroy 不会在遍历中途发生。池化相当于把这个保护去掉了，所以要自己补回来。顺带 DestroyActor 变成幂等的，原来“两具大肉体同时撞一具小肉体会重复加分”的问题也一起修了。

Q11B-7 旧肉体 3 秒后回收，这 3 秒里它被复用了怎么办？（压力题）

L2 延迟回收的计时器不能捕获对象引用，要捕获安排那一刻的借条。到期时用借条问池子“它还是不是那一任”，代数不对就忽略。我测过：如果尸体在这 3 秒内被提前回收、又被刷怪借走变成一只红队 AI，只检查“对象是否活跃”的写法会把这只活着的 AI 回收掉；用借条的写法会忽略。

L3 池是 LIFO 的，刚还回来的对象最先被借出，所以一旦有提前回收的路径，这个问题几乎必然发生。这和计时器句柄是同一个问题：只要“延迟操作”指向一个可能被复用的东西，就需要代数。Unity 的 Destroy(obj, 3) 没这个问题，是因为被销毁的对象永远不会回来。

Q11B-8 / Q11B-9 复位放哪？业务要不要知道对象来自池？

L2 分两类：离开游戏的清理放 EndPlay，销毁和入池都调用；清残留引用放入池时，避免闲置对象钉住别的东西；与新位置相关的放出池时。漏字段的问题靠两件事兜：一张按资源列的清理清单；一个冒烟测试——同一对象复用多次后检查关键状态。业务层不需要知道：CombatPawn 只调用 DestroyActor，是销毁还是入池由 World 根据它有没有归属的池决定。

Q11B-10 这个 Demo 真的需要对象池吗？

L2 不需要。每 3 秒刷一只，不池化也不会卡。我按源码的刷怪规则、加上存活时间和夺舍频率的假设模拟了一局 10 分钟：不池化要创建约 400 次、销毁约 390 次；池化后只创建 21 次，按峰值预热后局内是 0。这说明的是次数，真实耗时要在 Profiler 里看。我做这件事的目的是让框架支持池化，以后接子弹、特效这种高频对象时可以直接用。

Q11B-11 多线程能从池里借对象吗？（发散）

L2 Actor 池不行：激活、改父节点、改位置都是 Unity API，只能在主线程。纯数据的池（比如网络消息包、寻路请求）可以做成线程安全的：最简单是加锁；高频时用每线程一个本地缓存、批

量和全局池交换，这就是 tcmalloc 这类分配器“线程本地缓存”的思路，也和第 12 章“每个工作线程一个 SPSC 队列”是一个道理。

常见错误回答 ①

错误：“对象池就是 SetActive(false) 存起来、SetActive(true) 拿出来。”

问题：只说了激活状态。没说生命周期（Start 不会再调用、OnDestroy 不会触发）、状态复位、对外引用清理、借条安全——原 GameObjectWrapper 就停在这一步，H4 显示第二轮起出池的肉体已经是“死的”。

常见错误回答 ②

错误：“回收时我立刻把它放回池里，这样最省事。”

问题：回收经常发生在别人的遍历中（World 的 Tick、物理回调、计时器回调），立刻修改名单会重复 Tick 或越界（H2）。要么延迟到帧末，要么由遍历方保证安全。

本章速记

- 半成品四个缺口：**借条无代数、生命周期没接上、状态无复位点、死亡出口写死 Destroy**；后三个是框架问题。
- 四层：**L0 ObjectPool<T>（槽位 + 代数，从 1 开始）→ L1 ActorPool → L2 World.SpawnActor / DestroyActor → L3 Demo**；World 提供机制，GameMode 配置策略。
- Actor 新增：**EndPlay（销毁与入池共用）、OnReturnToPool（清残留）、OnTakeFromPool（准备上场）、IsPendingKill**；池化 Actor 只由 SpawnActor 注册；工厂**先失活再挂组件**。
- DestroyActor **只标记、幂等**，LateUpdate 统一回收（H2）；延迟回收**捕获借条**（H3）。
- SoulDecay：生成换成 SpawnActor，三处 Destroy 换成 DestroyActor，补复位；一局 10 分钟创建 **401 次 → 21 次**（H5）。

▶ **自测 1：工厂里如果不先 SetActive(false) 就 AddComponent<CombatPawn>()，预热 8 个对象会发生什么？**

▶ **自测 2：入池时 APawn.EndPlay 调用 Controller.UnPossess()，CombatPawn.OnUnPossess 又调用 World.DestroyActor(this, 3f)。为什么不会出问题？**

▶ **自测 3：ScheduleDestroy 里的 lambda 会分配吗？要不要优化？**

CHAPTER 12

ThreadSafeRingBuffer：lock、生产者—消费者与无锁的距离

名字里有 ThreadSafe，面试官一定会确认“安全到什么程度”。它是一个 lock-based 的有界队列，不是 lock-free；压力测试只验证了计数，没有验证顺序。这一章从 lock 讲到 SPSC 无锁，再讲到 false sharing。

本章目标

- 准确定性：lock-based thread-safe bounded queue，不是 lock-free
- 说清 C# `lock` 是什么、临界区里做了什么、成本在哪
- 理解 contention：为什么 4 个生产者比 1 个慢
- 说清 SPSC 为什么可以无锁、MPSC / MPMC 难在哪
- 理解 false sharing，并用实测数据说明它的影响
- 指出现有压力测试的三个缺陷，以及 Unity 里使用工作线程的边界

12.1 当前实现与使用状态

ThreadSafeRingBuffer.cs（节选）

```
public class ThreadSafeRingBuffer<T> {
    private readonly object _lockObject = new object();
    public bool TryEnqueue(T e) {
        lock (_lockObject) {
            if (_size ≥ _capacity) return false;
            _buffer[_tail++] = e; _tail %= _capacity; _size++;
        }
        return true;
    }
    public bool TryDequeue(out T e) { lock (_lockObject) { ... 与 RingBuffer 相同 ... } }
    public bool TryPeek(out T e)    { lock (_lockObject) { ... } }
    public int Count                { get { lock (_lockObject) { return _size; } } }
}
```

使用方	状态
ThreadSafeEventSystem（第 7 章 7.9）	零调用方
RingBufferTests.cs	5 个单线程用例（T = int）
RingBufferStressTest.cs	1 个生产者线程 + 1 个消费者线程，各 10 万个
任何 Demo	没有用到

所以这一章讨论的是“这个类本身是否正确、能撑多大并发”，而不是“项目里的多线程是否正确”——项目里没有多线程业务。

12.2 定性：它是 lock-based，不是 lock-free

术语	含义	本类
thread-safe	多个线程同时调用，结果仍然符合单线程语义（不丢、不重、不乱、不崩）	**是（每个操作整体互斥）**
lock-based / blocking	靠互斥锁保证；拿不到锁的线程等待	**是**
lock-free	不使用锁；任意时刻**至少一个**线程能在有限步内完成操作（一个线程被挂起不会让其他线程都停下）	**否**
wait-free	**每个**线程都能在有限步内完成	**否**
bounded	有容量上限，满了拒绝	**是（Try 语义，不阻塞等待）**

面试注意：Try 语义 ≠ 无锁

TryEnqueue 满了立即返回 false、不等待，这叫“非阻塞的接口”；但它内部仍然用 lock，拿锁时会等。不要因为方法名是 Try 就说它是无锁队列。

12.3 lock 做了什么，成本在哪

- **语义**：C# 的 `lock(obj)` 是 `Monitor.Enter / Monitor.Exit` 包在 try/finally 里，保证临界区互斥，同时提供内存屏障——临界区内的写入对下一个拿到锁的线程可见。
- **无竞争时**：一次原子操作（CAS）就能拿到锁，几十纳秒级别。
- **有竞争时**：先自旋一小段，仍拿不到就进入内核等待、被唤醒——涉及上下文切换，微秒级。多个线程抢同一把锁时，大部分时间花在“等”和“交接锁”上，这就是 **contention（锁竞争）**。
- **临界区很短**：本类每次只做几次读写，临界区本身几乎不花时间，开销几乎全在“拿锁 / 放锁 / 缓存行在核之间来回搬”。

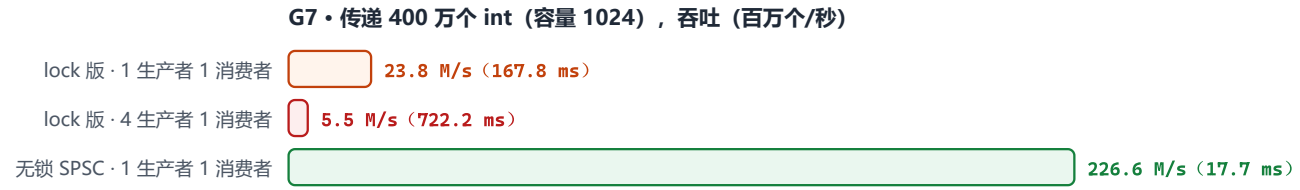


图 12-1 实测 G7（demos/infra_output.txt，C++ std::mutex 对应 C# lock 的语义，32 逻辑核 Windows 机器）。生产者从 1 个增加到 4 个，总吞吐反而降到约 1/4——这就是锁竞争；同样是一进一出，无锁 SPSC 约快 9.5 倍。三组都逐个校验了“每个生产者内部 FIFO”和校验和，全部通过

要强调的是：每秒 2380 万次对游戏的绝大多数跨线程通信来说已经绰绰有余（每帧几十到几百条消息）。lock 版本最大的问题不是吞吐，而是**延迟的不确定性**：主线程拿锁时如果正好赶上工作线程持锁被系统调度走，主线程就要等一个时间片。

12.4 为什么 SPSC 可以无锁

单生产者单消费者无锁环形队列（G7 中实现的版本，思路）

```
共享：   buf[capacity]（capacity 是 2 的幂）
        head：只有消费者写          tail：只有生产者写          ← 不再有两端共写的 size
生产者：t = tail（relaxed 读自己的）
        若 t - head(acquire 读对方的) == capacity → 满
        buf[t & mask] = x
        tail = t + 1（release 写）      ← 保证“写数据”先于“发布新 tail”被对方看到
消费者：h = head（relaxed）
        若 h == tail(acquire) → 空
        x = buf[h & mask]
        head = h + 1（release）         ← 保证“读完数据”先于“归还槽位”
head、tail 分别对齐到 64 字节，各占一条 cache line
```

- **每个变量只有一个写者**：这是能去掉锁的根本原因。第 8 章说过，size 是两端共写的，所以 RingBuffer 那套“用 size 区分满空”的设计无法直接改成无锁。
- **内存序**：仅仅“不加锁”是不够的。CPU 和编译器都可能重排读写，如果“写 buf”被重排到“写 tail”之后才对另一个核可见，消费者就会读到旧数据。release / acquire 配对建立“先行发生”关系。C# 里对应 `Volatile.Write / Volatile.Read`。
- **对齐**：head 和 tail 如果在同一条 cache line，两端每次写都会让对方的缓存失效——这就是 12.6 的 false sharing。

12.5 MPSC / MPMC 难在哪

模型	难点	常见做法
SPSC	只需正确的内存序	12.4 的写法；音频、日志线程
MPSC（多生产者单消费者）	多个生产者抢同一个 tail：要用 CAS 或 fetch_add 抢槽位，抢到槽位后还要“写完再标记可读”，消费者不能读到一个被抢到但没写完的槽	每个槽带序列号（Vyukov 有界 MPMC 队列的思路）；或每个生产者一个 SPSC，消费者轮询——游戏里常见（每个工作线程一个队列，主线程汇总）
MPMC	两端都有竞争；ABA（无锁栈 / 链表的经典问题，第 9 章）；内存回收（链表节点何时可以释放）；在高竞争下 CAS 失败重试也会“活锁”式地浪费	带序列号的环形数组；hazard pointer / epoch 回收；或者干脆用成熟库（.NET 的 `ConcurrentQueue<T>`、C++ 的 moodycamel::ConcurrentQueue）

面试注意：为什么不自己写无锁

“ThreadSafeRingBuffer 为什么不用 lock-free？”的好答案是：这个场景（工作线程偶尔给主线程发几条消息）lock 的吞吐完全够用，而无锁代码难写、难测、难证明正确；真需要时优先用 `ConcurrentQueue<T>` 或针对固定拓扑（每线程一个 SPSC）的简单结构。能说出 SPSC 的写法和内存序要点，证明你懂；选择不写，证明你有工程判断。

12.6 false sharing

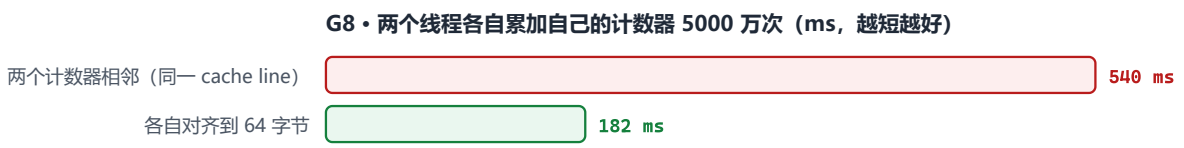


图 12-2 实测 G8。两个线程写的是**不同的变量**，逻辑上没有共享；但两个变量落在同一条 64 字节缓存行里，每次写都要让对方核的缓存行失效、再把整行搬过来，结果慢了约 3 倍

- **原理**：缓存一致性协议（如 MESI）以 cache line 为单位。一个核写了某行，其他核持有的这一行副本就失效。两个线程各写各的变量，只要在同一行，就会互相“踢”对方的缓存。
- **和本类的关系**：lock 版本里，锁对象、head、tail、size 都挤在一起，生产者和消费者每次都在同一组缓存行上竞争——即使临界区很短，缓存行也在两个核之间来回搬，这是 G7 里 lock 版慢的原因之一。
- **解决**：按写者分组并填充对齐（C++ `alignas(64)`，C# `[StructLayout(LayoutKind.Explicit)]` + `FieldOffset`）；每线程统计数据各自累加，最后汇总。

12.7 现有测试验证了什么

RingBufferStressTest.cs（节选）

```
var buffer = new ThreadSafeRingBuffer<int>(100);
Task producer = Task.Run(() => { for ( ... ) { while (!buffer.TryEnqueue(1)) Thread.Sleep(0); Ir
Task consumer = Task.Run(() => { while (consumed < N) { if (buffer.TryDequeue(out val)) {  ... ;
Task.WaitAll(producer, consumer);                    // 在 Start() 里同步等待
if (producedCount == N && consumedCount == N) Debug.Log("测试通过！数据完美守恒…");
```

缺陷	为什么
验证不了顺序和内容	入队的值恒为 1，乱序、重复、丢失后又多读一个，计数都可能对得上。G7 的做法是给每个值编号，逐个检查同一生产者内严格递增，并核对校验和
验证不了“丢失”	消费者“消费到 N 个才停”，如果真丢了数据，测试不是失败而是**永远卡住**，而它又在主线程 `Task.WaitAll`——编辑器会直接冻结
只有 SPSC	一个生产者一个消费者，碰不到多生产者抢 tail 的情况；而 lock 版本真正被设计来应对的就是多生产者
其他	`Thread.Sleep(0)` 只让给同优先级线程，忙等会占满一个核；没有挂在任何场景里；“未发生死锁或丢包”的结论超出了测试能证明的范围

同理，LogTest 的多线程测试（10 线程 × 500 条）只检查“没崩溃、文件变大了”，最后提示“请人工检查日志是否乱码”——它是一个手动观察工具，不是自动化测试。

12.8 Unity 里用工作线程的边界

- **Unity API 只能在主线程调用**：GameObject、Transform、组件、`FindObjectOfType`、大部分 `Application.*`。工作线程第一次访问 `XxxSystem.Instance` 时如果实例还没缓存，会触发 FindObjectOfType / new GameObject 而报错；应在主线程取好引用再交给工作线程。
- **Debug.Log 可以在工作线程调用**（Log.cs 的注释也这么说），但它会捕获堆栈，较慢。
- **正确的拓扑**：工作线程只做纯计算，结果通过线程安全队列交回主线程，主线程在固定时机消费——ThreadSafeEventSystem 的“派发在主线程 Update”就是这个思路。
- **更 Unity 的做法**：C# Job System + Burst 处理数据并行（NativeArray 只能放 unmanaged 类型，所以第 10 章说的“TimerData 不是 blittable”在这里会成为直接障碍）；IO 和网络用 async/await 回到主线程的同步上下文。

12.9 面试追问树

追问树 · 线程安全队列

```
Q12-1 ThreadSafeRingBuffer 是怎么保证线程安全的？
 ├── Q12-2 它是 lock-free 吗？lock-free 的定义是什么？
 ├── Q12-3 lock 的成本是什么？什么是 contention？
 │    └── Q12-4 你测过多生产者吗？                          ← G7：4 生产者吞吐降到约 1/4
 └── Q12-5 为什么不用 lock-free？
      ├── Q12-6 SPSC 怎么写成无锁？需要注意什么？           ← 内存序
      └── Q12-7 MPMC 难在哪？
Q12-8 false sharing 是什么？和你的队列有关系吗？
Q12-9 你的压力测试能证明什么？                             ← 质疑
Q12-10 Unity 工作线程能调用你的单例吗？                    ← 边界
Q12-11 C++ 里 std::mutex、自旋锁、atomic 怎么选？          ← 发散
```

Q12-1 → Q12-5　怎么保证线程安全？为什么不用 lock-free？

L1 • 30 秒　每个操作整体放在同一把 lock 里，入队、出队、Peek、Count 互斥，所以是 lock-based 的线程安全有界队列，不是无锁。

L2 • 正常回答　lock 在无竞争时只是一次原子操作，很便宜；竞争激烈时线程要自旋、甚至进内核等待，开销主要在交接锁和缓存行在核之间搬运。我用 C++ 按同样的结构测过：一进一出每秒约 2400 万次；四个生产者时总吞吐反而降到约四分之一。对游戏里工作线程偶尔回传结果的场景，这完全够用。

L3 • 深入　要无锁，先得改掉 size：它被两端同时修改。改成 head 只归消费者写、tail 只归生产者写，单生产者单消费者就能用 acquire/release 内存序代替锁，同样的测试快了约 9.5 倍。多生产者就要用 CAS 抢槽位，还要处理“抢到但没写完”的槽，复杂度上一个台阶；这时我会直接用 ConcurrentQueue，或者给每个工作线程一个 SPSC 队列。

L4 • 面试官攻击　“lock 版本在主线程用有什么风险？” → 吞吐不是问题，延迟的不确定性才是：如果工作线程持锁时被操作系统调度走，主线程拿锁就要等一个时间片，表现为偶发卡顿。所以主线程侧最好一次性把队列“换”出来（双缓冲交换，持锁时间极短），再在锁外慢慢处理——ThreadSafeEventSystem 的 swap 就是这个思路。

Q12-6　SPSC 写成无锁要注意什么？

L2　三点：每个变量只有一个写者；写数据之后用 release 发布 tail，读 tail 用 acquire，保证消费者看到新 tail 时也能看到数据，反过来 head 同理；head 和 tail 分开对齐到缓存行，避免 false sharing。容量取 2 的幂，用单调递增计数加位与取下标，满是 tail − head == capacity，空是 head == tail。

L4 攻击　“C# 里能写吗？” → 能，用 Volatile.Read / Volatile.Write 或 Interlocked；但要注意 IL2CPP 在 ARM 上的内存模型比 x86 弱，x86 上“碰巧正确”的代码在手机上可能出错，所以内存序必须显式写出来，不能依赖平台。

Q12-9　你的压力测试能证明什么？（质疑）

L2　只能证明“一个生产者、一个消费者时，生产和消费的数量相等、没有崩溃”。它入队的值恒为 1，证明不了顺序和内容；如果真丢了数据，它不会失败而是在主线程上永远卡住；也没有覆盖多生产者。我后来在 C++ 移植版里给每个值编号，逐个检查顺序和校验和，并测了四个生产者，这才算真正验证。

Q12-11　std::mutex、自旋锁、atomic 怎么选？（发散）

L2　单个整数的计数、标志用 atomic；临界区极短且竞争不激烈、并且确定持锁线程不会被换出（比如固定核心的引擎线程）时可以用自旋锁；其他情况用 mutex——它在竞争时会让出 CPU，

不会白白烧核。还要知道“锁的粒度”：一把大锁简单但竞争大，分段锁、读写锁、无锁结构是逐步降低竞争的手段，复杂度也逐步上升。

常见错误回答

错误：“我的队列加了锁，所以是线程安全的，压力测试十万次没丢包。”

问题：压力测试只计数、值恒为 1，证明不了“没丢包”（丢了只会卡住）；只测了单生产者。

更好的回答：“每个操作都在同一把锁里，所以线程安全；原来的压力测试只验证了 SPSC 下的计数，我后来补了带编号的顺序校验和多生产者测试。”

12.10 发散：面试官会从这里追到的知识

死锁四条件

互斥、持有并等待、不可剥夺、循环等待。ThreadSafeEventSystem 有两把锁（字典锁、交换锁），但两者从不嵌套持有，所以不会死锁——这是可以主动说明的点。

内存模型

x86 是强内存序（TSO），只有“写后读”可能重排；ARM 是弱内存序，几乎都可能重排。手游跑在 ARM 上，无锁代码必须显式使用 acquire/release。

条件变量与阻塞队列

如果消费者没数据时应该睡眠而不是轮询，要用 condition_variable（C++）/ Monitor.Wait（C#）/ BlockingCollection；Try 接口 + Sleep(0) 的轮询会占满一个核。

游戏引擎的任务系统

UE 的 TaskGraph、Unity 的 Job System 都是“工作窃取”的线程池：每个线程一个本地双端队列，空了去别人队尾偷——本质上是把 MPMC 拆成多个“几乎 SPSC”的队列来降低竞争。

本章速记

- **lock-based 线程安全有界队列，不是 lock-free**；Try 接口不等于无锁。项目里**没有任何业务在用。**
- G7：lock 版 1P1C **23.8 M/s**，4P1C **5.5 M/s**（竞争），无锁 SPSC **226.6 M/s。**

- SPSC 无锁三要点：**每个变量单写者、release/acquire、head/tail 分行对齐**；size 两端共写，所以原设计改不成无锁。
- G8：false sharing 让互不相关的两个计数器**慢约 3 倍。**
- 压力测试：值恒为 1、丢包会卡死主线程、只有 SPSC——**只证明了计数守恒。**

▶ **自测 1：为什么在 SPSC 无锁队列里，生产者写 tail 必须用 release，而不能用普通写?**

▶ **自测 2：G7 中 4 个生产者时总吞吐为什么比 1 个还低?**

CHAPTER 13

Log 与工具：编译期剥离、缓冲落盘与线上排查

日志看起来不是 Gameplay，但它是线上排查的唯一窗口。这一章讲清 [Conditional] 到底剥离了什么、为什么项目里的日志在手机上会全部消失、缓冲落盘为什么快以及它丢的是哪几行。

本章目标

- 说清 `[Conditional]` 与 `#if` 的区别，以及“调用连同参数一起被删除”的含义
- 发现项目宏只对 Standalone 平台定义带来的后果
- 说清缓冲落盘的收益（实测）与代价（崩溃丢日志、主线程 IO、锁内 IO）
- 找出 Log 与 LogTest 之间的三处不一致
- 回答“线上怎么排查问题”：分级、采样、环形缓冲、崩溃上报

13.1 两层开关：[Conditional] 和 #if

Log.cs（节选）

```
public static class Log {
    [Conditional("ENABLE_CONSOLELOG"), Conditional("ENABLE_LOGSAVE")]      // ① 调用点：两个宏都
    public static void N(string s) {
#if ENABLE_CONSOLELOG                                                      // ② 方法体：按宏编译
        UnityEngine.Debug.Log($"<color=green>系统信息:{s}</color>");
#endif
#if ENABLE_LOGSAVE
        AppendToFile($"[Notice] {s}");
#endif
    }
    // D(string) / E(string) 同理；E 用 Debug.LogError
}
```

	`[Conditional("X")]`	`#if X`
作用位置	标在方法上，影响**所有调用这个方法的地方**	包住任意代码块
判断依据	**调用方**所在程序集编译时是否定义了 X（多个 Conditional 是“或”关系）	当前文件编译时是否定义了 X
未定义时	方法本身还在，但调用语句被删除，**实参表达式也不会求值**	代码不存在
限制	方法返回值必须是 void	无

“实参不求值”是关键收益：`Log.N($"[GameState] 玩家得分！当前分数：{PlayerScore}")` 在宏未定义时，连字符串插值的分配都不会发生。如果只用 `#if` 包住方法体，调用方仍然会先拼好字符串再传进一个空方法。

> **这和 C++ 的日志宏是同一个思路**
>
> C++ 里写 `#define LOG(...) ((void)0)`，或者 UE 的 `UE_LOG` 在 Shipping 下按 Verbosity 编译期剔除，目的都是“连参数一起删掉”。函数版本的空实现做不到这一点（参数在调用前已经求值，除非编译器能证明没有副作用）。

13.2 宏只对 Standalone 定义：手机上日志全部消失

【源码事实】`ProjectSettings.asset` 里的定义是 `scriptingDefineSymbols: Standalone: ENABLE_CONSOLELOG`——只有 PC 平台定义了 ENABLE_CONSOLELOG，ENABLE_LOGSAVE 在任何平台都没有定义。

- 切到 Android / iOS 构建：两个宏都没定义，项目里 32 处 `Log.N/D/E` 调用**全部被删除**，包括 `Log.E`——比如 World 的“场景中存在多个裁判”“越权注册计分板”这类错误，在手机上不会有任何输出。
- 编辑器使用**当前活动构建目标**的宏：在编辑器里切换平台到 Android，编辑器里的日志也会一起消失，容易让人误以为代码没执行。
- 项目里另有 10 处直接调用 `Debug.Log`（Demo 与菜单脚本），它们不受这两个宏控制——同一个项目里两套日志开关。

面试注意：错误日志不应该和调试日志一起剥离

合理的分级是：Debug 级在发布版剥离；Notice 级按需保留；Error 级永远保留（并上报）。现在三个级别用同一组宏，等于“要么全有，要么全无”。UE 的做法是按 Verbosity（Fatal / Error / Warning / Display / Log / Verbose / VeryVerbose）分级，并能按日志类别和构建配置分别设置编译期与运行期的阈值。

13.3 缓冲落盘：快在哪，丢在哪

Log.cs（ENABLE_LOGSAVE 部分，节选）

```
private const int BUFFER_SIZE = 2048;                          // 字符数（StringBuilder.Length）
private static bool _deleteOldWhenRestart = true;
public static void InitLogSave() {
    if (_isInitialized) return;                                // 双重检查（字段不是 volatile）
    lock (_lock) {
        if (_isInitialized) return;
        _logPath = Path.Combine(Application.persistentDataPath, "Log.txt");   // 只能在主线程访
        if (_deleteOldWhenRestart && File.Exists(_logPath)) File.Delete(_logPath);   // ← 每次
        Application.quitting += Flush;                         // 只有正常退出才刷盘
        _isInitialized = true;
    }
}
private static void AppendInternal(string msg) {
    string time = DateTime.Now.ToString("HH:mm:ss.fff");     // 锁外格式化 ✓
    lock (_lock) {
        _buffer.AppendLine($"[{time}][T:{threadId}]{msg}");
        if (_buffer.Length ≥ BUFFER_SIZE) FlushInternal(); // 锁内做文件 IO
    }
}
private static void FlushInternal() { File.AppendAllText(_logPath, _buffer.ToString()); _buffe
```

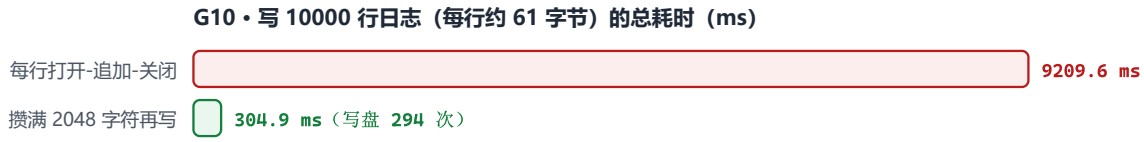


图 13-1 实测 G10（demos/infra_output.txt；Windows 11 + NTFS，含杀毒软件实时扫描，绝对值受环境影响很大，只看比例）。`File.AppendAllText` 每次都要打开、定位、写入、关闭文件，系统调用和文件系统开销远大于写入本身；按 2048 字符攒批后约快 30 倍（逐行写平均每行 921 μs）

代价	说明
崩溃丢日志	缓冲区最多攒约 33 行（G10：2048 ÷ 61）；只在攒满或 `Application.quitting` 时写盘，崩溃、被系统杀进程时不会触发 quitting——**最需要的那几行恰好丢了**
启动时删除旧日志	`_deleteOldWhenRestart = true`：崩溃后玩家重新打开游戏，上一次的日志在初始化时被删除。“崩溃 → 重启 → 拿日志”这条排查路径被切断
Log.E 不会立即写盘	错误和普通日志走同一个缓冲；LogTest 的“测试 Log.E 立即落盘”期望与实现不符
主线程同步 IO	谁写满了缓冲区谁就负责写盘；主线程打日志恰好触发时，这一帧多出一次文件 IO
锁内 IO	FlushInternal 在锁里执行：一个线程写盘时，其他所有线程打日志都要等
双重检查锁	`_isInitialized` 不是 volatile。在 .NET 的内存模型和 x86 上通常没问题，但按规范应该用 volatile 或 `Lazy<T>`；另外第一次在工作线程打日志时会访问 `Application.persistentDataPath`（主线程 API），注释也提醒了要在主线程先初始化
时间戳	`DateTime.Now` 每次都要做时区换算；高频日志可用 `DateTime.UtcNow` 或启动以来的单调时钟

成熟方案：专门的日志线程 + 无锁或双缓冲队列（主线程只入队，不碰文件）；Error 级立即刷盘；进程内保留最近 N 条的环形缓冲，崩溃时随崩溃报告一起上传；按启动时间生成文件名并保留最近几份，而不是删除上一份。

13.4 Debug.Log 本身的成本

- 每次调用都会捕获托管堆栈（可在 Player Settings 的 Stack Trace Logging 里按级别关掉），并把消息送到原生层的日志系统，是项目里最贵的“看不见的”开销之一。
- Log 给每条消息包了 `<color>` 富文本标签，调用方（如 CombatPawn）又包了一层，控制台里是嵌套标签；写进文件时这些标签会原样保留（文件路径下 AppendToFile 收到的是未包装的 `[Notice] {s}`，但 s 本身可能已经含有调用方加的标签）。
- 只要 ENABLE_CONSOLELOG 定义了，所有 Log 调用的字符串插值都会执行——第 10 章 GC 审计里“每次夺舍的日志字符串分配”就是这个。

13.5 Code Review：Log 问题表

级别	问题	影响
P1	宏只定义在 Standalone，Error 级与调试级一起剥离	手机包没有任何框架日志，包括错误
P1	启动删除旧日志 + 崩溃不刷盘	崩溃现场的日志不可得
P2	主线程同步写盘、锁内 IO	偶发卡顿；多线程打日志互相阻塞
P2	Debug.Log 与 Log 两套开关并存	统一剥离、统一上报都做不到
P3	DCL 字段非 volatile、DateTime.Now、富文本标签进日志	规范性与开销
测试	LogTest 是 OnGUI 手动面板；期望 Log.E 立即落盘（与实现不符）；多线程测试最后提示“请人工检查是否乱码”	手动工具，不是自动化测试

13.6 面试追问树

追问树 · Log

```
Q13-1 你的 Log 和直接用 Debug.Log 有什么区别？
  ├── Q13-2 [Conditional] 和 #if 有什么区别？          ← 实参不求值
  └── Q13-3 打包到手机上，你的日志还在吗？             ← 只对 Standalone 定义
Q13-4 为什么日志要先缓冲再写盘？
  ├── Q13-5 游戏崩溃了，最后几行日志在哪？             ← 丢失 + 启动时删除
  └── Q13-6 写盘在哪个线程？会不会卡主线程？
Q13-7 线上玩家反馈 bug，你怎么拿到现场信息？          ← 发散
```

Q13-1 → Q13-3　和 Debug.Log 的区别？手机上还有吗？

L1 • 30 秒　包了一层带分级的静态类，用 Conditional 特性做编译期开关：宏没定义时，调用连同参数一起被删掉，字符串拼接也不会发生；另有一个宏控制缓冲落盘。

L2 • 正常回答　Conditional 和 #if 的区别是：#if 删的是它包住的代码；Conditional 标在方法上，删的是所有调用点，而且实参不求值，这是零成本剥离的关键。限制是方法必须返回 void，判断以调用方程序集的宏为准。

L3 • 深入（主动说问题）　我复盘时发现宏只给 Standalone 平台定义了，打 Android 包时所有框架日志都会消失，包括错误级。错误日志应该永远保留并上报，只有调试级才剥离；现在三个级别共用一组宏，这是要改的。

Q13-4 → Q13-6　为什么缓冲？崩溃时最后几行在哪？

L2　逐行写文件每次都要打开、追加、关闭，系统调用开销远大于写入本身；我测过一万行，逐行写约 9.2 秒、攒 2048 字符再写约 0.3 秒（环境里有杀毒扫描，只看比例，约 30 倍）。

L3　代价是崩溃时缓冲区里最多三十几行没写出去——而且只在正常退出时刷盘，崩溃不会触发；更糟的是初始化时会删掉上一次的日志文件，崩溃后重启连之前写出去的部分也没了。写盘发生在“谁写满了缓冲就由谁写”，主线程可能因此卡一下，并且是在锁里写，其他线程都要等。改进是独立日志线程、错误级立即刷盘、保留多份历史文件，以及进程内环形缓冲随崩溃报告上传。

Q13-7　线上玩家反馈 bug，怎么拿到现场信息？（发散）

L2 几层手段：崩溃时由崩溃收集 SDK 上报调用栈和最近 N 条日志（进程内环形缓冲）；非崩溃问题靠关键流程埋点（如“开始夺舍 / 夺舍成功 / 比赛结束”）和错误日志上报，按版本、机型聚合；需要复现时用远程开关临时提高某个玩家的日志级别，或者打开录像 / 输入回放。还要注意日志里不能有隐私数据，量要可控（采样、限频）。

常见错误回答

错误：“发布版本我把日志宏关掉，所以日志零开销。”

问题：项目里还有 10 处直接 Debug.Log 不受宏控制；而且关掉的同时连错误日志也没了，线上出问题无从查起。

本章速记

- `[Conditional]`：删除**调用点**，**实参不求值**，按调用方程序集的宏判断；`#if`：删除代码块。
- 宏**只对 Standalone 定义**：手机包里 32 处 Log 调用全部消失（包括 Error）；编辑器跟随活动构建目标。
- 缓冲落盘约快 **30 倍**（G10），但崩溃**最多丢约 33 行**、只在正常退出时刷盘、**启动时删除旧日志**、主线程且锁内 IO。
- LogTest 是手动面板，期望 Log.E 立即落盘——**与实现不符。**

▶ **自测：`Log.N(BuildExpensiveReport())` 在两个宏都没定义时，BuildExpensiveReport 会被调用吗？如果把 [Conditional] 去掉、只保留方法体里的 #if 呢？**

PART FOUR

第四部分　Demo

框架是否好用，要看用它写出来的玩法。本批次先写主案例 SoulDecay（夺舍生存战）；GamePlayTest2 与通用数据流在后续批次补齐。

CHAPTER 15

SoulDecay：用“夺舍”压测 Gameplay Framework

一个 3 分钟能讲完的小玩法：灵魂每占据一具肉体就开始倒计时，必须不停夺舍别人的身体活下去。它几乎把框架的每个模块都用了一遍，也把几个隐藏问题暴露了出来。

本章目标

- 能用 30 秒介绍玩法，并说清“为什么用这个玩法测试框架”
- 画出开局时间线和一次夺舍的完整调用链（旧肉体、新肉体、AI 灵魂、UI 各发生了什么）
- 解释碰撞结算、寿命计时器与 UI 的协作方式，以及背后的设计取舍
- 找出 Demo 暴露的问题：游戏结束后继续计分、Die 不幂等、默认句柄、规则散落、物理冲突、材质实例
- 能回答“如果做成联网 / 启用对象池 / 写自动化测试”这类发散题

15.1 玩法：一句话、规则表

一句话：玩家是一缕灵魂，附身的肉体有保质期，到期就爆炸；用鼠标点击任意敌方肉体即可“夺舍”，抛弃旧身体；场上不断刷出红蓝两队 AI，体型大的撞到体型小的就把它碾碎。

项	源码事实（SurvivalGameMode / CombatPawn / SoulPlayerController）
阵营	`EFaction { Neutral, Player, RedAI, BlueAI }`；阵营挂在**灵魂（Controller）**上，通过接口 `IFactionMember` 表达
轻型肉体	球体，缩放 0.8，速度 8，寿命 5 s（玩家开局就是这种）
重型肉体	立方体，缩放 1.5，速度 3，寿命 30 s
刷怪	命名循环计时器 `AISpawner`，每 3 s（scaled）在 [−10,10]² 随机位置刷一只；50% 重型、50% 红/蓝；**没有数量上限**
寿命	只有**玩家**附身时才创建寿命计时器；AI 肉体没有寿命，活到被碾碎为止
夺舍	左键射线命中一具**有灵魂**、**不是自己当前肉体**、**阵营不同**的 CombatPawn → `Possess`；玩家得 1 分
碾碎	两具肉体碰撞，缩放**严格更大**且自己有灵魂的一方执行击杀；同阵营不互相碾压；击杀方阵营得 1 分
失败	玩家肉体寿命耗尽爆炸，或被更大的敌人碾碎 → `EndMatch()`
场景	地面 + 四面墙（Boundaries）、静态主相机、计分 UI、寿命文本、ESC 暂停菜单

玩法上的一个推论（面试时可以顺口说）

玩家开局是轻型（0.8），碾不了任何人；只有夺舍到重型肉体（1.5）才能反过来碾碎轻型 AI 得分，而且重型寿命 30 s，远长于轻型的 5 s。所以最优策略是“尽快找重型肉体”。重型与重型、轻型与轻型相撞什么都不会发生（必须严格大于）。

15.2 为什么用这个玩法来测试框架

普通 Demo 里 Possess 只在开局发生一次；夺舍把它变成**核心循环**，每隔几秒就发生一次，而且每次都同时涉及“抢占别人的 Pawn”“抛弃自己的 Pawn”“被踢出的 Controller 如何收场”。同时寿命计时器把 TimerSystem 的创建、停止、回调里销毁自己也变成高频操作。

框架能力	SoulDecay 怎么用	验证了什么	没验证什么
World / Actor	所有肉体和灵魂都在运行时 `AddComponent`，由 World 统一 BeginPlay/Tick	动态生成的 Actor 能正确进入 pending 并开始 Tick	大量 Actor 下 `List.Contains` 注册的 O(n) 成本（没有上限的刷怪会逐渐放大它）
Possess / UnPossess	夺舍 = 玩家 Controller 抢占 AI 的 Pawn	“先放弃自己的、再踢走对方的、最后双向绑定”的顺序；OnUnPossess 能拿到即将离开的 Controller	回调抛异常时的半绑定；Controller 被销毁时自动解绑
阵营（组合）	`IFactionMember` 接口挂在 Controller 上	“身份属于灵魂，不属于肉体”：同一具肉体换了灵魂，颜色和敌我关系随之改变	—
GameMode / GameState	SurvivalGameMode 声明 `GameStateClass = SurvivalGameState`，父类反射生成并担保注册	子类只声明类型即可得到专属计分板	写入权限收口：Pawn 和 Controller 直接调用 AddScore / EndMatch
TimerSystem	每具玩家肉体一个寿命计时器；循环刷怪计时器（委托缓存在字段）	高频创建/停止；回调里停止自己 + 停止别人 + Destroy（第 10 章重入路径）	AI 肉体的默认句柄（第 9 章 E1）——当前参数下不会触发
EventSystem	分数（Appearance）、寿命句柄（Logic）推给 UI	UI 只订阅事件，不引用 GameMode	MatchStateChanged 在本场景**无人监听**：游戏结束没有任何 UI 反馈
ObjectPool	写了 GetOrCreatePawn / RecycleObject	—	初始化被注释，全部是死代码
暂停	ESC → `timeScale = 0`	scaled 计时器冻结；两个 Controller 在 Tick 里检查 timeScale 停止思考	—

15.3 文件与职责

文件	继承 / 实现	职责
SurvivalGameMode.cs	AGameMode	开局：切到 InProgress、生成玩家肉体与灵魂并附身、启动 AISpawner；工厂方法 CreatePawn；EndMatch 时停刷怪；（未启用的对象池接入）
SurvivalGameState.cs	AGameState；同文件定义 EFaction、IFactionMember	三阵营分数；BeginPlay 广播一次 0 分；AddScore 修改并广播
CombatPawn.cs	APawn	移动；OnPossess 按阵营变色、玩家附身时创建寿命计时器并发事件；OnUnPossess 停计时器、玩家离开时枯萎并 3 s 后销毁；碰撞碾碎与加分；Die
SoulPlayerController.cs	AUnityController + IFactionMember	重写移动（交给 CombatPawn.Move）；左键射线夺舍并加分
SimpleAIController.cs	AController + IFactionMember	Perlin 噪声随机游走；失去肉体后 `Destroy(gameObject)` 自毁
SurvivalUIController.cs	MonoBehaviour	OnEnable 订阅分数 / 寿命句柄事件；Update 每帧用句柄拉取剩余时间，低于 1.5 s 红白闪烁，句柄失效显示 DIE
SoulDecayEvents.cs	AbstractEventArgs	`ScoreChangedEventArgs`（Appearance）、`PlayerLifespanTimerEventArgs`（Logic，携带 TimerHandle）
GameObjectWrapper.cs	IPoolable	出池 SetActive + 恢复 Tick；入池关 Tick + SetActive(false) + 清速度（未启用）

15.4 开局时间线

SoulDecayScene 启动（按源码推演；同一阶段内各对象的先后不保证）

```
帧 0 · Awake/OnEnable
  EventSystem.Awake / TimerSystem.Awake（场景预置；二者都没调 base.Awake，_instance 仍为空 → 第 6
  SurvivalUIController.OnEnable → EventSystem.Instance（FindObjectOfType 找到预置对象）→ Regist
帧 0 · Start
  SurvivalGameMode.Start（AActor.Start）→ World.RegisterAActor → World.Instance 为空 → 懒创建 W
帧 0 · Update（World.Update）
  pending → _actors → SurvivalGameMode.BeginPlay：
    AGameModeBase.BeginPlay → RegisterGameMode → InitGameState（反射生成 SurvivalGameState，担保
    StartMatch（InProgress，发事件）
    CreatePawn(0,0,0, 轻型) → AddComponent<CombatPawn>（Awake 取 Renderer）
    new GameObject("Player_Controller").AddComponent<SoulPlayerController>()
    playerSoul.Possess(playerBody)            ← 此时两者都还没 BeginPlay
      └ CombatPawn.OnPossess：变青色 → CreateTimer(5 s, OnBodyExploded) = (0,0) → Trigger(寿命句
    CreateTimer("AISpawner", 3 s, loop) = (1,0)
帧 1 · Start → Update
  SurvivalGameState / CombatPawn / SoulPlayerController 的 Start → pending → BeginPlay
    SoulPlayerController.BeginPlay（AUnityController）：new GameInput()、启用 Player 输入
    SurvivalGameState.BeginPlay：广播三个阵营的 0 分
  EventSystem 派发：阶段×2、寿命句柄、分数×3 → UI 记下句柄、刷新分数
  从这一帧起，玩家可以移动；UI 每帧显示 Last Time: 4.9 s ……
t = 3 s、6 s、9 s …  AISpawner → SpawnRandomAI → 新肉体 + AI 灵魂（先设阵营，再 Possess）
```

面试注意：“先 Possess，后 BeginPlay”

玩家的灵魂和肉体在帧 0 就完成了附身，而它们的 BeginPlay 要到帧 1 才执行。所以 OnPossess 里不能依赖 BeginPlay 中初始化的东西——CombatPawn 把 `GetComponent<Renderer>` 放在 Awake 里，DemoHeroPawn 的注释里也记录了这次修复。UE 的 `RestartPlayer` 顺序相反：先 Spawn（BeginPlay 已执行）再 Possess。

15.5 一次夺舍的完整调用链

玩家（在肉体 Old 里）点击红队 AI 的肉体 Target

```
SoulPlayerController.Tick → Mouse 左键 → ExecuteSoulTransfer
  Raycast 命中 Target：不是 Old、Target.Controller ≠ null、阵营 RedAI ≠ Player   ✓
  this.Possess(Target)
  ├─① ControlledPawn(Old) ≠ null → this.UnPossess()
  │     ├ Old.UnPossessed()
  │     │   └ CombatPawn.OnUnPossess    （此刻 Old.Controller 仍是玩家，先存为 leavingController）
  │     │       ├ StopTimer(Old 的寿命计时器)，_decayTimer = Invalid
  │     │       └ leavingController 是玩家 → 变黑、改名 "Dead Body"、关碰撞体、Destroy(Old, 3 s)
  │     │     Old.Controller = null
  │     ├ this.OnUnPossess(Old)              （SoulPlayerController 没重写）
  │     └ ControlledPawn = null
  ├─② Target.Controller(AI) ≠ null → AI.UnPossess()     ← UE 默认行为；源码里注释掉的那行就是它
  │     ├ Target.UnPossessed()
  │     │   └ CombatPawn.OnUnPossess：StopTimer(_decayTimer)   ⚠ AI 肉体的 _decayTimer = defaul
  │     │     离开的是 AI → 保持“新鲜”，只打日志
  │     └ SimpleAIController.OnUnPossess → Destroy(AI 灵魂)    （帧末真正销毁）
  ├─③ Target.PossessedBy(this) → Target.Controller = 玩家 → CombatPawn.OnPossess
  │       变青色 → CreateTimer(Target.MaxLifespan, OnBodyExploded) → Trigger(寿命句柄事件)
  ├─④ ControlledPawn = Target
  └─⑤ this.OnPossess(Target)
  GameState.AddScore(Player, +1) → Trigger(ScoreChanged)
下一次 EventSystem.Update：UI 收到新句柄 → 寿命文本切到新肉体
```

这条调用链回答了三个常被问的问题：

- **为什么 OnUnPossess 要先通知子类、再清空 Controller?** （APawn.UnPossessed 的注释原话：“先通知子类，拿到当前控制器的引用做一些判断”）因为 CombatPawn 要根据“离开的是谁”决定是枯萎还是保持新鲜。
- **为什么 AI 失去肉体要自毁，玩家不自毁?** AI 灵魂没有任何独立价值（没有输入、没有存档、没有 UI），留着只会每帧空转 Tick；玩家的 Controller 承载输入和“我是谁”，换身体时必须保留——这正是 Controller 与 Pawn 分离的意义。把“失去肉体后怎么办”放在各自 Controller 的 OnUnPossess 里，而不是让 Pawn 去判断，是正确的职责划分。
- **为什么阵营挂在 Controller 而不是 Pawn?** “身份属于灵魂”：同一具肉体，被红队 AI 控制时是红色敌人，被玩家夺舍后立刻变成玩家阵营。用接口而不是基类字段，是组合优于继承的体现：不是所有 Controller 都需要阵营。

推断：夺舍那一刻 UI 可能闪 1 帧 “DIE”

旧计时器在 World.Update（Controller.Tick）里被停掉，新句柄通过事件送给 UI。若这一帧 UI.Update 在 EventSystem.Update 之前执行，UI 手里还是旧句柄，`IsHandleValid` 为 false → 显示 “DIE” 一帧。是否出现取决于脚本执行顺序（第 1 章 1.4）。修法：事件改为同步通知，或 UI 不缓存句柄而是向 GameState 查询“玩家当前寿命”。

15.6 碰撞结算：双方都收到 OnCollisionEnter，怎么只结算一次

CombatPawn.cs:110-151（节选）

```
private void OnCollisionEnter(Collision collision) {
    if (Controller is IFactionMember mySoul) {                          // 只有活体能碾压
        CombatPawn otherPawn = collision.gameObject.GetComponent<CombatPawn>();
        if (otherPawn ≠ null && transform.localScale.x > otherPawn.transform.localScale.x) {
            if (otherPawn.Controller is IFactionMember otherSoul && otherSoul.FactionId == myS
                return;                                                 // 同阵营保护
            bool isCrushingPlayer = otherPawn.Controller is IFactionMember v && v.FactionId ==
            otherPawn.Die(isExplosion: isCrushingPlayer);               // 碾碎玩家 = 游戏结束
            if (World.HasInstance && World.Instance.GameState is SurvivalGameState gs)
                gs.AddScore(mySoul.FactionId, 1);                       // Pawn 直接写 GameState
        }
    }
}
```

Unity 的物理会给碰撞的**双方**各调用一次 OnCollisionEnter。代码用“严格比较体型”保证只有一方执行：A 大于 B 时只有 A 进入分支；体型相等时双方都不进入——所以相同体型永远不会互相碾压。这是一个简单有效的“单一裁决者”技巧。

【问题】Die 不幂等 → 可能重复计分

如果同一个物理步里两具大肉体同时撞上同一具小肉体：第一次结算调用 `victim.Die()`，Destroy 要到帧末才生效，受害者在这一帧里仍然存在；第二次结算时 `victim.Controller` 已经为 null，同阵营保护判断被跳过，再次 `Die()` 并**再加一次分。** `isDying` 只用来阻止“延迟销毁”，没有在 Die 的入口处拦截。修复：Die 开头 `if (isDying) return;`，并让 OnCollisionEnter 在受害者 isDying 时直接返回。

15.7 寿命计时器与 UI：push 句柄，pull 剩余时间

寿命数据流

```
CombatPawn.OnPossess（玩家）
  _decayTimer = TimerSystem.CreateTimer(MaxLifespan, OnBodyExploded)
  EventSystem.Trigger(new PlayerLifespanTimerEventArgs(_decayTimer))      ← push：只推一次“票据
SurvivalUIController.OnLifespanTimerChanged：_currentPlayerTimer = args.DecayTimerHandle
SurvivalUIController.Update（每帧）
  IsHandleValid(_currentPlayerTimer) ? GetTimeRemaining( ... ) → "Last Time: 3.2 s"    ← pull：每
                                     : "DIE"
```

方案	做法	优点	缺点
push 句柄 + pull 数值（本项目）	事件只传句柄，UI 每帧查询	不必每帧发事件；句柄带 generation，旧句柄自动失效，UI 不会读到别的计时器	UI 直接依赖 TimerSystem；UI 自己有 Update；切换时可能闪 1 帧
每帧 push 数值	Pawn 每帧发“剩余 x 秒”事件	UI 完全被动	每帧一个事件对象 + 闭包分配；队列压力
GameState 持有可观察属性	GameState 暴露 PlayerLifespan，UI 绑定	UI 只依赖 GameState，符合第 5 章的边界	需要 GameState 每帧更新或按需计算

面试里这一段的亮点是：**UI 敢长期持有一个 TimerHandle，正是因为它带 generation**——肉体爆炸或被抛弃后计时器被停止，UI 手里的句柄自动失效，显示 “DIE”，而不会读到下一个复用该槽位的计时器。这把第 9 章的设计和 Demo 连了起来。

15.8 游戏结束之后发生了什么

寿命耗尽 → GAME OVER

```
TimerSystem 阶段二B → OnBodyExploded → Die(isExplosion: true)
  ├ isDying = true
  ├ 玩家 + 爆炸 → World.AuthorityGameMode.EndMatch()            （Pawn 直接调用裁判）
  │    └ SurvivalGameMode.EndMatch → AGameMode.EndMatch → SetMatchState(WaitingPostMatch) → Tr
  │                                                          ⚠ 本场景没有任何监听者 → T
  │      → StopTimer("AISpawner")                             （重入：停止另一个计时器）
  ├ Controller.UnPossess() → OnUnPossess → StopTimer(自己)    （重入：停止正在执行的自己）
  │                          → 变黑、关碰撞体（isDying，不再延迟销毁）
  └ Destroy(gameObject)
UI：句柄失效 → “DIE”
```

现象（源码推演）	原因	该归谁管
没有“游戏结束”界面	MatchStateChanged 无人监听（GamePlayTest2 的 DemoMatchUI 监听了，SoulDecay 没有）	UI 订阅 GameState 的阶段事件
结束后分数还在变	剩余 AI 继续游走、互相碾碎；`AddScore` 不检查比赛阶段	GameMode 裁决：非 InProgress 时拒绝加分
玩家灵魂留在场上，没有身体也没有重开	SoulPlayerController 不自毁，也没有 Restart 流程	GameMode 的 RestartPlayer / 结算流程
AI 持续增长（结束前）	刷怪没有上限，只有互相碾碎会减少	GameMode 规则参数

这几条放在一起，就是第 5 章要讨论的核心问题：**规则没有收口到 GameMode**。Pawn 决定了“谁加分”“何时结束”，GameState 的 AddScore 是 public 的且不校验阶段，所以“比赛已结束”这条规则没有地方生效。

15.9 对象池为什么没启用？启用后会遇到什么

【源码事实】三个池子字段已声明、GetOrCreatePawn / GetOrCreateAIController / RecycleObject 已写好，但池子的初始化被注释，SpawnRandomAI 里调用池子的两行也被注释，没有任何地方调用 RecycleObject。【设计意图】从 GameObjectWrapper 出入池时开关 Tick 可以看出，作者已经意识到“入池对象不能继续被 World Tick”。

如果现在把它打开，会遇到一串**生命周期**问题——这也是“对象池最难的不是数据结构，而是复位”的最好例子：

问题	原因
第二次出池不会再 BeginPlay	AActor 在 `Start` 里注册，Start 每个组件一生只调用一次；`HasBegunPlay` 一直为 true。出池需要单独的“重新初始化”钩子
入池对象仍在 World 的名单里	SetActive(false) 不会触发 OnDestroy，World 不会注销它；只能靠 IsTickEnabled = false 阻止 Tick，但名单会一直变长
isDying 不会复位	CombatPawn 的状态字段没有 Reset；复用的肉体第一次被抛弃时不会延迟销毁
碰撞体被关掉、名字被改成 Dead Body、材质变黑	OnUnPossess 里的副作用都需要在出池时撤销（GetOrCreatePawn 里补了名字和碰撞体，但碰撞体是 `enabled = false` 而不是被删除，“为空才 AddComponent”的判断永远不成立）
死亡路径仍然 Destroy	Die 和 OnUnPossess 都直接 Destroy，没有改成 RecycleObject
旧 PoolObject 拷贝能回收别人的对象	PoolObject 只有 index 没有 generation（第 11 章，实测 E8）
参数不一致	GetOrCreatePawn 里轻型寿命 4.5 s、重型 8 s；CreatePawn 里是 5 s、30 s——死代码与活代码已经分叉

15.10 Demo 暴露的问题清单

级别	问题	位置	影响当前 Demo?
P0	AI 肉体的 `_decayTimer` 是 default (0,0)，死亡时会停掉槽位 0 的计时器	CombatPawn.cs:13 / 73 / 174	当前参数下不会触发（第 9 章 E1）
P1	游戏结束后继续计分；AddScore 不校验比赛阶段	CombatPawn.OnCollisionEnter、SurvivalGameState.AddScore	会：结束后画面上的分数仍在变化
P1	Die 不幂等，可能重复计分	CombatPawn.Die	需要同一物理步多重碰撞，偶发
P1	规则写在 Pawn / Controller：Pawn 直接 EndMatch、直接加分	CombatPawn、SoulPlayerController	功能正常；架构问题
P1	游戏结束无 UI 反馈，无重开流程	SurvivalUIController 未订阅阶段事件	会
P2	带 Rigidbody 的物体在 Update 里 `transform.Translate`，绕过物理（插值失效、可能穿透墙体再被弹回）	CombatPawn.Move	高速（轻型速度 8）时更明显
P2	`_renderer.material` 每次访问会为该 Renderer 实例化一份材质；Unity 文档要求自行销毁，否则这些材质实例要等到卸载未使用资源时才释放；也会打断合批	CombatPawn.OnPossess / OnUnPossess	每具肉体至少一份材质实例
P2	每次玩家附身：方法组委托、事件对象、事件闭包、日志字符串插值都会分配	CombatPawn.OnPossess	频率低，影响小
P2	刷怪无上限，World 注册 O(n)	SurvivalGameMode.SpawnRandomAI	长时间存活后逐渐变慢

P3	场景里有两个都叫“EventSystem”的物体（Unity UI 的 EventSystem 和自写的 EventSystem），类名也与 `UnityEngine.EventSystems.EventSystem` 相同	SoulDecayScene	易混淆，不影响运行
P3	重/轻肉体参数硬编码在 GameMode 里；死代码与活代码参数分叉	SurvivalGameMode	—

15.11 发散：如果把它做成真正的游戏

15.11.1 做成联网（状态同步，服务器权威）

联网版夺舍（扩展知识：项目没有实现网络）

```
客户端                                          服务器（GameMode 只在这里）
左键 → 本地射线得到目标 id
     → RPC：RequestPossess(targetId, 客户端时间戳) ──────→  校验：目标存在且存活？阵营不同？距离/冷却
                                                          （延迟补偿：按时间戳回溯目标位置再判断命中）
                                                          通过 → Possess：改 Controller/Pawn 归属
                                                               → 停旧寿命、开新寿命（服务器计时）
                                                               → PlayerState.Score += 1（复制给所有人）
     ←────── 复制：Pawn.Controller/Owner、阵营、寿命到期时刻 ──────
客户端：可先做表现层预测（变色、相机切换），被服务器拒绝时回滚
UI：读 PlayerState / GameState 的复制属性，而不是本地 TimerSystem
```

- **分数放哪**：每个玩家的分数放 PlayerState（每个玩家一份、复制给所有人），全局阶段放 GameState；本项目只有 GameState，三阵营分数混在一起。
- **寿命怎么同步**：不要每帧同步“剩余时间”，而是同步“到期的服务器时刻”，客户端用同步过的服务器时钟自己算剩余时间——这正对应第 10 章“存绝对到期时刻而不是累加 elapsed”。
- **碰撞碾碎**：只在服务器判定；客户端的 OnCollisionEnter 只做表现。

15.11.2 做成帧同步（lockstep）

客户端只上传“第 N 帧点击了目标 X”的输入；所有客户端在同一逻辑帧用同样的确定性逻辑执行夺舍、碰撞、计时。这要求：寿命用逻辑帧计数；物理碰撞改用确定性的自研碰撞（Unity 的 PhysX 物理不保证跨平台确定性）；Perlin 随机游走用同步的随机种子；同帧多个碰撞的结算顺序固定。

15.12 面试追问树

追问树 · SoulDecay

```
Q15-1 介绍一下你的 Demo
 └── Q15-2 为什么选这个玩法来测试框架？
      ├── Q15-3 夺舍时旧肉体、新肉体、AI 灵魂各发生了什么？
      │    ├── Q15-4 AI 失去肉体为什么自毁？玩家为什么不？
      │    └── Q15-5 为什么阵营挂在 Controller 上？
      ├── Q15-6 两个物体碰撞双方都触发回调，怎么只结算一次？体型相同呢？
      │    └── Q15-7 同一帧两个大的撞同一个小的呢？                ← 边界 / Die 不幂等
      ├── Q15-8 UI 怎么显示寿命？为什么敢长期持有 TimerHandle？
      └── Q15-9 游戏结束后发生了什么？                             ← 让你自己发现 bug
Q15-10 对象池为什么没启用？打开会出什么问题？
Q15-11 Pawn 直接调用 GameMode.EndMatch 合适吗？
Q15-12 做成联网版，夺舍请求怎么校验？分数放哪？
Q15-13 怎么给这个 Demo 写自动化测试？
```

Q15-1 / Q15-2　介绍一下你的 Demo，为什么选这个玩法？

L1 • 30 秒　SoulDecay 是一个“夺舍生存战”：玩家是灵魂，附身的肉体有寿命，到期爆炸，必须不停点击敌人的身体夺舍续命；红蓝 AI 不断刷出，大体型碾碎小体型得分。我选它是因为它把 Possess 从“开局一次”变成了“每几秒一次”的核心循环，同时高频创建、停止计时器，正好压测框架最核心的两块。

L2 • 正常回答　具体覆盖了：动态生成的 Actor 由 World 统一 BeginPlay/Tick；夺舍时 Controller 先放弃自己的肉体、再把目标的 AI 灵魂踢出去、最后双向绑定；阵营做成 Controller 上的接口，体现“身份属于灵魂”；GameMode 只声明 GameStateClass，父类反射生成计分板；寿命计时器到期的回调里会停止自己、停止刷怪计时器、销毁肉体，验证了计时器的重入安全；UI 通过事件拿到计时器句柄，每帧查询剩余时间。

L3 • 深入（主动说不足）　它也暴露了框架的问题：加分和判负写在 Pawn 里，游戏结束后分数还会变；AI 肉体的空计时器句柄是默认值，会和第一个计时器冲突，只是当前参数下不会触发；对象池写了但没接入。

Q15-6 / Q15-7　碰撞只结算一次？同一帧两个大的撞一个小的？

L2　双方都会收到 OnCollisionEnter，我用“严格比较体型”保证只有大的一方进入结算，体型相同则都不处理。

L4 攻击　“那两具大肉体同时撞一具小肉体呢？”　→ **修正回答**：这是我的代码的一个漏洞。Destroy 要到帧末才生效，第一次结算后受害者还在，第二次结算会再调用一次 Die 并再加一次分，因为 Die 没有在入口处判断 isDying。应该让 Die 幂等，并且在结算前检查受害者是否已经在死亡流程中。更彻底的做法是把“击杀”作为请求提交给 GameMode，由 GameMode 在一帧内去重后统一结算。

Q15-9　游戏结束后发生了什么？

L2　寿命到期的回调里 Die → EndMatch：阶段切到 WaitingPostMatch、停掉刷怪计时器；然后解绑灵魂、销毁肉体，UI 显示 DIE。

L3　但有三个问题：阶段事件在这个场景没人监听，没有结算界面；剩下的 AI 还在游走互撞，AddScore 不检查阶段，所以分数还在变；玩家的 Controller 没有身体也没有重开流程。根本原因是规则没收口——“比赛结束后不再计分”应该是 GameMode 的规则，但加分是 Pawn 直接调 GameState 完成的，GameMode 根本插不上手。

Q15-11　Pawn 直接调用 GameMode.EndMatch 合适吗？

L2　不合适。Pawn 应该只上报事实（“我死了，凶手是谁，死因是什么”），由 GameMode 决定这是否意味着比赛结束、谁加分。这样规则集中在一处：换一种模式（比如有复活次数），只需要换 GameMode，不需要改 Pawn。UE 里也是类似思路：伤害和死亡事件最终交给 GameMode 处理击杀与计分，计分结果写到 PlayerState / GameState 再复制出去。

L3　具体改法：Pawn 死亡时发一个 Logic 优先级的事件，或调用 GameMode 的 `NotifyKilled(victim, killer)`；GameMode 检查比赛阶段、去重、计分、判断是否结束；

GameState 的 AddScore 改成只有 GameMode 能调用。

Q15-13　怎么给这个 Demo 写自动化测试?

L2　分三层。① 纯逻辑：把“能否夺舍”（阵营、存活、是否自己）、“碰撞谁赢”（体型、阵营）、“加分规则”抽成纯函数，用 EditMode 测试覆盖所有组合，包括体型相等、同阵营、结束后加分。② 框架交互：PlayMode 测试里生成一个 Controller 和两个 Pawn，调用 Possess，断言双向引用、AI 灵魂被销毁、旧肉体 3 秒后销毁、计时器句柄失效。③ 场景冒烟：加载 SoulDecayScene，用 `Time.captureDeltaTime` 之类的手段固定帧时长，跑若干秒，断言没有异常日志、分数单调、结束后分数不变。

L3　当前代码要先做两件事才好测：TimerSystem / EventSystem 的核心逻辑从 MonoBehaviour 中拆出来，可以手动 Tick；随机数和输入通过接口注入，测试里可控。

常见错误回答

错误：“Demo 里用对象池管理了所有肉体，避免了 GC。”

问题：池子没有初始化，实际走的是 new + Destroy；而且就算打开，也会遇到 BeginPlay 不再触发、状态不复位等问题。

更好的回答：“我为肉体和 AI 灵魂写了对象池的接入代码，但发现 Actor 的生命周期（Start 只调用一次、SetActive 不会注销）和池化冲突，还没有接入；要接入需要给 Actor 增加出池/入池的生命周期钩子。”

本章速记

- 玩法：**灵魂附身有保质期，点敌人夺舍续命；大碾小得分**；轻型 0.8/8/5 s，重型 1.5/3/30 s；刷怪每 3 s 无上限。
- 为什么选它：把 **Possess 变成核心循环**，并高频创建/停止计时器、在回调里销毁自身。
- 夺舍链：**放弃旧肉体（停寿命、枯萎、3 s 后销毁）→ 踢出 AI 灵魂（AI 自毁）→ 双向绑定 → 新寿命 + 事件 → 加分。**
- 碰撞：**严格比较体型**保证单方结算；Die 不幂等可能重复计分。
- UI：**push 句柄 + pull 剩余时间**，句柄带 generation 才敢长期持有。

- 结束后：无结算 UI、**分数仍在变**、无重开——根因是**规则没收口到 GameMode。**

▶ **自测 1：玩家在轻型肉体里点击一具红队重型肉体，列出这一帧内被停止的计时器、被安排销毁的对象、分数变化。**

▶ **自测 2：为什么 CombatPawn 的 OnUnPossess 必须在 base.OnUnPossess() 之前读取 this.Controller?**
